ABAP INTERVIEW PREPARATION

ABAP interview handbook

Senior ABAP Cloud, RAP and Clean Core first; supporting ABAP and integration knowledge

353 questions across 25 study segments. Reviewed 13 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Open abap-interview-studio.pages.dev online, or download the repository and open START_HERE.html for offline study. The PDF is a portable reading and printing edition.

Study segments

Senior ABAP Cloud architecture decisions / 18 questions

RAP foundations and transactions / 24 questions

ABAP Cloud: senior RAP transactions and behavior design / 24 questions

CDS data modeling / 12 questions

Senior CDS, query contracts and security / 18 questions

RAP service design and advanced behavior / 15 questions

Senior RAP testing, performance and production / 18 questions

Clean Core levels and extensibility / 18 questions

Senior Clean Core governance and tradeoffs / 18 questions

ABAP foundations / 14 questions

Practical ABAP: values, expressions, and failure paths / 16 questions

OO design and Clean ABAP / 17 questions

Practical OO: contracts, state, and substitutability / 12 questions

Practical ABAP design patterns and trade-offs / 12 questions

ABAP SQL, HANA, and performance / 17 questions

OData Gateway and Fiori integration / 12 questions

BTP and cloud integration for ABAP developers / 10 questions

RICEFW, enhancements, and workflow / 15 questions

Testing security and delivery / 10 questions

Interview answers and project discussion / 7 questions

HTTP, REST, JSON and XML / 14 questions

Function modules, RFC, BAPI, parallel processing, and jobs / 12 questions

IDoc, ALE, AIF, and interface operations / 12 questions

Supporting skills: HANA decisions and source-to-release delivery / 3 questions

Core development mechanisms / 5 questions

Answer structure

Start with a definition and one useful example. Add the main trade-off or failure mode. If syntax or support depends on the SAP release, name that dependency and explain how you would verify it.

Use the practice workbook for scenarios and mock interviews. Use the quick revision PDF for the final review. References at the end of each segment link to official primary documentation. They support the concepts; examples and rehearsal advice are original

instructional material.

Senior ABAP Cloud architecture decisions

Treat each question as a design review: state the business invariant, choose a boundary, explain the rejected alternative, and name the evidence that would validate the decision. ABAP Cloud is the main development model here; deployment, API scope, and operational constraints still differ across SAP products. These scenarios demand decisions and verification rather than memorized framework definitions.

Choose an extension approach

Start with the business requirement, supported contracts and transaction boundary

First	**Can standard functionality or configuration meet the need?**	Use the standard capability and avoid unnecessary custom code.
Key user	**Does a delivered business context cover the adaptation?**	Use supported custom fields, logic or UI adaptation tools.
On-stack	**Does the logic need local transactional consistency?**	Use released local contracts and suitable ABAP Cloud extensibility.
Side-by-side	**Does the process need an independent or cross-system app?**	Use supported remote APIs and events; design retries and consistency.

Placement does not determine the Clean Core level. Check dependencies separately.

Start from the business need and transaction boundary before choosing an extension location.

See the visual atlas for the recall prompt, limits and related questions.

CLOUD-01 A new approval extension needs immediate document validation and an external supplier portal. Would you put everything on-stack or on BTP?

Keep the document check where it can reject the document before saving, using a supported local extension point. The supplier portal can be a separate BTP app if its users and release cycle justify that. Connect them through released remote interfaces. Make pending and failed steps visible. Choose based on timing, APIs, permissions, and recovery rather than putting everything in one place.

Reasoning and example

Suppose every internal posting waits for a live response from the supplier portal. A network outage can now stop all postings, even when the core document itself is valid. If the business permits it, save a valid submission locally and let the portal interaction happen afterward through a reliable integration process.

The user then needs to see whether the request is waiting, accepted, or failed. Track retries with a stable request identity and decide what withdrawal means after submission. Some rules must still reject the local document immediately, so keep those at a suitable supported local point. A separate application does not make remote work part of the same database transaction.

Avoid this mistake Deploying side-by-side does not make synchronous cross-system validation atomic or automatically clean-core compliant.

Interviewer probes Which business rule must be checked before the core document is committed? / How does the user recover when the portal accepts a request but the core system is unavailable?

Strong answer criteria Explains which business steps run in which system; Compares response time and independent release needs; Explains recovery when only part of the process succeeds

Verification evidence An architecture decision record, target API availability checks, and a failure-sequence test showing pending, retry, and withdrawal outcomes.

Read and verify this answer

SAP extension architecture guidance

CLOUD-02 A stakeholder says ABAP Cloud means moving the existing application to SAP BTP ABAP environment. How would you correct the proposal?

ABAP Cloud is a development model, not just a place to run code. It uses restricted language versions, released APIs, and models such as RAP on supported S/4HANA systems and SAP BTP ABAP environment. Moving to BTP is a separate decision. Check the data, business APIs, connectivity, and operational responsibilities available there before promising that an existing app will move unchanged.

Reasoning and example

A class currently reads sales data through a local S/4HANA interface. In a BTP ABAP system, the same business data and local interface may not exist. The new application might need a remote service, a supported replication design, or a different owner for that information. Valid ABAP syntax alone does not solve that difference.

List the application's dependencies and map each to an available target capability. Try the most important integration with realistic input before estimating the move. This separates two questions: can the code follow ABAP Cloud's rules, and is the proposed runtime the right place for the application? Both need evidence, even though they are often discussed together.

Avoid this mistake Cloud-compatible syntax does not establish identical application content, API availability, or deployment portability.

Interviewer probes Which dependencies become remote after the move? / What target-system evidence would you require before estimating migration?

Strong answer criteria Distinguishes model from runtime; Identifies lost data locality; Checks product-specific capabilities

Verification evidence A list of target-system capabilities mapping each dependency to a supported local or remote replacement, plus a tested integration example.

Read and verify this answer

SAP ABAP Cloud development model

CLOUD-03 A developer proposes dynamic calls to an unreleased SAP class because static ABAP Cloud checks reject it. What do you do?

Do not hide an unsupported dependency behind a dynamic call. Check the object's actual release state and look for a released alternative, supported extension point, or remote API on the target version. Revisit standard functionality if needed. An allowed classic boundary may be an option in a suitable private-edition landscape. It must be explicit and governed, not disguised as ABAP Cloud compliance.

Reasoning and example

A dynamically named class call may escape a superficial scan but still fail after an upgrade. The language or runtime may also reject it. Demonstrating that the call worked once does not prove that SAP supports the dependency or will keep its behavior stable.

Write down the exact capability that is missing and test supported alternatives with representative input. Where a classic exception is permitted, isolate it, identify its risk and owner, and keep the implementation within the product's actual rules. Public-edition restrictions do not disappear because another landscape allows classic development. An ATC suppression changes what is reported; it does not create a release contract for the object.

Avoid this mistake An ATC suppression or dynamic name does not create a supported API contract.

Interviewer probes What options remain in Public Edition when no released extension exists? / How would you track an API gap without blocking unrelated delivery?

Strong answer criteria Rejects hidden dependency bypass; Verifies target-specific alternatives; Separates private-edition exceptions from public-cloud rules

Verification evidence ADT API-state evidence, a documented capability gap, and a decision record showing supported alternatives and any approved bounded exception.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-04 Two teams want independent releases but share pricing helpers and persistence types. How would you structure packages and software components?

Give shared code a clear owner and a small, stable interface. Packages organize code and access; software components also affect delivery and lifecycle dependencies. Keep private database types with their owning component. Avoid circular dependencies and define delivery order. Supported component relations can allow targeted access, but access permission alone does not promise that an interface will stay compatible.

Reasoning and example

Two teams say they can release independently, but each uses the other's database row types. A field change can then require both components to activate together. Replacing those types with a narrow pricing request and result reduces the number of changes that callers must follow. The shared pricing capability also needs an owner and compatibility tests.

If that interface is still changing frequently, coordinated releases may be the honest choice for now. Creating more components does not itself remove the dependency. Document the direction of calls, permitted component access, and import order. Check that a provider change still supports previously delivered consumers before describing the components as independently releasable.

Avoid this mistake More software components do not create independent lifecycles when their dependencies still require lockstep delivery.

Interviewer probes When would you choose a component relation instead of a C1 API? / How would an incompatible dependency removal be detected?

Strong answer criteria Shows which component depends on which; Distinguishes permission to call an interface from a promise of compatibility; Matches component boundaries to how releases actually work

Verification evidence A dependency graph without cycles, explicit component relations, a transport/import sequence, and compatibility checks against the prior delivered API snapshot.

Read and verify this answer

SAP software component dependencies and API snapshots

SAP software component organization and lifecycle boundaries

CLOUD-05 The same business capability is exposed through a local released API and a remote service. Which should an on-stack application consume?

Compare what the two APIs actually do. A suitable local API avoids network calls and may fit the supported local transaction. A remote API has its own identity, failures, and transaction boundary. They may expose different operations despite similar names. Choose the contract that fits permissions, timing, and saving; do not assume local and remote access are interchangeable.

Reasoning and example

Calling the same SAP system through HTTP from a save routine can open another session. That session may compete for locks or commit separately, which makes a simple-looking self-call difficult to reason about. Check whether a suitable released local operation can be used in the required RAP phase instead.

On the other hand, a local read interface does not replace a remote command just because both concern purchase orders. Compare the operation, authorization, and failure behavior, then test concurrent use. A C1 local-use contract and a C2 remote-use contract authorize different kinds of consumption. If the caller may later move to BTP, keep that future network boundary in mind without pretending it already has local transaction guarantees.

Avoid this mistake C1 local consumption and C2 remote consumption are different purposes; one release contract does not authorize the other.

Interviewer probes Can the chosen local command participate in this RAP save phase? / What changes if the consumer moves to BTP next year?

Strong answer criteria Compares full contracts; Recognizes self-call transaction risks; Plans a replaceable consumption boundary

Verification evidence Documented local and remote API contracts plus a transaction and authorization test for the chosen consumption path.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-06 A CDS object compiles in the development system but fails after delivery to an older supported release. How would you prevent recurrence?

Define the products and releases the application supports before choosing features. Check CDS syntax, provider contracts, bindings, language rules, and released APIs on those targets. Test the lowest supported release. When a feature is unavailable, choose a compatible design, raise the minimum supported version explicitly, or maintain a separate variant. Activation on the development system proves only that system works.

Reasoning and example

A developer copies a projection example from current documentation. It activates locally, but an older delivery target does not support its provider contract. The transport fails before anyone can test the screen. Checking only the ABAP class would not have caught the CDS and service dependency.

Build a small compatibility test containing the related CDS, behavior, and service artifacts and execute a representative request on the lowest target. Record which features and API releases are available there. Keep broadly reusable business contracts separate from newer delivery features where that helps. Do not silently change the supported release promise after development; the design and support policy need to agree.

Avoid this mistake An object being active in development does not prove compatibility with every product edition or supported transport target.

Interviewer probes Where would you document the minimum feature level? / Which differences justify a versioned implementation rather than a shared code path?

Strong answer criteria Sets a concrete support matrix; Tests the lowest supported release; Includes dependent repository artifacts

Verification evidence A versioned compatibility matrix, target activation evidence, and service tests tied to the exact supported release and delivery revision.

Read and verify this answer

SAP ABAP Cloud development model

SAP released APIs, contracts, visibility, and states

CLOUD-07 Should a request, its approval history, and an external purchase order be one RAP business object?

Put data in one RAP business object when it has shared ownership, lifecycle, and rules that must hold in one transaction. Request items may belong to their request. Approval history may need separate retention rules. An external purchase order has another owner and transaction, so usually reference it through an API. Screen layout alone should not decide the composition tree.

Reasoning and example

Deleting a request should not accidentally remove approval evidence that the business must retain. It also cannot safely cascade a database deletion into another system's purchase order as if both were local rows. Define cancellation, retention, and external failure outcomes before drawing the composition hierarchy.

The opposite mistake is separating request lines that must be checked and locked together. That can make a simple total rule difficult to enforce consistently. Write down which facts must always agree at save time and who owns each piece of data. Keep tightly related local data together where appropriate, and use an explicit workflow or reconciliation process for relationships that cross system boundaries.

Avoid this mistake A composition relationship is not a convenient visual grouping and cannot provide atomicity across an external system.

Interviewer probes Which business rule requires the changes to commit together? / How do retention and cancellation rules affect which entities belong in the composition?

Strong answer criteria Starts with the business rules that must always hold; Identifies data ownership and retention needs; Explains how separate systems stay consistent

Verification evidence A list of business rules, allowed states and transitions, and reasons for the composition design, with deletion, cancellation, and remote-posting failure tests.

Read and verify this answer

SAP extension architecture guidance

CLOUD-08 A partner wants one ABAP solution to serve several customers. What isolation questions must be answered before calling it multitenant?

First verify the runtime's supported multitenancy model. Separate each customer's data and access across storage, configuration, caches, jobs, logs, and outbound connections. Automatic client handling covers only part of this. Define onboarding, upgrades, and offboarding too. Prove the boundary with tests where one tenant tries to read or affect another; a tenant column alone is not enough.

Reasoning and example

Two customers both have document number 100. If a shared cache uses only that number as its key, it may return the wrong customer's result where the runtime scope allows reuse. Database queries can be correctly restricted while the cache still leaks information. An outbound connection selected from the wrong tenant context can cause a similar problem.

Use the supported tenant context throughout the design and separate provider administration from customer permissions. Test background jobs and resources that live longer than one request. During offboarding, confirm that access and the intended data are removed without affecting another customer. Calling the solution multitenant requires this broader isolation evidence, not only a successful two-client database query.

Avoid this mistake Adding a tenant field or relying on automatic client filtering alone does not establish complete multitenancy isolation.

Interviewer probes Which caches or jobs outlive one request? / How do you prove offboarding removes access without damaging another tenant?

Strong answer criteria Covers non-database isolation paths; Uses supported tenant context; Includes tenant lifecycle operations

Verification evidence A tenant-boundary threat model, two-tenant negative integration tests, and an onboarding/offboarding runbook validated in the target runtime.

Read and verify this answer

SAP ABAP development and multitenancy guidance

CLOUD-09 A business team requests a new field, a rule, and a custom app. How do you decide between configuration, key-user, and developer extensibility?

Break the request into the field, rule, and application needs. Check standard configuration first, then supported key-user options. Use developer extensibility for requirements those options cannot meet safely. Verify where the field must appear and when the rule must run. Keep meaning, permissions, transport, and ownership consistent across the chosen mechanisms instead of rebuilding everything in custom code.

Reasoning and example

The new field must appear in a standard form, an API, and analytics. A supported key-user business context may already provide those publication paths. Creating a field in a separate custom table does not automatically connect it to them. Try the exact propagation on the target system before choosing the implementation.

The rule may fit a supported key-user hook if its data and execution limits match. Complex orchestration might need developer code instead. Assign an owner for changes to business-maintained rules and test their permissions and transport behavior. The right choice follows the requirement; key-user tooling is not inherently less serious, and developer tooling is not a reason to duplicate existing configuration.

Avoid this mistake Key-user extensibility is not inherently less serious engineering, and developer extensibility is not a reason to duplicate standard configuration.

Interviewer probes Who owns a rule that business users can change? / How will the field reach the required integration and output artifacts?

Strong answer criteria Checks standard options for each part of the requirement; Checks where the field appears and what the extension hook allows; Names who maintains and delivers each extension

Verification evidence A requirement-to-extension matrix and a target-system demonstration of field propagation, rule authorization, and transport behavior.

Read and verify this answer

SAP extensibility approaches and target environments

CLOUD-10 Can you expose the same RAP projection to a Fiori UI and long-lived external consumers?

Reuse the business model where it makes sense, but protect external consumers from routine UI changes. A Fiori UI may need draft and feature controls; an integration needs a stable API, paging, retry rules, and communication setup. Separate projections or services can support those different needs. Check the available binding features and test both consumers against the same business rules.

Reasoning and example

A UI team removes a property it no longer displays. If a nightly integration uses the same uncontrolled service contract, that harmless screen change can break the job. A draft-based editing process may also be a poor fit for an external command that expects a final posting result.

Define which fields and operations each consumer can rely on, and keep shared business checks in the common implementation where appropriate. Record service metadata and run compatibility tests before changes. For a breaking external change, provide an explicit version or migration path and deprecation policy. Generating an OData endpoint for a UI is not the same as committing to a durable external API contract.

Avoid this mistake An OData endpoint generated for a UI is not automatically a released, durable integration contract for external systems.

Interviewer probes Which behavior should be shared and which should be projected differently? / How would you introduce a breaking external-contract change?

Strong answer criteria Separates consumer contracts; Preserves shared business rules; Addresses external API lifecycle

Verification evidence Service metadata baselines, UI and integration contract tests, and documented compatibility and deprecation rules.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-11 A released CDS view contains all required fields. Why might it still be unsuitable for implementing a business command?

A released read view lets you read data; it does not automatically let you change that business object. Find the supported command API and check its permissions, locks, validation, and save rules. Do not update underlying tables because their fields are visible in a query. If no suitable command exists, revisit the process or record an API gap.

Reasoning and example

A view exposes a purchase order's approval status. Updating that status directly in the database may appear to approve the order, while skipping change history, outputs, follow-on checks, or related state. The field is part of a larger business operation, not simply a value that any reader may change.

Use a supported command and test both acceptance and rejection. Also consider a change made after your initial read: the command needs the correct concurrency handling rather than trusting an earlier status forever. If the released interface does not provide the required command, document that limitation. A useful read model cannot grant write ownership over its provider's persistence.

Avoid this mistake A released read API does not release its underlying tables for mutation or guarantee that its fields form a write contract.

Interviewer probes How would you handle a query result that becomes stale before the command? / What evidence distinguishes a supported command from a database shortcut?

Strong answer criteria Separates query and command semantics; Preserves provider ownership; Handles stale reads and concurrency

Verification evidence The command API's release and behavior documentation, plus concurrent-change and validation-failure tests.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-12 A large classic application cannot be replaced in one release. How would you introduce ABAP Cloud without creating two inconsistent applications?

Move one clear business capability at a time and keep one authoritative owner for each business state. Define how old and new callers are routed, how identifiers stay consistent, and how results are reconciled. A governed classic boundary may support coexistence in a suitable landscape. Avoid uncontrolled writes from both paths. Give the temporary bridge explicit retirement criteria.

Reasoning and example

The old report and the new RAP app both update the same document through different rules. Users can now reach states that one application accepts and the other rejects. A safer first step may change the screen or read path while both versions still use the same approved posting owner.

Plan when write ownership changes, how traffic is routed, and how you can reverse the cutover if needed. Compare results and reconcile records before retiring the old path. Set a measurable condition for removing the temporary adapter so it does not become an ownerless permanent platform. A RAP screen alone does not prove that the legacy posting transaction behind it now follows ABAP Cloud rules.

Avoid this mistake Using RAP for the new UI does not by itself modernize or validate the legacy posting transaction behind it.

Interviewer probes Which system owns writes during each transition phase? / What objective condition lets you remove the bridge?

Strong answer criteria Defines one authoritative writer; Plans coexistence and cutover; Sets retirement criteria

Verification evidence A phased ownership map, routing and rollback plan, reconciliation results, and an explicit bridge-removal milestone.

Read and verify this answer

SAP extension architecture guidance

CLOUD-13 A shared custom API is widely used but has no named owner. What would you change before accepting another dependency on it?

Name an owner before adding more consumers. That owner must maintain the API's meaning, compatibility, security, operations, and support policy. Find existing callers and record current behavior. Choose an appropriate release or component-access mechanism and explain the limits to users of the API. A technically callable object is not a reliable shared service without those responsibilities.

Reasoning and example

A shared pricing helper changes its rounding rule without changing a parameter. Every caller still compiles, but invoice totals now differ. The API owner needs to treat that as a behavior change and assess it with consumers, not declare compatibility because the ABAP signature stayed the same.

Capture examples for amounts, units, exceptions, and transaction assumptions, then use them as regression checks. Inventory remote or external consumers too; a local where-used list may not show them all. Set a change-notification and support process. If no team can sustain that responsibility, do not advertise the helper as independently reusable infrastructure merely by making it public or setting a release flag.

Avoid this mistake Public visibility or a C1 release flag does not replace accountable ownership and semantic compatibility testing.

Interviewer probes Who approves a behavioral change without a signature change? / How do you discover consumers outside the local where-used list?

Strong answer criteria Names who maintains the API promise; Tests behavior as well as signatures; Finds the real consumers

Verification evidence An API owner record, consumer inventory, support policy, and executable consumer-contract examples tied to a released baseline.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-14 A released BAdI exists, but it runs after the point where the requirement must reject a transaction. Is it still the right extension?

A released extension point can still run at the wrong time for your rule. Check when it executes, what data it sees, what it may change, and how it can reject work. If the rule must prevent saving, use a supported point that can do that. Otherwise revise the process; reporting an error after commit cannot undo the document.

Reasoning and example

The business says that an invalid approval reference must prevent document creation. You find a released notification BAdI that runs after saving. Raising a message there cannot turn the already committed creation into a rejected save. A later reversal is a separate business operation, with its own history and possible downstream effects.

Map the actual transaction phases and inspect the extension contract before choosing the hook. Test invalid input, rollback, and update failure on the target system. If no suitable preventive hook exists, discuss another supported design with the business. A familiar method name or released status proves neither the required timing nor permission to change the transaction's outcome.

Avoid this mistake A released extension point can still be the wrong place for a particular business rule.

Interviewer probes What data and modifications does the BAdI contract actually permit? / Would compensating after save meet the same business requirement?

Strong answer criteria Checks hook timing and permissions; Distinguishes prevention from compensation; Validates behavior with failures

Verification evidence A transaction-phase map, the extension-point contract, and a target-system test proving when rejection or follow-up occurs.

Read and verify this answer

SAP extensibility approaches and target environments

CLOUD-15 Your domain classes depend on Gateway messages, SQL row types, and HTTP clients. How would you improve the architecture without a wholesale rewrite?

Extract one business decision behind a small, clear input and result. Keep HTTP, database, and framework-specific mapping in adapters around it. Preserve existing callers and transaction ownership while making the change. This lets you test the rule without rebuilding the whole application. Adding wrapper classes helps only if the business logic actually stops depending on infrastructure details.

Reasoning and example

An eligibility calculation currently reads an HTTP request and creates Gateway messages directly. Let it instead receive the needed business values and return an eligibility decision with a clear reason. An adapter can translate that result into RAP messages or an integration response. Persistence and communication remain separate collaborators rather than hidden work inside the rule.

Before extracting it, capture cases for rounding, missing data, and failures so existing behavior stays protected. Move one responsibility at a time and rerun the real integration acceptance tests. Do not move commits or change the caller's transaction expectations accidentally. A new class per framework object adds little value if every supposedly reusable method still exposes framework types.

Avoid this mistake Wrapping every framework call in another class adds layers without isolating business policy if framework types still dominate the contract.

Interviewer probes Which extraction has the lowest transaction risk? / What should remain in the RAP behavior implementation?

Strong answer criteria Separates one clear business responsibility; Preserves callers and transaction ownership; Tests observable behavior before and after the change

Verification evidence A bounded before/after dependency map and passing business-policy tests plus unchanged integration acceptance results.

Read and verify this answer

SAP extension architecture guidance

CLOUD-16 A side-by-side service needs to act for users and also run scheduled reconciliation. Would you use one technical identity for both?

Treat user requests and unattended jobs as different access needs. Choose supported communication scenarios, authentication, and permissions for each. Use separate identities or scopes when required by those needs. Record who initiated a business action and plan credential rotation and tenant selection. One broad technical identity can give interactive users more access than they should have.

Reasoning and example

A reconciliation job may legitimately read documents across many business areas. A person using the portal should see only their permitted documents. If the portal forwards every request using the job's broad identity without the required authorization design, the UI can accidentally expose that wider access to every user.

Using a person's short-lived credentials for a nightly job creates a different problem: the job may stop when those credentials expire. Define the supported user-delegation and machine-to-machine flows separately. Test denied access, audit attribution, rotation, and tenant offboarding. Successful authentication proves which identity connected; it does not prove that the requested operation or tenant is allowed for that identity.

Avoid this mistake Authentication success proves identity, not permission for the requested business operation or tenant.

Interviewer probes Where is the initiating user recorded when a technical identity performs the command? / What happens during credential rotation or tenant offboarding?

Strong answer criteria Separates user and machine access; Preserves business authorization; Plans identity lifecycle and audit

Verification evidence A communication and authorization matrix with negative access tests, audit examples, and a credential-rotation test.

Read and verify this answer

SAP communication arrangements and scenarios

CLOUD-17 A released dependency is deprecated but still works. How do you decide when and how to replace it?

Deprecated means an API should be replaced according to its lifecycle guidance, not necessarily that it has already stopped working. Check the successor, support limits, affected callers, and supported target releases. Compare behavior as well as names and parameters. Plan tests, rollout, and monitoring before switching. Avoid adding unnecessary new dependencies on an API already scheduled for replacement.

Reasoning and example

A successor serves the same business purpose but returns different keys and uses another paging model. A mechanical method-name replacement may miss records or duplicate work. It might also change permissions or return a pending result where the old API completed synchronously. Test those differences with representative consumers and failure cases.

Check whether the successor exists on the lowest supported release. If not, the transition may require a deliberate compatibility path or a support-policy decision. Roll out with clear success measures and a supported way to reverse the change if needed. Keep old and new behavior visible until migration is complete, then remove the old dependency and its maintenance burden.

Avoid this mistake A release contract does not mean an API never evolves, and deprecation is different from decommissioning.

Interviewer probes What if the successor is unavailable on your lowest supported release? / Which semantic differences would block a direct replacement?

Strong answer criteria Distinguishes deprecated from removed; Checks the replacement behavior and target availability; Plans migration with measurable results

Verification evidence A dependency lifecycle register, old/new contract comparison, and regression results across the supported release matrix.

Read and verify this answer

SAP released APIs, contracts, visibility, and states

CLOUD-18 What would you require before approving a senior team's ABAP Cloud architecture for production?

Require evidence that the main business rules, permissions, transaction boundaries, and recovery paths work on the intended runtime. Check released dependencies, realistic workload, monitoring, retention, and deployment order. Give important assumptions an owner and a test result. Distinguish a prototype from a production-tested solution. Remaining risk can be accepted deliberately, with practical controls and clear responsibility.

Reasoning and example

A Fiori demo using mocked data looks convincing, but it cannot prove real RAP locking or permissions. Walk through a concurrent update, a dependency outage, a failed deployment, and a recovery exercise on the relevant environment. Link each result to the actual release candidate so old evidence is not mistaken for proof of new code.

Focus effort on failures that would harm the business rather than collecting diagrams alone. Confirm who monitors failures, who can retry safely, and how data remains consistent during recovery. A green syntax check or ATC run supports the review but cannot replace these checks. Production approval should state what was demonstrated and which controlled risks remain.

Avoid this mistake A clean syntax check, green ATC run, or attractive diagram is not an end-to-end production-readiness certificate.

Interviewer probes Which risk would you prove first with a technical spike? / What evidence would make you delay the release despite a successful demo?

Strong answer criteria Tests the risks with the greatest consequences first; Links evidence to the real runtime and source version; Defines who operates the app and how it recovers

Verification evidence A release review for the exact source version, including focused workload, permission, transaction, deployment, and recovery test results.

Read and verify this answer

SAP extension architecture guidance

Official references

SAP ABAP Cloud development model

SAP extensibility approaches and target environments

SAP extension architecture guidance

SAP released APIs, contracts, visibility, and states

SAP software component dependencies and API snapshots

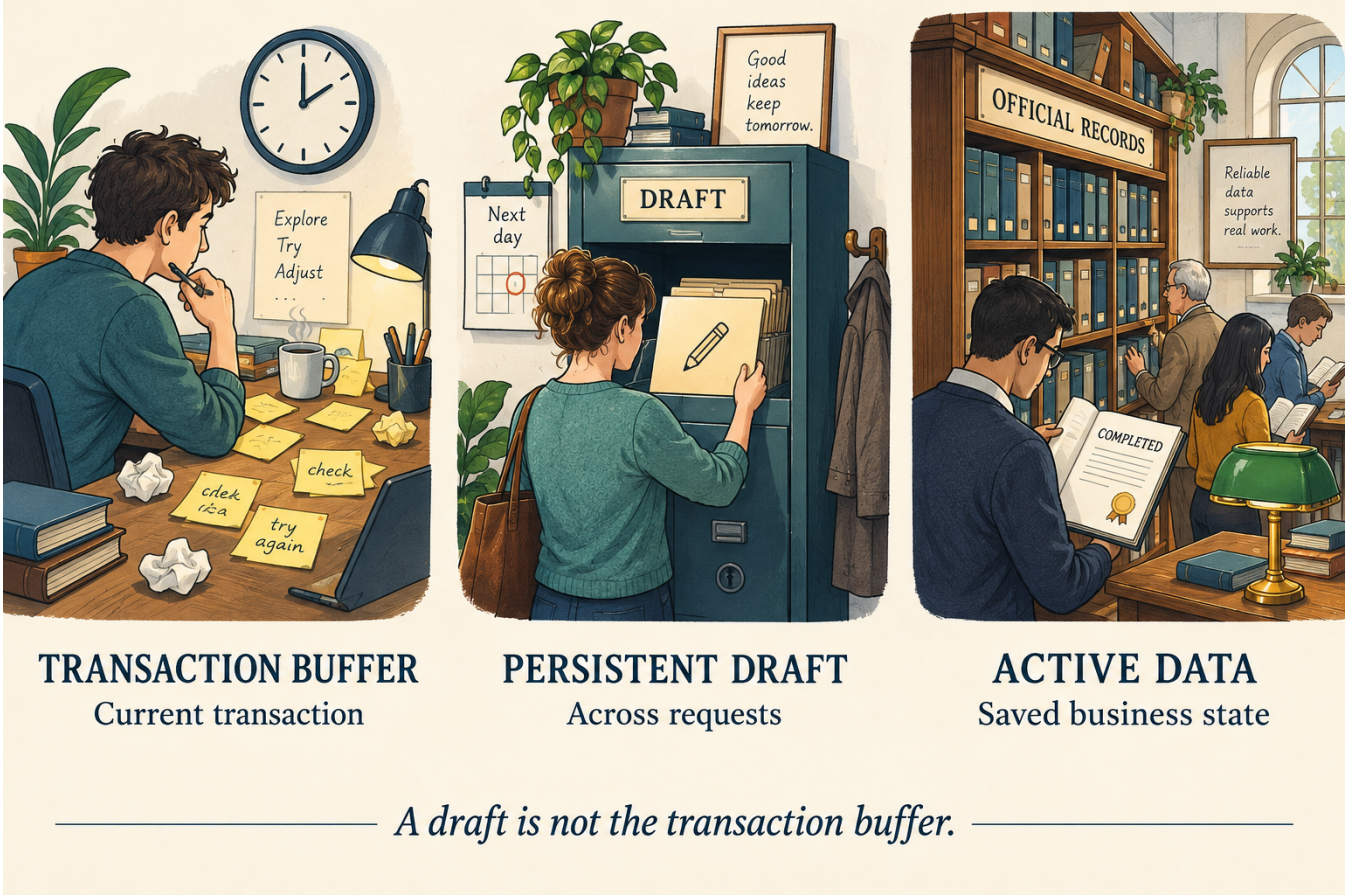
SAP software component organization and lifecycle boundaries

SAP communication arrangements and scenarios

SAP ABAP development and multitenancy guidance

RAP foundations and transactions

Start with the business-object boundary and transaction owner. Explain how the consumer changes the transactional buffer, how business rules run, and when the framework persists the result. These distinctions prevent the most serious mistakes in the original notes.



The desk represents the current transactional buffer; the draft cabinet represents persistent work across requests; the shared archive represents saved active data.

See the visual atlas for the recall prompt, limits and related questions.

RAP-01 What are the building blocks of a RAP application?

A RAP app has a data model, business rules, and a service. Tables store the data, and CDS describes what that data means. The behavior definition says what users can do; ABAP handlers add your business logic. Projections choose what a particular app exposes. The service definition and binding make those entities available through a chosen protocol.

Reasoning and example

Imagine a purchase-request app with a Submit button. CDS describes the request and its items. The behavior definition declares Submit, and its handler checks or changes the request as the operation requires. The transaction first holds those changes in memory. The RAP save sequence then writes valid changes to the database.

The projection exposes the fields and operations this app needs. The service definition selects the entities, while the binding chooses, for example, an OData service type. Read-access rules and UI annotations handle other concerns. Keeping these jobs separate makes it easier to find where a missing field, failed operation, or incorrect screen comes from.

The behavior definition (BDEF) is the operation contract. The behavior pool is its ABAP implementation container; generated local handler methods implement custom rules and operations. Managed RAP already supplies standard CRUD handling. strict(2) checks modeling requirements, not whether every referenced API is released or every permission is correct.

Avoid this mistake Behavior is not implemented by the service definition, and a binding alone does not make the endpoint production-ready.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-02 How do you choose managed versus unmanaged RAP?

Use managed RAP when the framework can handle your object's changes and database writes. Use unmanaged RAP when your implementation must handle those jobs, often because an existing business API already owns them. Managed RAP with unmanaged save is a middle option: RAP handles changes, and your code handles persistence. In every option, RAP still controls the transaction.

Reasoning and example

For a new travel-request app with your own tables, managed RAP usually avoids writing basic create, update, and delete handling yourself. You can focus on rules such as checking travel dates. For an existing application, a business API may already update several related tables correctly. Reusing it may justify unmanaged save or a fully unmanaged implementation.

Before choosing, check whether that API commits by itself, uses update tasks, or requires its own locks. A commit inside local RAP processing violates its runtime rules and may be rejected. Moving the call to a separate transaction creates another problem: it can save independently, and a later local rollback cannot undo it. Choose based on the API's behavior, not the application's age.

Avoid this mistake Unmanaged does not permit arbitrary COMMIT WORK or database changes anywhere in a handler.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-03 What is the transactional buffer?

The transactional buffer is the transaction's working copy of a business object. It includes changes that have not yet been saved. EML can read this current state through the business object. A direct database SELECT may still see the old saved values. Managed RAP supplies buffer handling; an unmanaged implementation must provide the required handling.

Reasoning and example

Suppose an order item has quantity 2 in the database. The user changes it to 5, and RAP must recalculate the order total before saving. Reading the item through the business object lets the calculation use the pending quantity of 5. Reading only the database can give 2 and produce an outdated total.

After a successful save, the database contains the new state. Until then, the buffer belongs to this transaction. It is not a place to keep unfinished work across unrelated HTTP requests. Draft provides separate persistent storage for that use case. This distinction helps explain why a correct database query can still be wrong inside a business rule.

Avoid this mistake The buffer is not a durable draft store and does not survive arbitrary stateless HTTP requests.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-04 Where may COMMIT ENTITIES be called?

Call COMMIT ENTITIES from an external ABAP program that owns an EML transaction. Inside a RAP handler or saver, let the framework finish the transaction. Do not call COMMIT ENTITIES or COMMIT WORK there. This rule also applies to actions, validations, and determinations. The OData request framework owns the save boundary for its requests.

Reasoning and example

A standalone ABAP program creates a request and its items with EML. It checks the modification responses, then calls COMMIT ENTITIES at the point where the complete business change should be saved. It must also check the save responses, because accepting a modification does not prove that saving succeeded.

Now put the same creation logic inside a RAP action. The action may make permitted changes, but it must not commit them itself. The framework still has other rules and changes to process. An early commit could split what should be one transaction, and violating the RAP contract can cause a runtime error. Moving the commit into a saver method does not fix that.

Avoid this mistake Moving the commit from validation into SAVE or CANCEL does not make it valid.

Read and verify this answer

SAP Learning on EML and transaction ownership

SAP RAP BO contract

RAP-05 What are the steps of the RAP save sequence, and where is the point of no return?

RAP saves changes in this order: FINALIZE finishes calculations, CHECK_BEFORE_SAVE checks the data, ADJUST_NUMBERS assigns final keys when late numbering is used, SAVE writes the data, and CLEANUP clears the buffer. The point of no return comes after CHECK_BEFORE_SAVE has passed for every involved business object. Remember it as: calculate, check, number, write, clear.

Reasoning and example

1. FINALIZE - finish the data. For an order, calculate the total from its items. In a managed BO, relevant on-save determinations do this work; in an unmanaged BO, use the corresponding saver method. This is the last phase for permitted changes to the transactional buffer. Ordinary EML MODIFY requests after FINALIZE cause runtime errors; the ban does not start at the beginning of FINALIZE.

2. CHECK_BEFORE_SAVE - make the final business checks. For example, reject an order whose delivery date is invalid. Managed BOs use validations; unmanaged BOs implement the corresponding saver check. Only when every involved BO passes do we cross the point of no return. A failed check stops the save before late numbering and persistence.

3. ADJUST_NUMBERS - assign final keys for late numbering. Our new order now receives document number 4711. Early numbering has already assigned the key, so it does not need this late assignment. Receiving a number does not yet prove that the database commit succeeded.

4. SAVE - write the changes. With normal managed save, RAP writes the BO data. With additional save, RAP still writes it, and your save_modified method adds work such as an audit row. With managed interaction plus with unmanaged save, your save_modified method replaces managed persistence. A fully unmanaged BO instead implements save in its local saver class. SAVE is the phase name; it does not mean every scenario uses a method named save.

5. CLEANUP - clear transaction-local buffers when the LUW finishes, including after rollback. This prevents old working data from leaking into the next transaction. CLEANUP_FINALIZE is a separate early-failure hook when FINALIZE or CHECK_BEFORE_SAVE fails in an involved BO; it can undo work done during finalization. It is not the last step of a successful save, and you should not assume every rejected save automatically discards all user input.

Example: two order items cost 40 and 60. FINALIZE calculates 100. CHECK_BEFORE_SAVE detects a bad delivery date, so the save stops: no late document number and no persisted order. With a valid date, late numbering can assign 4711 and persistence writes the order and items. The transaction owner completes the commit; cleanup then removes the working data. A handler or saver must not issue its own commit.

Senior detail: the point of no return means ordinary business checks must already have passed. It does not mean a database outage is impossible. SAP also documents a special failure-capable saver, CL_ABAP_BEHAVIOR_SAVER_FAILED, for supported cases such as BAPI integration. Check the exact scenario and release rather than treating that exception as permission to move normal validation into late save.

Avoid this mistake Do not ban all EML changes inside FINALIZE, put normal validation after the point of no return, or confuse save_modified in a managed BO with the save method of a fully unmanaged BO. CLEANUP_FINALIZE belongs to the early-failure path.

Read and verify this answer

SAP: Save Sequence Runtime

SAP: Save options

SAP: Implementing Additional Save

SAP: TYPE REQUEST FOR

SAP: CLEANUP_FINALIZE

SAP: BAPI integration and late numbering

RAP-06 Determination validation action or precheck?

Use a determination to fill or calculate values automatically. Use a validation to check whether data is acceptable for saving. Use an action for an operation a caller explicitly requests, such as Submit. Use a precheck to reject an incoming change before it enters the buffer. Choose by what must happen and when, rather than putting every rule in one handler.

Reasoning and example

In a request app, a determination can fill the default currency. A validation can reject an end date that is before the start date. Submit is an action because the user deliberately asks for a business step. A suitable precheck can reject a proposed field value before the update is accepted into the transaction.

A validation should report a problem, not silently repair the data. Also check the implementation type: in unmanaged RAP, declared determinations and validations apply to draft instances. For active instances, the appropriate unmanaged operation handlers and save checks must enforce consistency. Copying a managed example without that distinction can leave invalid active data unchecked.

Avoid this mistake An action is not simply another name for a CRUD event, and validations should not modify business data.

Read and verify this answer

SAP Learning on numbering and determinations

SAP RAP determinations and unmanaged scope

RAP-07 How do FAILED REPORTED and MAPPED differ?

FAILED says which instance or operation was rejected. REPORTED carries messages, including their severity and the field or instance they concern. MAPPED links a caller's creation identifier to the created object's identity where the operation supports it. These structures do different jobs. An error message in REPORTED alone does not necessarily make an operation fail.

Reasoning and example

A user saves two requests, and one has an invalid delivery date. A validation must identify the rejected request in FAILED and explain the problem in REPORTED. Adding a field reference lets the UI point to the date the user should fix. Reporting only an error-looking message is not a reliable way to reject the save.

For a successful create, the caller may have used a content ID before a final identity was available. MAPPED allows the caller to connect that input to the new object. Always use the response parameters actually provided by the handler contract. For example, a determination does not provide FAILED, so it is not a substitute for a rejecting validation.

Avoid this mistake Error severity in REPORTED is not a substitute for FAILED; determinations do not expose FAILED.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-08 What is EML and when do you use it instead of SQL?

EML lets ABAP code work with a RAP business object through its operations. READ ENTITIES reads its transactional state; MODIFY ENTITIES requests changes or actions. SQL reads or changes database data directly. It does not run the object's business behavior. Use EML when you need the business operation, and choose a suitable query interface when you only need database results.

Reasoning and example

Suppose a background program must submit an expense request. Updating its status column with SQL may skip checks, locks, and authorization handling that belong to Submit. Calling the exposed action through EML uses the business object's defined behavior and returns operation responses that the program must inspect.

The program also needs to handle the eventual save result. A successful action response is not proof of a completed commit. For a read-only report, a suitable CDS or SQL query may be the clearer and faster choice. EML is useful because it follows business behavior, not because every EML read is automatically faster than SQL.

An EML read normally starts from known entity keys or an association. It is not the managed list query with $filter, $orderby and paging that an OData consumer uses to find orders. Use a suitable query to find the records, then the BO operations to work with their transactional state.

Avoid this mistake EML does not mean that every read is automatically the best performance choice; use the correct semantic interface.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-09 What does IN LOCAL MODE mean?

IN LOCAL MODE is for a business object's own internal EML calls. Depending on the operation, it skips checks such as authorization, feature control, and prechecks so internal logic can maintain the object. It does not create a separate transaction or allow forbidden save behavior. It is not a general way for external callers to turn off another object's security.

Reasoning and example

A request total is readonly to users because the application calculates it. A determination inside that business object may need to update the total after an item changes. An internal EML update in local mode allows this kind of controlled maintenance even though a normal external update of the field is restricted.

The public operation still needs its proper permission checks and business rules. Review what the internal code changes and why it is trusted. A call to a different business object needs its own supported access contract; do not treat local mode as a reusable security bypass. The changes still belong to RAP's transaction and must follow its save rules.

Avoid this mistake It is not a general-purpose option for external callers to disable security on any BO.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-10 What do %cid %cid_ref %tky and %pid mean?

These fields identify objects at different stages of a RAP request. %cid labels a new instance, and %cid_ref refers to that creation, for example when adding its items. %tky is the transactional key, including relevant context such as draft state. %pid is a preliminary identifier for late numbering. None of these names means they are interchangeable.

Reasoning and example

Create a header and one item in one request. The header may not yet have its final document number. Give its creation a content ID, then refer to that ID when creating the item by association. RAP can connect them without requiring the caller to guess a final key.

Inside handlers, use the supplied transactional identity when returning results or messages. Rebuilding a key from only the document number can lose draft context. For late numbering, ADJUST_NUMBERS maps the preliminary identification in %pre to the final %key. A content ID belongs to request correlation; it is not a business number to store and show forever.

Avoid this mistake %tky means transactional key, not temporary key; %cid is not a persisted business document number.

Read and verify this answer

SAP Learning on EML and transaction ownership

RAP-11 Early numbering versus late numbering?

Early numbering gives an instance its final key during interaction. Late numbering keeps a preliminary identity and assigns the final key late in saving. A technical UUID and a business document number can be separate fields with different rules. Late numbering can avoid consuming numbers for some rejected changes. It does not, by itself, promise a gap-free business sequence.

Reasoning and example

A new request can receive a UUID immediately so its items have a stable technical parent. The business may want the visible approval number only when the request is accepted. That number can follow a separate supported numbering design rather than being used as the only technical identity.

Now consider two users saving at the same time. Reading the highest number and adding one can give both users the same result. Use suitable number-range and transaction handling instead. If the business requires no gaps, also examine failed saves, cancellations, and the exact numbering guarantees. Choosing late numbering is only one part of that design, not proof that every possible gap has disappeared.

Start by separating identity from the label a person sees. OrderUUID can be a stable technical key generated immediately, while OrderNumber is a later business number. Late numbering is needed when the actual entity key must be assigned late; a later display number alone does not automatically require it.

Avoid this mistake Late numbering alone does not guarantee a gap-free sequence, and SELECT MAX plus one is unsafe under concurrency.

Read and verify this answer

SAP Learning on numbering and determinations

SAP RAP late-numbering mapping

RAP-12 What problem does draft solve?

Draft saves unfinished editing work separately from active business data. It lets a user continue across HTTP requests or return after an interruption. RAP provides operations such as Edit, Resume, Activate, and Discard; Prepare can run selected checks and calculations. Activation moves valid draft changes into active processing. The request still needs its save sequence to finish.

Reasoning and example

Maya edits order 4703 from 80 EUR to 120 EUR and closes the browser. The draft keeps her unfinished 120 EUR edit, while active business data stays at 80 EUR. When she returns, activation rejects 120 EUR because this example permits at most 100 EUR. She corrects the draft to 90 EUR; only successful activation and save make 90 EUR the active value.

When the user activates it, required checks can reject missing or invalid data. Successful activation is part of the processing that produces active data; it does not authorize a custom handler to commit on its own. Keep draft and active identities clear in reads, messages, and actions. A custom Submit action may represent a separate business step and is not automatically the same as draft activation.

Avoid this mistake Every BO is not automatically draft-enabled, and a custom Submit action is not automatically draft activation.

Read and verify this answer

SAP Learning on draft

RAP-13 Can you rely on determination and validation order?

Do not make one independent determination depend on another running first unless the framework contract guarantees it. Calculate related values together or make their dependency explicit. Derivations should give a consistent result when repeated. RAP does have defined processing phases. That does not mean every handler inside a phase has an order you may rely on.

Reasoning and example

Suppose one determination converts an amount into the display currency and another calculates the total. If the total assumes conversion has already run, a later change may leave an incorrect result. A clearer design calculates both from the same current source values, or puts the dependent steps into one suitable operation.

Test more than the first create. Change the quantity twice, change the currency, and repeat the same update. The final total should remain correct and the handlers should not keep triggering each other unnecessarily. The issue is not that all RAP ordering is random. The issue is relying on an execution order that the documented contract never promised.

Avoid this mistake Calling the order random is imprecise; the important point is that an unsupported ordering assumption is unsafe.

Read and verify this answer

SAP RAP determinations and unmanaged scope

RAP-14 How does persistent field mapping work?

Field mapping tells RAP which persistence field belongs to a CDS element when their names differ. For example, RequestUUID can map to the table field request_uuid. The mapping connects names for the selected persistence implementation. It does not calculate new values or join tables. Business calculations and conversions need their own supported logic.

Reasoning and example

Your table uses request_uuid, but the CDS model exposes the clearer element name RequestUUID. The behavior mapping connects those names so persistence handling can read and write the intended field. Check every relevant field rather than assuming similarly named elements map themselves in every scenario.

Now imagine the screen shows amounts in another currency. Mapping Amount to a table column will not perform the currency conversion. That belongs in appropriate calculation or query logic. Projections and generated EML types also have their own names and identity fields, so use the correct type at each boundary. A field-name mapping does not remove those distinctions.

Avoid this mistake Mapping is not a database join or a universal value-conversion mechanism.

Read and verify this answer

SAP Learning on behavior implementation options

RAP-15 Can you call a BAPI from RAP, and what happens if it executes COMMIT?

Some BAPIs can be used in RAP; some cannot. A BAPI does not always execute its own COMMIT. Check the specific API. A local COMMIT WORK or BAPI_TRANSACTION_COMMIT inside RAP handler or saver code breaks RAP transaction ownership and can cause a runtime error. Moving that call to unmanaged save or save_modified does not make the COMMIT legal.

Reasoning and example

First inspect what the BAPI actually does: validation, authorization, locks, direct database writes, update tasks and internal COMMIT. Also check whether the API is permitted in the target ABAP language version. A released API is not automatically usable in every RAP phase.

For a compatible legacy API, an action can collect the intent in the transactional buffer. The documented save integration performs the actual write later. In a managed BO with unmanaged save, save_modified replaces managed persistence; with additional save it supplements managed persistence. A fully unmanaged BO normally uses its saver save method. These are different options, not permission to end the LUW yourself. RAP owns the final commit.

Example: an action requests an update through a suitable BAPI. Validate predictable errors before the point of no return, retain the request in the buffer, call the API in the documented late-save exit, map its responses correctly, and let RAP finish the transaction. For supported create scenarios with late numbering, SAP documents BAPI use in ADJUST_NUMBERS; check the exact release and failure-handling contract.

If the BAPI inevitably commits internally, do not paste it into save_modified or hide it behind a wrapper. Look for a suitable API or redesign the process around an explicit asynchronous boundary. That changes when the business operation completes and needs visible pending/failed states. BRFplus is a rules engine, not a transaction workaround; see RAP-17. Keep the real permission check at the supported authorization boundary and use the documented authorization-context pattern where needed.

Avoid this mistake Unmanaged means you implement more of the behavior or persistence. It does not mean unrestricted COMMIT. A wrapper, BRFplus call or RFC destination cannot magically preserve one atomic LUW.

Read and verify this answer

SAP BAPI consumption in RAP: late numbering and save sequence

SAP BAPI consumption in RAP: Authorizations

SAP: RAP action with integrated BAPI business logic

SAP exercise RAP640: implement unmanaged save

SAP: Transaction Model for Developing BAPIs

RAP-16 How do you group a multi-select Fiori action into one changeset with InvocationGrouping?

For a supported Fiori elements action, set invocationGrouping: #CHANGE_SET on its @UI.lineItem action annotation. The default #ISOLATED allows separate changesets for the selected instances. Both can still travel inside one HTTP $batch request. $batch is the envelope; a changeset is the atomic group inside it. Check the generated metadata and the actual network request.

Reasoning and example

Imagine selecting orders A, B and C and pressing Approve. B is invalid. With separate changesets, A and C may succeed while B fails. With one shared changeset, the intended result is all-or-nothing: B failing prevents A and C from being committed in that group.

Put the annotation on the action entry in a CDS metadata extension. Keep type: #FOR_ACTION and the correct dataAction name. In OData metadata, verify UI.InvocationGrouping uses UI.OperationGroupingType/ChangeSet. Then inspect the browser Network tab: seeing /$batch alone is not proof that there is one shared changeset. Test one deliberately invalid selected order and check persisted data.

This configures how the supporting UI invokes the action; it does not combine arbitrary requests from every client or promise one ABAP handler call. The handler must accept multiple keys correctly. A freestyle SAPUI5 client also needs the appropriate OData model group/submission configuration. Remote side effects that already committed cannot be undone by a local changeset rollback.

Avoid this mistake Do not equate #ISOLATED with no $batch, or #CHANGE_SET with automatic atomicity across remote systems. Verify the Fiori elements/OData version and service behavior.

Read and verify this answer

SAP: UI.lineItem.invocationGrouping

SAPUI5: action invocation grouping

RAP-17 Do BRFplus, bgPF or bgRFC make an internally committing BAPI safe in RAP?

BRFplus evaluates business rules; it does not fix a BAPI commit problem. bgPF is a background framework. Where supported, its uncontrolled operation variant can run logic that needs its own COMMIT. That work runs separately from the original RAP save, so it is not one immediate all-or-nothing transaction. Choose this only if the business process can accept a later result, retries and possible failure. An ordinary bgRFC unit still forbids COMMIT inside it.

Reasoning and example

Example: saving an order should start slow follow-up processing. With a supported bgPF integration, register the background work at the allowed point in RAP save so it becomes eligible after the initiating transaction succeeds. The user can see Pending, then Completed or Failed. Design retries, duplicate protection, monitoring and recovery. If the later work fails, the already committed order is not automatically rolled back.

Distinguish ordinary bgRFC units from the explicit bgPF uncontrolled operation contract. SAP provides IF_BGMC_OP_SINGLE_TX_UNCONTR and set_operation_tx_uncontrolled for the latter; its sample names COMMIT WORK as a use case. Check availability and restrictions in your target system. This does not allow COMMIT in the original RAP handler or saver. Intermediate commits in later processing can leave partial results, so retrying must not create duplicates.

BRFplus might decide which approval rule applies. It does not schedule background work or repair RAP transaction consistency. If someone says use BRF+ for the BAPI commit problem, clarify whether they meant bgPF or bgRFC, then review that design on its own merits.

Avoid this mistake Do not confuse BRFplus with bgPF, or a bgPF uncontrolled operation with an ordinary bgRFC unit. A supported background COMMIT does not make it atomic with the original RAP save.

Read and verify this answer

SAP: BRFplus business rules

SAP: Background Processing Framework

SAP: CALL FUNCTION IN BACKGROUND UNIT

SAP: Controlled SAP LUW

SAP: bgPF controlled and uncontrolled operation variants

SAP sample: bgPF controlled and uncontrolled execution

RAP-18 How do you enable and use create, update, and delete in managed RAP?

Declare create, update, and delete in the base behavior definition. Managed RAP supplies their standard implementation. A consumer calls those operations through EML or a service; RAP first changes the transactional buffer. Accepted changes are not yet saved. The transaction owner must complete the save successfully.

Reasoning and example

Imagine a small Order with OrderUUID and Comment. RAP generates the UUID on create. An external ABAP program creates an order with Comment = Draft, checks FAILED and REPORTED, and uses MAPPED to find the new UUID. It can then read or update that same order before saving. A pending delete removes an instance from transactional state; it is not a completed database deletion yet.

An external EML transaction owner uses COMMIT ENTITIES and checks the separate save response. A service request leaves transaction handling to the framework. A behavior handler must not issue its own COMMIT. On a rejected standalone operation, preserve the failure/messages and roll back the pending transaction; do not print a success message merely because MODIFY returned. For a service, also expose the permitted operations through the projection and implement the required permissions and business validations.

Avoid this mistake Managed CRUD avoids handwritten basic persistence, not authorization, validation, or checking save errors.

Read and verify this answer

SAP Learning: enable modifying operations

RAP-19 How do an order and its items form one managed RAP business object?

The order is the root; its items are composition children in the same BO. Define behavior for both entities. Create items through the parent association, and declare their supported update and delete operations. Children normally depend on the root for locking and authorization. A shared BO boundary lets RAP save their changes together.

Reasoning and example

For Order A, create two items through Order._Items. The parent connection tells RAP which order owns them. In the ordinary managed composition model, deleting the root also deletes its composition children. An associated Customer is different: it is a separately owned record and must not disappear when the order is deleted.

The CDS composition and association-to-parent must agree with the behavior aliases and dependencies. Expose the child and redirected associations in the service projection as needed. For a new parent and new children in one request, use the %cid/%cid_ref correlation described in RAP-10; do not guess a parent key that numbering has not supplied.

Avoid this mistake A normal association to a customer does not make the customer part of the order lifecycle.

Read and verify this answer

SAP Learning: composite business objects

RAP-20 Create or an action exists in the BO but is missing in the service. What do you check?

Follow the capability from the base behavior to the projection and then to the service actually called. A base operation is not automatically exposed by every projection. Check use create or use action, the projected entity, and the active binding metadata. Then distinguish a missing operation from one disabled by features, denied by authorization, or simply not shown by the UI.

Reasoning and example

Suppose the base Order declares create and action Submit. An app projection reuses update only. Calling a different endpoint or adding a button cannot expose the missing operations: first reuse the intended capability in the correct projection.

If metadata already contains the action, inspect operation availability, permissions, UI annotations and request responses. A disabled Submit button may be correct for an approved order. A 403 is a permissions issue, not proof that use action is absent. Refresh cached metadata only after confirming which service version the client uses.

Avoid this mistake Do not fix missing service behavior by adding a UI button or bypassing authorization.

Read and verify this answer

SAP Learning: expose behavior in a service

RAP-21 How do you read an order's items with EML, and what do RESULT and LINK contain?

Use READ ENTITIES with BY association and the source order keys. RESULT contains the item data you requested. LINK connects each source order identity to a target item identity. It is useful when reading children of several parents. Check FAILED and REPORTED instead of assuming every input produced children.

Reasoning and example

Order A has items A1 and A2; order B has B1. RESULT gives three item rows. LINK says A -> A1, A -> A2 and B -> B1. Use those links when grouping the response by its source, especially when target fields alone are not enough to identify the path. An order with no items has no child result; a failed order read must be diagnosed separately.

Inside the BO, this read can include pending child changes from the transaction. Keep the generated %tky identity, including draft state where present. Reading the database directly can miss an item just created in the buffer. RBA follows the declared association; it is not an arbitrary SQL join or a replacement for list-query filtering.

Avoid this mistake RESULT is entity data; LINK is the source-to-target relationship, not a second copy of those data fields.

Read and verify this answer

SAP Learning: read by association and links

RAP-22 Why does EML need derived types and %control instead of a database-table row?

An EML request describes an operation, not just stored fields. Derived types come from the CDS and behavior contract and include the identities and flags that operation needs. %control says which fields were supplied. An initial value can therefore mean either leave unchanged or deliberately clear the field, depending on its flag.

Reasoning and example

The stored Comment is Rush. An update sends Comment = an empty string. With %control-Comment = if_abap_behv=>mk-on, it asks to clear Comment. With the flag off, that field is not part of the update. The FIELDS addition can set these flags for listed fields.

Use TYPE TABLE FOR UPDATE for update input and TYPE TABLE FOR READ RESULT for read data. Carry %tky from the generated input or result so draft identity is not lost. %cid correlates a newly requested instance before its final identity is available; it is not a persistent business key. A table row copied from SQL does not express all these operation-specific details.

Avoid this mistake Use if_abap_behv=>mk-on for behavior flags. abap_true is not the same flag type.

Read and verify this answer

SAP Learning: EML derived types and flags

RAP-23 What must change when a managed RAP BO becomes draft-enabled?

Draft needs coordinated changes across the model, behavior, persistence and projection. Enable draft in the base behavior, provide compatible draft tables and administrative fields, and define the required draft actions. Configure entity ETags and the root total ETag with the correct lock dependencies. Reuse draft in the projection and expose the required associations and operations.

Reasoning and example

An order draft must survive closing the browser. Its draft persistence stores unfinished input separately from the active business record. For a composition, include the child entities and draft-aware associations too. Generate or adjust the draft tables from the actual definitions; do not copy an unrelated table layout.

Prepare should include the relevant validations so the consumer can receive state messages. Activation must transfer a valid state to active persistence. Test create, edit, reopen, discard and activate, plus a competing active change and expired draft lock. Verify both active and draft identities in EML. RAP-12 explains the lifecycle; ADV-01 explains why the total ETag and normal entity ETag protect different situations.

Avoid this mistake Adding with draft alone is not a complete draft implementation, and saving a draft is not activating a business record.

Read and verify this answer

SAP Learning: draft behavior implementation

RAP-24 What is a determine action, and how does it differ from a determination or ordinary action?

A determination is a rule that derives data when its trigger applies. A normal action implements a requested business operation, such as Submit. A determine action groups existing determinations and validations so a consumer can request their evaluation. In managed RAP it reuses those rule implementations rather than adding an ordinary action handler.

Reasoning and example

After a delivery date changes, a supported consumer can request a determine action that evaluates the related date validation. This can show feedback before the user chooses final save. Draft Prepare is a draft-specific determine action used in the activation flow.

Assigned rules still follow their trigger and optimization contracts unless an applicable always setting changes that. Listing rules is not a promise that each one runs every time or in the written order. Determinations run before validations; do not build dependencies on arbitrary order within either group. Expose the capability through the supported projection and consumer configuration. Evaluation alone does not mean COMMIT or successful persistence.

Avoid this mistake A determine action is not a place to write arbitrary action logic, and it does not replace save-time consistency checks.

Read and verify this answer

SAP Learning: determine actions

Official references

SAP Learning on behavior implementation options

SAP Learning on EML and transaction ownership

SAP Learning on numbering and determinations

SAP Learning on draft

SAP RAP BO contract

SAP RAP determinations and unmanaged scope

SAP RAP late-numbering mapping

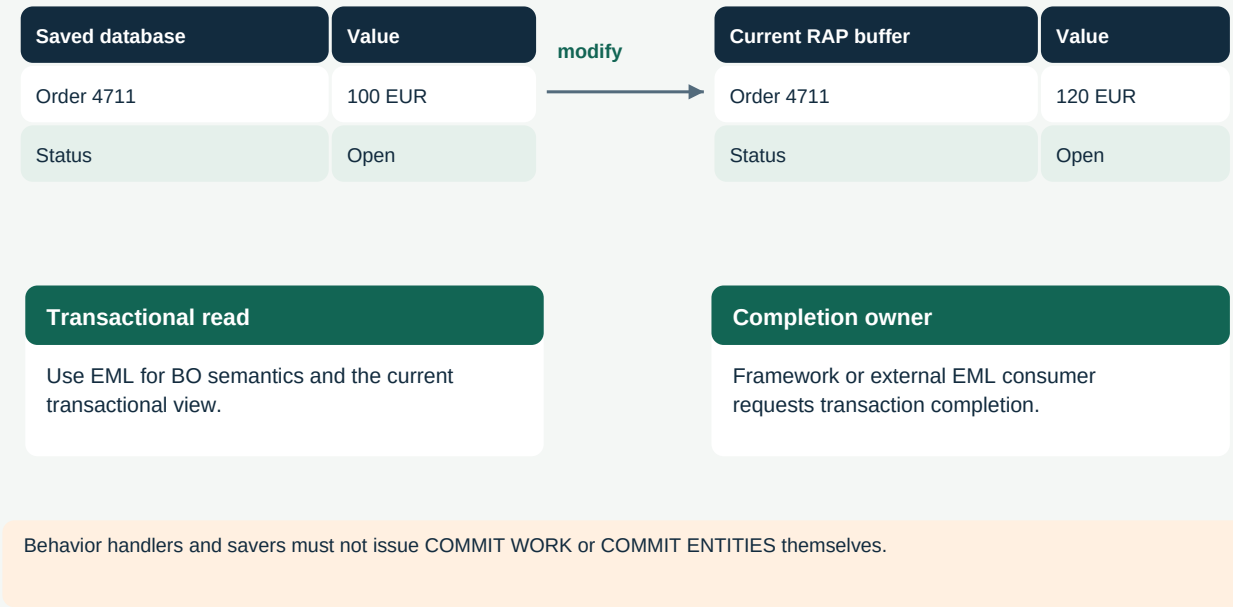
SAP BAPI consumption in RAP: late numbering and save sequence

ABAP Cloud: senior RAP transactions and behavior design

The main interview track: explain a decision, trace its failure paths, and name the evidence that would prove it. These original scenarios extend the supplied RAP notes beyond terminology. They are practice scenarios, not a measured ranking of interview frequency. The target product, ABAP release, language version, BO implementation type and consumer are intentionally unspecified: establish them before promising syntax or runtime support. Read each short answer aloud, then defend it against both follow-up probes.

A buffer is a working set, not a second commit owner

The same business object can have different in-memory and saved states



During modification, the transactional view can show 120 EUR while saved data still shows 100 EUR.

See the visual atlas for the recall prompt, limits and related questions.

SRAP-01 A new approval app has custom persistence, while its posting step must reuse an existing transactional API. Which RAP implementation type would you choose?

Start with managed RAP for the new approval tables. If an eligible API must perform the actual writes while RAP can manage the pending changes, consider managed RAP with unmanaged save. Use fully unmanaged RAP when existing logic must also handle interaction. Check the API's locks, permissions, numbering, buffering, and commits. Calling it a BAPI does not prove it fits.

Reasoning and example

Follow one successful approval and one rejected approval through the proposed design. Identify who holds pending changes and who writes each document. Additional save adds work alongside managed database writes; it is not a way to replace the owner of those writes. Make that choice explicit before implementing the handlers.

If posting happens in another system, it has a separate transaction. A managed approval object with a pending-posting state and reliable asynchronous processing may fit better than a synchronous action that promises everything commits together. Test what happens when approval saves but posting is unavailable, and how a retry avoids creating a second document. Complexity alone is not a reason to choose unmanaged RAP.

Avoid this mistake Choosing unmanaged merely because the application is complex, or assuming managed with unmanaged save removes RAP transaction rules.

Interviewer probes What concrete API property would rule out a synchronous RAP adapter? / Which implementation responsibilities would your choice leave with the framework?

Strong answer criteria Identifies who handles pending changes and database writes; Checks what the API actually does; Separates remote processing from one local transaction

Verification evidence Prepare a decision table and trace a create, failed update and rollback against the selected API on the target release.

Read and verify this answer

SAP: Save options

SRAP-02 An audit record must be saved with every managed order change. Should you use a determination, additional save, or unmanaged save?

For an audit that must be saved with a managed order change, additional save may fit. RAP still writes the order, and the supported save extension writes the audit without committing separately. A determination is for deriving object data, not recording an already completed save. Distinguish this business audit from technical logs that may need to survive a rejected transaction.

Reasoning and example

An audit record may need the user, operation, changed values, and an identifier linking it to the order change. Check which data the save extension actually receives. Changed-field input is not necessarily a complete before-and-after image; use the documented data options and obtain any required previous values correctly. Explicitly cleared fields matter too.

Test order creation, update, deletion, and a rejected save. The business audit should match the actual saved outcome, rather than claiming a change happened when it rolled back. Technical diagnostics for the failure may have a separate purpose and lifecycle. Do not switch all order persistence to unmanaged save merely to add logging, or directly rewrite managed order tables from additional save.

For a managed BO, with additional save keeps the framework's order persistence and adds save_modified for extra work. With unmanaged save replaces that persistence with your implementation of save_modified. A fully unmanaged BO has its own saver contract, commonly save. Neither variant allows your code to execute COMMIT WORK or BAPI_TRANSACTION_COMMIT. BRFplus does not change these rules. For a BAPI integration, review RAP-15 rather than assuming that any API is safe in the audit save hook.

Avoid this mistake Writing the audit in another LUW and claiming it is guaranteed to match the order, or updating managed order tables directly from additional save.

Interviewer probes How do you obtain a trustworthy before-image? / Should an audit failure reject the transaction, and in which phase can you detect predictable errors?

Strong answer criteria Distinguishes business audit from diagnostics; Preserves managed persistence and commit ownership; Checks available change-image semantics

Verification evidence Test successful create/update/delete and rejected save; compare order and audit persistence, including explicitly cleared fields.

Read and verify this answer

SAP: Save options

SAP: integrated BAPI action and late save

SRAP-03 A developer puts ordinary business validation in SAVE because the final data is available there. What would you change?

Move predictable business checks to the supported early save checks, after needed calculations and before late saving. Late save normally writes data that has already passed those checks. An error message is not a general rollback mechanism there. SAP does document special failure-capable saver options for some BAPI integrations. Verify that exact option and release instead of assuming all late failures work alike.

Reasoning and example

An approval amount exceeds the permitted limit. Detect that before database writes and return a message linked to the failing instance. The user can then correct the amount. Waiting until SAVE just because the final values are convenient there puts an ordinary business check at the wrong stage.

A database outage is a different technical failure and needs its supported recovery behavior. If a BAPI integration genuinely needs a documented failure-capable saver, inspect its generated signature and target-release guidance. That exception is not a reason to postpone checks that can run earlier, and it does not allow independently committed side effects. Avoid both absolutes: routine errors cannot all be returned from SAVE, but supported special cases do exist.

Avoid this mistake Saying either 'all errors can be returned from SAVE' or 'no supported late-save failure mechanism exists anywhere'.

Interviewer probes Where do final derivations belong relative to final consistency checks? / What evidence would justify using a failure-capable saver here?

Strong answer criteria Rejects predictable business errors before late save; Distinguishes invalid business data from technical failures; Explains the documented BAPI exception and its limits

Verification evidence Trace the save phases for invalid input and inspect the generated saver signature and SAP guidance for the installed release.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: BAPI consumption, late numbering and save sequence

SRAP-04 One action updates two local RAP BOs and then calls a remote payment service. Can the whole action be atomic?

Compatible local RAP objects can take part in the same RAP-controlled SAP transaction. A normal HTTP payment call cannot join that database transaction. If payment succeeds and local saving fails, local rollback will not undo payment. Model pending work and reliable delivery, or an explicit compensating process. Define timeout, retry, and reconciliation behavior instead of promising one atomic action.

Reasoning and example

Update the two local objects through supported behavior interfaces and check the permitted cross-object operations for the phase. Do not commit each object separately. Their shared local transaction can coordinate those compatible changes, but a remote payment has its own outcome and owner.

Store a stable business reference for the payment request. If the remote response times out, payment may still have completed; treat the result as unknown until a status check or reconciliation establishes it. A blind retry can charge twice. Show users an honest pending or recovery status, and test failures before and after both local save and remote response so every saved state has a recovery path.

Avoid this mistake Assuming that calling HTTP inside an action enrolls the remote system in the SAP database transaction.

Interviewer probes What happens after a payment timeout followed by a retry? / How would you recover after local commit succeeds but event delivery is delayed?

Strong answer criteria Draws the local transaction boundary; Models uncertain remote outcomes; Designs idempotency and reconciliation

Verification evidence Fault-inject before and after each local save and remote response boundary; verify that every persisted state has a documented recovery path.

Read and verify this answer

SAP: General RAP BO implementation contract

SRAP-05 A bulk EML update contains 200 instances, including missing keys and unauthorized changes. How do you report and process the result?

Handle all 200 inputs and keep each response linked to the right instance. FAILED identifies rejected work; REPORTED carries messages, including non-errors. MAPPED is for supported identity mappings, not a list of every success. Combine nested-call responses without losing earlier failures. The caller must still apply its commit policy and inspect the later save result.

Reasoning and example

Read the required current object state in suitable bulk operations instead of running a separate query for each row. Some keys may be missing, some updates may be unauthorized, and others may be valid. Every input needs an accountable outcome; unprocessed rows must not accidentally look successful because no error was added for them.

The consumer then decides whether the intended business unit permits eligible updates to continue or requires rejection of the whole unit. That decision depends on the supported operation and transaction boundary. Test several different failures in one input set and reconcile counts, identities, messages, and final database state. MODIFY ENTITIES accepting a change is not proof it has been committed.

Avoid this mistake Checking only the first input row, treating every REPORTED entry as failure, or declaring the operation committed after MODIFY ENTITIES.

Interviewer probes How would you associate a child error with its root request? / What do you check again after the external consumer issues COMMIT ENTITIES?

Strong answer criteria Handles all inputs and aggregates responses; Separates response roles and transaction phases; Avoids row-by-row dependency access

Verification evidence Run a mixed-input EML test with multiple distinct failures; assert key/message correlation, processed counts and final database state.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-06 An API consumer sends an empty comment to clear it, but the previous text remains. What do you investigate?

Check whether the caller supplied an empty value or omitted the field. In EML, the field list or %control says which fields should change. An empty value without that update instruction can correctly leave the old value alone. Trace payload conversion, adapter change flags, and defaulting logic. Do not mark every field supplied just to fix one cleared comment.

Reasoning and example

The caller sends only Comment with an empty value because it wants to remove the text. Amount and Currency are omitted and must remain unchanged. Verify that the service-to-EML mapping marks Comment for update, then check whether a determination fills the old default again. If a legacy API is called, map the corresponding change indicator too.

Test three cases separately: omitted comment, explicitly empty comment, and new nonempty text. Apply the same distinction to zero or false values where the field's business rules permit them. IS INITIAL alone cannot tell you whether the caller intended a change. Setting all control flags can fix the comment while accidentally clearing unrelated fields.

Avoid this mistake Using IS INITIAL to mean 'not provided' for every update field, or blindly setting all control flags on.

Interviewer probes How would you represent 'leave unchanged' versus 'set to zero'? / Where can a defaulting determination accidentally undo the requested clear?

Strong answer criteria Separates presence from value; Preserves partial-update semantics; Traces control mapping through adapters

Verification evidence Test omitted Comment, explicit empty Comment and nonempty Comment while asserting that all untouched fields remain unchanged.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-07 You must create three new orders with items in one EML request before final order numbers exist. How do you correlate the tree?

Give each new order a unique content ID for this request. Use %cid_ref to connect item creation to the correct newly created parent. Keep child identities and creation mappings too. Use the generated behavior types and read mappings by identifier, not row position. These content IDs link work within a request; they are not final document numbers or durable retry keys.

Reasoning and example

Create three orders, each with several items, before final order numbers are assigned. One item under the second order is invalid. Its message must still point to the correct child and parent, even if inputs are reordered or one creation is rejected. Guessing database keys or matching response-table positions makes this fragile.

Use the supported create-by-association correlation and retain draft or preliminary-key context where applicable. When final keys are assigned, the original parent-child relationships must remain correct. Check MAPPED rather than assuming its order matches the input. Finally, let the external transaction owner apply the intended all-or-nothing policy; correct creation mapping does not by itself decide whether the three orders must save together.

Avoid this mistake Assigning a fake database key to a new parent, correlating by row index, or confusing %cid with %pid.

Interviewer probes What changes if the BO uses draft and late numbering? / How do you correlate an error when the final parent key is still unavailable?

Strong answer criteria Connects new parents and children using content IDs; Keeps the full transactional identity; Tests errors in requests with several parents and children

Verification evidence Execute a reordered multi-parent deep-create case with one failed child and verify associations, mappings and error paths before and after save.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-08 A posting API returns the final document number only when called. How would you integrate numbering without publishing incorrect keys?

Keep preliminary identities until the supported save process assigns final document keys. Map %pre to the final %key during number adjustment. SAP documents a BAPI pattern that calls creation in ADJUST_NUMBERS with a suitable failure-capable saver. Check its availability, API release status, and error rules on the target system. Do not present an earlier identifier as a committed document number.

Reasoning and example

Create several documents concurrently and include a rejected API call. Verify that items and returned mappings still identify the right final documents. Do not publish a number in an email or external event before the business change is durably saved. Late numbering alone also does not guarantee a gap-free sequence; number-range and posting-API behavior still matter.

In the documented failure-capable late-save scenario, a late-save failure makes COMMIT ENTITIES return sy-subrc 8. An external EML consumer must issue ROLLBACK ENTITIES before continuing. Verify support for that exact pattern. This rule does not allow commit or rollback statements inside a behavior implementation, and ordinary ADJUST_NUMBERS must not be assumed to offer the special failure response.

Avoid this mistake Do not use SELECT MAX plus one, lose the preliminary-to-final key mapping, or assume ordinary ADJUST_NUMBERS supports recoverable failures without the documented special saver.

Interviewer probes How does the external EML consumer obtain the final key? / What must be demonstrated if the business insists on gap-free numbering?

Strong answer criteria Separates preliminary and final identities; Qualifies the BAPI saver exception; Does not promise gap-free numbers from RAP alone

Verification evidence Create several documents concurrently, include a failing API call, and verify final key conversion, associations and number-allocation behavior.

Read and verify this answer

SAP: BAPI consumption, late numbering and save sequence

SRAP-09 A user saves a draft, closes the browser, returns, and presses Activate. What state transitions and failures would you explain?

A saved draft is unfinished work, separate from active data. Activating a new draft creates active data; activating an edit draft applies changes to its active instance. Assigned Prepare checks, activation, and final saving must succeed. Discarding an edit draft leaves the existing active object intact. A business Submit action is a separate operation unless explicitly designed otherwise.

Reasoning and example

Ask what the user means by Save. A draft update can store unfinished editing data without completing the business document. The user can close the browser and later continue from that stored draft. If activation finds missing required data, the user should be able to correct the draft rather than lose the work.

Test both new and edit drafts through the intended UI or service. Discard a new draft and confirm it creates no active object; discard an edit draft and confirm the previous active object remains. Reopen in a fresh browser session and check messages as well as data. Other consumers that write active instances directly still need the appropriate checks; a draft-only flow cannot protect every write automatically.

Avoid this mistake Treating draft persistence as a final business commit or implementing Submit by directly updating draft tables.

Interviewer probes Which checks must still protect an API that writes active instances directly? / What remains after discarding an edit draft versus a new draft?

Strong answer criteria Distinguishes draft, activation and business status; Explains both new and edit drafts; Preserves a recoverable validation experience

Verification evidence Test new/edit draft creation, browser reopen, failed Prepare, successful activation and both discard variants; inspect active and draft results.

Read and verify this answer

SAP: Draft runtime

SRAP-10 A draft lock expires, another process changes an active child, and the first user resumes editing. How do you prevent silent loss of the child change?

Check the total ETag for the whole relevant active object, including child changes. After an exclusive draft lock expires, resume uses that version to detect changes made to active data. A normal entity ETag has a different request-level job. If a child update does not advance the required total version, an old draft may overwrite newer work unnoticed.

Reasoning and example

A user starts editing an order draft and leaves it open until the exclusive lock expires. Another process changes only an active item. When the first user resumes, the application should detect that the active object changed and offer the supported conflict recovery, rather than treating the old draft as current.

Test this exact child-only case with two sessions. Check all active update paths, including APIs outside the UI. In unmanaged RAP, inspect the implemented lock and version maintenance rather than assuming it is provided automatically. Direct table writes can bypass it entirely. Renewing a lock is not proof that a stored draft still matches active data; the total-ETag check supplies that missing comparison.

Avoid this mistake Checking only the root's own business fields or assuming an expired lock means the stored draft is automatically current.

Interviewer probes Which timestamp should change after a child-only modification? / How do you distinguish stale If-Match handling from draft resume protection?

Strong answer criteria Separates ordinary and total ETags; Includes child and alternative update paths; Defines a stale-draft recovery experience

Verification evidence Use two sessions, expire the draft lock, change only an active child, then resume and verify the conflict and unchanged active data.

Read and verify this answer

SAP RAP total ETag

SRAP-11 Two determinations calculate discount and total. Production results depend on which one executes first. How would you redesign them?

Calculate discount and total from the current source values without relying on an unspecified handler order. Repeating the calculation with the same input must give the same result. Cover every input that can affect it and avoid trigger loops. A shared calculation routine can help. For unmanaged active instances, first confirm that the chosen determination mechanism is actually supported.

Reasoning and example

If a rule subtracts a discount from an amount already discounted by a previous run, repeating it keeps lowering the total. Calculate from base price, quantity, and the intended discount instead. Changing Quantity and Discount together should produce the same final total as changing them separately. Update only the derived values that need changing.

Test repeated execution, combined edits, draft activation, and rules that trigger each other. Make sure they settle on a consistent result rather than loop or create extra children. Keep notifications and remote side effects out of repeatable calculations. A session flag saying run once can hide the symptom while leaving later valid changes uncalculated, so it is not a reliable replacement for correct derivation logic.

Avoid this mistake Depending on declaration order, using a session flag to run once, or sending a notification each time the determination runs.

Interviewer probes What happens when activation reuses already executed consistency work? / How would you detect and prevent a determination-trigger cycle?

Strong answer criteria Repeating a calculation gives the same result; Does not rely on an unsupported handler order; Checks triggers and what the implementation type supports

Verification evidence Test combined and separate field changes, repeated evaluation, draft activation and create-then-delete in one transaction; compare final values.

Read and verify this answer

SAP: Determinations

SRAP-12 A request changes CompanyCode to an unauthorized value and also makes its dates inconsistent. Would you use prechecks or validations?

Use a supported precheck to reject a disallowed incoming company-code change before it enters the buffer. Use the appropriate validation to reject inconsistent final dates. Keep the complete RAP authorization design as well; neither mechanism replaces it. For partial updates, combine supplied fields with the existing values needed for the check. Do not assume the request contains every field.

Reasoning and example

A user may be allowed to edit a request in company 1000 but not move it to company 2000. Check the proposed target context at the allowed operation boundary, including any permissions needed for the original context. The precheck must not accidentally accept an unauthorized change because it only examines the old company.

Separately, check the resulting date range. If only the end date is supplied, read the current start date needed for comparison. Test through both EML and the service so the rule is not just a UI convenience. For unmanaged active instances, implement consistency in the suitable handler or save checks; declared draft validations do not automatically protect active database writes.

Avoid this mistake Treating prechecks as a global security bypass or believing a UI mandatory marker enforces all backend invariants.

Interviewer probes Which state does the precheck see before the update is applied? / How would the rule behave for a partial update that omits both dates?

Strong answer criteria Separates incoming intent from final state; Keeps authorization explicit; Accounts for partial updates and unmanaged restrictions

Verification evidence Verify that a denied target context is rejected by the precheck before entering the transactional buffer. Separately verify that inconsistent dates are rejected by validation and cannot be persisted; test correction and successful retry through EML and the service.

Read and verify this answer

SAP RAP operation prechecks

SRAP-13 A CopyOrder action creates an order, but the UI stays on the source and the consumer cannot identify the copy. How do you review the action contract?

Decide whether CopyOrder should be a factory action that creates a new instance. Its MAPPED response links the caller's content ID to that creation; it is not an ordinary action-result parameter. A normal action with a declared result must fill that result correctly instead. Check supported cardinality and UI navigation on the target release. Returning $self does not automatically mean create a copy.

Reasoning and example

Define what copying means before debugging navigation. Generate a new identity, reset approval status, omit source audit ownership, and decide which children should be copied. Assigning the whole source structure can incorrectly copy permissions or business state even when the new key is returned correctly.

Then call the action with several inputs through EML and the UI. Verify that each created object maps to the right source request and that the UI navigates according to the supported action behavior. If one source must create several targets, check the allowed cardinality for this release and consumer. A hard-coded UI redirect can hide a broken action response instead of fixing its identification rules.

Avoid this mistake Assuming all factory-action cardinalities are identical on every release or solving wrong result mapping with a hard-coded UI redirect.

Interviewer probes What changes if one source must create several target objects? / Which source fields and children must never be copied automatically?

Strong answer criteria Chooses factory versus ordinary action deliberately; Preserves request-to-created-instance mapping; Defines copy semantics and release-dependent consumer behavior

Verification evidence Call the action through EML and the UI with multiple inputs; verify mappings, new identities, reset fields and navigation to the correct instance.

Read and verify this answer

SAP: Action definition

SRAP-14 A projection accepts a convenience field that must enrich an underlying create. Why might augmentation fit, and what are its boundaries?

Augmentation can add or translate input for supported operations at a RAP projection. Preserve values the caller already supplied and link generated work back to its original request. Check the permitted operations, control flags, and response behavior on the target release. It is not a general save hook. The base object's business rules must still protect every consumer.

Reasoning and example

A specialized projection accepts a display language when creating a request. Supported augmentation may add a related text creation so the underlying object receives both pieces of input. If the caller already supplied that text or a conflicting value, handle the conflict deliberately rather than overwriting it without explanation.

Use unique content IDs and the supported correlation mechanism so an error in generated child work can be traced to the original create. Test multiple instances, omitted convenience input, and generated-operation failures. Augmenting EML does not behave like an unrestricted normal modify call with arbitrary immediate results. Keep permanent consistency checks in the base behavior so another projection cannot bypass them by omitting the augmentation.

Avoid this mistake Using augmentation to bypass base validations or expecting an unrestricted local result from an augmenting operation.

Interviewer probes What if the caller already supplied the field your augmentation wants to set? / How is a generated child operation related back to the original request?

Strong answer criteria Uses augmentation to adapt projection input; Preserves supplied-field flags and request correlation; Keeps shared business checks in the base object

Verification evidence Test absent/present convenience fields, conflicting underlying input, multi-instance requests and generated-operation failures with the supported augmentation API.

Read and verify this answer

SAP: Operation augmentation

SRAP-15 A legacy function module issues COMMIT WORK internally. Can you make it safe by wrapping it in a RAP action?

No. The wrapper still executes the function module's commit, which conflicts with RAP owning the transaction. Moving that call into a saver does not remove the problem. Look for a supported API with compatible buffering and saving, or design a separate asynchronous transaction explicitly. Check the entire call chain, language permissions, API release state, and authorization rules before integrating it.

Reasoning and example

Calling a function module that commits inside local RAP processing violates the runtime rules and may cause a runtime error. Do not assume the commit will simply succeed and RAP will continue normally. Trace the full call chain to find explicit or implicit commits before selecting the integration approach.

Moving the call to RFC or HTTP in a separate transaction does not restore atomicity. That remote operation can commit while local RAP saving later fails, leaving a real document behind despite the local rejection. If separate processing is necessary, define pending states, safe retry, and reconciliation. Test where durable effects occur. Do not hide the committing call or disable checks while promising that RAP can roll everything back.

Avoid this mistake Do not hide a commit inside a wrapper or disable runtime checks. Moving it to another task does not make the separate transactions one LUW.

Interviewer probes How would you discover an implicit commit several calls below the adapter? / What business states would an asynchronous replacement expose?

Strong answer criteria Preserves RAP transaction ownership; Audits nested and implicit transaction effects; Offers a viable alternative integration boundary

Verification evidence Review the dependency call graph and trace successful and rejected requests; document the first durable effect and the actual transaction owner.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: BAPI consumption, late numbering and save sequence

SRAP-16 An older RAP BO works without strict mode. The team wants to release it for cloud consumption by adding strict(2). Is that sufficient?

No. strict(2) adds modeling and implementation checks; it does not release the object as an API. Review base and projection behavior together, including locks, permissions, ETags, and composition relationships. Check derived-type changes and eligible dependencies. A released API also needs a stable supported interface and lifecycle plan. Syntax success alone proves neither compatibility nor secure operation.

Reasoning and example

Capture the existing service metadata and representative EML and OData calls before enabling strict mode. Previously implicit behavior may need to be declared explicitly, and changes can affect what existing callers may do. Activate and test the whole dependent model on the target release, not just the base behavior definition.

Decide what the future released interface promises and which implementation details remain private. Check consumption and extension permissions separately, and define how incompatible changes will be handled. Documentation for a newer product version may describe features unavailable on this target. Run appropriate ATC checks and real consumer regressions, but do not treat strict mode as a Clean Core grade or an automatic API-release switch.

Avoid this mistake Treating strict(2) as a clean-core grade or API-release switch, or enabling it only on one layer and ignoring resulting contract changes.

Interviewer probes Which implicit behavior could disappear when strict mode is enabled? / What would you freeze in a released BO interface rather than in the base implementation?

Strong answer criteria Separates strict checks from API release; Reviews all affected layers; Plans compatibility and consumer regression evidence

Verification evidence Record before/after metadata, activate the full dependency set, run ATC and execute representative old consumer calls on the target release.

Read and verify this answer

SAP: Strict mode implementation requirements

SRAP-17 An unmanaged BO passes isolated tests but a second request reads leftovers after an earlier save rejection. Where do you look?

Inspect the custom buffer's lifetime and every cleanup path, including early save rejection and external rollback. Reads must show this transaction's changes, not leftovers from an earlier one. CLEANUP and CLEANUP_FINALIZE have different jobs. After an early rejection, remove the appropriate finalize-derived work while retaining the state needed for valid continued interaction. Do not clear all data blindly.

Reasoning and example

An unmanaged implementation stores pending rows in a static internal table. Tests pass when each starts in a fresh session, but a failed transaction leaves entries that the next transaction reads. The problem can affect deleted rows, preliminary keys, counters, and dependency data as well as ordinary updates. A static table is not automatically a safe cache.

Run failure-then-success and rollback-then-read sequences in a reused session. Also test correcting data after an early save rejection. Identify which values came from user interaction and which were added during FINALIZE, then apply the documented cleanup behavior for each phase. Cleaning only after successful saves leaks state; erasing every user change after rejection can break the intended retry behavior.

Avoid this mistake Implementing cleanup only after success or deleting all user changes after an early-save rejection without understanding the retry contract.

Interviewer probes Which data was created during FINALIZE rather than during user interaction? / How would an in-transaction READ differ from a direct SELECT after an unsaved update?

Strong answer criteria Keeps pending state inside its intended transaction; Distinguishes the cleanup phases; Reads current pending changes and preserves correct retry behavior

Verification evidence Run failure-then-success, rollback-then-read and repeated-save sequences within a reused session; assert no stale buffer entries or leaked temporary identifiers.

Read and verify this answer

SAP: Cleanup after early save failure

SRAP-18 An Approved event should trigger an external workflow. Why is calling its HTTP endpoint from an on-save determination unreliable?

An on-save determination can run before commit and can run again. Its remote HTTP call cannot be undone by a local rollback. Use supported transactional event publication or another deliberately designed delivery mechanism. The event should describe a committed business change. Delivery and processing can still be retried, so the consumer must prevent duplicate effects and support recovery.

Reasoning and example

An order is approved locally, but saving later fails. If a determination already called the workflow endpoint, a workflow may exist for an approval that never became permanent. Publish through the supported transaction-aware path so that rejected local work does not create that false business fact.

Give the event a stable identity and enough version or transition information for the consumer to recognize duplicates and stale work. A repeated event must not create a second workflow, and a delayed event may need checking against a later cancellation. Monitor the local change, publication, delivery attempts, and consumer outcome separately. A successful HTTP response or in-process event is not proof of exactly-once business completion.

Avoid this mistake Equating a successful HTTP response or a raised in-process event with durable, exactly-once end-to-end workflow execution.

Interviewer probes How would you recover an event that is delivered after the order is cancelled? / What identifier prevents duplicate workflow creation?

Strong answer criteria Separates local commit from remote effects; Handles duplicate and stale events safely; Monitors publication separately from consumer processing

Verification evidence Test rejected save, publisher outage, duplicate delivery and stale delivery; verify no workflow for rejected data and one effective workflow per transition.

Read and verify this answer

SAP: Business events

SRAP-19 A credit-limit check runs successfully on a draft, but active data is later saved through another consumer with an invalid amount. What is missing?

Protect the credit rule on every supported write path, including direct active updates. Check validation triggers, Prepare, and the implementation type. Managed save validation and fully unmanaged active consistency handling are different mechanisms. Also consider concurrent credit use. Passing a check earlier does not reserve credit or guarantee that the same amount is still available when saving.

Reasoning and example

A draft passes the credit check at 10:00. Before activation, another order uses part of the same customer's limit. The old check now describes an earlier situation. An API that writes active data directly can also miss a rule implemented only for a draft screen or draft-specific check.

Test both draft activation and direct active writes. Then run two transactions whose individual amounts fit the limit but whose combined amount exceeds it. Depending on the credit service, you may need an authoritative reservation, suitable locking, or a process that permits later rejection. Adding the same simple read twice does not eliminate the race between checking available credit and consuming it.

Avoid this mistake Duplicating the check only in the UI or assuming validation alone eliminates a cross-transaction race.

Interviewer probes What prevents two parallel orders from consuming the same available credit? / Which direct active-instance consumer tests would you add?

Strong answer criteria Protects the credit rule for all supported callers; Handles unmanaged active writes correctly; Recognizes that data can change between checking and using it

Verification evidence Exercise draft activation and direct active writes, then run two competing transactions whose combined amount exceeds the limit.

Read and verify this answer

SAP: Validations

SRAP-20 In a mass approval request, one instance fails. Must all other approvals be rolled back?

It depends on the business unit of work and the actual transaction boundary. An EML operation, a SAP LUW, an OData atomicity group, and a whole batch are not the same scope. One FAILED entry alone does not define the outcome for every other input. The transaction owner must inspect responses and apply the required save or rollback policy.

Reasoning and example

If the business approves a package only when every request qualifies, model that package's all-or-nothing requirement explicitly and test the save boundary. If approvals are independent, split work into appropriate independent units and show each result clearly. Neither policy should happen accidentally because of how a loop or HTTP envelope was written.

Run a mixed-success case through the actual EML consumer and OData request structure. Check the final database state after the chosen commit or rollback path. A successful interaction response is not proof of persistence, and an OData batch can contain separate atomicity groups. Align the technical boundary with the business expectation rather than claiming every batch succeeds or fails as one transaction.

Avoid this mistake Claiming that any FAILED entry universally rolls back every request, or that every OData batch is one atomic transaction.

Interviewer probes How does your answer change for separate atomicity groups? / What must an external EML consumer do after a partial interaction failure?

Strong answer criteria Identifies which changes must succeed together; Distinguishes accepted changes from a successful save; Matches transaction handling to the business requirement

Verification evidence Test a mixed-success action via EML and the actual OData request shape; inspect persisted state after both intended commit and rollback paths.

Read and verify this answer

SAP: Entity Manipulation Language

SRAP-21 After a quantity change, the backend total is correct but the Fiori page displays the old value. Is this a transaction bug?

First check whether the backend value is correct in the relevant draft or active state. If it is, inspect action results, side-effect metadata, and the UI's actual refresh requests. Side effects tell supported consumers what to reread; they do not calculate totals or commit data. Fix the refresh rule when that is the problem, rather than adding a commit.

Reasoning and example

Change an item quantity, then read the object through the service or EML in the same relevant state. If the total is already correct there, compare the browser's requests and response values. A missing refresh after a child change can explain why the screen shows the old total even though the calculation worked.

Verify that the metadata asks the supported client to refresh the needed total and action availability, and confirm those requests really occur. Reloading the entire page may hide the issue while losing edit context and fetching unnecessary data. If the total only becomes correct after activation, investigate the backend trigger timing too. Keep backend calculation and client display evidence separate throughout diagnosis.

Avoid this mistake Adding COMMIT WORK to make the UI update or treating a side-effect declaration as a substitute for a backend determination.

Interviewer probes What if the total is correct only after activation? / Which refresh targets and requests would you expect after an item quantity change?

Strong answer criteria Diagnoses backend state and client state separately; Preserves transaction ownership; Validates metadata against real consumer behavior

Verification evidence Compare EML/service values with browser requests before and after the change; verify draft context, refreshed totals and request volume.

Read and verify this answer

SAP RAP side effects

SRAP-22 Your team finds a useful SAP RAP base BO in ADT, but only a narrower BO interface is released. Which layer should your application consume?

Use the released BO interface and only the fields and operations it exposes for your allowed use. Visible source in ADT is not permission to consume the underlying base object. The interface lets the provider change internals while supporting callers. If an operation is missing, find another released API, request it, or revise the process. Do not bypass the interface with table writes.

Reasoning and example

Check whether the released interface includes the actions, fields, associations, and draft operations your application needs. A RAP BO interface exposes a supported subset of a business object; it is not simply an ordinary OO interface where you invent a handler. Target-release and cross-component visibility rules still apply.

Build the consumer using only that exposed subset and test its permissions and failure responses. If the base implementation has a useful extra operation, resist calling it merely because it activates today. C1 permission to consume the interface also does not imply permission to extend every underlying object. Record the actual capability gap so the design decision is explicit rather than hidden in an unsupported call.

Avoid this mistake Calling the base BO because it activates today, or equating C1 consumption permission with permission to extend every underlying artifact.

Interviewer probes What does the interface protect when the provider changes its persistence model? / How would you handle a required operation absent from the released interface?

Strong answer criteria Calls the released interface; Separates permission to use from permission to extend; Checks missing capabilities before implementing

Verification evidence Record API state and visibility in the target system; compile and test the consumer using only exposed fields, associations and operations.

Read and verify this answer

SAP: Business object interface

SRAP-23 Two requests execute Submit for the same order, and one client retries after a timeout. How do you prevent duplicate business effects?

Protect Submit as a business-state change using the appropriate locks and version checks. Then define what a retry means once submission has already succeeded. Return the original effective result or reject the repeat consistently. Use durable correlation for downstream work so retries do not duplicate it. A disabled button or request-local content ID cannot provide that protection across requests.

Reasoning and example

Two requests both read status Open before either has protected the transition. Without the right concurrency handling, both may start submission. Coordinate the transition and its durable follow-up intent so only the intended business effect is created. Test simultaneous calls as well as a later retry after the response to a successful commit was lost.

If callers provide an idempotency key, scope it to the operation and relevant business context. Detect reuse with incompatible parameters and decide how long the result is retained and found. A timeout should not force the caller to guess whether another Submit is safe. A singleton flag, browser debounce, or %cid does not store the cross-request business history needed for this decision.

Avoid this mistake Relying on a singleton flag, browser debounce, or request-local %cid as a durable cross-request idempotency key.

Interviewer probes What response should the retry receive if the first call committed but its response was lost? / How do you handle the same idempotency key with different parameters?

Strong answer criteria Coordinates state transition and concurrency; Defines retry outcomes explicitly; Prevents duplicate downstream effects durably

Verification evidence Run simultaneous Submit calls and replay after a simulated lost response; verify one effective transition and one intended downstream operation.

Read and verify this answer

SAP: General RAP BO implementation contract

SAP: Business events

SRAP-24 A validation reports the right error once, but the message remains after correction or appears on the wrong child. How would you debug it?

Check the message's transactional key, field markers, parent path, and state area. A state message must be removed or replaced when its condition is checked again. Confirm that correction triggers the right validation and uses the right draft instance. Add FAILED where rejection is required. Correct text alone cannot fix a message attached to the wrong child or stale state.

Reasoning and example

Two items have different delivery dates, and only the second is invalid. The message must identify that item and its parent path correctly. After the user fixes its date, revalidate the affected state area and clear the obsolete result using the supported pattern. Do not simply append another copy of the same message on every call.

Test invalid-to-valid-to-invalid edits, deletion of the failing child, and draft activation. Preserve unrelated errors from other checks rather than clearing all messages globally. Compare state messages that describe current invalid data with transition messages that describe one operation. Finally, verify the actual save rejection: REPORTED alone is not a substitute for the required FAILED response.

Avoid this mistake Appending error text on every call without resetting its state area, or assuming REPORTED alone rejects a save.

Interviewer probes Which message should survive a round trip and which should not? / How would you preserve an unrelated validation's error while clearing this one?

Strong answer criteria Correlates messages to transactional identity and fields; Manages state areas without erasing unrelated state; Separates messaging from operation rejection

Verification evidence Test invalid-to-valid-to-invalid edits on two children, child deletion and activation; assert message paths, stale-message removal and correct save rejection.

Read and verify this answer

SAP RAP state messages

Official references

SAP: General RAP BO implementation contract

SAP: Save options

SAP: BAPI consumption, late numbering and save sequence

SAP: Entity Manipulation Language

SAP: Draft runtime

SAP: Determinations

SAP: Action definition

SAP: Operation augmentation

SAP: Strict mode implementation requirements

SAP: Business object interface

SAP: Business events

SAP: Cleanup after early save failure

SAP RAP total ETag

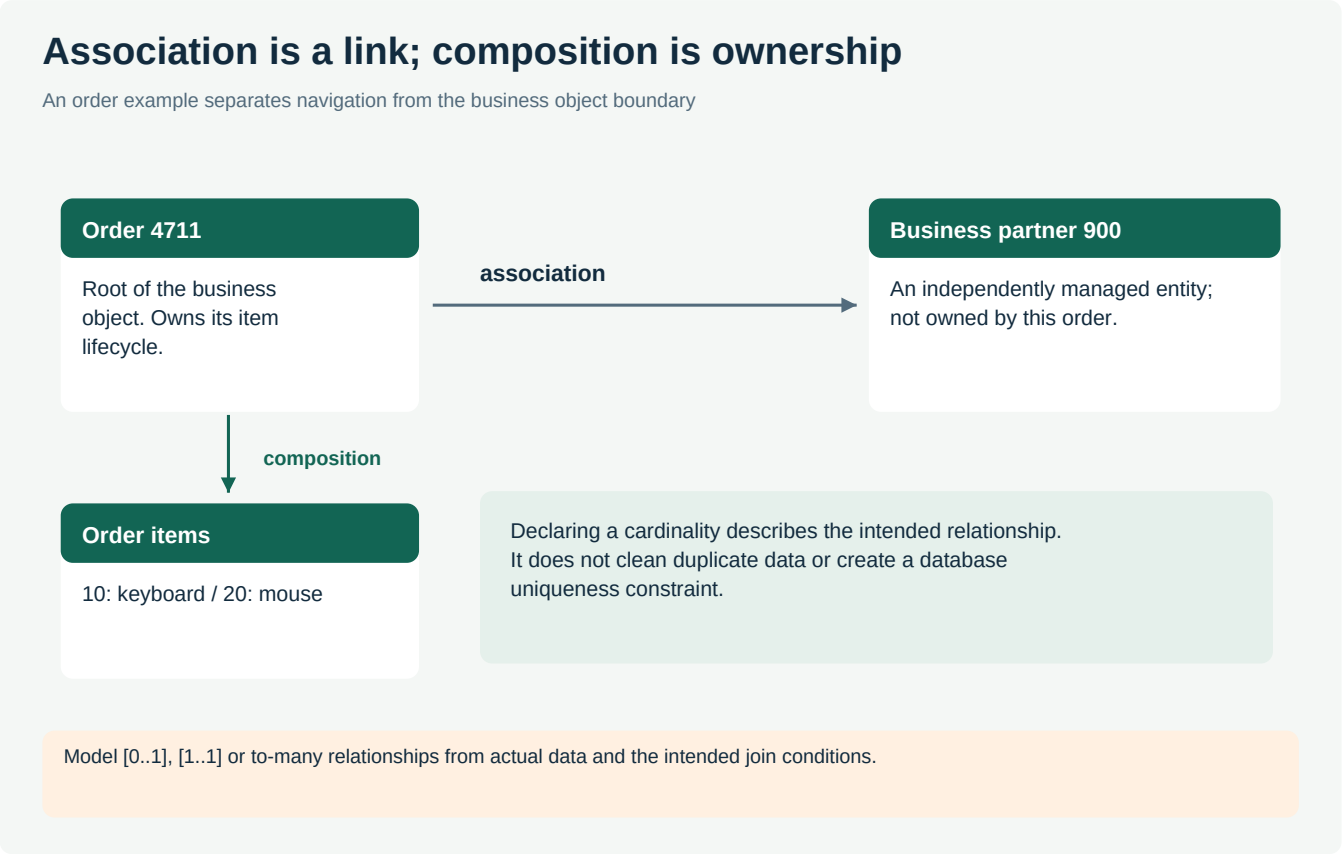
SAP RAP operation prechecks

SAP RAP side effects

SAP RAP state messages

CDS data modeling

Explain the data model before discussing annotations. A strong answer connects entity identity, association cardinality, authorization and the intended consumer. Distinguish a reusable CDS model from a transactional RAP projection and from an analytical query.



Items belong to the order BO; a partner reference crosses to an independently managed object.

See the visual atlas for the recall prompt, limits and related questions.

CDS-01 What does ABAP CDS add to a SQL query?

CDS turns a database query into a reusable ABAP data model. It can describe fields, relationships, currency meanings, annotations, and read-access rules. Several apps can use that model instead of rebuilding the same query. SQL still produces the database result. If users must create or change business objects, RAP behavior supplies those operations separately.

Reasoning and example

A sales-order list needs an order number, customer, and amount. A CDS model can expose these fields, connect the customer data, and identify the amount's currency. Appropriate access control can restrict which orders the caller reads. Another application can reuse this model instead of copying all the joins and rules.

That reuse also means the exposed fields and meanings matter to callers. Changing a field's meaning can break an app even when the query still runs. And a readable order model does not automatically support creating an order. For that, define the supported business operations and their behavior rather than treating a CDS SELECT as a complete transaction implementation.

Avoid this mistake A CDS definition does not automatically implement create, update and delete operations.

Read and verify this answer

SAP Learning on CDS concepts

CDS-02 What is the difference between a classic CDS view and a CDS view entity?

A classic CDS view creates both a CDS entity and a separately named DDIC SQL view. A CDS view entity does not create that separate SQL-view object. For new development, prefer view entities where the target release and consumers support them. Migration needs care because existing programs may call the old SQL-view name directly.

Reasoning and example

Suppose a classic view declares a SQL-view name that several reports use in SELECT statements. Replacing the CDS definition with a view entity changes more than the source syntax: those reports still expect the separately named object. Find the consumers and plan how they will use the replacement.

Check release support and the requirements of tools that consume the view before converting it. Test representative reads after changing the callers, including their fields and result identity. Do not copy AbapCatalog.sqlViewName into the view entity, because it belongs to the classic pattern. The useful interview distinction is which repository objects exist and who depends on them, rather than simply calling one syntax old and the other new.

Avoid this mistake Do not copy AbapCatalog.sqlViewName into a view entity or assume all older systems support view entities.

Read and verify this answer

SAP Learning on CDS concepts

CDS-03 Association or join and how do you select cardinality?

A join combines sources in the current query. An association defines a relationship that a consumer can follow, often producing a join when used. Cardinality describes how many related rows can really match the ON condition. It must match the data. Declaring a relationship to-one when several rows can match can lead to incorrect results.

Reasoning and example

Order 4710 has amount 100 EUR and a status with English and German texts. Joining only on Status finds both texts, so the order appears twice and a sum can incorrectly show 200 EUR. Match Status and the requested Language. If the text source is truly unique for that pair, a to-one target cardinality may be correct. Test missing texts and unexpected duplicates too.

The order's own key is a separate issue. Declaring OrderUUID as a key does not remove rows duplicated by an incomplete join. Likewise, writing a to-one cardinality does not enforce uniqueness in the text source; the ON condition and actual target data must support it. Also measure performance where needed. An association is useful for modeling relationships, but is not automatically faster than writing a join.

Avoid this mistake Associations are not automatically faster than joins and do not enforce referential integrity.

Read and verify this answer

SAP Learning on CDS concepts

CDS-04 Association versus composition in RAP?

Use an association for a relationship between entities. Use a composition when a child belongs to a parent within one RAP business object, such as an order item belonging to its order. That ownership affects the object's lifecycle and modeled lock and authorization dependencies. When exposing projections, redirect these relationships to the matching projected entities.

Reasoning and example

An order item has its own table row, but it normally exists as part of an order. A composition expresses that ownership and supports operations such as creating items through their parent. Review the modeled delete behavior and how children depend on the root for locking and authorization. Those choices must match the business object's lifecycle.

A customer linked to the order is different: deleting the order should not normally delete the customer. That relationship belongs as an association rather than a composition. In the app's projections, make sure navigation points to the projected order and item entities. A foreign key alone does not decide whether the target is an owned child.

Avoid this mistake Not every foreign key should become a composition; lifecycle ownership is the deciding factor.

Read and verify this answer

SAP Learning on compositions

CDS-05 What do interface composite consumption and projection mean?

These names describe different modeling jobs. Interface views aim at reuse, and consumption views serve particular consumers. VDM basic, composite, and consumption annotations describe a modeling convention. A RAP projection exposes a supported view of one underlying entity for a use case. Its provider contract limits what it can do. None of these labels publishes a service by itself.

Reasoning and example

Two apps may share the same request model but need different fields and operations. Keep common meanings in the reusable model. Give each app an appropriate projection, and redirect its relationships consistently so navigation reaches the intended projected entities. This avoids copying core business definitions for every screen.

Do not add an extra CDS layer just to satisfy a naming diagram if it adds no useful responsibility. Likewise, labeling a view composite does not make it eligible for every projection feature or create a service binding. Check the provider contract before adding joins or expressions. The key explanation is what each layer does for a consumer and what changes it protects that consumer from.

Avoid this mistake A VDM composite annotation is neither a prerequisite for service binding nor permission to perform arbitrary joins in a RAP projection.

Read and verify this answer

SAP Learning on CDS concepts

CDS-06 What are CDS abstract and custom entities?

An abstract entity defines a structured type, for example the input of an action. It does not produce a database query result. A custom entity defines query data that an ABAP provider supplies instead of generated SQL. Neither means an abstract ABAP superclass. Choose according to whether you need a data shape or an implemented query.

Reasoning and example

A Submit action might need a comment and requested approval date. An abstract entity can describe that input shape; the action is still declared in the behavior definition. There is no table of action parameters to read merely because this type exists.

For a supplier search backed by a remote service, a custom entity can describe the result fields while an ABAP query provider fetches the data. That provider must respect the requested filters, ordering, paging, count, and permissions as applicable. Returning the remote service's first page without honoring the caller's request can make the list misleading. The custom entity supplies the model, not an automatic implementation of those query rules.

Avoid this mistake Do not describe an abstract entity as a BO that other CDS entities inherit and persist.

Read and verify this answer

SAP CDS abstract entities

CDS-07 Does DCL on a base view automatically protect every higher view?

No. Access control on a lower CDS view does not automatically protect every view built on it. Design the read rules for the entity that callers actually access. Supported DCL inheritance can reuse conditions with the required mappings. The #CHECK annotation alone creates no rule. RAP permissions for changing objects are a separate check.

Reasoning and example

Assume the base order model restricts Lea to sales organization A, and a service exposes a higher consumption view. Test that exposed view through its actual read path. If order B1 from organization B appears alongside A1, the protected base view has not proved the service is protected. Check the customer value help separately so it does not reveal B-Customer either.

Inspect the effective DCL on the accessed entity and any inherited conditions and field mappings. Include users with different permissions and realistic filters in the test. Then test update or approval permission separately in RAP. A correct read filter limits visible rows; it does not automatically decide whether the user may execute every action on those rows.

Avoid this mistake DCL on one layer does not automatically secure an arbitrary view-on-view stack.

Read and verify this answer

SAP Learning on DCL inheritance

CDS-08 Where do UI annotations and value helps belong?

Put supported UI and value-help annotations in metadata extensions when the model allows it. Keep data meanings, such as the link between an amount and its currency, in the data model. Check which annotations and extension layers the release supports. A value help makes input easier, but the backend must still check submitted values and permissions.

Reasoning and example

A Fiori elements list shows a request amount and offers a cost-center value help. A metadata extension can describe the list columns and the supported value-help setup. The CDS data model still needs to say which currency belongs to the amount so consumers interpret the value correctly.

Now call the service directly and submit a cost center that the value help would never offer. The backend must reject it when it violates a business rule or permission. Hiding a field or narrowing a picker is not enough. Also check annotation layer precedence: a customer or partner extension may interact with existing metadata, so moving every annotation mechanically can change the resulting UI.

Avoid this mistake UI.hidden and a value help are not security or data-integrity controls.

Read and verify this answer

SAP Learning on CDS concepts

CDS-09 What is a virtual element and what are its limits?

A virtual element is calculated in ABAP for a supported query scenario, often through a SADL calculation exit. It is not a stored database column. That matters when callers want to filter or sort by it. Those operations need explicit supported handling, and support varies by service and release. Use database-side modeling when efficient database filtering is essential.

Reasoning and example

A list might display a status description calculated from data that is not stored in its main table. A virtual element can be suitable if the supported query flow calls the calculation and supplies the needed input fields. Request only the inputs required rather than fetching unrelated data for every row.

Now suppose users need to sort thousands of rows by that value before paging. A calculation performed after reading a page cannot automatically produce the correct global order. Check supported sorting or filtering handling, or choose a different model. A UI requestAtLeast hint can help a particular UI request fields, but it is not a guarantee for every possible consumer.

Avoid this mistake requestAtLeast is a UI request hint, not a universal guarantee that every consumer calculates a virtual element.

Read and verify this answer

SAP RAP: implementing sorting of virtual elements

CDS-10 How do analytical CDS models differ from transactional models?

An analytical model answers questions such as totals by month or company. It organizes measures, dimensions, and supported aggregation rules. A transactional RAP model identifies individual objects and exposes operations such as Create or Approve. A model designed for one job is not automatically correct for the other. Always account for currencies, units, and the actual query engine.

Reasoning and example

An order app may display individual orders and allow approval. A spending report instead groups values by supplier and month. If some orders are in EUR and others in USD, adding the raw amounts gives a number without a valid common currency. Group them appropriately or use the intended conversion before interpreting a total.

Check what the selected analytical engine, service protocol, and client actually support. A cube or analytical query has responsibilities that an ordinary transactional list does not. Merely adding a VDM annotation or sending $apply does not prove that grouping, conversion, and aggregation have the intended meaning. Test a small dataset with known mixed currencies before trusting the totals.

Avoid this mistake A VDM annotation or an OData $apply request alone does not guarantee correct analytical semantics.

Read and verify this answer

SAP analytical projection views

CDS-11 How do CDS keys and database keys relate?

A CDS key tells consumers which fields identify a row. It does not force the query result to be unique. A database key enforces uniqueness in a table, but joins and grouping can change the result. RAP and OData need a stable, truly unique identity. Check the complete result, not just the key declarations.

Reasoning and example

Imagine flight rows identified by carrier, connection, and flight date. If a CDS entity marks only carrier and connection as keys, two flights on different dates appear to have the same identity to the client. A list may confuse them or address the wrong row even though the database has two valid records.

Include the fields needed to identify one result row and check what joins do to that uniqueness. Joining a flight to several related rows can multiply the result again. Marking another field as key does not remove duplicates or repair the underlying query. Test with realistic repeated business values, not just sample data where every connection happens to appear once.

Avoid this mistake Marking an element key does not remove duplicate rows.

Read and verify this answer

SAP Learning on CDS concepts

CDS-12 Is a maintenance view a RAP view and how are defaults supplied?

A classic maintenance view belongs to DDIC table or view maintenance, often used through SM30. A RAP app uses a CDS model and behavior to provide a different kind of maintenance application. In RAP, determinations or supported defaulting functions can supply defaults. The chosen UI and metadata control presentation. First clarify which maintenance approach the question means.

Reasoning and example

A team asks for a small configuration maintenance screen. If it means an existing SM30 dialog, examine its maintenance events and supported classic setup. If it means a Fiori RAP app, design the CDS entities and behavior, including who may change entries and how default values are supplied. The word maintenance does not make these the same technology.

For example, a RAP determination may fill an initial currency while backend checks still reject invalid input. UI annotations can help present the fields, but they do not replace that behavior. Do not invent generic Metadata.ignore or Metadata.defaultValue annotations as a shortcut: verify the actual supported defaulting and visibility mechanisms for the chosen app.

Avoid this mistake Metadata.ignore and Metadata.defaultValue are not generic annotations that implement maintenance behavior.

Read and verify this answer

SAP Learning on CDS concepts

SAP Learning on DCL inheritance

Official references

SAP Learning on CDS concepts

SAP Learning on DCL inheritance

SAP Learning on compositions

SAP analytical projection views

SAP CDS abstract entities

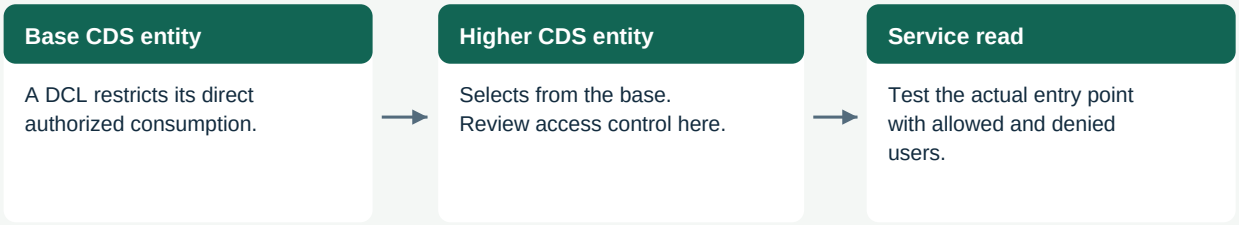
SAP RAP: implementing sorting of virtual elements

Senior CDS, query contracts and security

Diagnose data leaks, identity defects and query errors through the actual consumer boundary. Preserve business semantics, authorization and supported contracts when redesigning the model.

The exposed entity needs its own access decision

A base-layer DCL is not automatically inherited as an access filter



Mechanism	Purpose
DCL	Read access for the consumed CDS entity
RAP authorization	Permission to execute operations
Feature control	Availability in the current context

@AccessControl.authorizationCheck: #CHECK is not an authorization rule by itself.

Review security where consumers enter the model, rather than assuming that lower layers protect every upper view.

See the visual atlas for the recall prompt, limits and related questions.

MODEL-01 A new consumption entity exposes orders outside the user's sales organization. The base CDS has DCL. What do you investigate?

Trace the entity that the service actually reads and its access rules. DCL on the base view does not automatically protect every higher view or navigation path. Define the required restrictions, using supported inheritance and mappings where appropriate. Test both allowed and denied data. A UI filter or #CHECK annotation alone cannot create the missing security rule.

Reasoning and example

Create similar orders in sales organizations A and B, then use an identity allowed only for A. Compare direct base reads, the exposed projection, and association navigation. This reveals where the intended restriction stops being applied, rather than assuming the layer with DCL secures all later layers.

Include counts, value helps, and alternative service paths. A hidden organization's key or count can still disclose information even when its rows are absent from the main list. Preserve legitimate access and any deliberate documented exceptions while fixing the rule. Keep RAP update and action permissions separate: proving which rows a user can read does not prove which operations that user may perform.

Avoid this mistake Do not add #CHECK and assume it creates a missing rule.

Interviewer probes Which entry point actually evaluated DCL? / How would a field rename affect inherited restrictions?

Strong answer criteria Traces the exposed read path; Defines restrictions at the correct boundary; Tests negative and positive identities

Verification evidence Capture the real service responses for two organizational scopes, including navigation and counts.

Read and verify this answer

SAP Learning: authority checks and DCL

MODEL-02 An approver may update organization A. Can an update move an order from A to B using only existing-instance authorization?

Permission to update an order in A does not automatically allow moving it into B. Check both the current object and the proposed target values at the appropriate operation boundary. Supported prechecks can help reject disallowed input before it enters the buffer. Cover creates, updates, and derived changes. Keep the complete authorization and business validation design, not just the existing-instance check.

Reasoning and example

An approver can edit an order in organization A, so the check against its current state succeeds. The update payload changes Organization to B, which the user is not allowed to choose. Without a target-value check, permission for A becomes an unintended route into B.

Test A-to-A changes, denied A-to-B changes, initial values, and several inputs in one request. If a determination derives the organization, that result must also remain within the intended rules. Internal local-mode updates should sit behind a properly checked public operation, not provide an indirect way to submit forbidden values. Creation needs its own checks because there is no existing instance whose authorization settles the question.

Avoid this mistake Do not equate authorization for the current object with authorization for any future object state.

Interviewer probes What is checked for creation? / What happens if a determination changes the organization?

Strong answer criteria Separates current and proposed state; Checks the public mutation boundary; Considers derived and bulk changes

Verification evidence Demonstrate a rejected cross-scope update through the actual service with no unauthorized persisted change.

Read and verify this answer

SAP RAP authorization control

MODEL-03 A custom query returns only authorized rows but an unrestricted total count. Is it secure?

No. The count must describe the same authorized, filtered result as the returned rows, before paging. A custom provider cannot assume remote or manually assembled data is protected automatically by database access control. Apply the same restrictions to data, count, and supported aggregates. If an operation cannot be answered securely and correctly, reject it clearly instead of returning misleading information.

Reasoning and example

A user can see five permitted orders, but the service returns a total count of 900 including hidden orders. The list has filtered the rows, yet the count still reveals information. Use one consistent definition of the permitted result before calculating its count or selecting a page. Do not simply fetch everything remotely and hide only the displayed rows.

Test a user with no access, a count-only request, filters, ordering, and any navigation or aggregation the service supports. A count-only request must still apply permissions even when no rows are requested. If the remote source cannot support the required secure selection, choose a suitable safe implementation or explicitly reject the unsupported request.

Avoid this mistake Filtering only the displayed page leaves count, aggregates and existence checks exposed.

Interviewer probes How do you handle count-only requests? / What if the remote API cannot apply your authorization predicate?

Strong answer criteria Uses one authorized result definition; Protects count and aggregates; Rejects unsupported semantics explicitly

Verification evidence Use restricted identities and compare authorized data, count and aggregate responses over the same fixture.

Read and verify this answer

SAP: Requesting and Setting Data or Count

MODEL-04 A query-provider filter cannot be converted to simple ranges. A developer catches the exception and returns everything. What is the correction?

Do not turn a filter-conversion failure into an unrestricted read. Preserve the original AND/OR grouping using a supported expression representation or another safe translation. If the provider cannot support the filter, reject it clearly. Keep mandatory authorization restrictions separate from user filters so neither disappears on an error. Handle dynamic field names and values through supported safe APIs.

Reasoning and example

The filter means either region A with amount above 100, or region B with Urgent true. Converting that into separate region and amount ranges can change which records match. A request that should return selected urgent B records may instead return too much or hide legitimate matches.

Create test rows that distinguish the original grouped expression from the simplified one. Include an unsupported function and verify that its failure returns the documented error rather than all records. Authorization restrictions must remain in force regardless of filter parsing. If the backend cannot express the requested condition faithfully, an explicit unsupported-request response is safer and more useful than a successful-looking incorrect dataset.

Avoid this mistake A swallowed filter exception must not become an unrestricted selection.

Interviewer probes Can the backend express the same predicate? / How do mandatory authorization filters survive translation failure?

Strong answer criteria Preserves AND/OR grouping; Keeps mandatory access rules in force; Clearly rejects unsupported requests

Verification evidence Compare expected row sets for nested filters and prove failure never broadens authorized access.

Read and verify this answer

SAP: Implementing Filtering in an Unmanaged Query

MODEL-05 A paged custom entity repeats some rows after an insertion. How would you make the contract predictable?

Use a deterministic sort order with a unique tie-breaker. Then decide what consistency the caller needs across pages. Stable ordering does not freeze the data: an insertion can still shift offset-based pages. Use a supported continuation or snapshot approach where required, or document the weaker behavior of a live list. A complete export needs stronger planning than ordinary UI browsing.

Reasoning and example

Twenty orders share the same creation date. Sorting only by that date leaves their relative order unclear, so add an appropriate unique key as a tie-breaker. Now insert an order between page requests: even with that stable order, an offset can move and cause a repeated or missed row. These are two different problems.

Clarify whether the screen should show a changing current list or the caller needs a reproducible extraction. Test equal sort values, descending order, insertions, deletions, empty pages, and changing counts. Choose a supported mechanism that meets the requirement and state its limits. Adding ORDER BY fixes ambiguous ordering, but cannot alone promise a frozen dataset across separate requests.

Avoid this mistake Adding ORDER BY alone does not guarantee a frozen dataset across requests.

Interviewer probes What is the tie-breaker? / Does the use case require a snapshot or just stable ordering?

Strong answer criteria Separates ordering from consistency; Chooses a contract for concurrent changes; Tests page boundaries with duplicate sort values

Verification evidence Run successive requests while inserting and deleting records and verify the documented paging behavior.

Read and verify this answer

SAP: Interface IF_RAP_QUERY_PAGING

MODEL-06 Adding a language-dependent text association doubles order totals. Would DISTINCT be a good fix?

Find out what one result row should represent and why the join produces more than one matching text. Check language, validity dates, and any other condition needed for uniqueness. Fix that relationship or aggregate at the correct level. DISTINCT may hide rows without fixing totals. Declaring to-one cardinality does not make duplicate matches disappear.

Reasoning and example

An order for 100 EUR joins to two language texts. The resulting query now contains two copies of the order amount, so a later sum becomes 200 EUR. The problem is the relationship's actual multiplicity, not simply the appearance of repeated labels. A false to-one declaration can also mislead query optimization.

Use fixtures with two languages, missing texts, and overlapping validity periods. Decide which text should apply and express the supported condition, or separate text retrieval from aggregation where appropriate. Check both detailed rows and the consuming total. DISTINCT can discard legitimate detail or leave the incorrect aggregate in place, so demonstrate the intended row identity and amount after the real fix.

Avoid this mistake Do not fix an incorrect join by simply declaring a to-one cardinality.

Interviewer probes What should one result row represent? / Which condition ensures that only one related row matches?

Strong answer criteria Defines what one result row means; Checks how many related rows actually match; Verifies totals in the consuming query

Verification evidence Show expected rows and totals for multiple texts, missing texts and overlapping validity intervals.

Read and verify this answer

SAP CDS modeling course

MODEL-07 A CDS report sums 100 EUR and 100 USD into 200. What should the model and service do?

Do not add amounts in different currencies as though they were the same unit. Group totals by currency or perform an agreed conversion first. Define the target currency, rate type, date, and rounding rule with the business. Preserve the currency and unit links in the model. An annotation describes those links; it does not automatically perform conversion.

Reasoning and example

A report contains 100 EUR and 100 USD. Showing 200 without a currency policy gives a misleading total. The business may want separate currency totals, or a total in EUR using a specified exchange-rate date. Those are different valid requirements and should be chosen deliberately.

Test mixed currencies, a missing rate, and values that require rounding. Check whether rounding happens per item or after aggregation, because totals can differ. Keep original values where reconciliation needs them and inspect how the real analytical or transactional consumer interprets the fields. The same principle applies to quantities with different units. A correctly annotated amount still needs correctly designed calculation behavior.

Avoid this mistake Do not confuse semantic annotations with conversion logic.

Interviewer probes Which date determines the rate? / At which stage is rounding applied?

Strong answer criteria Keeps amounts and quantities linked to their currency or unit; Explains the agreed conversion rules; Reconciles rounded results with original values

Verification evidence Compare expected totals for same-currency, mixed-currency and missing-rate fixtures.

Read and verify this answer

SAP CDS modeling course

MODEL-08 A transactional projection needs a calculated field that its provider contract does not permit. How do you redesign it?

Check what the projection's provider contract allows on the target release. Put the calculation in a suitable underlying model, a separate read model, or a supported virtual element if its limits fit. Preserve stable identity, permissions, and update behavior. Do not force arbitrary joins into a transactional projection just for display. Test filtering and sorting as well as reading the field.

Reasoning and example

The UI asks for a calculated risk band and wants to filter orders by it. An ABAP-calculated display value might look correct for the current page but not support the required server-side filter. Decide where that value belongs: stored business state, a database expression, or a read-only enrichment with clearly limited query support.

Identify who recalculates it when source data changes. Then test read, filter, order, and update through the intended binding on the actual release. The transactional entity must still identify the right object and preserve behavior mapping and authorization. A field appearing in a preview proves display in that path, not a complete supported query-and-update design for every consumer.

Avoid this mistake A field that displays in preview is not proof of a supported filterable transactional contract.

Interviewer probes Does the field participate in filtering or ordering? / Who owns recalculation when source data changes?

Strong answer criteria Checks what the projection allows; Preserves correct object identity; Supports the filtering and sorting the consumer needs

Verification evidence Exercise read, filter, order and update against the intended binding on the target release.

Read and verify this answer

SAP CDS modeling course

MODEL-09 The business wants to sort a million-row list by a virtual element computed through a remote call. What would you propose?

Do not calculate a million remote scores row by row just to sort a list. A virtual element does not automatically support efficient database sorting or filtering. Consider a suitable stored read model with refresh rules, a supported database calculation, or limited enrichment without global sorting. Agree how fresh results must be and how failures appear. Measure with realistic data.

Reasoning and example

A remote provider supplies each customer's risk score. Sorting all orders by that score may require far more work than displaying scores for one small page. Ten demo rows hide the cost of remote calls, data transfer, and rebuilding the result before paging. Virtual-element logic does not automatically move to HANA.

Ask whether scores can be stored and refreshed under an agreed policy, or whether users can work with a bounded result instead. Define what happens when the provider is unavailable and how stale values are marked or reconciled. Measure latency, calls, and transferred rows for both data and count requests. Choose a design that makes the sort correct at the required scale, not merely visually convincing.

Avoid this mistake Do not claim that adding a virtual element automatically pushes its logic to HANA.

Interviewer probes What freshness is acceptable? / How many remote calls occur for one page and its count?

Strong answer criteria Identifies unsupported query operations; Controls remote-call volume; Makes freshness and failure explicit

Verification evidence Measure request count, rows transferred and latency with a realistic dataset and a failing remote dependency.

Read and verify this answer

SAP RAP: implementing sorting of virtual elements

MODEL-10 Two rows in a service share the declared CDS key after a join. Why is this more than a display issue?

A service key must identify exactly one exposed instance and stay stable. Declaring CDS fields as key does not force a joined query to be unique. Duplicate identities can break navigation, caching, selection, and updates. Fix the query or key design and assess existing consumers. If a public key's meaning must change, plan an explicit API transition.

Reasoning and example

An order-item entity exposes only OrderId as its key. Two items in the same order therefore look like the same resource to the client. The list may show rows, but opening or updating one can target the wrong item. Client-side deduplication hides the symptom while losing legitimate data.

Test multiple items with read-by-key, navigation, stable URLs, and update targeting. Repeat reads in different orders to expose cache or selection confusion. Determine whether one row should represent an item or an aggregate, then choose the matching identity and query. If callers already rely on the incorrect public key, coordinate a migration instead of silently changing what an existing URL means.

Avoid this mistake Do not hide duplicate-key rows with client-side deduplication.

Interviewer probes Can the public key change compatibly? / Is the entity a business object instance or an aggregate?

Strong answer criteria Proves one exposed key identifies one result row; Explains the effect on callers; Plans how callers move to a corrected identity

Verification evidence Validate uniqueness, stable canonical URLs and correct read/update targeting for multiple child records.

Read and verify this answer

SAP CDS modeling course

MODEL-11 A value help reveals customer names that the order list hides. Where should the fix live?

Protect the value-help entity and its text and navigation paths as read entry points in their own right. Linking it from a protected order list does not automatically restrict it. Test direct service access with the actual user context. Also validate selected values when writing an order. Clients can submit a customer key without ever opening the picker.

Reasoning and example

A restricted user cannot see another organization's orders but can search its customer names in the value help. The auxiliary search is leaking information even though the main list looks correct. Check its access rules, text associations, search results, and counts using the same restricted identity. Hiding the picker does not protect direct requests.

Then send an update containing a forbidden customer key directly to the service. The write path must reject it under the required business and permission rules even if the search now hides it correctly. Read restrictions stop disclosure; write checks stop an invalid or unauthorized reference being saved. Both are necessary because the UI is only one possible consumer.

Avoid this mistake Hiding the value-help control is not a backend security fix.

Interviewer probes Can the entity be consumed directly? / What checks a value submitted outside Fiori?

Strong answer criteria Protects auxiliary data exposure; Checks direct service access; Validates proposed business references

Verification evidence Execute denied-user searches and direct payload changes using allowed and forbidden customer keys.

Read and verify this answer

SAP Learning: authority checks and DCL

MODEL-12 A batch job asks to use privileged EML because normal authorization rejects it. How do you evaluate the request?

First check the job's legitimate scope and whether normal permissions can represent it. Privileged EML requires a provider-supported privilege design; it is not a generic fix for rejected calls. Verify exactly which checks it bypasses and which trusted callers may reach it. Keep business checks and auditability. PRIVILEGED, IN LOCAL MODE, and a technical user are different mechanisms.

Reasoning and example

A repair job is authorized to reconcile one organization. Giving it a broadly callable privileged helper could accidentally let normal consumers perform unrestricted posting. Review the provider's authorization contexts and enabled privileged entry points, then expose only the narrow operation the job actually needs.

Test the approved repair and a denied attempt from an ordinary caller. Record who requested the repair and which business records it affected. If ordinary scoped authorization can meet the requirement, prefer that simpler model. If privilege is justified, document the precise bypass behavior on this provider and release. The fact that a background identity is technical does not by itself explain or authorize every bypass.

Avoid this mistake Do not assume PRIVILEGED, IN LOCAL MODE and a technical user are equivalent.

Interviewer probes Which authorization context is disabled? / Why cannot ordinary scoped authorization satisfy the requirement?

Strong answer criteria Requires the provider to support the requested privilege; Limits who can call the trusted operation; Keeps business checks and an audit trail

Verification evidence Review privilege configuration and demonstrate allowed repair plus rejected unauthorized invocation.

Read and verify this answer

SAP EML local and privileged modes

MODEL-13 A developer uses IN LOCAL MODE in a helper to avoid feature checks. What makes the helper safe or unsafe?

Check that the helper belongs to the object's own behavior implementation and trace every public operation that reaches it. Local mode can skip defined checks for internal maintenance, depending on the operation. It does not approve arbitrary user input or remove RAP transaction rules. Validate public inputs first, preserve business rules internally, and assess calls to other objects separately.

Reasoning and example

A determination calculates a readonly total from approved input and updates it through internal EML. That can be a valid use of local mode. Now a new public action accepts a Total parameter and passes it directly to the same helper. The helper has become an indirect way for callers to overwrite a field they could not normally change.

Test those action inputs and examine which controls the specific local-mode operation bypasses. Keep caller-controlled values separate from trusted derived values, and verify the public operation's permission and input checks. A helper's name does not make it safe. Its safety depends on how it is reached, what values it accepts, and which changes it can make.

Avoid this mistake Do not fix missing authorization by routing all work through local mode.

Interviewer probes Which controls does this specific operation bypass? / Can a caller influence the supposedly derived values?

Strong answer criteria Traces user input to the internal update; Limits internal changes to their intended purpose; Distinguishes own-object and other-object calls

Verification evidence Use adversarial action inputs and verify that only approved derivations reach readonly or protected fields.

Read and verify this answer

SAP EML local and privileged modes

MODEL-14 A CDS unit test passes, but the same entity leaks data for a restricted user. What may the test be missing?

Check whether the test actually evaluates access control. CDS doubles used to test joins may isolate or disable DCL, so a passing query test is not automatically a security test. Use supported access-control test setup with meaningful permission data where available. Add real service tests with restricted identities. Keep query correctness and deployed authorization evidence separate.

Reasoning and example

A unit test proves that a join returns three expected rows from fixture data. That is useful, but it says nothing about whether a user should see all three. Add cases for no access and two different organizational scopes, and inspect the test configuration to confirm that DCL is enabled and exercised.

Finally call the real service with restricted users. That checks the consumed entity, binding, and identity propagation that the isolated test may not cover. Capture positive cases too, so a fix does not simply block everyone. A green double-based test and a passing restricted-user service test prove different things; neither result should be inferred from the other.

Avoid this mistake A green database-double test does not prove deployed authorization.

Interviewer probes Is DCL enabled in the test environment? / Which real identity is used by the service?

Strong answer criteria Inspects double behavior; Adds denied and scoped authorization cases; Separates unit and deployed evidence

Verification evidence Show test configuration plus restricted-user service results; do not infer one from the other.

Read and verify this answer

SAP CDS access-control tests

MODEL-15 An integration user can access all tenants or clients through a custom read provider. What boundaries must be reviewed?

Review client handling, tenant isolation, business permissions, and remote credentials separately. Local CDS restrictions do not automatically protect data assembled through custom or remote calls. Do not trust an unrestricted tenant selector from request input. Use the environment's supported tenant-aware configuration and check the identity seen downstream. The correct isolation model depends on the actual runtime.

Reasoning and example

The caller supplies a tenant or company parameter, and a custom provider forwards it through a remote connection with broad privileges. The remote response may contain another customer's records even if the local CDS model appears restricted. Inspect who controls that selector and how it is checked before building the remote request.

Test two scopes that share the same document numbers. Verify request construction, remote identity, and returned data so matching business IDs do not hide a cross-scope leak. Review permitted communication configuration without exposing credentials. A client field, destination name, or successful login covers only part of the chain. The whole path must preserve the intended tenant and business access boundaries.

Avoid this mistake Do not treat a client field, a destination name or authentication alone as complete tenant isolation.

Interviewer probes Who controls the remote tenant selector? / Which identity does the remote system evaluate?

Strong answer criteria Separates technical and business scope; Constrains untrusted selectors; Checks end-to-end identity propagation

Verification evidence Run cross-scope negative tests and review the permitted communication configuration without exposing secrets.

Read and verify this answer

SAP RAP authorization control

MODEL-16 A projection hides an operation, but another consumer can still call the base BO. Is hiding sufficient?

Hiding an operation in one projection limits that consumer's interface, not every possible caller. Check who can use the base object or another released interface. Enforce the required permissions in the appropriate behavior implementation and align release visibility and package access. Feature control can explain availability, but alternative entry points still need authorization. Review every supported change path.

Reasoning and example

The Fiori projection omits Delete, while a background program uses a broader local interface that includes it. That may be intentional for a controlled cleanup job, but the job's permission must be explicit. The missing UI button does not prove an ordinary user cannot delete through another supported route.

Build a simple list of consumers, operations, and allowed identities. Test unauthorized delete attempts through each exposed mutation path, including local and remote consumers where applicable. Check where the decisive permission decision occurs and whether base access is properly limited. Fixing only the UI service shape leaves a gap whenever a different caller can reach the underlying operation.

Avoid this mistake Do not use the UI service shape as the only security policy.

Interviewer probes Which consumers are permitted to use the base contract? / Where does the decisive permission check execute?

Strong answer criteria Inventories alternate entry points; Separates surface restriction from permission; Tests local and remote consumers

Verification evidence Provide a consumer/operation/identity matrix and evidence of denied mutation through each exposed path.

Read and verify this answer

SAP BO interfaces

MODEL-17 A released entity gains a new field that is expensive and sensitive. Is an additive change automatically safe?

An added field can still change security, cost, or behavior for existing consumers. Check default reads, broad selections, serialization, metadata-driven clients, and permissions. A sensitive or expensive field may belong in a more restricted interface. Verify the release contract's evolution rules and test unchanged callers. Old code still compiling is not enough to prove the addition is safe.

Reasoning and example

A new personal-data field appears automatically in a broad select or export. No caller changed, yet users now receive information that needs a different access decision. Another added field calculates remotely for every returned row, making an existing list much slower. Both are compatibility concerns beyond the ABAP signature.

Compare old and new payloads, permission results, and latency for representative requests. Test explicit field selection, default selection, and UIs that build screens from metadata. Decide whether the field should be exposed separately, selected only when needed, or protected differently under supported mechanisms. Keep existing consumers' expectations in view before describing an additive change as harmless.

Avoid this mistake Do not assess compatibility using only whether old source code still compiles.

Interviewer probes Does an existing broad select expose it? / How does it affect authorization and request cost?

Strong answer criteria Checks changes in meaning, access, and runtime cost; Controls what existing reads expose by default; Tests existing callers without changing them

Verification evidence Compare old/new payloads, access results and latency for representative existing requests.

Read and verify this answer

SAP BO interfaces

MODEL-18 An analytical requirement is added to a transactional BO. Would you put every aggregate into its projection?

Choose an analytical read model that represents the required totals and groups. Do not distort an editable object's identity to fit every aggregate. Reuse reliable data meanings where possible, but define aggregation, currency, permissions, and freshness explicitly. Keep changes on the transactional object's supported operations. A separate read model needs ownership and consistency rules, not duplicated business logic nobody maintains.

Reasoning and example

An approval app edits individual items, while management wants monthly spend by organization. One monthly total is not the same kind of entity as one editable item. Treating both as a single transactional projection can make keys, permissions, and behavior confusing. Build a read model that clearly represents the monthly grouping instead.

Reconcile its totals with authorized transactional fixtures, including mixed currencies and late changes. Decide whether a live query meets response-time needs or whether a refreshed read model needs a defined delay and reconciliation process. Keep shared rules owned and consistent across the models. The fact that an aggregate comes from writable business data does not make the aggregate itself a sensible writable object.

Avoid this mistake Do not make an aggregate writable merely because its source is a transactional BO.

Interviewer probes What should one aggregate row represent? / How do both models preserve permissions and currency meaning?

Strong answer criteria Distinguishes editable objects from grouped results; Reuses data meanings deliberately; Defines freshness and reconciliation

Verification evidence Reconcile grouped totals to authorized transactional fixtures and validate the intended analytical consumer.

Read and verify this answer

SAP analytical projection modeling

Official references

SAP RAP authorization control

SAP Learning: authority checks and DCL

SAP EML local and privileged modes

SAP CDS access-control tests

SAP CDS modeling course

SAP BO interfaces

SAP custom-entity application example

SAP analytical projection modeling

SAP RAP: implementing sorting of virtual elements

RAP service design and advanced behavior

Use this segment for deeper follow-up questions. Connect each feature to a concrete requirement and explain its limits. Release-specific BDL syntax and UI behavior must be checked against the employer's ABAP and SAPUI5 versions.



Alice and Ben read version 7. Alice updates successfully, producing version 8. Ben must not overwrite that newer state using the stale version 7 precondition.

See the visual atlas for the recall prompt, limits and related questions.

ADV-01 How do pessimistic locks and ETags complement each other?

A lock stops conflicting processing while it is held. An ETag lets a write request check whether the data has changed since the client read it. They solve different problems and are often needed together. RAP declares the relevant lock and version handling in behavior. Draft scenarios also need total-ETag handling for relevant changes to active data.

Reasoning and example

Ana and Ben read order 4704 at quantity 2 with ETag V1. Ana saves quantity 3, producing ETag V2. Ben tries to save quantity 4 with his older V1. The If-Match check rejects that stale update, leaving quantity 3 intact. Ben must refresh and decide whether quantity 4 is still the change he wants. The ETag did not reserve the order while he was reading it.

A pessimistic lock addresses conflicting processing during the period it is held. Draft editing and changes to active data require the matching draft lock and total-ETag design. Test stale updates and draft resume behavior separately. A lock is temporary coordination; an ETag is a version check, and neither is an audit log.

Avoid this mistake Locking and logging are different concepts; an ETag is not a log and does not reserve the object.

Read and verify this answer

SAP draft concept

ADV-02 What is the difference between global and instance authorization in RAP, and where do DCL and prechecks fit?

Global authorization asks: may this user perform Update in general? Instance authorization asks: may this user update this particular record? Here, instance means a business-object record, not an ABAP class instance. DCL controls which rows a read returns. A precheck can reject incoming values before they enter the transactional buffer.

Reasoning and example

A user may change projects in Company Code 1000, but not 2000. The global check permits Update as an operation. The instance check permits project A in 1000 and rejects project B in 2000. A global pass is not permission to change every project.

The BDEF can declare authorization master (global, instance). get_global_authorizations receives requested_authorizations without record keys. get_instance_authorizations also receives keys and returns decisions identified by %tky. AUTHORITY-CHECK with BUKRS DUMMY ignores that field; the instance check instead supplies the existing project's CompanyCode.

Instance authorization checks existing data, not the new values submitted by the current request. For Create, or an Update that changes CompanyCode, check the proposed value in a precheck when that boundary needs protection. Otherwise, being allowed to edit a project in 1000 could be mistaken for permission to move it to 2000. CDS DCL remains responsible for read filtering.

Feature control has another job: availability under business rules. A closed project may be uneditable even for an authorized user. Test both an unauthorized user on an editable project and an authorized user on a closed project through the service, not only through disabled buttons.

Avoid this mistake Global authorization, instance authorization, DCL, prechecks and feature control solve different problems. A global pass or disabled button does not secure every record or incoming value.

Read and verify this answer

SAP: Authorization definition and prechecks

SAP Learning: implementing authority checks

ADV-03 What does authorization none on an action mean?

Where the behavior syntax supports it, authorization: none exempts that action from its RAP authorization checks. It does not make the whole service anonymous. Service authentication and checks inside called APIs can still apply. Use this only when the action's security design justifies the exemption. Do not add it simply to make an authorization error disappear.

Reasoning and example

Consider an action whose result is intentionally available to every authenticated user of that service and needs no object-specific permission. Its design may justify an explicit RAP authorization exemption. Document what data it reads or changes and why access at the service boundary is enough for that action.

If the action calls another API, that API may still check its own permissions and reject the call. Conversely, exempting a sensitive approval action can remove a necessary protection even when login is still required. Test with a user who can access the service but should not perform privileged work. Verify the exact syntax and supported action contract on the target release.

Avoid this mistake Do not propose it as a universal fix for authorization errors or expose a sensitive action by accident.

Read and verify this answer

SAP Learning on authorization

ADV-04 Instance static and factory actions?

An instance action works on an existing identified object, such as approving one request. A static action does not need an existing instance as its input target. A factory action creates instances and returns the required creation mapping. Choose the type that matches the business operation. Their input, result, and identity rules are different.

Reasoning and example

Approve needs a request to approve, so it is an instance action. A calculation preview may be a static action when it uses supplied input without needing an existing request. CopyRequest may be a factory action because its purpose is to create a new request and tell the caller which object was created.

Check the generated handler signature and return the result and mappings that the chosen action type requires. Returning $self from an ordinary instance action does not turn it into a factory action. Likewise, static describes how the RAP action is used; it is not permission to ignore the RAP transaction or treat the handler like an unrelated static utility method.

Avoid this mistake Static does not mean a static ABAP method that can ignore RAP identity and response contracts.

Read and verify this answer

SAP action definitions

ADV-05 How do action parameters and results work?

An action's behavior definition describes its input and result. A CDS abstract entity can define a structured input such as a comment and date. The generated handler receives that input and must return the declared result and response information. Result cardinality and instance binding matter. A UI can offer a parameter dialog only where the consumer supports it.

Reasoning and example

Suppose Submit accepts an approval comment. Define that parameter shape, declare it on the action, and use the generated handler signature to read it. The abstract entity describes the input fields; it does not itself contain or implement the action. The handler still carries out the business step under RAP's rules.

A different action, CalculatePreview, may return estimated totals without saving a new object. Its result type and cardinality should describe that outcome clearly. Test missing or invalid parameters and inspect what the caller actually receives. Do not assume that defining parameters guarantees the same input dialog in every client; metadata and consumer support determine the user experience.

A handler can receive several requested keys. Process all of them, collect failures and messages, and build the declared result for the successful requests. For an instance action returning [1] $self, the result identifies each successful source instance and places its returned entity data in %param. Updating a row and filling result are separate responsibilities; neither proves that the later save succeeded.

Avoid this mistake Parameters are not defined by putting an action inside an abstract entity.

Read and verify this answer

SAP action definitions

ADV-06 What is augmentation and how does it differ from additional save?

Augmentation changes or adds supported input at a RAP projection boundary before the underlying business object handles it. It helps adapt a particular consumer's input. Additional save performs extra persistence-related work in a suitable managed save scenario. These mechanisms run for different reasons. Neither is a general hook for arbitrary work before every save.

Reasoning and example

A specialized projection may accept input that needs to be translated into fields understood by the shared business object. A supported augmentation can make that translation so the underlying behavior receives the right changes. Check the projection's supported modifying operations and keep the core business rules in the appropriate underlying implementation.

Writing an audit record alongside the managed object's persisted changes is a different concern. It may require an additional-save design that follows the transaction rules. Putting it into augmentation just because that code runs during a modification confuses input handling with saving. Similarly, a virtual display field does not become persistent simply because an augmentation exists; design its supported data flow explicitly.

Avoid this mistake Augmentation is not a generic before-save hook for copying every virtual element into a table.

Read and verify this answer

SAP operation augmentation

ADV-07 What are side effects in a RAP UI?

Side-effect metadata tells a supported UI what to read again after a change or action. For example, changing quantity may require refreshing the total and available actions. The backend must still calculate the correct values. Side effects only coordinate refreshes. Support depends on the backend release, service metadata, and SAPUI5 consumer.

Reasoning and example

The user changes an item's quantity from 2 to 3 at a unit price of 15 EUR. A backend determination changes the draft total from 30 EUR to 45 EUR, but the screen may still show 30 EUR until it reads that value again. Suitable side-effect metadata tells the UI that the quantity change affects the total and, where supported, related messages or permissions.

Check the actual network requests and displayed result after both field edits and actions, including draft processing. If the backend total is wrong, adding a refresh cannot fix the calculation. If the backend is correct but the UI stays stale, inspect the metadata and client support. A non-UI API caller does not automatically perform these extra reads just because the metadata exists.

Avoid this mistake A side-effect declaration does not implement the calculation or force all non-UI consumers to issue extra reads.

Read and verify this answer

SAP RAP side effects

ADV-08 State messages versus transition messages?

A state message describes a problem in an object's current data, such as a missing required date. In supported draft scenarios it can remain until the state is checked again. A transition message describes the current operation or request. Attach messages to the correct instance, field, and state area. Clear old state messages when they no longer apply.

Reasoning and example

A user saves a draft without a delivery date. A state message can keep the date marked as invalid while the user continues editing. When the date is corrected and the relevant check runs again, handle the state area properly so the old error disappears. Otherwise the screen can show an error for data that is now valid.

A one-time message such as Submission completed belongs to the operation, not to a permanent invalid state. Test repeat edits, correction of errors, and a fresh request to avoid stale messages. Also remember that a message and rejection are different jobs: putting an error in REPORTED does not by itself guarantee that RAP rejects the save.

Avoid this mistake Every error should not be stored as a persistent state message, and REPORTED alone does not block saving.

Read and verify this answer

SAP RAP state messages

ADV-09 Service definition versus binding and UI versus Web API?

A service definition chooses which entities and names are exposed. A service binding chooses the protocol and service type, such as OData V4 UI or Web API. UI services support relevant UI behavior; Web APIs serve programmatic consumers. Creating these objects is only part of delivery. Authorization, publication, communication setup, and deployment must also be handled.

Reasoning and example

Start with the caller: is it a Fiori elements app or an external integration? Expose the needed entities, choose a suitable binding, then inspect the generated metadata and exercise the actual operations. A service that reads correctly in an ADT preview has not yet proved that the production consumer can authenticate or call it.

Follow the target environment's deployment and communication setup, and test permissions with the intended user or communication identity. Confirm that transported objects and the real endpoint behave as expected. A CDS modeling annotation such as VDM.viewType: #COMPOSITE does not create or activate a binding. Model classification and service delivery are separate responsibilities.

Avoid this mistake VDM.viewType: #COMPOSITE neither creates nor activates a service binding.

Read and verify this answer

SAP service binding documentation

ADV-10 What must a custom query provider implement?

A custom query provider must answer what the caller actually asked for. That includes supported filters, sorting, paging, selected fields, and count, while enforcing permissions. A count request and a data request can arrive independently. Count the authorized, filtered result before paging. Treat zero rows requested differently from PAGE_SIZE_UNLIMITED, and use a stable sort order.

Reasoning and example

Lea asks for open orders in organization A, sorted by CreatedAt, skipping one and taking two. Of 12 source orders, four match both her permission and filter: 101, 104, 108 and 110. Use OrderID to break equal CreatedAt values. The page contains 104 and 108, while the total count remains 4. Keep that ordering stable so rows do not move unpredictably between pages. If a remote source is involved, map errors and account for its paging and consistency limits.

Test a count-only request separately; it must not depend on entering a branch that fetches data. Also test an empty page and a page size of zero. Zero means no rows, not an unrestricted read. Compare it explicitly with IF_RAP_QUERY_PAGING=>PAGE_SIZE_UNLIMITED. Handle unsupported options deliberately instead of silently returning a plausible but incorrect list.

Avoid this mistake Returning any internal table is not a correct implementation of IF_RAP_QUERY_PROVIDER.

Read and verify this answer

SAP: Implementing an Unmanaged Query

SAP: Requesting and Setting Data or Count

SAP: Interface IF_RAP_QUERY_PAGING

ADV-11 How do RAP business events differ from ABAP Objects events?

ABAP Objects events use handlers within ABAP execution. RAP business events describe business occurrences using RAP's event rules. Local and remote consumption have their own supported lifecycle. Sending events outside the system also needs configured infrastructure and a publication design tied to the transaction. Consumers should be prepared for retries and duplicate delivery rather than assuming exactly one arrival.

Reasoning and example

A request is submitted successfully, and another system needs to start follow-up work. Publish the business event through the supported transactional mechanism so the external process does not act on a change that later rolls back. A local callback during processing is not proof that the business change has been durably saved.

At the consumer, use the event's business identity and a suitable duplicate-handling design. If the same submission event is delivered again after a retry, it should not create a second approval task. Verify the supported ordering and delivery behavior instead of assuming it. Locally consumed RAP events also should not all be described as ordinary synchronous SET HANDLER callbacks.

Avoid this mistake Do not collapse every locally consumed RAP event into synchronous SET HANDLER semantics or promise universal ordering.

Read and verify this answer

SAP: Develop Business Events

ADV-12 How do you extend an existing RAP BO safely?

First check what the business object officially allows you to extend on your system's release. Then use those supported extension points to add the field or behavior. Follow the change through storage, CDS, projection, service, and any UI metadata. Check permissions and saving too. Being able to see or edit an object's source does not mean SAP supports extending it.

Reasoning and example

Suppose the business wants an Approval Reference field on an existing request. Check the object's extension documentation first: can you add a stored field, expose it through the service, and add any needed rule? If one part is unsupported, adding only a CDS field will not deliver a working feature.

After implementing the supported changes, enter a value, save the request, and reopen it. Confirm the value is stored and returned by the service. Test invalid values, restricted users, and the normal process without the new field. Existing consumers must keep working. Extension support depends on the particular business object and release, so do not copy assumptions from a different RAP example.

Avoid this mistake Being able to create a CDS extension does not prove every BO behavior or API contract is extensible.

Read and verify this answer

SAP: RAP Extensibility Enablement

ADV-13 How do static and dynamic feature controls differ?

Static restrictions are declared in the behavior definition, such as a readonly field. Dynamic feature controls decide availability from current context, such as making approved requests uneditable. They describe business availability, not who has permission. Check the controls supported for the operation and release. For one operation, choose global or instance feature control; do not declare both.

Reasoning and example

A generated UUID should not be freely edited by the caller, so a static readonly declaration fits. The request description may be editable while the request is open but readonly after approval, which needs a state-dependent rule. Use the applicable generated feature-control handler to express the supported restriction.

Now test an API call as well as the screen. Permission checks must still prevent an unauthorized user from changing an otherwise editable request. Also validate the business data: mandatory field metadata is not a complete substitute for backend consistency rules. Keeping static restrictions, dynamic availability, authorization, and validation separate makes it easier to explain why a request was rejected.

Compare static field controls: readonly prevents consumer writes; readonly:update permits supplying the field on create but prevents later consumer updates; mandatory declares a required input for supporting consumers. Do not rely on a UI required marker alone to reject invalid persistence: implement the necessary backend validation. Dynamic features may make an operation unavailable for the current business state; authorization independently decides whether the caller is allowed.

Avoid this mistake A dynamic UI state does not prove security, and mandatory metadata does not replace all backend business validations.

Read and verify this answer

SAP feature control

ADV-14 What is a normal RAP function, and when should you use it instead of an action?

A RAP function returns data or a calculation without changing the business object. Use it to preview an order total, for example. It runs in a FOR READ handler and returns the value in result. It must not update the buffer, save data or trigger a business side effect. Use an action when the operation should change business state.

Reasoning and example

Suppose an order has Quantity 3 and UnitPrice 10. A PreviewTotal function reads the current values through EML and returns Amount 30. The caller sees the calculation, but the order is not changed or saved. If the user changes Quantity to 4 in the same transaction, a new preview can return 40. Read the transactional state rather than assuming the database already contains that edit.

A normal instance function receives the selected instance keys. Its result links each input %tky to the calculated %param. Return a result for each successful input and a FAILED response for a rejected one. A static function is the variant that does not require an existing instance; it uses request correlation such as %cid. Static does not mean background processing.

Declare the function in the BDEF and expose it with use function in the projection when the service needs it. Read-only does not mean public: configure and test the applicable authorization and feature controls. A function must not send an approval email, call a remote posting operation or create new state messages. Check result types, cardinalities, binding and client support on the target release. This RAP function is not an ABAP function module.

Avoid this mistake Do not implement a preview by changing the BO and then trying to undo it. A function returns a result; a determination changes derived BO data, and an action can perform a business change.

Read and verify this answer

SAP: RAP function definition

SAP: FOR READ handler method

ADV-15 How does a default function suggest values for RAP action parameters?

A default values function proposes initial input for an action dialog. It does not execute the action. For example, opening ApplyDiscount can suggest 10 percent for a large order and 5 percent for a small one. The user can edit the suggestion. When the action is finally called, validate the submitted value again.

Reasoning and example

Attach a default function to the action declaration, for example GetDefaultsForApplyDiscount. Its name follows the GetDefaultsFor convention. RAP derives its input and result types from the operation; generate the handler in ADT rather than inventing a separate parameter contract. For an instance action, preserve each input %tky and fill the suggested parameter fields in %param. If a projection exposes the action, expose the default function there too.

Example: order A totals 1200 EUR and order B totals 200 EUR. A simple teaching rule suggests DiscountPercent 10 for A and 5 for B. The default function reads both orders and returns those suggestions; it does not change either order. A user then enters 80 instead of 10. The action must check its allowed discount range and permissions and reject 80 if the rule forbids it.

The supporting UI requests the default values before execution. A direct API caller can supply different input or skip the default request, so defaults are not an enforcement mechanism. A default can also become outdated before the action runs. Test multiple selected instances, missing keys, a cancelled dialog and invalid submitted values. Check backend release, OData binding and Fiori elements support rather than assuming every client opens the same dialog.

Avoid this mistake Do not confuse an input suggestion with a saved field value, determination or validation. Opening or cancelling the dialog must not perform the action.

Read and verify this answer

SAP: defaulting input parameters for operations

SAP sample: default values functions for Fiori elements

Official references

SAP Learning on authorization

SAP service binding documentation

SAP draft concept

SAP RAP documentation

SAP operation augmentation

SAP action definitions

SAP feature control

SAP RAP side effects

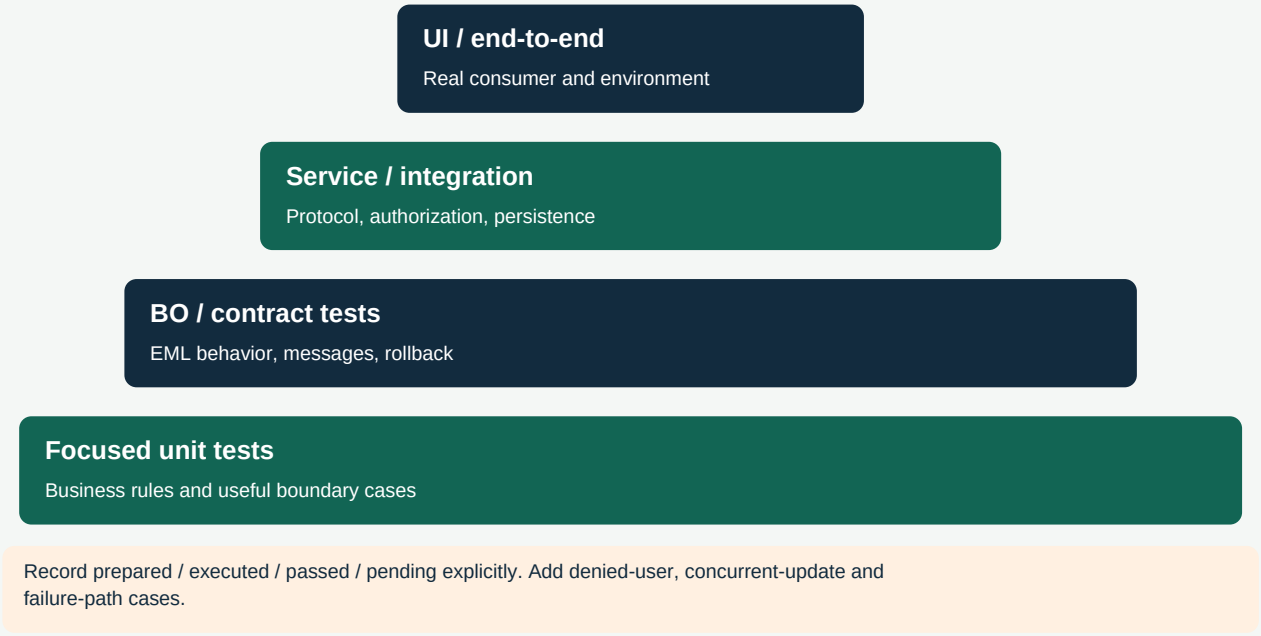
SAP RAP state messages

Senior RAP testing, performance and production

Defend the evidence behind a cloud service: observable behavior, failure recovery, real consumer security, measured performance and reliable delivery. All scenarios are hypothetical; no SAP execution is implied.

Different tests produce different evidence

A passing test double does not prove a deployed service works



Record prepared / executed / passed / pending explicitly. Add denied-user, concurrent-update and failure-path cases.

The widening base suggests many focused checks, but there is no mandatory numerical ratio.

See the visual atlas for the recall prompt, limits and related questions.

OPS-01 How would you test a Submit action without replacing the business object you are supposed to verify?

I run the real Submit action and keep the business rules I want to test. I replace only outside dependencies, such as another service or business object, with suitable test doubles. Then I call the action, often through EML, and check its results, errors and saved state. Mocking Submit itself would not test Submit.

Reasoning and example

A purchase request may be submitted only when every item has an account assignment. I create one valid request and another with an assignment missing. Both tests execute the real Submit implementation. I check that the valid request reaches the intended state and that the invalid one is rejected with the correct message.

If a different business object supplies a spending limit, I can replace that dependency with a supported double and choose its returned limit. That keeps this test focused on Submit. I also verify the saved outcome through the test framework's supported save boundary. A predefined successful response from a fake Submit would prove none of these rules.

Avoid this mistake Do not label a test of a mocked Submit response as a test of Submit.

Interviewer probes Which real business rule must this test execute? / Which outside dependency can be replaced without replacing that rule?

Strong answer criteria Names the real behavior being tested; Replaces only outside dependencies; Checks the result, errors and saved state

Verification evidence Show the dependencies and a test that fails when the real Submit check is deliberately broken.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-02 When would SQL doubles, CDS doubles and RAP BO doubles give misleading confidence?

Test doubles replace particular dependencies; they do not prove the whole application works. SQL doubles help isolate database access, CDS doubles help test CDS logic, and RAP BO doubles can replace another object's EML behavior. I choose the double for the rule being tested. Real persistence, permissions and service behavior still need suitable integration checks.

Reasoning and example

A pricing calculation reads a table. A SQL double can supply controlled prices so the test checks the calculation without depending on production data. That test does not prove an external action is still compatible. Likewise, a fake EML error response can test how a consumer handles failure, but it does not test the provider that would produce the real error.

I state what each test actually executes and replaces. A generic BO buffer double does not run arbitrary custom provider logic, and access-control behavior needs explicit checks. Focused integration tests then cover the real save, authorization and service path. A more advanced-looking double does not automatically supply stronger evidence.

Avoid this mistake A more sophisticated double does not automatically provide broader integration evidence.

Interviewer probes Which behavior is replaced by the double? / Which important behavior still needs a real integration test?

Strong answer criteria Chooses doubles for the dependency being replaced; Explains what the test does not execute; Checks the real integration separately

Verification evidence Record what each test covers, then verify one relevant path through the real provider and service.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

OPS-03 A bulk EML action processes ten requests and rejects two. What should its test assert?

I check which specific requests succeeded or failed, using their identities rather than row positions. I verify FAILED entries, message targets and the resulting data. I also test whether the chosen operation should allow partial success or require everything to succeed together. A successful interaction is not enough if saving later rejects the change.

Reasoning and example

I send ten requests with deliberately nonsequential keys. Two have different invalid values. The test checks that exactly those identities appear as failed, that each message points to the right request and that valid requests receive the intended treatment. Reordering inputs helps reveal code that accidentally matches results by row number.

Next, I test a failure during save after the interaction step appeared successful. The consumer must not report that object as permanently completed. I inspect buffered and saved state according to the supported test boundary. The expected all-or-nothing or partial-result rule must be defined first; counting ten messages cannot tell me whether ten operations were handled correctly.

Avoid this mistake Ten messages do not prove ten correct results. Match outcomes to the actual request identities.

Interviewer probes Must all requests succeed together, or is partial success allowed here? / How will the test detect a failure that happens only during save?

Strong answer criteria Matches every result to its input identity; Checks interaction results and saved data separately; Tests the intended all-or-nothing or partial-success rule

Verification evidence Show which requests should succeed or fail, where their messages point and which rows remain after each failure case.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-04 A draft test passes when run alone but fails after another test. What would you inspect?

I look for state left behind by the earlier test. That can include draft or active data, transaction buffers, locks and configured test-double responses. Each test should set up its own state and clean up through supported test mechanisms. I do not add random commits to hide the problem, because that can change what the test proves.

Reasoning and example

Two tests use request 4711. The first leaves a draft or a configured double response behind, while the second assumes a clean active request. Running the second alone works; running the pair fails. I run them in the opposite order and inspect setup and teardown to find the shared state.

Distinct fixture keys can help confirm the cause, but the final fix should still manage the test environment correctly. I check active and draft identities separately, release supported resources and reset doubles as required. A fixture for an active instance is not automatically a valid draft fixture. An order-dependent pass is weak evidence of correct draft behavior.

Avoid this mistake A test that depends on execution order is unreliable evidence for draft behavior.

Interviewer probes Are active and draft transactional keys distinct? / Which setup/teardown resource is shared?

Strong answer criteria Distinguishes active and draft identities; Resets the test resources correctly; Proves tests do not depend on their order

Verification evidence Run the tests separately, in both orders and in the suite. Keep results showing they behave consistently.

Read and verify this answer

SAP RAP test frameworks and boundaries

OPS-05 How do you prove optimistic concurrency rather than merely calling the same update twice?

I use two independent clients that read the same version before either saves. The first update should succeed; the second should be rejected when it sends the old ETag. I check that the first user's data remains intact. Draft locks and the draft total ETag need separate tests because they protect different parts of the editing flow.

Reasoning and example

Alice and Ben both read one entity instance and keep its ETag. Alice changes it and saves. Ben then updates that same instance with his old token. The expected result is a stale-version rejection, not an overwrite. If Ben rereads first, he gets the new token and the test no longer exercises a stale writer.

For draft, I separately test editing while an exclusive lock is valid and resuming after it expires. A child-only change to active data should be considered by the configured draft total ETag check. That is different from the entity-level If-Match test. Two updates inside one shared transaction do not reproduce two users with independent versions.

Avoid this mistake Two updates in one shared transaction are not a realistic two-user concurrency test.

Interviewer probes Which token protects this entity update? / What separate case should the draft total ETag test cover?

Strong answer criteria Uses independent clients or sessions; Sends an intentionally old version token; Checks that rejected updates leave data intact

Verification evidence Record each client version, request, response and final value from the controlled two-user test.

Read and verify this answer

SAP total ETag and draft concurrency

OPS-06 Fiori still displays an old total after a successful action. How do you separate UI refresh from backend correctness?

I first read the business object again to see whether the server saved the right value. If the value is wrong there, I investigate the backend calculation. If it is right, I check what the UI refreshes after the action. The action result, side effects and deployed metadata must tell the client what changed.

Reasoning and example

An item quantity changes and the server recalculates the order total, but the original Fiori screen still shows the old amount. I compare a direct read, a new UI session and the original session's network requests. If both fresh reads show the correct total, the likely problem is that the original screen did not request the changed field.

I inspect the supported side-effect declaration and action result, then test a targeted refresh with the actual deployed metadata. I do not add a second total calculation in the browser to hide a backend problem. Another commit or a full-page reload may mask the symptom without fixing the missing refresh behavior.

Avoid this mistake A page reload workaround does not explain whether the backend or refresh contract was defective.

Interviewer probes What does a fresh service read return? / Which request should refresh the affected property?

Strong answer criteria Checks the value actually held by the backend; Inspects metadata and network requests; Keeps the business calculation on the server

Verification evidence Record the action and later reads, showing values before and after in the actual app.

Read and verify this answer

SAP RAP side effects

OPS-07 A RAP list becomes slow after enabling count and expanding children. What do you measure first?

I measure where the request spends its time: selecting rows, counting, loading children or doing extra ABAP and remote work. I use representative data and the affected user's permissions. Then I change the largest proven cost and repeat the same test. I preserve filtering, count correctness and access checks while reducing unnecessary work.

Reasoning and example

A page shows only 20 orders, but its count scans millions of allowed records and its expansion loads many child items. I compare a page-only request, a request with count and one with expansion. Traces show database time, application time, rows and any remote calls, so I can see which addition causes the delay.

I repeat the measurement with a restricted user because authorization filtering can change the database work. If expansion is the issue, the consumer may not need every child on the first screen. If count dominates, changing a small handler calculation will do little. I compare before and after using the same parameters and volume.

Avoid this mistake Do not optimize the shortest visible SELECT before finding the dominant request cost.

Interviewer probes Is count more expensive than the page? / What changes for an authorization-restricted user?

Strong answer criteria Measures the separate parts of the request; Uses realistic data and user permissions; Keeps correct results and access checks while improving speed

Verification evidence Keep comparable before-and-after traces showing total time, rows and calls.

Read and verify this answer

SAP SQL analysis in ADT

OPS-08 A custom entity calls a remote API once for every returned row. How would you remove the N+1 pattern?

I look for a supported bulk request instead of one remote call per row. I fetch only the references needed for the current result and limit request size. A cache is an option only with clear freshness and access rules. I still handle missing results, throttling and partial failures rather than hide them behind a faster response.

Reasoning and example

A page reads 50 orders and then calls a remote service once for each supplier. Together with the original read, that is 51 calls. A supported bulk endpoint may fetch the needed supplier references in one request. I remove duplicate keys and keep the batch within the service's limits instead of downloading every supplier.

The bulk response may still contain missing references or individual errors. I test those cases and rate limits, not just the reduced call count. If caching is needed, I define how long values remain valid and which user or tenant may reuse them. A shared cache must not expose one customer's data to another.

Avoid this mistake An unbounded cache can trade latency for stale or cross-user data leakage.

Interviewer probes What is the maximum request size? / How are partial remote failures represented?

Strong answer criteria Uses bulk requests with size limits; Defines cache freshness and who may reuse data; Handles missing results and rate limits

Verification evidence Measure remote calls and transferred rows, including tests for missing data and rate limits.

Read and verify this answer

SAP: Implementing an Unmanaged Query

OPS-09 A service works in development but fails after transport. What evidence should the handoff have included?

A transport moves code, but the service may also need configuration, roles, communication setup and frontend deployment. I verify the active version and dependencies in the target system. The handoff should name these requirements and their owners. A working development preview or successful import does not prove that the real production user can use the service.

Reasoning and example

The RAP behavior activates after import, but an action fails because the destination or communication arrangement is missing. In another system, the endpoint exists but the intended user's role does not grant access. These are different failures from a defect in the business calculation, so I inspect the actual target setup before changing code.

The handoff should list the code and non-code prerequisites, installation order and a small end-to-end check. I verify the imported revision, service exposure, communication settings and consuming identity. Each item needs a responsible person and an observed result. Publishing a binding locally in development does not establish a usable production endpoint.

Avoid this mistake A successful transport import does not prove that the service works for its real users.

Interviewer probes Which required items are configuration rather than source? / Who verifies the real consuming identity?

Strong answer criteria Lists code and required configuration separately; Checks the version and setup in the target; Names who owns each operating requirement

Verification evidence Record actual endpoint, role, communication and service-test results in the target readiness checklist.

Read and verify this answer

SAP RAP service binding

OPS-10 How would you evolve a released RAP service when an action needs a new mandatory input?

A new required parameter can break every existing caller that does not send it. I check whether a valid optional value or default can preserve old behavior. If not, I plan a new operation or version and a controlled migration. The change must meet the real business rule while giving consumers an explicit path to update.

Reasoning and example

An approval action now needs a legal justification. Old clients have no field for it. Supplying an empty default would keep calls running but might violate the new requirement, so technical compatibility alone is not enough. I agree a supported transition that respects when the new business rule must take effect.

That may involve a successor action, an announced retirement date and updates to both UI and integration clients. I test generated clients, metadata-driven screens and direct API consumers separately. I track actual use of the old operation before retiring it where the policy permits a transition. Successful server activation does not prove that existing requests still work.

Avoid this mistake A changed action can activate successfully while still breaking existing callers.

Interviewer probes Is a default legally/business valid? / How do you know which consumers still use the old contract?

Strong answer criteria Explains why old callers may break; Plans a clear migration; Tests existing consumers

Verification evidence Run old and new client tests, and track use of the old operation before retiring it.

Read and verify this answer

SAP RAP service binding

OPS-11 A remote approval succeeded, but the RAP caller timed out. How should an operator recover?

A timeout leaves the result unknown until I check it. I use the request's stable reference to find the remote approval before submitting again. If it exists, recovery should reuse that result. If it cannot be determined, the case needs a visible status and an owner. Local rollback cannot erase an approval already committed remotely.

Reasoning and example

The remote workflow creates approval 9001, but its response never reaches the RAP caller. A new creation request could start approval 9002 for the same submission. I query by the original reference or use the service's documented idempotency rule so a retry returns or reuses the first result. The local app records the resolved workflow link.

If the remote outcome cannot be queried, I keep a durable recovery record and use the agreed operator or compensation procedure. I also test a late duplicate callback and a callback for a request version that has since changed. A delayed response must not approve stale local data or repeat an already completed decision.

Avoid this mistake A timeout is not evidence that the other system performed no work.

Interviewer probes Which stable reference identifies the original request? / Who investigates when the remote outcome cannot be queried?

Strong answer criteria Distinguishes unknown from failed; Finds the original result before unsafe retry; Handles duplicate and outdated callbacks

Verification evidence Simulate a lost response. Verify one intended business effect and a visible status from which recovery is possible.

Read and verify this answer

SAP RAP business-object contract

OPS-12 A failed save leaves no business record, but support still needs diagnostics. How would you log it safely?

I separate business audit records from technical diagnostics and user messages. A log written inside a rolled-back transaction may disappear too. I use supported monitoring for diagnostics that must survive, rather than add a commit inside RAP behavior code. Logs should identify the failed request without exposing secrets or claiming that a rejected business change succeeded.

Reasoning and example

An order fails validation. FAILED identifies the rejection and REPORTED supplies the relevant message, but no order is saved. Support still needs to connect that user error to the failed request. I use the available service or application monitoring facilities with their documented save and privacy behavior for that diagnostic purpose.

A business audit entry saying 'order submitted' has a different meaning and should accompany a successful business transition, not survive a rejected submission as a false fact. I test success and rollback and inspect which records remain. Adding COMMIT WORK in a handler just to keep a log would break RAP's ownership of transaction completion.

Avoid this mistake Do not add COMMIT WORK inside a handler just to preserve a log entry.

Interviewer probes Should this record remain after rollback? / Does it describe a technical failure or a completed business event?

Strong answer criteria Separates business audit from technical diagnostics; Lets RAP control transaction completion; Keeps sensitive values out of logs

Verification evidence Test success and rollback, then inspect which supported messages and logs remain.

Read and verify this answer

SAP RAP business-object contract

OPS-13 A production incident starts after a release. How do you contain it without making the data worse?

I first limit the harmful processing through authorized operational controls and preserve the evidence. Then I identify the affected records and what has already committed. I choose a code fix, rollback or replay only after understanding those effects. Reverting code does not reverse business data, and replaying a backlog can repeat the damage.

Reasoning and example

A new Submit implementation starts duplicate remote approvals. I stop the affected scheduled initiation through the authorized control and record the release, request references and time window. Then I compare local requests with the remote approvals to find what was already created. Simply deploying the old code would not remove those duplicates.

After fixing duplicate handling, I test recovery on a controlled sample and reconcile existing approvals using supported business procedures. Only then do I resume the backlog, watching error counts and the rate of processing. The recovery plan needs a clear condition for resuming and an owner for unresolved records. Replaying everything before the fix could create even more approvals.

Avoid this mistake Do not rerun the whole backlog while the duplicate-creation defect remains active.

Interviewer probes Which business changes have already been saved? / What result proves processing can safely resume?

Strong answer criteria Stops harmful work before replaying it; Repairs code and business data separately; Defines when processing can safely resume

Verification evidence Keep an incident timeline, a list of checked affected records and the result of a controlled replay after the fix.

Read and verify this answer

SAP RAP business-object contract

OPS-14 What belongs in a RAP release gate beyond a passing ATC run?

ATC is one check, not proof that the feature works. I also test the business rules, failures, permissions and integrations affected by the change. Changes involving saves or concurrency need matching rollback and conflict tests. I record the version and environment for each result, and clearly mark anything not run rather than count it as passed.

Reasoning and example

A release changes an action parameter and publishes a new approval event. I test existing callers against the new action, verify the event after a successful save and check duplicate delivery. I also test a user who must not submit. These checks address the risks of this change; they are more useful than repeating an unrelated checklist.

For a label-only adjustment, the required evidence would be much smaller. For this release, I also verify relevant performance, dependency compatibility and target configuration. ATC can find issues covered by its configured checks, but it cannot prove every business rule or external connection. The release record should show executed results, pending checks and their owners.

Avoid this mistake Choose tests for the failure risks this change creates; a generic checklist alone is not enough.

Interviewer probes Which new failure mode did this change introduce? / Which result is pending rather than passed?

Strong answer criteria Chooses checks for the risks of this change; Separates source checks from runtime tests; Records tested versions and pending checks

Verification evidence Link each changed behavior to actual test results, responsible people and remaining risks.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP RAP feature and release overview

OPS-15 A team advertises 95% coverage while never asserting FAILED or persisted state. How would you improve it?

Coverage tells me which statements ran, not whether the results were correct. I add checks that fail when an important business rule is broken. For RAP, that can include rejected identities, messages, saved state and unwanted side effects. I then use coverage to find remaining gaps instead of adding meaningless code or repeating happy-path calls.

Reasoning and example

A test executes a validation for a missing account assignment and reports high coverage. However, it never checks FAILED, the message target or whether the invalid request can be saved. If I temporarily remove the rejection and the test still passes, it does not prove the validation works. I strengthen those assertions and restore the implementation.

I apply the same idea to successful processing: the test must check the expected state and absence of unintended changes. Coverage remains useful for locating paths we never executed, but the percentage is not the acceptance criterion by itself. A smaller set of failure-sensitive tests can provide stronger evidence than many calls without meaningful checks.

Avoid this mistake High statement coverage can still miss a broken business rule.

Interviewer probes Would the test fail if the business check were removed? / Which business rule matters more than increasing the coverage percentage?

Strong answer criteria Checks meaningful outcomes; Uses tests that fail when the rule is broken; Does not treat coverage as proof of correctness

Verification evidence Demonstrate a test failing when rejection or saving is deliberately broken, then restore the code.

Read and verify this answer

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

OPS-16 How would you prepare an extension for a platform upgrade?

I list the APIs and extension points the application actually uses, then check what changes in the target release. I prioritize important business flows and known exceptions. Tests run against a representative target system and include affected consumers, permissions and behavior. Released APIs reduce some upgrade risk, but they do not remove the need to test our application.

Reasoning and example

An extension calls a classic dependency through a wrapper and also exposes a released service to another system. I give each dependency an owner and check its target-release status, including any deprecation or successor. A wrapper does not make an underlying upgrade risk disappear. Existing external clients need testing against the upgraded service too.

I then run focused regression tests for the affected metadata, draft behavior, authorization, performance and configuration. If an API must be replaced, the plan includes moving consumers before retiring the old path. Last year's ATC result is not evidence for this year's target release. I retain the tested versions and actual results for the important flows.

Avoid this mistake Do not treat last year's clean ATC report as evidence for the new target release.

Interviewer probes Which dependencies create the biggest upgrade risk? / Does the application assume behavior the API documentation does not promise?

Strong answer criteria Lists actual API and extension dependencies; Tests risks for the target release; Assigns owners for exceptions and deprecated APIs

Verification evidence Record target versions, dependency classifications and regression results for the most important flows.

Read and verify this answer

SAP RAP feature and release overview

OPS-17 A cloud job processes 500,000 records and times out. How would you redesign it?

I split the work into limited-size packages that can restart safely. Each package has stable record IDs, a clear commit boundary and recorded progress. I use background or application-job features supported by the target system and limit parallel work. Recovery checks what actually completed, including changes that may have saved just before the job failed.

Reasoning and example

A pricing job finishes 20 packages and crashes during the next. On restart, it should not blindly repeat the completed work. I use recorded progress plus stable business references to distinguish completed, failed, pending and uncertain results. I also check whether a user edited a record after the original calculation so recovery does not overwrite newer data.

A critical test crashes between the business update and checkpoint handling. The design must detect the existing effect or repeat it safely; a checkpoint alone may not prove exactly-once execution. I tune package size and concurrency against database locks and remote limits. Moving one huge loop into a background job does not solve its restart problem.

Moving a job to background avoids a dialog waiting limit; it does not make an unbounded selection or retained internal table use less memory. Reduce the selected population and retained data, or design bounded, restartable processing. See SQL-17 for package lifetime and memory checks.

Avoid this mistake Moving a huge synchronous loop into a background job does not solve its transaction or restart design.

Interviewer probes What is one package that can recover independently? / How does recovery avoid overwriting newer user changes?

Strong answer criteria Limits package size and competing work; Defines how to find completed work after a crash; Uses supported cloud job features

Verification evidence Simulate a failure inside a package, then verify restart, resource limits and final business totals.

Read and verify this answer

SAP application job catalog and template

OPS-18 How would you communicate senior readiness when you can review code locally but cannot run SAP checks?

I say clearly what I reviewed, what I prepared and what I actually ran. Local review can find defects and produce useful tests, but it cannot prove SAP activation or real runtime behavior. I provide exact target-system steps and expected results. Missing evidence stays pending; I do not replace it with a guessed pass or unrelated performance number.

Reasoning and example

For a RAP approval change, I can review the source and prepare tests for a valid submission, a denied user and a rejected save. I can also provide a two-session concurrency scenario and a service smoke test. The handoff states which source revision to transfer, which roles are needed and what each result should show.

The person with target access then runs syntax and activation checks, ATC and the relevant tests, returning logs tied to that revision and environment. Until then, prepared test code is not an executed result. A UI running against mocks also does not prove integration with RAP. Clear limits make the evidence more useful, not less credible.

Avoid this mistake Do not present prepared test code or a mocked UI as successful SAP integration.

Interviewer probes Which claim can you prove now? / What exact target result would close the remaining risk?

Strong answer criteria Clearly separates prepared and executed work; Provides runnable steps and expected results; Links results to the source revision and environment

Verification evidence List prepared, executed and pending checks, with expected outcomes and the person responsible for each.

Read and verify this answer

SAP RAP test frameworks and boundaries

Official references

SAP RAP test frameworks and boundaries

SAP transactional buffer double support

SAP CDS access-control testing

SAP SQL analysis in ADT

SAP RAP service binding

SAP RAP feature and release overview

SAP RAP business-object contract

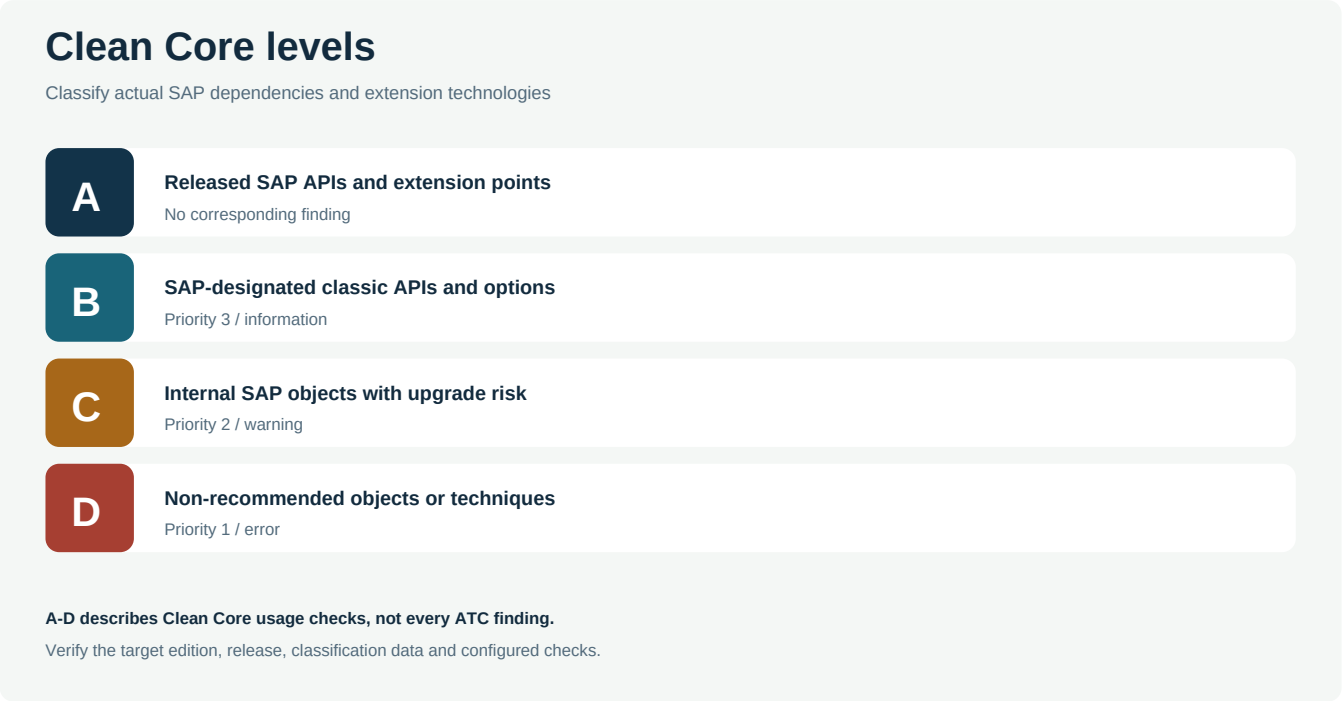
SAP total ETag and draft concurrency

SAP RAP side effects

SAP application job catalog and template

Clean Core levels and extensibility

Clean Core interviews test architectural judgment, not just memorization of A through D. Explain the target edition, actual API and extension technology, upgrade risk and governance. The levels complement the choice of key-user, on-stack or side-by-side extensibility.



Read each level as a classification of the actual SAP dependency or extension technology.

See the visual atlas for the recall prompt, limits and related questions.

CORE-01 What is Clean Core in a practical ABAP answer?

Clean Core means extending SAP through supported interfaces so custom changes are easier to keep working during upgrades. First check whether standard configuration already meets the need. When code is needed, choose suitable APIs and extension points, and track any exceptions. Custom code can still exist. Clean Core also covers areas such as business processes, data, and integration.

Reasoning and example

The business needs an extra approval check. First see whether standard configuration supports it. If not, look for a supported extension point at the moment when the check must run. Record which API or extension contract you rely on and test the approval process when the SAP system is upgraded.

In an interview, explain that decision using a real requirement and its outcome. Saying that all custom code should disappear does not explain how the requirement is met. Neither moving the code to BTP nor making its classes neat automatically removes dependence on SAP internals. The useful result is a supported extension whose behavior and upgrade responsibility are clear.

Avoid this mistake Clean Core is not the same as Clean ABAP, zero custom code or moving everything to BTP.

Read and verify this answer

SAP Clean Core best practices

CORE-02 What are Clean Core levels A B C and D?

The levels describe which SAP dependencies and extension techniques custom code uses. A uses released APIs and extension points under their supported contracts. B uses SAP-designated classic APIs and supported classic options. C uses SAP internals that may change. D uses explicitly non-recommended objects or techniques. Check the actual classification for the product and use involved.

Reasoning and example

Imagine four ways to meet a requirement. A released API that supports the operation is the preferred A path. A designated classic API may provide a justified B path in a landscape where classic development is allowed. Calling an internal object introduces C risk, while an explicitly non-recommended technique belongs to D.

The response should match the risk: prefer A where suitable, assess B, give C a named owner and replacement plan, and prioritize fixing D. Do not classify by familiarity or naming. A function module everyone has used for years is not automatically B, and an unreleased object is not automatically D. Look up the actual dependency and supported operation.

Remember: B is a SAP-designated classic API; C is an internal mechanism without a stability promise; D is an explicitly non-recommended object or technique. SAP describes B as Clean Core, C as conditionally Clean Core, and D as not Clean Core. Reading an internal SAP database table and making an unsupported direct write are different uses and can have different levels. CORE-13 shows where to look up noAPI.

Avoid this mistake A familiar API is not automatically B, and every unreleased object is not automatically D.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-03 How do the levels map to ATC messages?

For the configured Clean Core usage checks, A has no corresponding finding. B gives priority 3 information, C gives a priority 2 warning, and D gives a priority 1 error. This mapping applies to those checks. Other ATC checks may use priorities for different problems. A green run is useful only when the right checks and classification data actually ran.

Reasoning and example

A team sees no Clean Core findings after an ATC run and calls the application Level A. Before accepting that, check the variant, check implementation, classification data, and target release. If the relevant usage check was not included, silence says nothing about those dependencies.

When a priority 2 finding appears, inspect the object and the relevant Clean Core classification rather than treating the number as a universal description of every ATC issue. A security finding with the same priority needs its own interpretation. Use the findings to guide dependency review, and test the application separately for correctness and security. Clean Core classification is not a complete application quality certificate.

Look for Usage of APIs in the Clean Core check group. SAP points to SAP Note 3565942 for the relevant checks; availability depends on the ATC system's updates and configuration. Include applicable Allowed SAP enhancement technologies and Critical statements checks: an API catalog alone does not detect every non-recommended technique.

Avoid this mistake No ATC findings does not prove that the entire application is secure, correct or fully Level A.

Read and verify this answer

SAP Clean Core ATC levels and setup

SAP: Clean Core classification catalog and ATC checks

CORE-04 Why is a classic API Level B rather than Level A?

A classic API can be designated by SAP for supported classic use without being released for ABAP Cloud use. That is why it can be Level B rather than A. The Clean Core classification and the ABAP language version's access rules are related but different. Check the exact object and allowed use; a BAPI name alone proves neither.

Reasoning and example

A private-edition extension needs a function module that SAP designates as a classic API. That designation may make it a reasonable B dependency in an allowed classic implementation. It does not mean an ABAP for Cloud Development class can call it directly, or that public-edition developer extensibility permits the same design.

Check both questions separately: what is the dependency's classification, and what may this consumer use? Inspect the actual API state, classification data, product, and operation. If a permitted wrapper design is needed, account for its underlying dependency too. Calling an interface established or public in conversation is not the same as proving it has the required release contract.

Avoid this mistake Do not describe all classic APIs as public-cloud-ready or as formally C1-released.

Read and verify this answer

SAP Clean Core ATC levels and setup

SAP Cloudification Repository

CORE-05 What does responsible Level C use look like?

Using Level C means accepting a dependency on SAP internals that may change. First explain why a supported alternative does not meet the need. Isolate the dependency, name its owner, test the affected process, and plan its replacement. Recheck it when SAP changes or releases alternatives. Your product restrictions and exception policy still decide whether the use is allowed.

Reasoning and example

Suppose an important report needs information that no suitable supported API currently exposes. In a landscape where the proposed internal read is permitted, keep it behind a small, clear interface so the rest of the app does not depend directly on that internal object. Record why the exception exists and which business process depends on it.

During an upgrade, test that process and inspect changes to the dependency and its classification. If SAP later offers a suitable API, revisit the exception and replace the internal read where feasible. A signed exception does not freeze SAP's implementation or make the risk disappear. C is a managed risk decision, not unrestricted permission to call anything.

Avoid this mistake C is not an unlimited permission to call anything, nor is it the same risk category as a modification.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-06 What belongs to Level D and what would you do first?

Level D covers explicitly non-recommended objects or techniques, including problematic modifications and unsupported direct changes to SAP business data. Confirm the exact classification and operation first. Then replace the risky path with a supported business API or extension point. Start with active, high-impact writes and modifications. Use tests and a staged rollout so the replacement preserves the business process.

Reasoning and example

A custom report directly updates an SAP document table and seems to work. The standard business transaction may also update related records, enforce permissions, lock the document, and create change history. The single table update can leave those other parts inconsistent even when the changed field looks correct on screen.

Find the supported API or extension point for the business operation and compare its outcome with the required process. Build regression tests around the full document, not just that one column. Introduce the replacement carefully and monitor the result. Successful execution in a development system is evidence that SQL ran, not evidence that the business transaction is supported or complete.

Avoid this mistake The fact that a direct update works in development does not establish a supported business transaction.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-07 Are A through D the same as the old three tiers?

No. The earlier three-tier model described ABAP Cloud development, API enablement through wrappers, and classic extensions. Levels A to D classify the actual dependencies and techniques in more detail. They are especially useful when discussing private-edition and on-premise custom code. A tier number does not translate directly into a Clean Core level.

Reasoning and example

A classic extension may use a SAP-designated classic API and qualify at B for that dependency. Calling it tier 3 does not automatically make it C. Likewise, a wrapper used to enable an ABAP Cloud consumer can still contain an internal C dependency or a non-recommended D technique.

When reviewing the solution, trace what it really calls and how it extends SAP. Use the applicable current classification instead of converting an old architecture diagram mechanically into grades. You can still explain the three tiers as background for why wrappers exist, but keep placement and dependency assessment distinct. That produces a more useful answer than memorizing tier 1 equals A and so on.

Avoid this mistake Do not map tier 1 to A, tier 2 to B and tier 3 to C mechanically.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-08 Does wrapping an unreleased SAP API make it Level A?

No. A released custom wrapper can give its callers a controlled interface where the platform allows that design. The wrapper still calls the underlying SAP object, with that object's classification and upgrade risk. Review the full call chain. Releasing your own interface does not make an internal or non-recommended SAP dependency become Level A.

Reasoning and example

An ABAP Cloud class calls a released custom wrapper, and inspection finds an internal SAP function behind it. The consumer has a clear custom contract, but the internal dependency and its upgrade risk remain. Check the actual release and classification evidence; the wrapper alone cannot support a Level A claim.

The same reasoning applies to a nominated classic B dependency, an internal C dependency or a D technique: hiding it behind one method does not remove it. Record the dependency, owner, and replacement condition. If a suitable released SAP API becomes available, assess moving to it and retiring the wrapper. The wrapper's useful job is to control and isolate access, not to change the facts about the code underneath.

Avoid this mistake A wrapper is not a mechanism for laundering an internal or prohibited dependency into an entirely Level A solution.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-09 How do public private and on-premise editions change the answer?

Start with the SAP edition and release. Public-edition developer extensibility uses supported ABAP Cloud features and released contracts. Private edition and on-premise can also allow classic development, depending on release. Levels A to D help assess those dependencies. They do not allow forbidden objects or language features in a restricted environment. ABAP Cloud is not limited to BTP.

Reasoning and example

Two teams may ask for the same extra validation but work in different landscapes. One uses public edition; the other has a private-edition system with existing classic extensions. The permitted implementation options can differ even when the business requirement is identical. Check available APIs, extension points, and language-version rules before proposing a solution.

For the private-edition team, a classic option may be available but still need a dependency and risk assessment. For the public-edition team, calling it Level B or an approved exception does not unlock a classic technique the environment forbids. Explain both support and suitability. Where the code runs alone is not enough to identify its development model or its Clean Core level.

Avoid this mistake Private cloud does not mean that every classic customization is recommended, and ABAP Cloud does not mean only BTP.

Read and verify this answer

SAP Clean Core best practices

SAP ABAP Cloud FAQ

CORE-10 How do you choose key-user on-stack and side-by-side extensibility?

Choose the simplest supported option that meets the business need. Key-user tools fit supported field and process adaptations. On-stack developer extensibility fits logic that needs close access to local business data or the transaction. Side-by-side fits a more independent app or cross-system process. Use supported remote interfaces and plan how separate systems recover when one fails.

Reasoning and example

An extra field may already be supported by a key-user business context, so writing a new application adds little value. A validation that must reject the current document before it saves may fit a released local BAdI. A supplier portal used across several systems may fit a separate BTP application with its own release cycle.

Compare the required timing, users, data, and failure behavior. A remote portal cannot automatically share the local document's database transaction, so define pending and failed states where needed. These are placement choices, not automatic Clean Core grades. Each chosen implementation still needs a review of the APIs and extension technologies it actually uses.

Avoid this mistake These are placement and development choices, not automatic A-D grades or fixed implementation times.

Read and verify this answer

SAP Clean Core best practices

CORE-11 Why is side-by-side not automatically Clean Core?

Running outside SAP does not remove dependence on SAP. An external app can still use unsupported endpoints, read internal tables, or copy fragile business rules. Clean integration needs supported contracts, security, versioning, monitoring, and clear failure handling. BTP can help separate application lifecycles, but the quality of the integration still decides how safely the systems can change.

Reasoning and example

A portal on BTP reads data through an undocumented endpoint that changes during an SAP upgrade. The portal is outside the ERP system, but its unsupported dependency still breaks it. Moving the code changed its location without making the interface stable. A supported business API provides a clearer contract to build and test against.

If the portal also consumes events, handle retries without creating duplicate business actions and monitor failures. Test changes to the API and business meaning before upgrades. Independent deployment is useful only within the limits of the agreed integration contract. Copying SAP's core business logic into the portal may introduce another source of maintenance and consistency problems.

Avoid this mistake Moving custom code out of the ERP server does not erase coupling.

Read and verify this answer

SAP Clean Core best practices

CORE-12 What are release contracts C0 C1 and C2?

C0 is a release contract for extension. C1 is for use within the system, and C2 is for remote use. Check the exact object's release state, contract, and allowed visibility before using it. These contracts say how an object may be consumed. They do not grant business permissions or promise that every operation you need exists.

Reasoning and example

You find a CDS entity that is released for local ABAP use. That can support a local read, but it does not automatically give an external application an HTTP endpoint. A remote API needs the appropriate remote contract and service setup. The reverse also matters: a published remote API does not necessarily release its internal ABAP implementation class for local callers.

For an extension, inspect whether the object actually permits the required kind of extension under the relevant contract. A public class definition or a familiar I_ name is not enough evidence. Then check business authorization separately, because a supported API can still reject a caller who is not allowed to perform the operation.

Avoid this mistake A public class or an I_ CDS name is not proof of a released API contract.

Read and verify this answer

SAP API release contracts

CORE-13 Where do you find Classic API, Not Recommended and noAPI classifications, and how do you verify an API choice?

Open the SAP Cloudification Repository Viewer and choose the Classic API Clean Core classification catalog. Search for the exact object; State → No API corresponds to state noAPI in the JSON data. That means Not Recommended, not merely unreleased. Also check the target system's API state and the intended operation, then assess actual usage with maintained ATC checks.

Reasoning and example

Try RFC_READ_TABLE in the linked catalog. The checked SAP data identifies objectType FUNC, objectKey RFC_READ_TABLE and state noAPI. This is evidence of an explicitly non-recommended object; the name being familiar or remote-enabled does not make it a suitable API. Recheck current classification data when assessing another release.

Not Released is not the same as Not Recommended. An object without an ABAP Cloud release can be a designated Classic API (B) or an internal object (C). D also includes non-recommended techniques such as unsupported direct writes to SAP tables; an object lookup alone cannot assess how your code uses it.

For a purchase-document change, verify that the candidate API supports writes, the required fields and your target product. A released read-only CDS entity with a matching name is not a write API. Run ATC Usage of APIs and the relevant enhancement/critical-statement checks on the actual code. CORE-03 explains the scoped priority mapping; a successful syntax check alone does not establish support.

Avoid this mistake Not Released ≠ Not Recommended. Catalog status, allowed language-version access and support for the intended operation are separate checks.

Read and verify this answer

SAP Cloudification Repository Viewer

SAP classification data: objectClassifications_SAP.json

CORE-14 Can a solution using both A and B dependencies be called entirely A?

No. If the solution depends on a Level B interface, that dependency is still part of the solution. You can report an A-compatible consumer and a B wrapper separately, but the whole chain is not entirely A. Keep object-level findings visible. A large percentage of A objects must not hide one critical higher-risk dependency.

Reasoning and example

An application has many classes using released interfaces and one wrapper calling a designated classic API. Reporting only the classes in the consumer layer makes the result look cleaner than the actual system. Include the wrapper and explain which business operation depends on it and what maintenance risk remains.

Prioritize by business impact as well as counts. One unsupported write path used for every payment can matter more than many low-impact read findings. Show both the distribution and the critical dependencies, then choose remediation based on exposure, replacement options, and cost. Averaging grades or counting only the top layer does not explain whether the important business process will survive an upgrade.

Avoid this mistake Percentages of Level A objects should not hide one critical D write path.

Read and verify this answer

SAP Clean Core best practices

CORE-15 What if no released API covers the business requirement?

First recheck standard features, configuration, and supported extension points on the target release. Consider a different process or placement if it meets the requirement. Where classic development is allowed, assess designated classic APIs and any remaining internal dependency under the exception policy. Request a supported API when needed. Do not assume a classic workaround is possible in public edition.

Reasoning and example

A team cannot find a released API for a required write. Confirm the exact missing operation before declaring an API gap: a standard process or released extension point may achieve it another way. Compare alternatives with the business, including whether immediate completion is truly required or a different workflow would work.

If an allowed temporary exception remains necessary, record its owner, rejected alternatives, test coverage, and replacement condition. Review it when SAP offers a successor. Do not start with a modification simply because it is technically convenient. And in a restricted environment, an exception document cannot enable a forbidden technique; the design must still fit what that product supports.

Avoid this mistake Do not immediately choose a modification, and do not assume public edition permits a classic workaround.

Read and verify this answer

SAP Clean Core best practices

CORE-16 What would your Clean Core governance pipeline contain?

Use ATC checks that match the system and include API usage, extension technologies, security, and code quality as appropriate. Review findings instead of hiding them. Give exceptions an owner and a date or condition for review. Track fixes and rerun checks for upgrades. Combine the automated results with architecture review and real business-process regression tests.

Reasoning and example

A transport contains a new internal API call. The pipeline should identify it with the relevant check and route it for a decision under the team's policy. Some findings may block delivery, while a justified exception may be accepted with a clear owner and follow-up date. Store that decision rather than disabling the whole check.

Before an upgrade, update the relevant check and classification information and rerun the affected process tests. A previously accepted dependency may now have a supported replacement or a changed risk. The priority number helps triage work, but it does not tell you the full business impact. Runtime evidence and responsibility for the exception remain necessary.

Avoid this mistake A priority number alone is not a complete technical-debt decision or a substitute for runtime tests.

Read and verify this answer

SAP Clean Core ATC levels and setup

CORE-17 Is Clean ABAP the same thing as Clean Core?

Clean ABAP is about code that people can understand, change, and test. Clean Core is about supported extension choices and dependencies that are easier to maintain across SAP changes. They help each other but measure different things. A well-written class can still depend on SAP internals. Renaming variables does not change that dependency's Clean Core level.

Reasoning and example

Review a custom approval class in two passes. First ask how it connects to SAP: does it use supported APIs and extension points, or directly modify an internal table? That determines the extension and upgrade concern. Then review its responsibilities, naming, interfaces, errors, and tests to judge how maintainable the implementation is.

You might find an easy-to-read class with a dangerous write dependency, or a supported API used through confusing, untested code. Each needs a different improvement. Replacing the unsupported write helps Clean Core; separating responsibilities and adding meaningful tests helps Clean ABAP. A strong interview answer explains both changes and why one does not automatically solve the other.

Avoid this mistake Refactoring variable names does not change the Clean Core level of an unsupported dependency.

Read and verify this answer

SAP Clean Core best practices

CORE-18 How would you modernize a large custom-code portfolio?

Start by finding which custom code is actually used and valuable. Confirm and retire genuinely unused parts, then classify dependencies in active code. Fix the most important risky paths first and adopt suitable supported APIs step by step. Measure business regressions and upgrade effort. Do not rebuild every classic report in RAP just to make the technology look uniform.

Reasoning and example

A portfolio contains hundreds of reports, but a small number handle critical daily work. Confirm usage and ownership before deleting anything. Pick one important active dependency, such as an unsupported business-table write, and document the current business outcome. Replace it behind a clear interface and test the entire affected process.

Roll out the change with evidence that users can still complete their work, then repeat for the next priority. Some reports may remain perfectly suitable as reports, while a transactional application may benefit from RAP. The modernization decision should follow business value and risk. An automatic conversion promise hides the need to understand existing behavior, integrations, and exception cases.

Avoid this mistake RAP is not a mandatory replacement for every report, interface or classic application.

Read and verify this answer

SAP Clean Core best practices

Official references

SAP Clean Core best practices

SAP Clean Core ATC levels and setup

SAP Cloudification Repository

SAP ABAP Cloud FAQ

SAP TechEd Clean Core exercise

SAP API release contracts

Senior Clean Core governance and tradeoffs

Clean Core decisions require evidence about dependencies, supported extension contracts, business impact, and ownership. These scenarios use A-D as extensibility classifications while keeping API release contracts and ATC severities separate. A senior answer should make uncertainty visible, defend a practical decision, and define how the organization will detect and reduce the remaining risk.

Clean Core levels

Classify actual SAP dependencies and extension technologies

A	**Released SAP APIs and extension points** No corresponding finding
B	**SAP-designated classic APIs and options** Priority 3 / information
C	**Internal SAP objects with upgrade risk** Priority 2 / warning
D	**Non-recommended objects or techniques** Priority 1 / error

A-D describes Clean Core usage checks, not every ATC finding.

Verify the target edition, release, classification data and configured checks.

Read each level as a classification of the actual SAP dependency or extension technology.

See the visual atlas for the recall prompt, limits and related questions.

GOV-01 A project claims Level A because its frontend uses RAP. How would you assess the extension?

Check the whole extension, including wrappers and the SAP objects they call. RAP and Fiori do not decide the Clean Core level. A depends on released contracts; B uses classified classic options; C uses internals; D uses non-recommended dependencies or techniques. Record the actual classification for each path on the target system. Do not call a mixed solution entirely Level A.

Reasoning and example

A RAP handler calls a custom wrapper, which then calls an internal SAP class. Looking only at the handler's released custom interface misses the underlying dependency. Assess the consumer and wrapper separately where appropriate, but include both in the delivered solution's dependency record. A customer namespace does not erase the risk.

Link each finding to the business capability it affects and the person responsible for maintaining it. Confirm the applicable API state, classification, and ATC result for the same source version. A passing UI test shows that a tested interaction worked, not that every call underneath uses Level A interfaces. State clearly what is included in any classification claim.

Avoid this mistake RAP, BTP deployment, a customer namespace, or a green UI test is not proof that every underlying dependency is Level A.

Interviewer probes What exactly is included in the claimed Level A scope? / How would you find dependencies hidden behind customer adapters?

Strong answer criteria Defines scope before classification; Traces indirect dependencies; Separates framework choice from API evidence

Verification evidence An object-level dependency inventory with target-system release/classification evidence and the applicable ATC result for the same revision.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-02 An API is released with C1. A team assumes that means Clean Core Level C and permission to expose it remotely. How do you respond?

These are two different naming systems. C0 means Extend, C1 means Use System-Internally, and C2 means Use as Remote API. They are release-contract purposes, not Clean Core levels A to D. Check the object's visibility for the intended caller too. A C1 release does not by itself authorize remote exposure or classify the dependency as Level C.

Reasoning and example

A locally released class may be suitable for an on-stack application without being a remote API. If you build a custom remote service over it, you become the provider of a new interface. You must define authentication, business permissions, errors, transaction behavior, and how callers can rely on future versions.

The local API has not magically changed to C2, and your service does not inherit guarantees that its provider never made. Inspect API-state properties and documentation for the exact supported use. Keep the separate questions clear: what release purpose is allowed, who may consume it, and what Clean Core classification applies to the implementation's actual dependencies?

Avoid this mistake C0/C1/C2 and Clean Core A/B/C/D are different classifications; release for one purpose does not release every possible usage.

Interviewer probes Which visibility must the API expose for this caller? / What responsibilities arise if you build a custom remote service over a local API?

Strong answer criteria Names contracts accurately; Checks purpose and visibility; Recognizes new provider responsibilities

Verification evidence ADT API-state properties, provider documentation, and a consumer design showing local use or an explicitly owned remote contract.

Read and verify this answer

SAP release contracts, visibility, and API lifecycle states

GOV-03 A team wraps an internal SAP function module and releases the wrapper. Has the technical debt disappeared?

No. A wrapper can protect callers from some changes, but the underlying SAP dependency still has its original release status and upgrade risk. Record both the wrapper's public promise and its implementation dependency. Where allowed, give it an owner, regression tests, and a replacement plan. A useful wrapper hides unstable details; merely renaming the same internal interface adds little protection.

Reasoning and example

If the wrapper exposes the internal function module's structures unchanged, a provider change can still spread to every caller. A smaller request and result designed around the business need can contain more of that change inside the wrapper. Test those promised inputs, outcomes, and failures rather than only checking that the call returns.

A wrapper over a classified classic API can fall within the applicable Level B guidance. Wrapping an internal object does not make it a designated classic API. Keep the target-release assessment visible and define when a released replacement should be adopted. Your own release flag is a promise you maintain, not a way to change SAP's classification of its underlying object.

Avoid this mistake A customer release flag does not turn an unreleased SAP dependency into a released SAP API or erase its classification.

Interviewer probes Which unstable details must stay behind the wrapper? / Who replaces the dependency when a released alternative becomes available?

Strong answer criteria Records the underlying dependency risk; Provides a small stable interface; Names who replaces the dependency

Verification evidence A wrapper dependency record, contract tests, an owner, and a documented replacement trigger with the current target-release assessment.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-04 An urgent release needs an ATC exemption for an internal dependency. What makes the exception governable?

Make the exception specific: business need, exact finding, source scope, target edition, alternatives, risk owner, and supporting tests. Give it an expiry or review trigger and a realistic fix plan. An ATC exemption changes how a finding is handled. It does not make forbidden code legal or turn the dependency into Level A. The release decision must accept the remaining risk explicitly.

Reasoning and example

An approved exception covers one read-only adapter. A later change adds a write through another internal call. That new risk must not silently inherit the old approval. Tie the exception to the relevant objects and finding category, rerun checks after changes, and revisit it when the provider or target release changes.

Keep security and transaction defects visible as separate decisions; they are not automatically accepted by a Clean Core waiver. State which tests reduce the risk and who will replace the dependency. An indefinite blanket exemption makes future changes hard to assess. A bounded exception lets the team deliver a justified need while preserving a clear limit on what was actually accepted.

Avoid this mistake An approved exemption is not a reclassification to Level A and must not become a general permission to bypass ABAP Cloud restrictions.

Interviewer probes What event invalidates this exception? / How do you prevent the approved scope from growing unnoticed?

Strong answer criteria Limits what the exception covers and how long it lasts; Names who accepts and owns the risk; Distinguishes exempting a finding from permission to use an API

Verification evidence An exception record linked to the exact ATC finding, source revision, release target, regression evidence, owner, and expiry or review trigger.

Read and verify this answer

SAP ATC configuration and transport-release governance

GOV-05 A Level B solution works in Private Edition. Can the same approval justify deploying it in Public Edition?

No. A Level B decision does not override Public Edition's supported APIs, language rules, or extension options. Reassess the business need for that target using configuration, key-user tools, developer extensibility, and supported remote integration. Identify what cannot move and compare a supported redesign with keeping the current deployment. Even Level A does not guarantee identical capabilities across editions.

Reasoning and example

A Private Edition app uses a classic enhancement or SAP GUI-dependent component. It may have a justified role there but no supported equivalent in the proposed Public Edition scope. Moving its screen to BTP does not make an unavailable backend operation suddenly callable. Test the required business operation against actual target APIs.

Separate the parts that can be reused from those needing redesign. Record each missing capability and its effect on the process, then let the business compare alternatives with evidence. Prior approval of a dependency in one landscape is not permission in another. Classification helps assess risk; product availability and technical restrictions still determine which solution can be built.

Avoid this mistake Level B is not a universal deployment permission, and Level A design does not guarantee identical API coverage across editions.

Interviewer probes Which part of the solution is genuinely portable? / What evidence would justify keeping a private-edition dependency?

Strong answer criteria Checks target-edition constraints; Separates classification from portability; Offers supported redesign options

Verification evidence An edition capability comparison with verified target APIs, extension points, and a documented functional gap for each nonportable component.

Read and verify this answer

SAP extensibility options by environment and language version

GOV-06 You have thousands of custom objects and a limited upgrade budget. How would you prioritize remediation?

Prioritize by actual use, business importance, dependency risk, target-upgrade changes, and effort. Validate unused-code candidates before retiring them. Investigate high-impact non-recommended or changed internal paths early, and give remaining work owners and acceptance criteria. Do not rank only by object count or execution frequency. One critical posting dependency may matter more than hundreds of small utilities.

Reasoning and example

A year-end report has no executions in a three-month trace. Deleting it based on that trace alone can cause an outage at closing time. An emergency posting tool may run only once a year and still be essential when it is needed. Combine usage evidence with process-owner confirmation and the limits of the observation period.

Then compare target-upgrade changes with business impact and replacement cost. A daily financial dependency that changes in the target release may deserve urgent attention, while an unaffected lower-risk utility can wait. Record why each item has its priority and how success will be tested. A large findings count is not yet a useful remediation plan.

Avoid this mistake Do not equate low execution frequency with low business importance or treat every finding as equal remediation work.

Interviewer probes How would you avoid retiring seasonal functionality? / What makes one Level C dependency more urgent than another?

Strong answer criteria Combines usage with business importance; Uses evidence about changes in the target release; Prioritizes work with clear owners

Verification evidence A ranked remediation portfolio with usage-window limitations, business-owner decisions, dependency impact, estimated effort, and target-release tests.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-07 No released API supports a required business operation. What would your escalation contain?

Describe the business operation you need, the target product and release, and why existing supported options do not meet it. Include transaction behavior, permissions, expected volume, and failure needs. Provide a small reproducible example where helpful. Ask for a supported capability rather than simply a table release. Meanwhile, choose an explicit interim process, scope change, or permitted governed alternative.

Reasoning and example

Instead of saying release this table, explain that the application must close a document with the right validations, permissions, and failure response. Show which APIs and extension points were checked and the exact operation each cannot provide. That gives the provider a requirement it can act on without committing to your proposed internal implementation.

The project still needs a plan while the request is assessed. It may delay this feature, adjust the process, or use an edition-appropriate temporary exception with a retirement condition. Opening an API-gap ticket does not authorize direct table access, and it does not guarantee a delivery date. Keep the gap visible rather than hiding it inside a workaround.

Avoid this mistake An API-gap ticket does not authorize direct table access or promise that the requested API will arrive by the project's deadline.

Interviewer probes What evidence shows that existing APIs really do not fit? / How do you keep delivery moving without hiding the gap?

Strong answer criteria Requests a supported business operation; Explains why supported alternatives do not fit; Defines what the project will do while waiting

Verification evidence A clear API-gap record with searches for the target release, required operation and failure behavior, a reproducible example, and the chosen interim scope.

Read and verify this answer

SAP extensibility options by environment and language version

GOV-08 The business asks to reproduce a heavily customized legacy process exactly. How do you challenge that without dismissing the requirement?

Separate the business result that must be preserved from the old screen or implementation. Demonstrate how standard configuration and supported extensions meet that result, including exception cases. Keep legal and contractual requirements explicit. Compare the ongoing cost of each retained customization with its value. Let the business owner decide using concrete behavior, cost, and risk rather than slogans.

Reasoning and example

The old approval screen combines several steps that standard applications separate. The real control may be that the requester cannot approve their own request and that every decision is recorded. Rebuilding the exact screen through internal hooks may add upgrade risk without improving either control. Walk through those requirements with the process owner.

Include unusual cases such as withdrawal, delegation, and rejected approvals so standard functionality is not accepted on a happy-path demo alone. Where a real requirement is missing, compare a supported extension or process adjustment with preserving the deviation. Fit-to-standard should help make that tradeoff visible, not dismiss a legitimate need or assume visual parity is the only measure of success.

Avoid this mistake Fit-to-standard is not a slogan for refusing real requirements, and exact visual parity is not always the real business invariant.

Interviewer probes Which requirement is mandatory and who can prove it? / What exception case could invalidate the proposed standard process?

Strong answer criteria Separates outcomes from historical design; Uses concrete process demonstrations; Makes cost and ownership visible

Verification evidence A fit-gap matrix with business-owner acceptance, exception-path demonstrations, and lifecycle cost estimates for retained deviations.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-09 Your team wants to release a custom API used by three other teams. What must be ready before setting its release state?

Before release, define the API's supported inputs, outcomes, errors, transaction behavior, and permissions. Give it an owner willing to maintain compatibility. Keep stable types and behavior public and private storage details private. Capture a delivered baseline and set compatibility, deprecation, and successor rules. If the API is still experimental, coordinated component access may fit better than a premature stability promise.

Reasoning and example

Three teams already call a pricing API. Removing an optional field or changing rounding may look like cleanup to the provider but break one caller's business result. Keep a snapshot of what was delivered and use the applicable compatibility checks for detectable technical changes. Consumer tests cover meanings and outcomes that a signature comparison cannot prove.

Do not replace the old baseline just to silence a genuine incompatibility finding. Plan how current users migrate when a breaking change is needed, and communicate support periods. Supported access permission can enable internal collaboration, but releasing a stable API is a stronger ongoing commitment. Set the release state only when the provider is prepared to meet that commitment.

Avoid this mistake Releasing an API is a provider commitment; broad access permission is not equivalent to a stable contract.

Interviewer probes Which changes can be detected statically and which need consumer tests? / How would you deprecate the API without breaking current users?

Strong answer criteria Defines what the API owner must maintain; Keeps a trustworthy record of the delivered interface; Checks both technical compatibility and business behavior

Verification evidence An API specification, owner and consumer list, prior-delivery snapshot, compatibility results, and consumer-contract test evidence.

Read and verify this answer

SAP release contracts, visibility, and API lifecycle states

SAP software component dependencies and compatibility snapshots

GOV-10 The direct dependency scan is clean, but an upgrade still breaks the extension. What could the governance model be missing?

Check what the scan did not cover: indirect calls, dynamic selection, configured implementations, remote interfaces, and data assumptions. Verify its package scope, check version, and classification data too. Connect static findings with a dependency map and process tests. Clean direct references are useful evidence, but they do not prove that every runtime path will survive an upgrade.

Reasoning and example

A custom facade chooses an internal implementation from configuration, so scanning only its direct references misses the actual dependency. A remote service can also change independently of the ABAP transport. Include these paths in the map and assign owners who can explain and test them.

Another upgrade failure may come from data behavior: a query expected one text row, but changed data now gives several and doubles a total. No forbidden direct call is needed for that defect. Rehearse representative configured processes on the target upgrade and document what the checks and tests covered. A green result is meaningful only within that stated scope; it is not proof that all hidden assumptions are safe.

Avoid this mistake A green direct-reference check is not proof that the whole solution has no unsupported or semantically fragile dependencies.

Interviewer probes How would you discover configuration-driven dependencies? / Which runtime paths must the upgrade rehearsal cover?

Strong answer criteria Looks for indirect, dynamic, and configured dependencies; Checks whether the scan covered the needed objects; Tests complete business processes

Verification evidence An expanded dependency graph, documented scan scope and tool versions, and upgrade regression evidence covering configured runtime paths.

Read and verify this answer

SAP software component relations and consistency checks

GOV-11 How would you enforce Clean Core in delivery without blocking every legacy transport immediately?

Record existing findings as a clear baseline and prevent new unapproved risk. Use checks appropriate to the target environment for language, APIs, extension techniques, compatibility, security, and correctness. Keep existing exceptions bounded and prioritize their fixes. Make the gate's scope and thresholds reproducible. Give developers feedback early enough to fix issues before release day.

Reasoning and example

A layout change to a legacy report should not automatically demand an unplanned rewrite of the whole application. It also must not quietly introduce a new internal write dependency. Compare the current findings with the accepted baseline, then review new findings and any change that makes an existing exception riskier.

Store the check variant, versions, priorities, scope, and exception policy so teams cannot redefine green by weakening checks. Protect and review policy changes. Plan remediation for existing exposure with named owners instead of suppressing it forever. The baseline explains known risk; it is not permission to add unlimited new debt, and transport-blocking priorities are a configured team policy rather than universal ATC behavior.

Avoid this mistake A baseline is not approval for unlimited new debt, and transport blocking thresholds are configured policy rather than a universal property of every ATC check.

Interviewer probes What happens when a touched legacy dependency becomes riskier? / How do you stop teams from weakening the shared check variant?

Strong answer criteria Distinguishes baseline from new risk; Uses reproducible early gates; Governs exception and policy changes

Verification evidence A versioned gate policy, representative pass/fail cases, a protected check-configuration record, and a baseline-to-current findings comparison.

Read and verify this answer

SAP ATC configuration and transport-release governance

GOV-12 A vendor upgrade changes an object used by an approved Level C extension. How do you organize the response?

Reassess the exception for the new vendor release. Find affected callers and processes, reproduce the failure, and compare old and new behavior. Check whether a suitable released replacement now exists. If the internal dependency remains justified, adapt its isolated boundary and test the full affected process. Coordinate vendor and custom delivery, including rollback or business recovery where data changes are involved.

Reasoning and example

An internal object's signature is unchanged, but its defaulting or validation now behaves differently. A syntax-only repair cannot show whether postings, permissions, messages, and resulting document states still meet the business need. In another case, activation fails because a type changed; fixing that type still needs behavioral regression tests.

Use the actual upgraded environment and record both vendor and custom source versions. Compare representative successful and rejected transactions, and investigate whether the defect depends on specific data. If a supported successor now covers the need, revisit keeping the exception at all. Last year's approval or successful upgrade is evidence about the previous situation, not a guarantee for this release.

Avoid this mistake A previous exception or successful prior upgrade does not prove compatibility with the next vendor release.

Interviewer probes How do you distinguish an API change from a data-dependent defect? / Would you keep the exception if a suitable released successor is now available?

Strong answer criteria Reassesses release-specific exposure; Tests semantic behavior; Coordinates recovery and replacement

Verification evidence A target-upgrade impact record, reproduced failure, reviewed boundary change, and process regression results tied to both vendor and custom revisions.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-13 What would a credible two-year custom-code modernization roadmap look like?

Build the roadmap around business capabilities and reduced maintenance risk. Retire confirmed unused functions, replace important risky dependencies, and use supported ABAP Cloud options for suitable new work. Valuable classic code can remain under clear governance. Give each phase owners, measurable outcomes, and operational acceptance. Reserve capacity for upgrades and API changes instead of promising a full RAP rewrite by a date.

Reasoning and example

One phase removes an unused interface after confirming business ownership. Another replaces internal reads with a released query. A third redesigns a critical posting path. These changes need different tests and deliver different benefits, so counting converted classes gives a poor picture of progress.

For each phase, say what risk or cost will decrease and how that will be demonstrated. Examples include fewer unsupported write paths, a successful target-upgrade rehearsal, or lower effort to adapt a shared interface. Budget for integration and operational acceptance, not only new screens. A roadmap can support gradual coexistence while still setting clear retirement goals for temporary bridges and unnecessary dependencies.

Avoid this mistake Modernization is not measured reliably by RAP object count or by replacing all classic code regardless of business value.

Interviewer probes What quarterly outcome would demonstrate reduced upgrade risk? / How do you fund remediation that has no visible new UI feature?

Strong answer criteria Prioritizes outcomes for business capabilities; Keeps justified classic and modern code working together; Measures the reduction in maintenance and upgrade risk

Verification evidence A capability-based roadmap with owners, usage decisions, target dependency reductions, migration acceptance criteria, and budgeted upgrade work.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-14 A classified classic API meets the need, while a released API covers only half the operation. Would you accept Level B?

Yes, where the target permits it, a verified Level B API can be a reasonable choice. Compare the complete supported operation with the partial released alternative, including permissions, transaction safety, and maintenance. Verify the whole dependency path and state the B dependency honestly. Keep a reason to reassess later. This choice does not override Public Edition or restricted ABAP Cloud access rules.

Reasoning and example

The released API supports only part of the posting operation. Completing the rest with direct table updates may be less safe than using a properly classified classic API that performs the full transaction. Test the actual business result, required fields, rejection paths, and authorization rather than choosing a label without considering completeness.

Check the exact object's classification and extension technique; the name BAPI does not prove Level B. If the consumer uses a restricted language version, verify the allowed access design separately. Record why the decision fits this target and what successor or upgrade change would trigger review. Accepting B can be a responsible documented choice, but should never be presented as direct unrestricted Cloud access or an entirely A solution.

Avoid this mistake Not every old API is Level B, and a Level B assessment does not make it directly callable from restricted ABAP Cloud code.

Interviewer probes What makes the exact API qualify as classic rather than internal? / What would cause you to revisit this decision after an upgrade?

Strong answer criteria Supports the Level B choice with exact evidence; Preserves the complete business transaction; Respects product and language restrictions

Verification evidence Specific object-classification evidence, a complete operation test, a comparison with released alternatives, and a scheduled reassessment trigger.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-15 A required report uses an internal read object with no current replacement. Would you accept Level C, and under what conditions?

First prove that no suitable supported read meets the need and that the target allows the proposed use. If C is accepted, isolate the internal dependency, limit what callers depend on, and name its owner. Set a review trigger and replacement plan. Read-only reduces some write risks but still has confidentiality, result-correctness, and upgrade risks. Test those explicitly.

Reasoning and example

An internal view changes a join after an upgrade. The report still activates but now returns duplicate rows and a wrong business total. Another change could broaden returned data for a restricted user. A test that merely executes the SELECT successfully misses both problems.

Capture representative expected totals, row identities, filters, and denied-access results. Recheck them on the target release and use available SAP change information as supporting evidence, not a complete guarantee. Keep the exception tied to this precise dependency and purpose, with a date or event that forces review. A justified internal read is not blanket approval for other internal objects or non-recommended Level D techniques.

Avoid this mistake Read-only does not mean risk-free, and Level C is not a blanket exemption for all internal objects or non-recommended Level D techniques.

Interviewer probes What result change could escape an activation check? / Which precise event forces this exception back into review?

Strong answer criteria Treats read semantics as consequential; Isolates and owns exposure; Defines focused compatibility evidence

Verification evidence A bounded exception with dependency classification, authorization and result baselines, target-upgrade checks, and replacement-review triggers.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-16 How would you assess a solution that uses a released CDS query, a classified classic BAPI, an internal class, and a modification?

Assess each path separately on the actual target. The released query may support A use, the classified classic BAPI introduces B, the internal class introduces C, and a non-recommended modification can introduce D. Do not average them into one reassuring score. Link each to business use, ownership, and alternatives, then prioritize fixes using impact as well as classification.

Reasoning and example

The modification may sit in an old unused path, while the internal class is called during daily financial posting. Both dependencies belong in the factual assessment, but those usage differences affect what to fix first. Confirm historical and seasonal use with the process owner before deciding to remove anything.

For a replacement, test the full intended operation and its errors, not just successful activation. Keep the classification evidence, target version, owner, and business impact beside each finding. A single letter for the entire solution can hide these facts. Classification describes the exposure; remediation priority also needs use, criticality, available alternatives, and the cost of changing the business process safely.

Avoid this mistake A single letter for a mixed solution can hide scope; classification and remediation priority are related but are not the same decision.

Interviewer probes Which capability would you remediate first and why? / What evidence is required before removing the modification?

Strong answer criteria Classifies each dependency instead of averaging grades; Separates classification from fix priority; Uses business importance and actual usage

Verification evidence A scoped assessment table linking each dependency to classification evidence, process impact, usage, ownership, and a justified remediation decision.

Read and verify this answer

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

GOV-17 The sponsor says Clean Core is a developer concern and rejects funding for integration monitoring and data cleanup. How would you negotiate?

Explain the business failures that monitoring and data quality prevent. Clean Core includes more than extension code: poor master data, unsupported integrations, and unmanaged process variants also make change difficult. Propose a small set of owned controls with clear outcomes and costs. Phase improvements where reasonable. Make any deferred business risk and its operating owner explicit.

Reasoning and example

An order API is properly released, but invalid customer mappings make messages fail repeatedly. Nobody owns the queue, so business orders remain incomplete even though the code uses supported interfaces. Another wrapper or ATC exception will not repair the mapping or ensure someone retries failed work.

Agree who maintains mappings, responds to alerts, reconciles results, and prioritizes cleanup. Use a concrete incident or realistic failure scenario to compare the cost of controls with the cost of unprocessed business work. Define measurable service expectations and distinguish essential controls from improvements that can wait. A good extensibility grade does not certify data quality, secure integration, or effective day-to-day operation.

Avoid this mistake A strong extensibility classification does not certify clean data, standard processes, secure integrations, or effective operations.

Interviewer probes Which control delivers the highest near-term business value? / Who is accountable when an interface remains technically successful but business processing is incomplete?

Strong answer criteria Connects monitoring and data controls to business results; Proposes a practical phased plan; Names business and operational owners

Verification evidence A responsibility matrix, costed minimum control plan, measurable operational targets, and documented acceptance of deferred risks.

Read and verify this answer

SAP Clean Core dimensions beyond extensions

GOV-18 An interview panel shows an A-D chart with ATC priorities. What would you clarify before treating it as a release gate?

Explain that the A-D priority mapping belongs to particular Clean Core checks. Ask which variant, checks, classification data, object scope, and target release produced the results. Other ATC priorities can describe security or performance issues instead. Define what the release gate proves, which exemptions remain, and which runtime tests are still needed. Save enough run information to repeat the assessment.

Reasoning and example

One project has fewer findings because its variant omitted an enhancement check or used different classification data. Comparing the raw count with another project can therefore reward missing checks rather than safer code. Store the configuration, source version, scope, and exemptions so the comparison is like for like.

Even matching static checks do not prove that a posting transaction behaves correctly. Add the relevant runtime evidence and distinguish a resolved dependency from a finding that was merely exempted. Do not treat every priority 1 as Level D or every priority 2 as C across all ATC checks. Absence of findings also does not prove complete Level A coverage when assessment scope is incomplete.

Avoid this mistake Do not interpret every priority 1 finding as Level D, every priority 2 as Level C, or absence of findings as proof of complete Level A coverage.

Interviewer probes What would you store so this assessment can be reproduced? / How would you distinguish an exempted finding from a resolved dependency?

Strong answer criteria Explains which check gives the priority mapping; Records enough information to repeat the assessment; Distinguishes static checks from runtime tests

Verification evidence The exact ATC variant and check configuration, classification/version information, source and scope identifiers, exemptions, and complementary runtime validation results.

Read and verify this answer

SAP ADT release notes: ABAP_CLEAN_CORE_READINESS check variant

Official references

SAP Clean Core levels, classic APIs, wrappers, and edition boundaries

SAP release contracts, visibility, and API lifecycle states

SAP software component dependencies and compatibility snapshots

SAP software component relations and consistency checks

SAP ATC configuration and transport-release governance

SAP ADT release notes: ABAP_CLEAN_CORE_READINESS check variant

SAP extensibility options by environment and language version

SAP Clean Core dimensions beyond extensions

ABAP foundations

Practice explaining data structures and transactional correctness before discussing frameworks. These answers distinguish internal memory, database persistence, and application locks. Classic update-task examples apply to supported Standard ABAP environments; ABAP Cloud and RAP impose additional API and transaction restrictions.

Find key 50: three internal-table access paths

The same seven records; choose the table and key for the actual access pattern.

STANDARD

Linear free-key example

Row	Key	Check
1	30	1
2	70	2
3	10	3
4	20	4
5	50	match
6	40	
7	60	

Inspect rows until the match.

SORTED

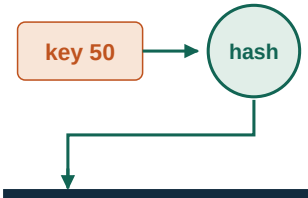
Ordered key lookup

Row	Key	Probe
1	10	
2	20	
3	30	
4	40	1st
5	50	3rd
6	60	2nd
7	70	

Illustrative probes: 40, 60, 50.

HASHED

Complete unique key



Bucket	Record key
...	...
h(50)	50 [match]
...	...

Bucket route is conceptual.
No primary index access.

Also consider secondary keys, inserts, memory, and range access.

Not a timing benchmark: sorted probes and hash buckets illustrate the lookup idea.

A free-key linear search, an ordered lookup, and a complete hashed-key lookup find the same record through different access paths.

See the visual atlas for the recall prompt, limits and related questions.

ABAP-01 How do you choose between standard, sorted, and hashed internal tables?

I choose the table by how I will read it. A standard table is useful for processing rows in sequence or by position. A sorted table keeps rows in key order and helps with key ranges. A hashed table is useful for finding one row by its complete unique key. I also consider how often rows change and how much memory the table needs.

Reasoning and example

Imagine importing orders from a file. If I must report errors in file order, I can keep the rows in a standard table. If I repeatedly find one order by its unique ID, a hashed lookup table can avoid searching every row. If I process all orders for one customer together, a sorted key starting with customer is useful.

The important question is which lookup the program actually performs. Searching a hashed table by a different field does not receive the same benefit as using its complete hash key. I test realistic volumes and include the cost of inserting and changing rows, not only reading them.

Avoid this mistake A hashed primary key is unique; that does not mean every field value must be unique or that arbitrary partial-key searches are constant time.

Read and verify this answer

SAP internal table reference and examples

ABAP-02 What are primary, secondary, default, and empty internal table keys?

The primary key is the main key in the table definition. A secondary key gives another way to find or order its rows. A default key is chosen from the row type, so it may leave out fields I need. An empty key says that I do not need a primary field-based key for this standard table. Secondary keys cost memory and extra work when data changes.

Reasoning and example

An import table may need to keep file order while also supporting repeated lookups by customer. I can use a standard table with an empty primary key and a sorted secondary key for customer. The rows still have positions in the standard table; an empty key does not remove that index.

When reading by customer, I select the intended secondary key explicitly where required. I check whether faster reads justify maintaining another key during imports and changes. This key belongs to the internal table in ABAP memory. It does not create a database index or make a database query faster.

DELETE ADJACENT DUPLICATES removes only neighboring matches. First SORT by the intended duplicate criteria and any rule for which row to retain, then specify those criteria explicitly with COMPARING. For example, sorting by customer and changed_at descending before COMPARING customer keeps the newest row per customer, provided ties have a defined rule. This deliberately changes the original order.

Avoid this mistake An internal table key is not a database index, and an empty key does not remove the standard table's index.

Read and verify this answer

SAP internal table reference and examples

ABAP-03 When is BINARY SEARCH correct, and how do table expressions handle missing rows?

BINARY SEARCH works only when a standard table is sorted in ascending order that matches the search fields. Otherwise it can miss an existing row. A normal table expression raises CX_SY_ITAB_LINE_NOT_FOUND when no row matches. I decide whether a missing row is expected and handle it accordingly. For an existence check only, I can use line_exists.

Reasoning and example

Suppose I sort orders by customer and order ID. A binary search must use fields in an order compatible with that sort. Sorting by order ID alone and then searching by customer is not safe. A sorted table can make the intended order harder to break accidentally.

For an optional description, VALUE with OPTIONAL can return an initial value when the row is missing. For a required order, that would hide an error, so I report the missing order instead. When I need the row itself, I normally do one lookup and handle its outcome rather than check existence and repeat the same lookup. line_exists does not set sy-subrc or sy-tabix. Use its boolean result directly; do not inspect those system fields as if READ TABLE had run.

Avoid this mistake Adding BINARY SEARCH to an unsorted table can return incorrect results; modern table-expression syntax does not automatically optimize an unsuitable key.

Read and verify this answer

SAP internal table reference and examples

ABAP-04 How do a work area, field symbol, and data reference differ?

INTO copies a table row into a work area. Changing that work area alone does not change the table row. ASSIGNING gives a field symbol that points at the row, so changes go straight to it. A data reference stores a reference to a data object that I access explicitly. I check that a field symbol is assigned or a reference is bound before using it.

Reasoning and example

To mark imported orders as checked, I can loop over lt_orders ASSIGNING <ls_order> and change a non-key status field. The change is immediately visible in the internal table. With INTO ls_order, I only change the work area and need a separate table update if that is intended.

I choose the form that makes ownership clear. Copying a row is useful when I need to calculate without changing it yet. Direct access is useful for deliberate updates. I must still respect key restrictions: changing a protected key field through a field symbol can cause an error rather than safely reorganize the table.

Avoid this mistake Do not change protected table-key components through an alias; key restrictions depend on table category and the access in progress.

Read and verify this answer

SAP internal table reference and examples

SAP ABAP Keyword Documentation

ABAP-05 What is the difference between a domain, data element, structure, and table type?

A domain describes technical properties, such as a value type and length. A data element gives a field its business meaning and labels. A structure groups fields into one row. A table type describes an internal table, including its row type and keys. None of these definitions alone creates stored database records.

Reasoning and example

Suppose buyer ID and supplier ID both use the same character length. They may share a domain but use different data elements so the screen labels and documentation explain the correct meaning. A request structure can then combine an ID, amount and currency. A table type describes a list of those request rows.

Creating a variable with that table type gives the program an internal table in memory. Saving rows requires a separate persistence operation against an appropriate database object or business API. Also, domain fixed values describe allowed values in certain contexts; the application must still validate rules that are not automatically enforced by its input path.

Avoid this mistake A DDIC structure is not a database table, and domain fixed values are not a universal substitute for application validation.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-06 How do a foreign key, check table, and value table differ in classic DDIC?

A check table contains the values that a field is meant to refer to. A DDIC foreign key describes that relationship and supports applicable input checks. I do not assume it is a database constraint that protects every write. A domain value table is only a proposal when defining the relationship. Setting a value table alone does not complete the check.

Reasoning and example

A country field may refer to a country customizing table. Defining the foreign key tells Dictionary-based tools how the fields belong together and can support input validation. That helps a user choose a valid country in supported screens, but a different program can still write invalid data if it bypasses the required application checks.

I therefore inspect the actual write path before relying on the relationship. A service should reject an unknown country according to its business rules. I also keep the terms separate: the foreign key describes a relationship; a database index helps find rows. One does not automatically provide the function of the other.

Avoid this mistake Foreign key does not mean secondary key or secondary index, and a value table alone does not guarantee valid database content.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-07 How do you explain an SAP LUW versus a database LUW?

A database LUW is one database transaction, ending with commit or rollback. An SAP LUW is the whole business change that the application treats as one unit. In classic dialog processing, it can span several database LUWs. SAP locks and the save process help keep that business change consistent. The application or framework that owns the transaction decides when to save.

Reasoning and example

A user opens an order, changes several fields and then presses Save. Classic dialog steps can cause database transaction boundaries before that final business decision. The application cannot simply keep ordinary database locks open while the user thinks. It uses the appropriate SAP enqueue lock and save mechanism to protect the business operation.

This is why a helper method must not commit just because its own work is finished. The larger operation may still fail. In RAP, the framework controls the transaction and save sequence, so classic COMMIT WORK patterns cannot simply be inserted into behavior methods. I first identify who owns the complete save.

Avoid this mistake A database commit caused by a work-process boundary is not permission to split a business operation into independently committed fragments.

Read and verify this answer

SAP LUW reference and examples

ABAP-08 Who should issue COMMIT WORK, and what does AND WAIT guarantee?

The caller responsible for the complete business operation should commit. A helper must not save half the work unexpectedly. In classic update processing, COMMIT WORK AND WAIT waits for high-priority updates: sy-subrc is 0 for success and 4 for failure. Without AND WAIT, sy-subrc is 0 without confirming the asynchronous update result. Neither form waits for every external follow-up.

Reasoning and example

Imagine a process that creates a document and then adds required accounting data. If the document helper commits immediately, a later accounting error can leave an incomplete business result. The helper should follow the agreed transaction rules and let the owner decide when the complete operation is ready to save.

AND WAIT is useful for checking the relevant classic update result, but it does not prove that an email, background job or remote system has finished. In a RAP behavior implementation, I do not add COMMIT WORK or BAPI_TRANSACTION_COMMIT. I check both RAP restrictions and any legacy API, because an internal commit hidden inside that API is also a problem.

Avoid this mistake COMMIT WORK AND WAIT is not a distributed transaction guarantee and does not authorize commits inside RAP handlers.

Read and verify this answer

SAP LUW reference and examples

SAP: COMMIT WORK and update result

ABAP-09 Why do SAP enqueue locks exist in addition to database locks?

A database lock protects database work during a database transaction. An SAP enqueue lock says that a business object is being changed and lets cooperating applications detect conflicts. It can cover a longer business interaction, such as editing a document. Every writer must follow the same locking rules. I also handle errors and cancellation so lock ownership is released or transferred correctly.

Reasoning and example

Two users open the same order. The first obtains the appropriate SAP lock; the second receives a clear conflict instead of silently overwriting the first change. The first user then reads the state, edits it and saves through the agreed transaction process. Cancellation must also end the edit cleanly.

An enqueue lock only works when programs respect it. An unrelated SQL update can ignore that agreement, so the lock is not a physical barrier around every table write. In classic systems I may inspect SM12 for diagnosis. The exact lock scope and update ownership matter; in cloud applications I use the supported APIs and tools for that environment.

For classic generated ENQUEUE/DEQUEUE modules, _SCOPE controls dialog/update ownership; do not assume every lock ends at the same COMMIT. Initial key fields together with initial X_<field> flags can request a generic lock covering multiple records. Check that breadth deliberately. If an operation needs several locks, acquire them in a consistent order across callers and clean up on failure.

Avoid this mistake An enqueue lock does not physically prevent unrelated SQL code from updating a table if that code ignores the locking protocol.

Read and verify this answer

SAP LUW reference and examples

SAP: Function modules for lock requests

ABAP-10 How do you choose and propagate class-based exceptions?

I choose an exception type that tells callers how failure must be handled. CX_STATIC_CHECK makes handling or declaration a syntax-check concern. CX_DYNAMIC_CHECK uses runtime rules when exceptions leave a procedure. CX_NO_CHECK can propagate without a RAISING declaration. I catch an exception where I can recover or explain it better, and keep PREVIOUS when wrapping the original cause.

Reasoning and example

A document reader may fail because a dependency is unavailable. The business service can raise a document-read exception with a useful message while keeping the technical exception in PREVIOUS. Support can then inspect the original cause, and the caller can still tell failure from an ordinary missing document.

I avoid an empty CATCH that makes the program continue as if reading succeeded. I also do not replace an established framework error format without checking callers. All three exception categories can be caught. CX_DYNAMIC_CHECK does not mean an exception may cross any undeclared procedure interface without consequences; its propagation rules still have to be respected.

Avoid this mistake NO_CHECK does not mean an exception cannot be caught; DYNAMIC_CHECK does not mean it can cross any procedure interface without declaration.

Read and verify this answer

SAP exception category and propagation rules

ABAP-11 Which modern ABAP expressions are useful, and when should you avoid them?

Modern expressions can make simple work easier to read. VALUE builds values, CORRESPONDING copies matching fields, COND chooses a value, and CONV converts a type. Inline declarations and string templates also reduce clutter. I use the syntax supported by the target release. If one expression becomes difficult to explain or debug, I split it into named steps.

Reasoning and example

Suppose an import creates a result row from several input fields. A short VALUE expression can show the resulting row in one place. CORRESPONDING is useful when fields have matching names, but I check any mapping and what happens to unmatched fields. CONV performs a technical conversion; it does not prove that a price or business identifier is valid.

For a calculation with several decisions, intermediate variables can explain the work better than a deeply nested COND or REDUCE. I choose readability over the fewest lines. These expressions execute ABAP logic; using newer syntax does not automatically move processing to the database or make the program faster.

Avoid this mistake Modern expression syntax is not synonymous with ABAP SQL and does not automatically improve runtime performance.

Read and verify this answer

SAP internal table reference and examples

SAP ABAP Keyword Documentation

ABAP-12 How do you handle amounts, quantities, dates, and timestamps correctly?

An amount needs its currency, and a quantity needs its unit. I use suitable numeric types and explicit conversion and rounding rules. A date identifies a calendar day; a timestamp identifies an instant when its time basis is known. I keep time-zone context when showing it to users. Display formatting must not silently change calculation rules.

Reasoning and example

An order contains 100 EUR and 100 USD. Adding the numbers to show 200 without a currency conversion is wrong. I either show separate totals or convert using the required rate and date. I also test currencies with different decimal rules and quantities that use different units.

For time, a deadline stored as an instant can be Monday for one user and Sunday for another. I convert it with the intended time zone instead of treating a local clock string as universal. Tests should include daylight-saving changes and rounding boundaries, because those cases often reveal errors that ordinary values hide.

Avoid this mistake A numeric field alone does not explain its currency or unit, and a local date-time string is not an unambiguous global instant.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-13 When would you use RTTI, RTTC, or dynamic ABAP?

RTTI lets a program inspect a type at runtime. RTTC lets it create type descriptions for data whose structure is decided at runtime. These tools help genuinely generic code, such as configurable exports. For ordinary business code with known fields, static types are usually easier to understand and check. Any dynamic field or SQL choice still needs validation.

Reasoning and example

An export tool lets an administrator choose a supported set of columns. It may inspect the source structure and build an output structure with just those columns. Runtime type tools can help because the final shape is not fixed when the program is written. An invoice calculation with a stable list of fields usually does not need that complexity.

I map choices to known fields and reject unsupported names. If dynamic SQL is involved, escaping a value is not enough to approve a table name or a whole condition. Static types remain my default because the compiler, editor and reviewer can see more errors before the program runs.

Avoid this mistake Dynamic programming is not inherently more reusable, and escaping values alone does not validate arbitrary dynamic SQL identifiers or syntax.

Read and verify this answer

SAP ABAP Keyword Documentation

ABAP-14 How would you investigate an ABAP failure without changing production behavior?

I first collect the failed operation, inputs, time, user and system version. Then I use the relevant logs, dump and call stack to find where it failed. I reproduce the problem safely with controlled data. The fix must address the cause and keep authorization, locks and transaction behavior correct. I add a test for the failing case before declaring it fixed.

Reasoning and example

A job fails while the same operation succeeds in a dialog session. I compare the execution user, authorizations, input data and processing context instead of assuming the code path is identical. In a classic system, ST22 may show the failing statement and call stack. Traces and debugging require suitable permission and a limited scope.

Once I can explain the failure, I test the smallest useful correction and the surrounding success path. Catching every exception or granting broad access may remove the visible error while creating a worse problem. If evidence only suggests a cause, I say it is a hypothesis and identify the next check that can confirm it.

Avoid this mistake Suppressing an exception or granting broad authorization may hide the symptom while leaving the defect and data-integrity risk intact.

Read and verify this answer

SAP ABAP Keyword Documentation

Official references

SAP internal table reference and examples

SAP LUW reference and examples

SAP exception category and propagation rules

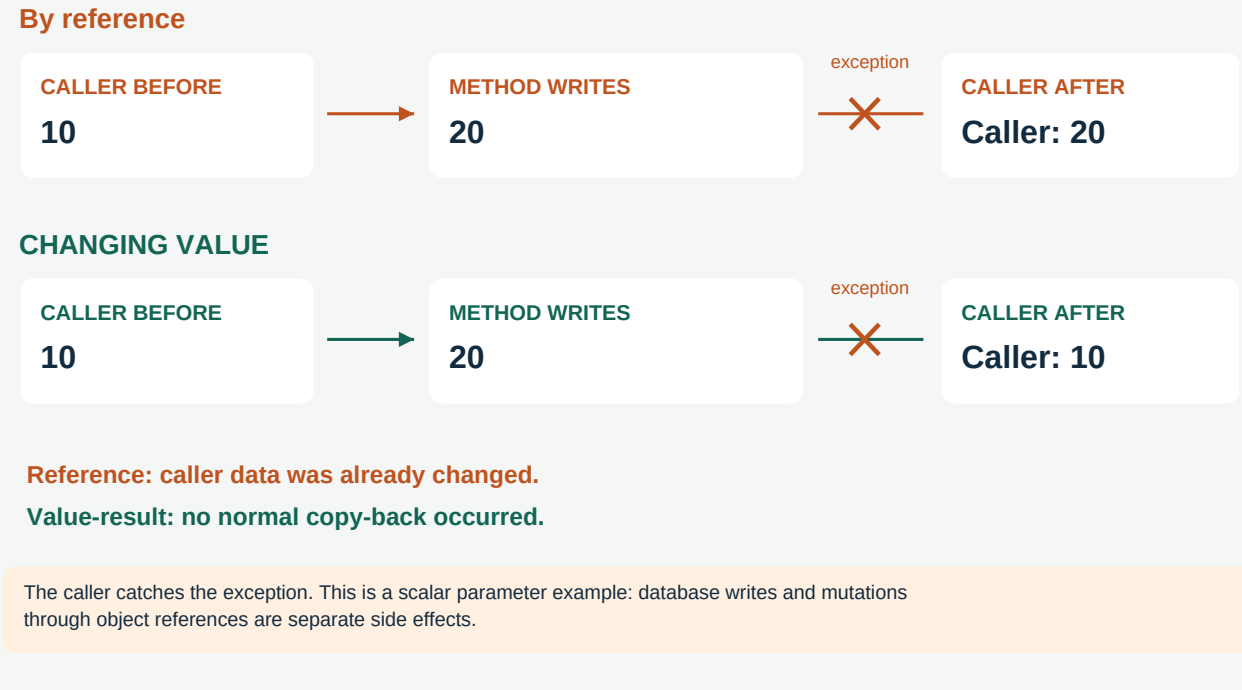
SAP ABAP Keyword Documentation

Practical ABAP: values, expressions, and failure paths

Use these worked scenarios after the basic language questions. They test the consequences of parameter passing, aliasing, partial updates, and failure handling rather than syntax recall alone. Modern additions require a supporting target release and language version; examples are teaching scenarios, not executed SAP evidence.

An exception separates reference and value-result paths

A plain CHANGING integer starts at 10; the method sets 20, then raises an exception



The caller catches the exception. This is a scalar parameter example: database writes and mutations through object references are separate side effects.

Compare the same assignment under two transfer modes. The final caller value is visible evidence of the difference.

See the visual atlas for the recall prompt, limits and related questions.

BASE-01 What changes when a method receives a CHANGING parameter by value instead of by reference?

By reference, the method works on the caller's data directly. With VALUE on a CHANGING parameter, it works on a local value and copies it back when the method finishes normally. If an exception leaves the method, that normal copy-back does not happen. This can protect the caller's value, but it does not roll back database writes or changes to shared objects.

Reasoning and example

Start with a plain integer containing 10. A method changes its CHANGING parameter to 20 and then raises an exception that leaves the method. With reference passing, the caller can already see 20. With value passing, the caller retains 10 because the local result was not copied back. On normal completion, both forms can return the changed value.

That difference is important when changing an existing method signature. Tests must cover both successful completion and escaping exceptions. If the value contains an object reference, changing the referenced object is a separate effect: the copy still points to that object. VALUE is not a general undo mechanism for everything the method does.

Avoid this mistake Pass-by-value is not a transaction rollback and does not clone objects behind reference parameters.

Interviewer probes How would this change with an object reference parameter? / Can an IMPORTING reference variable still expose a mutable object?

Strong answer criteria Separates input/output direction from value/reference passing; Explains when a local value is copied back; Identifies effects on the database and shared objects

Verification evidence A small method-call test showing caller values after normal return and exception.

Read and verify this answer

SAP: Pass by value

BASE-02 A copied structure contains a string, a nested table, and an object reference. Is it an independent snapshot?

Not completely. The copied string and nested internal table behave as independent values when changed. A copied object reference still points to the same object. Changing that object through either structure can therefore affect what the other sees. For a true snapshot, I decide how to handle every mutable referenced object instead of assuming ordinary assignment clones it.

Reasoning and example

Suppose ls_request contains a note, an item table and a reference to a pricing object. I copy it to keep a before-change version. Editing the copied note or item values does not change the original value components. But calling a setter on the copied pricing reference changes the same pricing object used by the original structure.

If the snapshot must preserve the old price, I store the calculated price as a value, use an immutable object or implement a deliberate copy of the required object state. A full clone needs rules for shared identities and cycles. Runtime memory-sharing optimizations do not change these value-versus-reference rules; a deep ABAP type is not automatically a deep object clone.

Avoid this mistake A deep ABAP type is not a promise that ordinary assignment recursively clones the entire object graph.

Interviewer probes Which components need copying for an audit snapshot? / How would you test sharing without inspecting memory addresses?

Strong answer criteria Separates copied values from copied references; Explains why both references still point to one object; Defines what a snapshot must preserve

Verification evidence Mutation tests for a copied string, nested row, and referenced object's attribute.

Read and verify this answer

SAP: Sharing between dynamic data objects

BASE-03 A large temporary internal table is no longer needed. When would you choose CLEAR or FREE?

For an internal table without a header line, CLEAR and FREE both remove the rows. CLEAR keeps the initial memory allocation available; FREE releases that allocation too. I use CLEAR when I expect to refill the table soon, and FREE when it is finished for now. Neither removes the variable declaration or guarantees that the operating system immediately shows lower process memory.

Reasoning and example

A package loop fills a small work table, processes it and starts the next package. CLEAR is often a sensible reset because the table will be used again. Repeatedly freeing and allocating it may add unnecessary work. After a one-off large reconciliation, FREE may be appropriate when the table will no longer be needed.

I distinguish memory used by current rows from memory retained for future use, and measure in the relevant ABAP session. CLEAR does not promise to retain every byte the table ever needed. FREE also does not destroy an object that another reference still reaches. A single process-memory number cannot explain all of those separate lifetimes.

Avoid this mistake Do not claim CLEAR retains every byte ever allocated, or that FREE destroys referenced objects still reachable elsewhere.

Read and verify this answer

SAP: CLEAR

BASE-04 A dynamic ASSIGN fails, but IS ASSIGNED is still true. How can the program update the wrong field?

The field symbol may still point to the previous successful target after a failed dynamic ASSIGN. IS ASSIGNED only says it points somewhere; it does not prove the latest lookup worked. I check the ASSIGN result immediately. Where supported, ELSE UNASSIGN makes failure clear. Otherwise I unassign before the lookup and handle failure before writing through the field symbol.

Reasoning and example

A generic mapper first assigns component PRICE and writes an amount. It then tries the misspelled component PRCIE using the same field symbol. If that lookup fails but the old assignment remains, checking only IS ASSIGNED can allow the next write to change PRICE again. The program has updated a real field, but the wrong one.

I test the sequence of one successful lookup followed by one failed lookup. An isolated failure test can miss the stale assignment. The mapper should reject the unknown component or follow a clearly defined skip rule, and it must not write through the old alias. I verify release support before using ELSE UNASSIGN syntax.

Avoid this mistake An assigned field symbol is not evidence that the most recent dynamic ASSIGN succeeded.

Read and verify this answer

SAP: ASSIGN, ELSE UNASSIGN

BASE-05 A READ TABLE fails, yet a later IF sy-subrc = 0 takes the success branch. What do you inspect?

sy-subrc holds the result of the most recent statement that set it. It is not permanently attached to READ TABLE. Another statement between the read and the IF may have replaced the value. I check it immediately or save it in a named variable. Table expressions have their own exception or predicate behavior, so I do not assume they work like READ TABLE.

Reasoning and example

A READ TABLE does not find an order. Before checking the result, the program uses FIND to prepare a log message, and FIND succeeds. The later IF sy-subrc = 0 now checks the FIND result, so it enters the success branch even though the order lookup failed.

I move the decision directly after the read or preserve the read result before doing other work. The regression case includes a missing order and successful log preparation, because that is the sequence that caused the bug. Not every statement changes sy-subrc, so I inspect the actual intervening statements rather than guess. A modern table expression must be handled according to its documented missing-row behavior.

Avoid this mistake Not every statement resets sy-subrc, and a table expression is not a drop-in sy-subrc-producing READ TABLE.

Read and verify this answer

SAP: SY-SUBRC

BASE-06 An import must reject precision loss rather than silently round or truncate. Where does EXACT help?

EXACT checks that an assignment or calculation can produce its target value without losing information. It raises an appropriate exception when that is impossible. CONV uses normal conversion rules and is not the same check. I catch conversion failures at the input boundary and explain them to the caller. I still validate business rules, because an exact value can still be forbidden.

Reasoning and example

An import receives a text value longer than the fixed-length target field. A normal conversion may discard characters, while a lossless conversion can reject that input. For a packed result with two decimals, a calculation such as one divided by three cannot be represented exactly, so EXACT is useful if rounding is prohibited.

If the business actually requires rounding, I apply the agreed rounding rule explicitly instead of treating every inexact result as invalid. I also check rules such as a positive amount, a known currency or an allowed identifier. EXACT can detect loss of technical information; it cannot decide whether a perfectly representable negative amount is acceptable for this document.

Avoid this mistake EXACT does not prove that a technically representable value is an allowed business value.

Read and verify this answer

SAP: EXACT examples

BASE-07 Two invoice totals differ by one cent although both use decimal types. How do you diagnose rounding?

I check where rounding happens, which rounding mode is required and the precision of intermediate values. Rounding each line and then adding can differ from adding first and rounding the total. Decimal types do not decide that business rule for me. I keep the currency context and write tests for values near the rounding boundary.

Reasoning and example

Assume the agreed rule is half-up rounding to two decimals. Two positive line results are each 0.005. If each line is rounded first, each becomes 0.01 and the total is 0.02. If I add the precise values first, the sum is 0.010 and rounds to 0.01. Both calculations follow their chosen order, but only the specified order is correct for the document.

I compare the implementation with that rule, including intermediate assignments that may already round a value. I test multiple small lines, negative amounts where allowed and currency-specific requirements. Changing the display to two decimals or choosing decfloat34 does not by itself settle when or how the business wants rounding.

Avoid this mistake Choosing decfloat34 or formatting two decimals does not settle the business rounding policy.

Interviewer probes How do negative half-way values affect the selected mode? / Where is currency conversion applied relative to tax rounding?

Strong answer criteria Checks precision at each calculation step; States when and how rounding occurs; Keeps the correct currency and conversion rules

Verification evidence Worked positive/negative boundary examples with independently calculated expected totals.

Read and verify this answer

SAP: Rounding function ROUND

BASE-08 Why can replacing MOVE-CORRESPONDING with CORRESPONDING erase an existing structure field?

Plain CORRESPONDING builds a new result. Fields missing from the source therefore become initial in that result. MOVE-CORRESPONDING between structures keeps unmatched fields already in the target. A CORRESPONDING expression with BASE can start from existing values, but matching source fields still overwrite them, even with initial values. I choose the form based on the intended update behavior.

Reasoning and example

The existing request has status Approved and a note. The incoming structure contains only a new note. MOVE-CORRESPONDING updates the note and leaves the status. Assigning a plain CORRESPONDING result replaces the whole target with a new value, so the unmatched status becomes initial. Supplying the existing structure as BASE preserves that unmatched status.

Now consider a source type that also contains status, but with a blank value. Status matches by name, so it can overwrite the BASE value with blank. BASE is not an automatic ignore-empty or PATCH rule. If an API must distinguish an omitted field from an explicitly cleared field, I define and test field-presence handling separately.

Avoid this mistake BASE preserves unmatched components; it does not mean ignore every initial source value or implement PATCH automatically.

Read and verify this answer

SAP: Working with structures

BASE-09 Unsorted orders must be processed once per customer with their member rows. How does GROUP BY change the loop?

LOOP AT with GROUP BY collects rows with equal group keys even when they are not adjacent. The outer loop then runs once per group. LOOP AT GROUP can process that group's members. This avoids relying on the input being sorted for traditional control-level processing. I still define the required group and member order rather than assume grouping provides the business sort order.

Reasoning and example

The input customer sequence is A, B, A. Grouping by customer creates two groups, and both A orders belong together despite being separated in the input. The group loop runs twice. Inside the A group, a member loop can calculate its total or process each of its orders.

I use an explicit group key when it makes the code easier to follow. If I request WITHOUT MEMBERS because only distinct customers are needed, I cannot later loop over the missing group members. This processing happens on an internal table in ABAP memory, not as database aggregation. If the output must be customer-sorted, I express and test that ordering instead of relying on encounter order.

Avoid this mistake Grouping an internal table is not database aggregation, and its default group order is not automatically business sort order.

Read and verify this answer

SAP: LOOP AT itab, GROUP BY

BASE-10 A FILTER expression is proposed for a standard table with an empty primary key. Is that enough?

An empty primary key alone is not enough for the basic FILTER form. It needs a suitable sorted or hashed key. A standard table can work if it has a suitable secondary key selected with USING KEY. The condition must also follow the chosen key rules. For an arbitrary condition, a FOR expression or a normal loop may be clearer.

Reasoning and example

A standard table keeps orders in arrival order but also has a sorted secondary key starting with customer. A key-based FILTER can use that secondary key to select a customer according to the supported predicate rules. The table does not need to lose its arrival-order use just to gain another access path.

A condition combining unrelated fields with a complex OR is a different problem. I do not force it into FILTER just because the syntax looks shorter. A straightforward loop or suitable FOR expression can express the intended rule more clearly. I check the actual key definition and target-release syntax, then measure if performance matters; the keyword alone does not prove a faster implementation.

Avoid this mistake FILTER is not a general-purpose synonym for every WHERE expression supported by LOOP or FOR.

Read and verify this answer

SAP: FILTER, basic form

BASE-11 A REDUCE expression calculates a maximum, but all-negative input returns zero. What went wrong?

The initial accumulator value is wrong for this input. Starting a maximum at zero makes zero a candidate, even if every real value is negative. I start from a valid member or represent no result explicitly. I also define what empty input means. REDUCE makes the loop concise, but it does not choose the correct starting value or numeric type for me.

Reasoning and example

For values -8 and -3, the maximum is -3. An accumulator initialized to zero stays at zero because neither input is larger. The result looks like a valid number but does not occur in the input and is not its maximum. Initializing from a valid first member avoids that invented candidate.

An empty table needs a separate decision: perhaps return an optional result, or raise a meaningful exception if at least one value is required. I test empty, all-negative, mixed and single-value inputs. I also check intermediate type limits and rounding. A reviewer should be able to see initialization, comparison and update clearly; if the expression hides them, I use a named step or loop.

Avoid this mistake REDUCE does not supply mathematically correct initial values or empty-set semantics on the developer's behalf.

Read and verify this answer

SAP: Table reductions, structured result

BASE-12 A report accepts a filter value and a sort-column name. Is escaping quotes sufficient for safe SQL?

No. A filter value and a column name are different kinds of input. I pass values through host variables in static ABAP SQL where possible. For a configurable column or dynamic clause, I map approved choices to known SQL elements and reject anything else. Escaping quotes in a value does not make arbitrary SQL text safe or authorized.

Reasoning and example

A customer name containing an apostrophe should be treated as a value, not assembled into executable SQL text. A requested sort option is different: I map an accepted choice such as newest to a known timestamp column and direction. I do not accept any string and insert it as ORDER BY syntax.

Mandatory authorization restrictions are built separately from optional user filters. The user must not be able to remove them by changing a filter or sort parameter. I test unknown columns, invalid directions and inputs containing SQL-like text, as well as ordinary names. Safe quoting protects one kind of value; it does not approve table names, operators, whole conditions or access to restricted data.

Avoid this mistake Input that is safely quoted as a literal is not therefore safe as SQL syntax or an authorized data access.

Interviewer probes How would you test an input that is valid syntax but unauthorized? / Which parts of the SQL shape can remain completely static?

Strong answer criteria Separates data values from SQL names and syntax; Maps choices to explicitly allowed SQL elements; Keeps mandatory authorization conditions in place

Verification evidence Negative tests for quoted values, forbidden columns, injected operators, and unauthorized records.

Read and verify this answer

SAP: Prevent SQL injection

BASE-13 A resource release is placed only in CLEANUP. Why can it remain unreleased after success?

CLEANUP is not a finally block that runs after every TRY. It is used during the relevant exception propagation to an outer handler. It does not run just because normal processing reaches ENDTRY. I arrange release on success and on the required error paths. I also check who owns the resource so I do not release framework-owned locks too early.

Reasoning and example

A method opens an application-owned temporary handle and processes data successfully. If release exists only in CLEANUP, reaching ENDTRY normally leaves that handle open. An exception caught inside the same TRY also does not make CLEANUP a universal after-CATCH step. I therefore make the successful release and exceptional release paths explicit.

I test normal completion, a locally handled exception and an exception that leaves for an outer handler. Release can fail too, so I handle that without hiding the original error or letting an exception escape CLEANUP. The same reasoning applies to lock ownership: cleanup must respect the intended lifetime, not remove a framework lock while the business transaction still needs it.

Avoid this mistake Do not assume CLEANUP automatically runs after a CATCH in the same TRY, or release framework-owned locks prematurely.

Interviewer probes What changes when the exception is caught inside the same TRY? / Who owns release when a caller supplied the resource?

Strong answer criteria Explains why CLEANUP is not a general finally block; Tests successful and failed execution; Releases only resources owned by this code

Verification evidence Instrumented resource tests show one correct release on every intended exit.

Read and verify this answer

SAP: Exception system response

BASE-14 An optional limit parameter is omitted in one call and explicitly set to zero in another. How can a method distinguish them?

IS SUPPLIED checks whether the caller passed the parameter. IS INITIAL checks its value. Those are different: an omitted parameter and an explicit zero can both contain zero. I use presence only when omission has a defined meaning in the API. A wrapper must preserve that meaning, because forwarding its own initial variable makes the downstream parameter supplied.

Reasoning and example

A helper treats an omitted limit as use the configured default, while a supplied zero means return no rows. Checking IS INITIAL cannot distinguish these calls. IS SUPPLIED can, so the helper first decides which meaning the caller requested and then applies the value rule.

A wrapper introduces another risk. If it always forwards its own optional parameter variable, the next method sees a supplied argument even when the original caller omitted it. I test the direct calls and the wrapper path, and forward conditionally or pass explicit presence information where appropriate. OPTIONAL permits omission; it does not automatically carry that omission through every later method call.

Avoid this mistake OPTIONAL does not automatically preserve absent-versus-present semantics across another method boundary.

Read and verify this answer

SAP: IS SUPPLIED

BASE-15 A single-line INSERT INTO TABLE hits an existing unique primary key. Does it update the existing row?

No. In this single-row internal-table INSERT case, a duplicate unique primary key prevents insertion and is reported through sy-subrc. The existing row is not updated. I decide whether duplicates should be rejected, merged or replaced and implement that decision explicitly. Other insertion forms and secondary-key conflicts can behave differently, so I do not generalize this return-code rule.

Reasoning and example

An import contains order 7 twice, first with amount 100 and then with amount 200. Inserting both into a hashed table with a unique order key does not replace 100 with 200 automatically. If the program ignores the failed insertion, it may silently keep the first row and hide conflicting input.

I define the import rule before coding. Identical duplicates might be acceptable, in which case I compare the relevant values. Conflicting duplicates should produce a useful error with the order and input position, unless a deliberate replacement rule exists. I check sy-subrc immediately for this operation and test both cases. Uniqueness prevents duplicate keys; it does not explain which business value should win.

Avoid this mistake Uniqueness enforcement is not a complete deduplication policy, and INSERT is not an automatic upsert.

Read and verify this answer

SAP: Populating internal tables

BASE-16 An authorization helper returns an empty range table. Why is WHERE field IN range unsafe as the only access restriction?

An empty ABAP selection range means no restriction for IN. It does not mean the user is allowed no values. If an authorization lookup returns no allowed plants, I return no data or reject the request before querying. I keep mandatory access restrictions separate from optional user filters. Nonempty ranges also have inclusion and exclusion rules, not just list membership.

Reasoning and example

A user has no permission for any plant. The helper returns an empty range, and a query uses WHERE plant IN that range as its only access check. The initial range does not narrow the selection, so the query can expose records the user should never see. The program must handle the no-permission result explicitly first.

I test a user with no allowed plants, one allowed plant and relevant exclusions. I also test that adding or omitting a user search filter cannot remove the mandatory restriction. An empty optional filter may correctly mean show all permitted data; an empty authorization result means no permitted data. They must not share an ambiguous representation without an explicit check.

Avoid this mistake An empty ABAP selection range is not the same as an SQL list containing zero allowed values.

Interviewer probes How do you distinguish an optional filter from mandatory authorization? / What result do you expect from a range containing only exclusions?

Strong answer criteria Explains why an empty range does not restrict IN; Returns no data when there are no permissions; Separates optional search filters from required access checks

Verification evidence A denied-user test proves zero data and zero aggregate leakage with an empty authorization result.

Read and verify this answer

SAP: Comparison with selection tables

Official references

SAP: Pass by value

SAP: Sharing between dynamic data objects

SAP: CLEAR

SAP: ASSIGN, ELSE UNASSIGN

SAP: SY-SUBRC

SAP: EXACT examples

SAP: Rounding function ROUND

SAP: Working with structures

SAP: LOOP AT itab, GROUP BY

SAP: FILTER, basic form

SAP: Table reductions, structured result

SAP: Prevent SQL injection

SAP: Exception system response

SAP: IS SUPPLIED

SAP: Populating internal tables

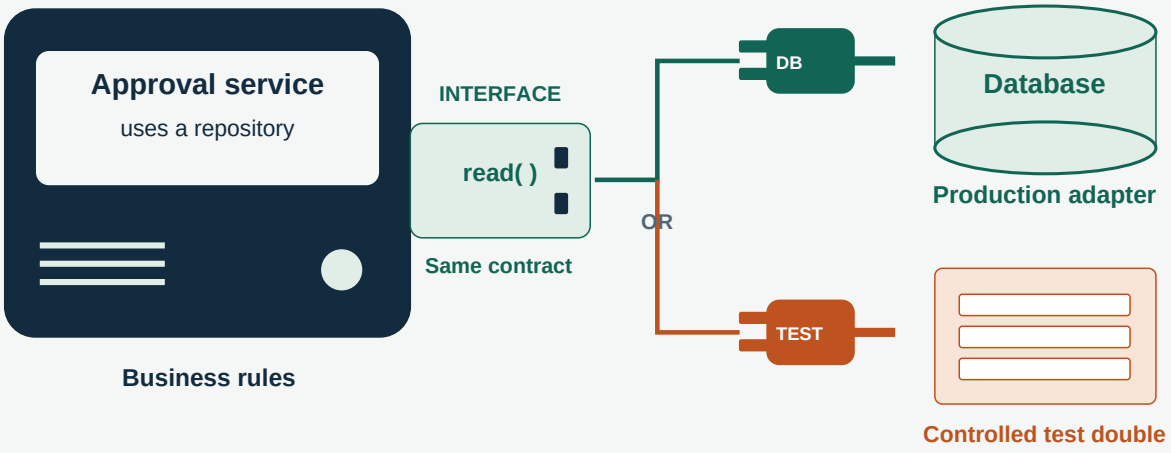
SAP: Comparison with selection tables

OO design and Clean ABAP

Explain object orientation through responsibilities, replaceable dependencies, and observable behavior. Memorized definitions become useful when you can name a concrete tradeoff. The examples are hypothetical design exercises, not claims about personal project experience; project naming conventions are separate from the SAP language.

One contract; production and test can plug in

Constructor injection supplies a collaborator. Polymorphism selects its implementation at runtime.



A test checks service behavior; a real integration check still verifies database behavior.

Inject one suitable implementation per instance. An adapter does not make an unreleased API released.

The service depends on a repository contract. Production and test implementations fit the same socket; the composition boundary supplies the appropriate collaborator.

See the visual atlas for the recall prompt, limits and related questions.

OO-01 Explain abstraction, encapsulation, inheritance, and polymorphism with one example.

Abstraction gives callers a simple operation, such as send, without showing every technical step. Encapsulation keeps an object in a valid state by controlling access to its data. Inheritance lets a subclass specialize a base class while keeping its promises. Polymorphism lets the same call use different implementations, such as an email sender or a test sender.

Reasoning and example

An approval coordinator needs to notify a user. It calls send through a notification interface and supplies a message. The production implementation sends email, while a test implementation records the message for an assertion. The coordinator does not need to know which concrete class it received.

The sender protects its own valid setup, such as a required destination, rather than exposing every attribute for arbitrary changes. If inheritance is used, a subclass must still behave as a valid sender. These ideas work together: callers see a small useful operation, implementation details stay inside the sender, and a new delivery method can be added without rewriting the approval decision.

Avoid this mistake Encapsulation means protecting valid behavior and state, not merely generating getters and setters for every attribute.

Read and verify this answer

SAP ABAP object orientation examples

OO-02 What can an ABAP interface contain, and when would you use one?

An ABAP interface describes what an implementing class provides. It can contain methods, types, constants, attributes and events, not just method names. I use a small interface when callers need one clear capability and different implementations can provide it. This also lets a test supply a controlled replacement. Specialized interfaces, such as BAdI interfaces, can have additional rules.

Reasoning and example

A checkout needs a price, so its price-provider interface can define how to request one and what happens when no price is available. One class reads local data; another calls an external service. The checkout uses the same operation, while the implementations keep their technical work private.

The contract must include behavior as well as types. Returning an empty price on network failure is different from reporting that no price exists. I avoid putting unrelated operations into the interface simply because one concrete class already has them. A narrow interface makes both the caller and its test double easier to understand, and it reduces the effect of future changes.

Avoid this mistake An interface is not an abstract class, and a collection of similarly named methods alone does not establish a meaningful common contract.

Read and verify this answer

SAP interfaces and implementation semantics

OO-03 How do ABSTRACT and FINAL affect classes and methods?

An abstract class cannot be created directly, but it may contain working shared code. It does not have to contain an abstract method. An abstract method leaves implementation to a subclass before that subclass can be concrete. FINAL on a class prevents subclasses. FINAL on a method prevents further redefinition of that method.

Reasoning and example

A document-processing base class could provide common logging while leaving a processing step abstract. A concrete subclass implements that step and can then be instantiated if no abstract methods remain. This is useful only when the shared process and the allowed variation are deliberate parts of the design.

For a service that should not be specialized through inheritance, FINAL makes that limit explicit. It can still implement interfaces and receive other objects as dependencies. I follow ABAP rules rather than copying assumptions from another language: redefining an inherited method preserves its interface, and ABAP does not solve this with same-name methods that have different parameter lists.

Avoid this mistake Do not import another language's virtual keyword or method-overloading rules into ABAP; use ABAP's actual redefinition and signature rules.

Read and verify this answer

SAP ABAP object orientation examples

OO-04 How do upcasting and downcasting work in ABAP Objects?

Upcasting uses a general reference type for an object, such as its superclass or interface. Downcasting asks to use a more specific type. Neither creates a new object. A downcast fails if the actual object does not support the requested type, so I check it or handle CX_SY_MOVE_CAST_ERROR. Frequent downcasts can mean the caller needs a better interface.

Reasoning and example

A payment-interface reference can point to a card-payment object. Through that reference, I can use operations declared by the interface. If I really need a card-specific operation, a valid downcast exposes it, but the reference still points to the same object with the same state.

If the reference actually points to a bank-transfer implementation, casting it to the card class cannot turn it into a card. The cast fails. I test that path when a cast is necessary. More often, if ordinary payment processing keeps checking concrete classes, I move the shared capability into a suitable interface so callers can work without knowing every implementation.

Avoid this mistake Casting a reference changes the accessible static view, not the object's dynamic type or stored data.

Read and verify this answer

SAP ABAP object orientation examples

OO-05 What is the difference between instance and static methods and constructors?

An instance method works with a particular object and its data. A static method belongs to the class and has no current object of its own. The instance constructor prepares a newly created object. The class constructor prepares class-level state when the runtime first needs the class in that internal session. I keep that hidden initialization small and predictable.

Reasoning and example

Two approval-service objects can each hold a different repository reference. Their instance methods use their own repository. A static helper, by contrast, cannot assume that either object is its current instance. Class-level data can be shared by those instances within the internal session, so changing it may affect both.

I use the instance constructor to receive required dependencies and establish a usable object. I avoid loading large datasets or calling remote systems unexpectedly from a class constructor. Also, declaring CLASS-METHODS does not itself prevent object creation. Creation visibility or an abstract class controls that, and class data is not automatically one shared value across the whole SAP system.

Avoid this mistake ABAP does not make a class uninstantiable simply because it has static methods, and class data is not automatically shared system-wide.

Read and verify this answer

SAP ABAP object orientation examples

OO-06 When do you prefer composition over inheritance?

I prefer composition when one object needs help from another: the service has a repository or a formatter. Those helpers can change independently. Inheritance fits a real specialization where the subclass keeps the base class promises. I do not build a class hierarchy just to reuse a few methods. Composition often makes testing and replacing a dependency simpler.

Reasoning and example

A report service reads data, checks access and formats output. It can receive a repository, an authorization policy and a formatter as separate helpers. A test replaces the repository while using the real calculation. A new output format does not require a new version of the entire report hierarchy.

A large base report class may instead force every subclass to inherit database, output and security decisions it does not need. That makes a small change affect unrelated reports. I use inheritance when a shared process and its extension steps are genuinely stable. The practical test is whether a subclass can be used wherever the base type is expected without surprising its caller.

Avoid this mistake Code reuse alone is weak justification for inheritance; a subclass that violates caller expectations is not a sound substitute.

Read and verify this answer

SAP Clean ABAP style guide

SAP ABAP object orientation examples

OO-07 How do dependency injection and dependency inversion differ?

Dependency inversion means business code depends on the capability it needs, usually an interface, rather than one technical implementation. Dependency injection means someone else supplies that implementation, often through the constructor. The first is a design rule; the second is a way to connect objects. Together they make dependencies visible and easier to replace in tests.

Reasoning and example

An approval service needs to read a request. It accepts a repository interface in its constructor instead of creating a database repository inside approve. Production setup supplies the real repository; a unit test supplies a fake that returns a known request. The approval rule itself runs unchanged in both cases.

Passing an object is not enough if the service ignores it and creates another dependency internally. Also, injecting a large concrete database class can still leave business code dependent on database details. I keep the required operations small and meaningful, and test both successful reads and failures through that interface. This reveals what the service really relies on.

Avoid this mistake Injecting a concrete class everywhere may aid construction while still leaving the business layer tightly coupled to infrastructure.

Read and verify this answer

SAP Clean ABAP style guide

OO-08 How would you apply SOLID without overengineering?

I use SOLID to keep changes understandable. A class should have a clear job, and new variations should not require editing every caller. Replacements should keep the same promises, interfaces should contain what their callers need, and business rules should depend on useful abstractions. I add structure for a real need. More classes do not automatically mean better design.

Reasoning and example

A tax calculation mixed with database reads and printing is difficult to test. Separating the calculation lets me give it known values and check the result without a printer or live database. A narrow rate-provider interface may help if rates can come from different places or the source can fail.

That does not mean every five-line helper needs its own interface and factory. I ask which change or test becomes easier because of each extra layer. Single responsibility does not mean exactly one method per class. Open/closed also does not forbid fixing existing code; it encourages a design where expected variations can be added without repeatedly disrupting stable business logic.

Avoid this mistake Single responsibility does not mean one method per class, and open/closed does not mean existing code must never be changed.

Read and verify this answer

SAP Clean ABAP style guide

OO-09 What is a Factory, when do you use it, and how does it work with Strategy?

A Factory creates the right object for the caller. Strategy is the object that contains one version of a rule. For example, a Factory chooses a domestic or export price calculator; both expose the same calculate method. Use a Factory when choosing or setting up objects is repeated or complicated. For one simple class, NEW at the entry point may be enough.

Reasoning and example

Imagine three reports need prices for domestic and export orders. Without a Factory, every report repeats the same IF and knows both concrete class names. Move that choice into a create_calculator method returning the calculator interface. A report then asks the returned object to calculate; it does not need to know which class was created. An unknown country must give a clear error, not silently choose a price rule.

Strategy separates the calculations. Factory separates object creation. The Factory can return a new object each time; it does not imply Singleton. Inject the resulting interface into the business service so a unit test can supply a fake calculator.

A static method with CASE is commonly called a simple Factory. The formal Factory Method pattern lets subclasses decide which product to create. Abstract Factory creates related families, such as a serializer and sender for the same provider. Choose the simplest variant that solves the actual problem.

Avoid this mistake Do not call every method that returns an object Factory Method without distinguishing a simple factory from the formal pattern.

Read and verify this answer

SAP Learning: Implementing Factory Methods

SAP examples: Factory Method and Abstract Factory

OO-10 What is a Facade, when is it useful, and how does it differ from Adapter?

A Facade gives a complicated subsystem a small, easy-to-use interface. Instead of making every caller run prepare, check and submit, expose one submit_request operation. An Adapter solves a different problem: it converts one interface or data format into another. You can use both. Neither pattern changes the transaction rules of the code it calls.

Reasoning and example

Suppose a delivery preview needs three services: read the order, calculate package sizes, and estimate delivery cost. A Facade offers preview_delivery(order_id). It coordinates those services and returns one result. The UI no longer repeats the sequence, and changing the packaging service affects fewer callers. This is useful when several screens or jobs use the same multi-step operation.

If an old carrier API expects technical structures, an Adapter maps your request to those structures. The Facade coordinates the overall task; the Adapter translates one dependency. Keep the dependencies behind interfaces so tests can verify the sequence and stop on a failed check without contacting a carrier.

For a posting Facade, explicitly document who owns COMMIT and what can be rolled back. A Facade is not a new LUW. Do not use it just to rename one trivial method, or hide unreleased SAP dependencies and claim they are now released.

Avoid this mistake Wrapping an unreleased SAP object does not by itself make that underlying dependency released, cloud compliant, or clean-core Level A.

Read and verify this answer

SAP examples: Facade

SAP: a RAP facade over existing BAPIs

OO-11 What is Singleton in ABAP, how do you implement it, and when would you use it?

Singleton lets callers reuse one object instead of creating a new one each time. In ABAP, a typical implementation uses CREATE PRIVATE, a static reference, and get_instance. The first call creates the object; later calls return the same reference. That instance belongs to an internal session, not the whole SAP system. Use it only when sharing that lifetime is intentional.

Reasoning and example

Imagine a report repeatedly needs the same read-only configuration. A Singleton can read it once and reuse the same cache during that internal session. Calling get_instance twice returns the same object. A different user or internal session can have another instance, so this is not a system-wide lock or shared cache.

Do not store the current order, current approver, or other request-specific data in a long-lived Singleton. The next operation can see stale values; one test can affect another test. Define refresh and invalidation rules for configuration too. CREATE PRIVATE only prevents arbitrary construction: the class must also retain and reuse its instance.

If shared lifetime is not needed, create an ordinary service and pass it into the caller through an interface. That is easier to replace in tests. Singleton answers how many instances to reuse; Factory answers how to create an object; Facade answers how to simplify a subsystem.

Avoid this mistake A singleton does not synchronize business updates across sessions, work processes, application servers, or users.

Read and verify this answer

SAP: implementing a Singleton buffer

SAP Learning: controlling object creation

OO-12 How do ABAP OO events differ from durable business events?

An ABAP OO event calls registered handlers in the current runtime context. It is useful for local notifications, such as a UI interaction. A durable business event must survive failures and support delivery after the business change is saved. That needs a supported event or integration mechanism. RAISE EVENT alone does not provide persistence, retry or reliable external delivery.

Reasoning and example

An ALV can raise a local event when the user selects a row. A controller registered with SET HANDLER reacts and updates the display. If that session ends, the local event is not a saved message waiting for another system. This is appropriate for an immediate screen interaction.

Starting an external approval workflow after a document is saved is a different requirement. The design must coordinate save and publication, identify duplicates, monitor failed delivery and support retries. Otherwise the workflow could start for a document that later fails to save, or never start after a temporary outage. Similar event terminology does not make the two delivery mechanisms interchangeable.

Avoid this mistake RAISE EVENT by itself does not persist a message or guarantee workflow creation after a commit.

Read and verify this answer

SAP ABAP object orientation examples

OO-13 What would you change first in a difficult legacy ABAP method?

I first find out what the method promises to its callers and who owns its save. Then I make small changes: clearer names, less duplication and separate calculation from database or remote calls. I protect risky behavior with focused tests before larger changes. The goal is easier understanding without changing the business result accidentally.

Reasoning and example

A long posting method calculates a discount, reads configuration and saves a document. I first extract the discount calculation if its inputs and result can be made clear. I leave commit, authorization and locking behavior in place until I understand their callers and tests. That gives reviewers a small change they can verify.

Next I use tests around important amounts, rejection paths and existing edge cases before moving more code. Merely converting every loop into an expression can obscure the actual change. Team naming also helps consistency: this project uses lt_ for tables, ls_ for structures and co_ for constants. Those are project conventions, not special ABAP syntax or universal Clean ABAP requirements.

Avoid this mistake A broad stylistic rewrite without behavior evidence can introduce regressions and make review harder, even if the result looks more modern.

Read and verify this answer

SAP Clean ABAP style guide

OO-14 How do you make an OO design testable, and what should ABAP Unit assert?

I keep the business rule real and make outside dependencies replaceable. Tests supply known data, time or service responses and check the result a caller can observe. I cover normal cases, limits and failures. Coverage can reveal untested code, but it cannot prove the assertions are useful. Real database, authorization and service behavior still need integration checks.

Reasoning and example

An approval rule sends amounts above a limit to a manager. I test below, exactly at and above that limit, plus the case where no manager is available. A repository double supplies the request, and the test checks the real decision and reported error. The expected result comes from the rule, not from repeating the implementation in the test.

I also simulate a read failure so it cannot become an accidental approval. Separate tests in the target environment verify the actual repository and security behavior. A unit test with fake persistence is valuable, but it cannot prove a real service call, save or role assignment works correctly.

Avoid this mistake A test that only asserts that a method returned, or mirrors its implementation step by step, provides weak evidence.

Read and verify this answer

SAP Clean ABAP style guide

OO-15 How does MVC separate responsibilities?

The Model holds application data and relevant behavior. The View displays information and accepts user input. The Controller handles interactions and coordinates what happens next. MVC separates these jobs so changes are easier to manage. It does not require one fixed call sequence, and it does not move backend business rules into the browser.

Reasoning and example

In a SAPUI5 application, a view binds a list to a model. When a user presses Submit, the controller gathers the intended action and calls the backend. The backend checks the business rules and permissions, then returns success or an error. The model and bindings help the screen reflect the result.

A disabled Submit button is useful feedback, but it cannot be the only enforcement of the rule. Someone can call the service directly. I keep required validation in the backend and use the UI for a clear experience. This division also helps testing: a controller interaction test and a backend business-rule test prove different parts of the application.

Avoid this mistake MVC does not mean copying all backend business logic into the UI controller or assuming the view must always be manually rebuilt.

Read and verify this answer

SAPUI5: Use the MVC Concept

OO-16 Singleton, Factory or Facade: how do you choose with a simple example?

Ask what problem you have. Need the right object created? Use a Factory. Need one deliberately shared instance? Consider Singleton. Need one easy entry point to several services? Use a Facade. These patterns can work together, but none is required just because the application uses ABAP Objects.

Reasoning and example

For a delivery preview, a Factory can choose a carrier calculator. A Facade can then read the order, calculate package sizes and ask that calculator for a price. A read-only configuration cache might be shared within the internal session, but that is a separate lifetime decision.

Start with ordinary classes. Add a Factory when creation logic repeats, a Facade when callers repeat a multi-step sequence, and Singleton only when one shared instance has a clear benefit and safe lifetime. In an interview, explain the problem and trade-off before naming the pattern. A report with one fixed calculation may need none of them.

Avoid this mistake A Factory does not require Singleton, and a Facade does not automatically add COMMIT, retries, authorization or transactional safety.

Read and verify this answer

SAP Learning: Factory Methods

SAP examples: OO design patterns

OO-17 Two NEW calls create two objects. How often do constructor and class_constructor run, and in what order?

constructor prepares each new object, so two successful NEW calls run it twice. class_constructor prepares the class once in an internal session, before its first relevant use. If the first use is creating an object, class initialization happens before instance initialization. Use constructor for each object's required data and dependencies. Keep class_constructor small: its work happens implicitly.

Reasoning and example

Create an order helper for order 7, then another for order 9. Each CONSTRUCTOR receives its own ID through IMPORTING and stores it in that object. If this is the first class use, the order is CLASS_CONSTRUCTOR → CONSTRUCTOR for 7 → CONSTRUCTOR for 9. The example shows a class counter of 1 and an object counter of 2. CLASS_CONSTRUCTOR has no parameters and does not repeat after COMMIT. Once means one internal session, not the whole SAP system.

Calling a static method or accessing a static attribute can initialize the class before any object is created. Merely referring to a type or a constant does not trigger class_constructor. It has no parameters and is called by the runtime, not explicitly by application code.

With inheritance, static constructors initialize the hierarchy from the superclass down. An explicitly implemented subclass constructor calls super->constructor( ) before using the new object's instance components, passing any required parent parameters. The subclass constructor is not a redefinition. If it is not declared, the inherited constructor interface and implicit parent initialization rules apply. Do not hide remote calls or a large data load in this automatic startup path.

Avoid this mistake Once per internal session does not mean once per SAP system, once per user forever, or once per SAP LUW. A static constructor is not a substitute for passing request-specific data to an object.

Read and verify this answer

SAP: instance and static constructors

SAP Learning: encapsulation and initialization

Official references

SAP Clean ABAP style guide

SAP ABAP object orientation examples

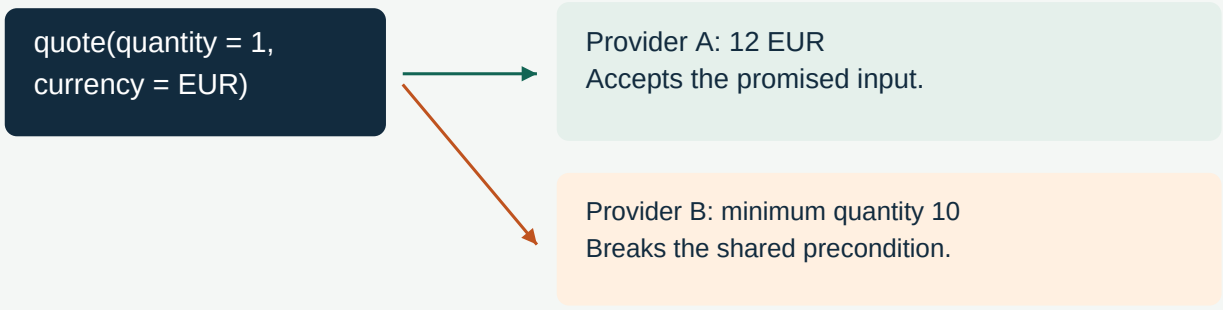
SAP interfaces and implementation semantics

Practical OO: contracts, state, and substitutability

Work through these decisions after the OO fundamentals. The emphasis is on observable promises: valid state, compatible substitutes, honest failure, ownership, and useful tests. Design recommendations are applications of the cited language rules and SAP guidance, not claims that the compiler enforces a complete business contract.

Same call shape does not prove the same behavior

Shared quote contract: accept every positive quantity; return the requested currency



Change request	Contract-preserving direction
Different inherited implementation	REDEFINITION keeps the signature
Optional simulation capability	Separate capability; honest unsupported result

The interface supports common dispatch across unrelated classes. It does not enforce business substitutability. Shared code alone does not justify a superclass.

A valid quantity-one call exposes a behavioral violation invisible in matching parameter types. The lower table separates redefinition from a genuinely new capability.

See the visual atlas for the recall prompt, limits and related questions.

OODEEP-01 A method named reserve returns a reservation ID. Which promises must be explicit beyond its parameter types?

I explain what inputs are accepted, what success means and what can change before a failure. The caller also needs to know whether the reservation is already saved, whether repeating the call is safe and who commits. Parameter types cannot answer those questions. I document these rules and test them so another implementation cannot quietly change the caller's expectations.

Reasoning and example

A caller needs to reserve two items together. The first implementation returns an ID but leaves saving to the caller. A replacement implementation returns an ID and commits immediately. Both can have the same ABAP signature, yet the second leaves the first reservation saved if reserving the second item fails.

I therefore state whether reserve only prepares the change or makes it permanent. I also define repeated calls, invalid inputs and error results in ABAP Doc or the relevant API documentation. Tests include failure on the second item and inspect the first item afterward. Returning an ID is useful, but it is not proof that a save has happened or that rollback remains possible.

Avoid this mistake RETURNING an identifier does not itself promise committed persistence, and a method name cannot replace a transaction contract.

Interviewer probes What state is allowed after reserve raises an exception? / What is the result when the same business request is retried?

Strong answer criteria Defines the state after success and failure; States who commits; Defines what repeated calls do

Verification evidence Contract tests plus documentation covering buffered success, persistence boundary, and repeated requests.

Read and verify this answer

SAP Learning: Documenting ABAP code

OODEEP-02 A new price-provider implementation rejects quantities below ten although the interface accepts any positive quantity. Is it a valid substitute?

No, because callers are allowed to request any positive quantity. Refusing quantities below ten adds a new restriction and breaks valid callers. The replacement must keep the original promise, or expose its narrower capability separately. I compare errors and side effects as well as successful results. Implementing the same ABAP interface proves compatible types, not compatible behavior.

Reasoning and example

A checkout calls the price-provider interface for one unit. The existing provider returns a quote, as documented. A new bulk provider rejects that request because its own minimum is ten. The program activates, but the checkout now fails for an input it was always allowed to send.

I can keep the shared behavior, choose the provider only through an explicit bulk-specific capability, or change the shared API through a planned migration. I do not hide the new restriction behind the old interface. A common test suite should run the same accepted-input, result and failure cases against each implementation. That checks the promises a caller relies on, rather than merely whether both classes compile.

Avoid this mistake Implementing an interface or passing a cast check proves type compatibility, not behavioral substitutability.

Interviewer probes Can a substitute silently return a different currency? / Which common tests would run against every provider?

Strong answer criteria Identifies the new restriction on valid inputs; Checks results, errors and other changes; Runs the same behavior tests for each provider

Verification evidence A contract suite exercises minimum positive quantity, currency, failures, and repeat calls for both providers.

Read and verify this answer

SAP Learning: Using interfaces

OODEEP-03 Two unrelated services share a capability and some validation code. Should their callers depend on an abstract superclass?

I would use an interface for the shared capability and a helper object for shared validation. That lets unrelated services reuse what they need without pretending they belong to one class family. An abstract superclass is useful when there is a real shared process and deliberate specialization. Similar code alone is not enough reason, especially because an ABAP class has only one superclass.

Reasoning and example

A document exporter and a notification sender both support cancellation, but their work is otherwise unrelated. A small cancellation interface lets a coordinator call either one. If both need the same cancellation policy, they can each use a policy object without inheriting all of one another's shared setup.

By contrast, a family of parsers may genuinely share a fixed sequence with a few defined steps implemented by subclasses. An abstract base class could fit that situation. I ask what the caller needs and what implementation is actually reusable before choosing. Putting shared code into a base class is convenient initially, but a forced hierarchy can make later independent changes much harder.

Avoid this mistake An abstract superclass is not automatically a better contract because it can carry reusable implementation.

Interviewer probes Which invariant would the superclass enforce? / How would the design accommodate a class already inheriting from a framework base?

Strong answer criteria Separates what callers need from shared implementation code; Checks whether a subclass really is a valid kind of its base; Allows unrelated behaviors to change separately

Verification evidence A responsibility sketch shows the caller's minimal interface and the owner of shared validation.

Read and verify this answer

SAP Learning: Defining interfaces

OODEEP-04 A subclass needs a different implementation and an extra mandatory input for an inherited method. Can REDEFINITION add it?

No. REDEFINITION changes an eligible inherited instance method implementation while keeping its interface. It cannot add a new mandatory parameter. ABAP does not solve this with same-name methods having different signatures. If the new input changes what the operation means, I design a separate operation or a deliberate API change. Existing base-type callers must still work as promised.

Reasoning and example

A base method formats a document using its declared inputs. A specialized formatter now needs an extra profile. I cannot simply add that profile to the redefined method signature. A separately named operation may be clearer, or a required profile can be supplied during valid object setup if that still fits the original method rules.

The important test is a caller holding only the base reference. It must not suddenly need a downcast and a hidden setup call to use an operation that previously worked. I check successful results, errors and initialization together. Even adding an optional parameter elsewhere does not automatically make a changed behavior compatible; the caller's expectations still need review.

Avoid this mistake Redefinition is not method overloading, and optional parameters do not automatically make every behavioral change compatible.

Read and verify this answer

SAP: Extending a subclass

SAP: ABAP Doc class example and method uniqueness

OODEEP-05 A local test class needs access to private state. What does granting friendship expose, and what does it not imply?

Friendship allows access to the granting class's private and protected components. It goes one way; the granting class does not automatically gain the same access back. A superclass friend is not automatically a friend of its subclasses, while subclasses of a friend can gain access. I keep the grant narrow and prefer testing public behavior when that is enough.

Reasoning and example

A local test class needs a legacy seam that cannot yet be reached through the public API. Granting friendship to that focused final test class can be narrower than making production attributes public. The test can then access the granting class's internals without opening them to every ordinary caller.

I still inspect who receives the permission. Granting friendship to a broadly implemented interface can give access to its implementing classes, which is much wider than one test class. The inheritance directions matter too: a friend class and a subclass of the class granting friendship are different situations. I do not reduce the rule to friendship is inherited or is never inherited; I check the actual classes involved.

Avoid this mistake Do not summarize friendship as simply inherited or not inherited; the direction and which class is subclassed matter.

Read and verify this answer

SAP: Friendship between classes

OODEEP-06 Two references point to one resource-owning object. Does clearing one reference close the resource or destroy the object?

Clearing one reference only clears that variable. If another reference still points to the object, the object remains reachable. Memory cleanup is also different from closing a connection or releasing a business lock. For resources that must be released promptly, I define an explicit owner and close or release operation. I run it on the required success and error paths.

Reasoning and example

A coordinator and a cache both refer to one session object. Clearing the coordinator reference leaves the cache reference usable, so it cannot guarantee that the session resource was closed. Even when an object later becomes unreachable, relying on garbage collection is not a precise business-resource release plan.

If the coordinator owns the resource, an explicit close operation may be appropriate. Other holders then need a defined response if they try to use it after closure. If the coordinator merely borrowed it, closing it may break the real owner. I test both ownership and failure paths instead of assuming CLEAR on a reference provides object destruction or lock release.

Avoid this mistake Reference clearing is not an object-level close operation and must not be used as a lock-release guarantee.

Interviewer probes Who may close a shared or borrowed object? / What should a second close or a call after close do?

Strong answer criteria Separates object reachability from timely resource release; Identifies who owns or borrows the resource; Defines behavior after close

Verification evidence Tests cover multiple aliases, explicit close, repeated close, and use after closure.

Read and verify this answer

SAP: Initializations

OODEEP-07 An object can be created with a missing repository and fails much later on its first business call. What would you change?

I would require the repository when creating the service and reject an invalid reference immediately. That avoids an object that looks usable but fails later because setup was forgotten. Constructor injection makes required helpers visible. A factory can coordinate more complex creation. I still check the reference explicitly; declaring a constructor parameter does not guarantee the caller passed a bound object.

Reasoning and example

An approval service currently has a public empty constructor and a later set_repository call. One caller forgets that setter, and approve fails far away from the mistake. Requiring the repository and policy during creation makes the missing setup visible at the point where the object is built.

I validate the required references and return a meaningful construction failure where appropriate. I do not need to perform every remote call in the constructor. If opening an external session is a separate step, I define that lifecycle clearly and test calls before and after opening. The goal is a predictable usable state, not heavy hidden initialization that surprises callers.

Avoid this mistake Constructor injection does not validate an initial reference automatically, and heavy I/O is not required for good initialization.

Interviewer probes When would a factory improve error handling over constructor work? / How do you prevent this object escaping before setup succeeds?

Strong answer criteria Defines what a usable new object requires; Rejects missing dependencies at creation; Separates setup from later business operations

Verification evidence Creation tests prove invalid dependencies fail immediately and valid instances need no undocumented setup call.

Read and verify this answer

SAP Learning: Encapsulation and consistency

OODEEP-08 A test passes alone but fails after another test changes CLASS-DATA. Why does NEW not reliably isolate it?

NEW creates a new instance, but it does not reset CLASS-DATA. That class-level state can be shared by instances in the same internal session and can remain after a test finishes. I remove unnecessary shared mutable state or provide a controlled reset. If a cache is required, I define its keys and refresh rules. It is not automatically synchronized across servers.

Reasoning and example

The first test loads a policy into a static cache. The second prepares different policy data and creates a new service, but the service still reads the old class-level cache. The second test fails only when run after the first, which is a sign that test state is leaking.

An injected provider created for each test can remove that unwanted sharing. If production genuinely needs caching, I test cache invalidation and all relevant context in the key, such as the intended user or organizational scope. A reset can help isolate legacy tests, but it does not fix an incorrect production cache lifetime. Changing test order only hides the problem.

Avoid this mistake CLASS-DATA is not per instance, but it is also not a globally synchronized cache across application servers.

Interviewer probes Does a class constructor rerun for every NEW? / Which context must appear in a security-sensitive cache key?

Strong answer criteria Explains how long class-level data remains; Does not assume NEW resets shared test state; Defines cache keys, lifetime and refresh rules

Verification evidence The suite passes individually, together, and with varied order without stale shared state.

Read and verify this answer

SAP: CLASS-METHODS

OODEEP-09 A repository raises a low-level exception. Should a business service catch CX_ROOT and return an empty result?

Usually no. Returning an empty result can make a technical failure look like a valid not-found result. I catch the failure where I can recover or turn it into a meaningful service error. If I wrap the exception, I keep PREVIOUS so the original cause remains available. Missing data, a rejected business request and an unavailable dependency should remain distinguishable.

Reasoning and example

A pricing repository cannot be reached. If the service catches CX_ROOT and returns an empty price, the caller might assume no price exists and apply a default such as zero. A network problem has now become an apparently valid business result, which is more dangerous than a clear error.

I define separate handling for a confirmed missing price and a failed attempt to read the price. The service can expose a useful unavailable message while preserving the low-level cause for diagnosis. Tests simulate both cases and verify that neither accidentally produces a successful free item. A broad catch can be appropriate at a final error-handling point, but not as permission to invent success data.

Avoid this mistake Broad catch-and-default handling is not resilience when it changes an unknown outcome into valid-looking data.

Interviewer probes When is catching CX_ROOT justified at an application boundary? / What information belongs in user feedback versus technical diagnostics?

Strong answer criteria Keeps missing data different from a failed read; Converts errors into useful service-level meaning; Retains the original exception cause

Verification evidence Tests assert distinct not-found and technical-failure paths, including the previous exception chain.

Read and verify this answer

SAP Learning: Working with exception classes

OODEEP-10 A getter returns a reference to a mutable internal collection. Has private visibility protected the object's invariants?

No. Private storage is not protected if a getter returns a reference that lets callers change the same data directly. I decide whether the getter returns an independent value, read access or intentionally shared state. Business changes should normally go through methods that check the rules. Returning a table value removes that table alias, but object references inside its rows can still be shared.

Reasoning and example

A cart add_item method rejects negative quantities. Its get_items method returns a writable reference to the real internal item table. A caller uses that reference to insert a negative quantity, bypassing add_item completely. The attribute is private, but the cart can still enter a state its own rules forbid.

Returning a suitable value copy can prevent direct changes to that table. I still inspect row components: a copied object reference points to the same object, so it may expose another way to change internal state. Alternatively, I offer the specific read operations callers need. Tests should try the exposed access paths, not only check that the attribute declaration says PRIVATE.

Avoid this mistake A copied reference is not a copied referent; changing the static view alone does not create immutable state.

Interviewer probes Does returning a table value solve reference components inside its rows? / Which mutation should remain an operation of the owning object?

Strong answer criteria Finds how a reference can bypass validation; Defines who may change the data; Checks object references inside copied rows

Verification evidence Attempted external mutation cannot bypass the owner's quantity rule or alter a promised snapshot.

Read and verify this answer

SAP: Assignment rules for reference variables

OODEEP-11 A widely used provider interface needs an optional new capability. How do you avoid forcing unrelated implementations into fake behavior?

I would consider a separate interface for the optional capability. Providers that really support it can implement it, while existing callers keep using the original interface. Adding a method affects implementers even if old callers never use it. If the main interface must change, I plan the meaning and rollout. Returning fake success is not a valid way to pretend support.

Reasoning and example

A price-provider interface is widely used, and a new caller needs simulation. One external provider cannot simulate. Adding simulate to every provider and having that one return an empty successful result hides the limitation and can mislead the caller.

A separate simulation capability lets setup code choose a suitable provider or report that the capability is unavailable. Ordinary pricing callers need not change. I test supported and unsupported selection as well as the simulation itself. A no-op implementation is acceptable only if doing nothing truly meets the documented rule. Interface evolution requires checking both sides: existing callers and every class that must implement the new operation.

Avoid this mistake Adding a method is not automatically harmless merely because existing callers do not invoke it.

Interviewer probes How will existing implementers activate after the interface change? / When would a separate version be clearer than optional capability discovery?

Strong answer criteria Checks effects on callers and implementing classes; Does not return fake success for unsupported work; Plans rollout and explicit unsupported-capability handling

Verification evidence An implementation inventory and compatibility tests cover old clients and providers without simulation.

Read and verify this answer

SAP: Interfaces in ABAP Objects

OODEEP-12 An OO unit test mocks calculate_total and then asserts the mocked total. What behavior has it proved?

It mainly proves that the mock returns the value configured by the test. It does not prove the real total calculation. To test calculation, I keep calculate_total real and replace outside inputs such as a price source. A different test may mock the calculator when it is testing checkout coordination. The important question is which real behavior the test claims to verify.

Reasoning and example

For a discount rule, the test supplies known prices and calls the real calculation with quantities below, at and above the threshold. It asserts totals derived from the business rule. If the calculation method itself is mocked to return 90 and the test checks 90, a broken implementation could still pass unchanged.

A checkout test has another purpose: it may verify that a total is requested before posting or that a calculation error prevents posting. There, a calculator double can be appropriate. I name the subject and expected behavior clearly so reviewers can tell these cases apart. High coverage or many mocks alone cannot establish useful test evidence.

Avoid this mistake The same component can be a real subject in one test and a doubled dependency in another; coverage alone does not settle test quality.

Interviewer probes Which defect should make this test fail? / What real integration evidence remains necessary after a double-based unit test?

Strong answer criteria Names the real behavior being tested; Replaces dependencies rather than the tested rule; Derives expected results independently from the implementation

Verification evidence A deliberately changed business rule makes its calculation test fail, while orchestration tests remain separately meaningful.

Read and verify this answer

SAP: Adding test doubles

Official references

SAP Learning: Documenting ABAP code

SAP Learning: Using interfaces

SAP Learning: Defining interfaces

SAP: Extending a subclass

SAP: ABAP Doc class example and method uniqueness

SAP: Friendship between classes

SAP: Initializations

SAP Learning: Encapsulation and consistency

SAP: CLASS-METHODS

SAP Learning: Working with exception classes

SAP: Assignment rules for reference variables

SAP: Interfaces in ABAP Objects

SAP: Adding test doubles

Practical ABAP design patterns and trade-offs

Use patterns to explain a concrete design decision, not to recite a catalog. These original senior scenarios build on OO-09 through OO-12 and distinguish pattern mechanics from RAP transaction, API-release and security contracts. SAP's pattern examples are exploratory demonstrations, not production templates. Verify syntax and dependencies on the target release. Recommendations below are contextual analysis, not claims that SAP mandates a particular application architecture.

One variation: replace a rule or specialize a step?

Worked choice: approval policy varies by country; process ownership must stay stable

STRATEGY / composition

Approval coordinator -> rule interface

Country A rule
Limit: 500 EUR

Country B rule
Limit: 900 EUR

700 EUR: A rejects; B accepts under this fixture.

TEMPLATE METHOD / inheritance

1. **Read inputs**
2. **Calculate policy**
3. **Return decision**

Stable skeleton; subclass supplies the chosen step.

Assembly decision	Boundary to keep
Factory selects policy	Validate country and policy version
Constructor receives rule	Rule returns a decision; does not commit

The amounts are fictional policy fixtures. Choose a real variation point; a reusable calculation alone does not require an inheritance hierarchy.

The same 700 EUR request receives different policy decisions through the same Strategy contract. Template Method instead fixes a call skeleton and exposes deliberate subclass steps.

See the visual atlas for the recall prompt, limits and related questions.

PAT-01 Approval rules vary by country, while the application workflow should remain stable. Strategy or Template Method?

I would use Strategy for country-specific approval rules. The coordinator keeps the workflow, and the selected strategy returns the rule decision. Template Method fits a shared sequence where subclasses intentionally fill in a few fixed steps. Here the rule can vary without changing the coordinator class. I would only choose inheritance if the shared sequence and permitted overrides are clear requirements.

Reasoning and example

The application receives an order for 8,000 EUR and supplies the appropriate country rule. In this example, the DE rule requires a manager above 5,000 EUR, while the FR rule uses 10,000 EUR. The same amount therefore requires a manager for DE but not for FR. Each rule receives the required order data and returns a decision, such as the required approver. It should not secretly commit, send email or change global data. The coordinator stays responsible for applying the decision and continuing the workflow.

I test that coordinator with several rules and verify the same success and failure expectations. If countries later need different process sequences, I review the workflow itself instead of adding increasingly powerful subclass hooks. Strategy works best when the changing part is a replaceable rule. Template Method works when a stable base algorithm deliberately controls which steps subclasses may customize.

Avoid this mistake Choosing inheritance solely to share code, or allowing a rule implementation to take over transaction ownership.

Interviewer probes Which variation changes at runtime rather than through subclassing? / How would you prevent a new rule from changing the workflow's transaction boundary?

Strong answer criteria Separates country rules from workflow coordination; Explains why composition or inheritance fits this case; Keeps rules replaceable and separately testable

Verification evidence Run identical coordinator contract tests with two country rules and a test double; assert decision and side-effect boundaries.

Read and verify this answer

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: Template Method

PAT-02 A posting integration needs matching serializer and sender implementations for each provider. Is one static factory sufficient?

A simple factory may be enough for a small fixed set of providers. Abstract Factory helps when several objects must belong to a compatible family, such as a serializer and sender for one provider. Factory Method means a creation operation that subclasses can override. I choose the smallest design that prevents incompatible combinations. Returning an object from a static method does not automatically make it Factory Method.

Reasoning and example

Provider A expects one message format and endpoint behavior; provider B expects another. If the caller independently chooses a serializer and sender, it could combine A format with B sender. A factory that returns a validated pair can prevent that mistake without a large hierarchy when only a few combinations exist.

If each provider supplies several related products that evolve together, a provider-factory interface may make the family clearer. I test that every configured provider returns compatible objects and that unknown configuration fails explicitly. Silently choosing a default can send a business document to the wrong destination. The purpose is safe construction, not using the largest possible pattern.

Avoid this mistake Confusing a static creation helper with inheritance-based Factory Method or introducing an Abstract Factory for one trivial object.

Interviewer probes What concrete invalid combination must construction prevent? / When would adding a creator subclass be better than changing a configuration map?

Strong answer criteria Distinguishes simple factory, Factory Method and Abstract Factory; Names which related objects must be compatible; Checks unknown configuration and future changes

Verification evidence Test each supported provider family and an unknown provider; verify no serializer/sender cross-pairing can be constructed.

Read and verify this answer

SAP ABAP pattern examples: Abstract Factory

SAP Factory Method pattern

PAT-03 A legacy posting API has awkward parameters and a three-step prepare/check/post sequence. Would you build an Adapter, a Facade, or both?

An Adapter can translate the application request and errors into the legacy API format. A Facade can offer one posting operation over prepare, check and post. Both may be useful, but they must preserve failures, authorization and commit rules. I inspect the actual underlying calls. Neither wrapper makes an unreleased API available to ABAP Cloud or removes an internal commit.

Reasoning and example

The application sends a simple posting request. An adapter maps it into the required function-module structures and converts responses into meaningful results. A facade coordinates prepare, check and post, and stops before posting when the check fails. I test mapping separately from that sequence so errors are easy to locate.

Important cases include an explicit zero amount, a warning that must be handled according to the API rules and a remote timeout with an unknown outcome. A timeout is not always proof that nothing was posted. The caller needs enough information to reconcile rather than repeat the operation blindly. The wrapper must expose those facts instead of making the API look safer than it is.

Avoid this mistake Treating any wrapper as both a supported API and a transaction adapter without inspecting its dependencies.

Interviewer probes Which errors belong to the domain contract and which retain technical detail? / What evidence would make the legacy API unsuitable despite a neat interface?

Strong answer criteria Separates field/error translation from coordinating several calls; Preserves commit rules and API release restrictions; Tests important values and error mapping

Verification evidence Trace all three steps with injected failures and inspect dependency release state; assert no hidden commit or swallowed business rejection.

Read and verify this answer

SAP ABAP pattern examples: Adapter

SAP ABAP pattern examples: Facade

PAT-04 You want timing, diagnostic logging and retries around an outbound service. Would stacked Decorators be safe?

Decorators can add logging or timing around the same service interface. Their order matters. Retrying needs extra care because repeating a request can repeat a business effect. I only retry when the operation rules make that safe, and I distinguish a rejection from an unknown remote outcome. I also decide whether logs and timing describe each attempt or the complete operation.

Reasoning and example

Put logging outside retry and it can record one logical call with several attempts inside it. Put logging inside retry and it can record every attempt. Either can be useful, but the records should share an identifier so support can understand what happened instead of counting attempts as separate business requests.

I test exceptions and return values through the full decorator chain. Logs must avoid sensitive payloads. A diagnostic logging failure should not silently turn a successful business call into failure unless a required audit rule deliberately demands that. Automatic retries on every POST are unsafe: after a timeout, the remote system may already have applied the first request.

Avoid this mistake Adding automatic retries to every POST or assuming decorator order cannot change observable behavior.

Interviewer probes Which wrapper should count attempts and which should measure total latency? / How does a timeout after remote success affect the retry policy?

Strong answer criteria Preserves the service results and errors; Explains how wrapper order changes observation; Does not assume retry automatically prevents duplicate effects

Verification evidence Test wrapper orders, a transient read failure, uncertain write completion and logging failure; inspect returned outcome and number of remote effects.

Read and verify this answer

SAP ABAP pattern examples: Decorator

SAP Clean ABAP: explicit failure flow

PAT-05 Three handlers can route a request to an approver. The first matching rule should win. How would you implement and test Chain of Responsibility?

I would use an ordered Chain of Responsibility when the first matching handler should choose the route. Each handler must distinguish handled, not applicable and failed. I define what happens when no rule matches and test the configured order. A technical failure must not quietly fall through to another approver. If all validations must run, I use a different execute-all process.

Reasoning and example

A high-value rule, a regional rule and a fallback can all potentially apply to one request. If the high-value rule must win, it appears before the general fallback. The selected handler returns an approver decision, while the transaction coordinator applies it. The rule should not secretly save data during selection.

Tests include overlapping matches, no match and a failure while a rule reads required information. I record which rule selected the approver so support can explain the route. Treating a failed lookup as not applicable can accidentally send the request to a less restrictive fallback. First-match routing is useful, but it does not mean every validation in the chain has run.

Avoid this mistake Treating an exception as 'not applicable', or confusing first-match routing with execute-all validation.

Interviewer probes What happens when two handlers match? / How is a configuration error different from a valid request with no applicable route?

Strong answer criteria Defines rule order and each routing outcome; Separates first-match routing from running all validations; Explains the selected route and controls data changes

Verification evidence Test overlapping rules, no match, handler failure and reordered configuration; assert exactly which handlers execute and which route is chosen.

Read and verify this answer

SAP ABAP pattern examples: Chain of Responsibility

PAT-06 A local ABAP event updates an ALV, and the team proposes another handler to start an external workflow. What boundary has changed?

The requirement changed from a local notification to reliable work in another system. An ABAP OO event can notify registered listeners in the current execution context. It does not save a message or guarantee delivery after commit. An external workflow needs a supported publication mechanism plus retry and recovery rules. I separate display updates from the saved business transition that should start the workflow.

Reasoning and example

An ALV listener reacts to a local event and refreshes the screen. It can also be accidentally registered twice or left registered when obsolete, so I manage its lifetime and test error handling. None of that registration creates a durable subscription that survives the runtime context.

For an external workflow, I need to know that the document change was saved and that publication can recover after an outage. The receiving side must identify repeated or delayed messages and avoid creating duplicate workflows. A local handler that directly calls the external system can run before a later save failure or lose delivery on an error. Observer alone does not solve those business guarantees.

Avoid this mistake Assuming SET HANDLER creates a durable subscription or that Observer guarantees exactly-once business effects.

Interviewer probes Which part of the design survives an application-server restart? / What if one local listener fails after another already changed its state?

Strong answer criteria Separates local notification from messages that survive failures; Connects external work to saved business changes; Defines handling for listener and delivery failures

Verification evidence Test local listener registration separately from rejected save, duplicate event and delayed workflow delivery cases.

Read and verify this answer

SAP Observer pattern

SAP RAP business events

PAT-07 A command object represents Submit and can be queued or retried. Does using Command make execution idempotent or undoable?

No. Command packages an action and its inputs; it does not automatically make repeats safe or provide undo. I define a stable operation ID, storage, authorization and repeated-execution rules separately. An undo operation only works when the business has a valid reverse action. A posted document may require a compensating document rather than deletion or a simple rollback.

Reasoning and example

A Submit command stored only as an object in memory disappears when its runtime context ends. For a reliable queue, I store a suitable versioned data record and use it to reconstruct an allowed executor. A saved object reference is not a durable message format. At execution time, I check the required business state and authorization.

I also test a repeated operation ID with the same payload and with a conflicting payload. The first may return the existing result; the second may need rejection, according to the defined rules. If submission already posted a document, local rollback cannot erase that completed effect. Recovery may need a separate, audited reversal operation.

Avoid this mistake Assuming execute can safely run twice or that every business command has an exact inverse.

Interviewer probes What result should a retry return after the first response was lost? / How would you evolve a persisted command payload across application releases?

Strong answer criteria Separates packaging an action from safe execution; Defines a saved operation ID and repeat-call behavior; Uses a business reversal where simple undo is impossible

Verification evidence Simulate duplicate execution, conflicting payload reuse, lost response and executor upgrade; assert one effective transition and traceable recovery.

Read and verify this answer

SAP ABAP pattern examples: Command

SAP LUWs in ABAP

PAT-08 A repository interface improves testability, but its RAP implementation reads managed persistence directly. What can go wrong?

A testable interface can still hide an incorrect implementation. Direct table reads can miss changes waiting in the RAP transaction buffer. I first define whether the repository should read saved data or the current transaction state. For BO logic that needs current changes, I use suitable EML access. I do not write managed tables directly and bypass the RAP behavior checks.

Reasoning and example

An operation changes an item, and a validation then reads it through a repository before save. If the repository selects the database table, it can return the old value even though the current RAP transaction already contains the new one. Newly created or deleted instances make the difference even more obvious.

A reporting read of persisted data may legitimately use a different approach. I give those needs distinct names or responsibilities so callers understand which state they receive. Tests should cover update-then-read, create-then-read and delete-then-read before save using the real BO path. A convenient fake returning expected rows will not reveal a production implementation that reads the wrong state, and a second home-made buffer adds more consistency problems.

Avoid this mistake Equating a mockable interface with correct behavior or recreating RAP's transactional buffer in a second general-purpose cache.

Interviewer probes Which caller actually needs persisted rather than buffered state? / What integration test would fail if the repository used SELECT instead of EML?

Strong answer criteria States whether a read returns saved or current buffered data; Respects RAP save ownership and pending changes; Tests the interface and the real BO integration

Verification evidence Compare unsaved create/update/delete reads through the adapter with the BO contract, then run rollback and save paths on the target system.

Read and verify this answer

SAP ABAP pattern examples: data-access isolation examples

SAP RAP implementation contract

PAT-09 A singleton caches the current approver and makes tests pass alone but fail as a suite. How do you redesign its lifetime?

I would remove the current approver from long-lived shared mutable state. I would pass the relevant context explicitly and use an operation-scoped service, or a cache with clear keys and refresh rules. An ABAP singleton normally belongs to an internal session, not the whole system. Its lifetime can still span several operations or tests. Forcing test order does not fix that design.

Reasoning and example

One test asks for an approver for company A and populates the singleton. A later test for company B reuses the cached value and gets the wrong approver. Similar trouble can occur when a job processes several contexts. The cache stored current approver without saying current for which request or policy.

I make that context explicit in the operation and, if caching is needed, include every relevant distinction in its key and invalidation rules. Tests get controlled service instances. A reset method may temporarily isolate legacy tests, but production must still have the correct lifetime. Clearing a cache between tests cannot repair an application that mixes request-specific decisions during normal use.

Avoid this mistake Claiming singleton means one system-wide instance or solving hidden state solely by forcing test order.

Interviewer probes What exact lifetime is required by the cached value? / How would two simulated users in one test expose the defect?

Strong answer criteria States the actual singleton lifetime; Passes request-specific context explicitly; Separates isolated tests from a correct production cache

Verification evidence Run tests in multiple orders and reuse one session across different users or contexts; assert no stale decisions and deterministic dependency replacement.

Read and verify this answer

SAP ABAP pattern examples: Singleton

SAP Clean ABAP: state lifetime

PAT-10 A service obtains every dependency from a global factory inside its methods. How would you introduce dependency injection without building a framework?

I would build the service near the application entry point and pass its required interface-typed helpers through the constructor. Business methods then use those supplied helpers instead of finding them globally. A small factory can assemble them if setup is complex. This makes tests predictable without requiring a dependency-injection framework. Framework-created classes may need a supported seam or a delegated service instead.

Reasoning and example

An approval method currently calls a global factory to obtain a repository and posting client every time. A unit test cannot easily control what it receives and may accidentally reach a live dependency. I move that construction into application setup and give the service the two objects explicitly.

The test supplies a known repository and a client that cannot post externally, then exercises the real approval decision. A separate integration check verifies production wiring. If RAP or another framework controls construction, I do not invent unsupported constructor parameters; I use a suitable delegated application service or supported test seam. Replacing the factory with another global lookup would still hide the same dependencies.

Avoid this mistake Replacing one hidden global singleton with a service locator and calling the result fully decoupled.

Interviewer probes Which object owns construction in a framework-created RAP handler? / How would a newly added dependency become visible to existing tests?

Strong answer criteria Separates constructing dependencies from business decisions; Supplies dependencies explicitly within framework rules; Checks production setup and safe replacements in tests

Verification evidence Instantiate the application service with doubles, assert no live dependency access, and test the production factory's intended object graph independently.

Read and verify this answer

SAP Clean ABAP: dependency injection

SAP ABAP test isolation examples

PAT-11 A small report has one stable calculation. A reviewer requests Strategy, Abstract Factory and Repository 'for clean architecture'. How do you respond?

I would ask what concrete problem each pattern solves. One stable calculation may need only a clearly named method and good tests. An external service may deserve an interface because it can fail or needs replacement in tests. I add Strategy or factories when real variations make them useful. The aim is understandable change, not a high count of classes.

Reasoning and example

A small report calculates a tax base with one supported rule. A method with explicit inputs, an explicit result and boundary tests can be easy to read and maintain. Adding Strategy, Abstract Factory and Repository around that pure calculation may introduce many files without removing any actual difficulty.

If the rate comes from a remote service, that dependency is a real reason for a seam: tests need controlled values and failures. If a second tax rule arrives later, extracting a Strategy can then make sense. I leave a clear reason for that future change. I neither reject patterns automatically nor apply them as decoration; each added layer should make a named responsibility easier to understand or test.

Avoid this mistake Equating more classes with greater maintainability or rejecting all patterns because the current implementation is short.

Interviewer probes Which expected change would justify the next abstraction? / What testing benefit can you demonstrate before adding another interface?

Strong answer criteria Connects each extra layer to a real problem; Keeps the design simple while making outside calls testable; Names a concrete future change that would justify another layer

Verification evidence Compare change scope and tests for the current design and proposed abstraction; retain only boundaries with a demonstrated maintenance or testing benefit.

Read and verify this answer

SAP Clean ABAP: abstraction level

SAP Clean ABAP: focused responsibilities

PAT-12 A factory selects country approval Strategies from configuration. How do you keep that combination safe, explainable and testable?

I map supported configuration values to known approval strategies and pass the selected strategy to the coordinator. Missing or unknown configuration should fail clearly, not default to approval. Each rule returns the same kind of result. I test selection separately from calculation. Dynamic class creation still needs approved class choices and allowed dependencies; it cannot bypass ABAP Cloud restrictions.

Reasoning and example

Configuration selects a country rule and, when needed, a policy version. A factory validates that choice and creates the corresponding strategy. Tests check every supported mapping and an unknown value. Separate tests verify each rule, while common cases check that all strategies keep the same result and failure promises.

For a long-running approval, I decide whether it keeps the policy version from submission or deliberately uses the current policy. Otherwise a configuration change could alter the rule halfway through without an explanation. When audit requires it, I record which policy produced the decision. Arbitrary class names from untrusted input and a fallback that approves after configuration failure are both unsafe shortcuts.

Avoid this mistake Allowing arbitrary class names from untrusted input or implementing a 'default approve' path when configuration lookup fails.

Interviewer probes What should happen to a pending request after the configured policy changes? / Which tests distinguish factory-selection failure from a correct rule rejecting the request?

Strong answer criteria Separates choosing the strategy from running it; Rejects unsupported configuration safely; Defines policy versions and shared behavior rules

Verification evidence Test supported and unknown countries, policy changes during pending approvals and identical contract assertions against every Strategy.

Read and verify this answer

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: object creation alternatives

Official references

SAP ABAP pattern examples: Strategy

SAP ABAP pattern examples: Template Method

SAP ABAP pattern examples: Abstract Factory

SAP Factory Method pattern

SAP ABAP pattern examples: Adapter

SAP ABAP pattern examples: Facade

SAP ABAP pattern examples: Decorator

SAP Clean ABAP: explicit failure flow

SAP ABAP pattern examples: Chain of Responsibility

SAP Observer pattern

SAP RAP business events

SAP ABAP pattern examples: Command

SAP LUWs in ABAP

SAP ABAP pattern examples: data-access isolation examples

SAP RAP implementation contract

SAP ABAP pattern examples: Singleton

SAP Clean ABAP: state lifetime

SAP Clean ABAP: dependency injection

SAP ABAP test isolation examples

SAP Clean ABAP: abstraction level

SAP Clean ABAP: focused responsibilities

SAP ABAP pattern examples: object creation alternatives

ABAP SQL, HANA, and performance

Start with correct result semantics, then measure execution cost. Be ready to explain row multiplication, empty selection drivers, NULL values, and transaction-sensitive buffering before recommending a faster query. HANA-specific techniques and tool availability depend on the target edition and release.

A join repeats the header once for every matching item

Example: sum the order totals before and after a one-to-many join. All amounts are EUR.

ORDERS: 2 rows

Order	Header total	Currency
10	100	EUR
20	80	EUR

ITEMS: 3 rows

Order	Item	Product
10	1	A
10	2	B
20	1	C

JOIN ON order ID

RESULT: 3 rows

Order	Item	Header total	Currency
10	1	100	EUR
10	2	100	EUR
20	1	80	EUR



Fix the grain: aggregate at the intended business level before combining results.
SUM(DISTINCT amount) is not a general fix: different orders can have equal totals.

Outer join caveat: a WHERE predicate on the missing right side can remove unmatched left rows.

Joining two order headers to three items repeats order 10's total, inflating a naive sum from EUR 180 to EUR 280. For outer joins, matching conditions in ON can preserve unmatched left rows, while right-side conditions in WHERE can remove them.

See the visual atlas for the recall prompt, limits and related questions.

SQL-01 How do ABAP SQL, Native SQL, CDS, and AMDP differ?

ABAP SQL is the usual way to read and change database data from ABAP. CDS defines reusable data models and queries. Native SQL uses database-specific SQL, so the developer must handle more details. AMDP manages HANA database procedures and functions written in SQLScript through ABAP classes. I start with the simplest supported option and check client, authorization and release rules.

Reasoning and example

A report needs totals for one company and month. ABAP SQL may express the whole selection and sum clearly. CDS becomes useful when the same data model should be reused by several consumers. Neither requirement automatically needs a database procedure.

If a necessary calculation cannot be expressed well in supported ABAP SQL or CDS, I consider AMDP and measure the result. I also check which dependencies are allowed in the target environment. Writing SQLScript inside an ABAP-managed method does not turn it into ABAP SQL. In particular, direct table reads must not be assumed to apply the access control of a CDS entity that would otherwise protect those records.

ADBC is an ABAP API for Native SQL, useful when a permitted database-specific operation cannot be expressed through ABAP SQL. It does not add ABAP SQL client handling: a client-dependent table needs the correct explicit client restriction. Bind data values, allowlist any configurable identifiers, and perform the required authorization checks. Native SQL also does not inherit a CDS entity's DCL just because it reads the same tables. ABAP Cloud requires the language and released APIs allowed in the target environment.

Avoid this mistake ABAP syntax around an AMDP method does not turn its SQLScript body into ABAP SQL or automatically apply CDS access control.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP AMDP use and selection guidance

SAP: client handling guidelines

SQL-02 What is the empty-table trap in FOR ALL ENTRIES?

An empty FOR ALL ENTRIES driver is dangerous: the WHERE condition is ignored, so the query can read all rows in scope. I check that the driver is not empty before selecting. I also decide how duplicates should be handled. FOR ALL ENTRIES removes duplicate result rows, which can matter when selected columns do not uniquely identify records.

Reasoning and example

A report receives a list of customer IDs and should read their orders. If the list is empty, I normally return an empty result without running the selection. Adding another condition in the same WHERE clause is not a replacement for that guard: the empty-driver rule still applies.

I can remove repeated driver keys to reduce unnecessary work, but I separately check the output. Two different orders may have the same selected customer and amount. If I omit the order identity, result duplicate elimination can collapse those rows. I test empty input, repeated IDs and distinct records with equal displayed values before deciding the query is correct.

Avoid this mistake An empty FAE driver does not mean no rows; a separate filter in the same WHERE clause is not a safe replacement for the guard.

Read and verify this answer

SAP FOR ALL ENTRIES selection semantics

SQL-03 When would you use a join instead of FOR ALL ENTRIES?

A join is useful when related database rows can be selected together with clear matching rules. FOR ALL ENTRIES can fit when the driver data already exists in ABAP. Neither is always faster. They can return different duplicates and row counts, so I check the required result first. Then I compare database work, transferred data and performance with realistic inputs.

Reasoning and example

An order-to-item join produces a row for each matching item. That is useful when the report needs the item details, but it repeats order fields. If the report only needs orders that have an item meeting a condition, an EXISTS selection may describe the requirement more clearly than joining and removing duplicates afterward.

FOR ALL ENTRIES also has its own duplicate elimination and empty-driver behavior. Replacing it with a join requires tests for those differences, not just a runtime comparison. I include large customers, customers with no matching items and repeated driver values. A faster statement is only an improvement if it still produces the business result the caller expects.

Avoid this mistake Replacing FAE with a join can change duplicates and row counts even when the new query runs faster.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP FOR ALL ENTRIES selection semantics

SQL-04 How can a LEFT OUTER JOIN accidentally behave like an inner join?

A LEFT OUTER JOIN keeps left-side rows even when no right-side row matches. The missing right side contains SQL NULL values. A WHERE condition demanding a right-side value can remove those rows again. I put a condition in ON when it defines which right rows may match. I use WHERE when I really want to filter the final result.

Reasoning and example

The report must show every order, with an English description where available. I join the description using the order key and language condition in ON. An order without English text remains in the list, with no matching description. The user can still see that the order exists.

If I put the right-side language condition in WHERE instead, the unmatched row does not meet it and disappears. The report now hides some orders, even though the SQL still says LEFT OUTER JOIN. I test an order with English text, one with only another language and one with no text. That makes the intended behavior visible before tuning the query.

Avoid this mistake Moving conditions between ON and WHERE is not always semantics-preserving for outer joins.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-05 What is the difference between SQL NULL and an ABAP initial value?

SQL NULL means a value is missing or unknown. An ABAP initial value is a real value, such as zero or blank. Comparing something with NULL does not behave like comparing it with zero. I use IS NULL to check for NULL and COALESCE when a replacement is intended. Supported null indicators can preserve the distinction when reading into ABAP.

Reasoning and example

A left outer join reads orders and optional payments. One order has a payment of zero; another has no payment row. Those are different facts, but a plain transfer into an ABAP amount field can make both look like zero. I preserve the missing-row information if the report needs to distinguish them.

For example, I can include a suitable existence indicator or use supported null handling rather than guessing from the amount. COALESCE replaces NULL with a chosen value, but that deliberately loses the distinction unless I keep it elsewhere. It does not replace an ordinary stored zero or blank. I test both cases so missing data is not shown as a confirmed amount.

Avoid this mistake COALESCE replaces NULL, not every initial or blank value, and WHERE column = NULL is not a correct null test.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-06 How do GROUP BY, WHERE, and HAVING work together?

WHERE filters the rows before grouping. GROUP BY collects rows with the same chosen fields, and functions such as SUM calculate a result for each group. HAVING filters the resulting groups. I let the database do this work when it is appropriate. I still check currencies and joins, because a fast sum can be wrong if it adds incompatible or repeated amounts.

Reasoning and example

To show monthly sales totals above a threshold, I first use WHERE to select the month. I group by company and currency so euros and dollars remain separate. SUM calculates each total, and HAVING keeps only groups above the required threshold. The application receives a small result instead of every sales row.

Before trusting it, I check any join added to the query. If one sales item matches three partner rows, its amount may be counted three times. A small fixture with one known item and several partners reveals that error. The right fix is to model the required rows correctly, not simply to accept the database total because aggregation ran successfully.

Avoid this mistake Database aggregation is not automatically correct if different currencies are mixed or joins multiply the rows being aggregated.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-07 What do cardinality and uniqueness mean for join correctness?

Cardinality describes how many matches a relationship can have: none, one or many. The actual join fields and data decide which rows are returned. Missing key fields can multiply rows or join unrelated records. I check the complete identity and test missing and multiple matches. A CDS key or a to-one declaration does not repair incorrect data or matching logic.

Reasoning and example

A document number may be unique only together with company code and year. Joining by number alone can connect an item to a different document with the same number. Text tables introduce another common issue: one record can have several languages, so omitting language can multiply the result.

I prepare test data with the same number in different years and with several text languages. Then I compare the returned rows with the intended matches. DISTINCT may hide duplicated output rows, but it does not correct a wrong selected description or an inflated sum. I fix the join condition and only declare cardinality that the model and data actually support.

Avoid this mistake DISTINCT can conceal an incomplete join condition while leaving aggregates or selected attributes logically wrong.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-08 Does SELECT SINGLE return a predictable first row?

SELECT SINGLE does not mean the latest, smallest or first business record. It is suitable when the conditions identify one required row, or when any match is acceptable. If several rows match and their order matters, I use a supported limited query with ORDER BY. I include a unique tie-breaker so equal sort values still give a predictable result.

Reasoning and example

A status table contains several events for one order. Reading one matching row without ordering can return an old event. If I need the latest status, I sort by event time and a unique sequence or ID, then limit the result to one row using supported syntax.

The unique extra sort field matters when two events have the same timestamp. Without it, either tied row may be chosen. I test those equal timestamps and an order with no events. I do not rely on the row that happened to be returned in development: a changed index, execution plan or data distribution can change that result without any change to the ABAP statement.

Avoid this mistake The primary key definition or a convenient current execution plan is not an implicit ORDER BY guarantee.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-09 What do you optimize first in a slow data selection?

I first confirm the correct result and measure a repeatable slow case. Then I look for too many rows, unnecessary columns, repeated queries and joins that create extra work. Reducing database calls and data transfer often helps more than changing ABAP syntax. I measure each change with realistic data and verify that permissions and business results stay correct.

Reasoning and example

A report loops over thousands of items and issues one SELECT for each item. Even a fast individual query can become expensive when called thousands of times. Reading the required data together can reduce those round trips, provided the new query preserves the intended matching and authorization rules.

I compare total runtime, number of calls and transferred rows before and after the change. I also test missing records and a large customer whose data is much less evenly distributed than the development sample. Selecting every column or returning an unlimited result can still waste memory after the repeated calls are removed. The goal is less necessary work, supported by measurements.

There is no universal sequence such as indexes first, then CDS, then AMDP. Start with the measured bottleneck: ABAP work, database work, transfer, locks or a remote dependency. Choose a change that removes that cost and preserves behavior. Pushdown is useful when appropriate set processing belongs near the data, not merely because the target database is HANA.

Avoid this mistake A query that is fast on ten development rows may be unsuitable for millions of production rows or a different key distribution.

Read and verify this answer

SAP ABAP SQL examples and semantics

SAP HANA secondary indexes

SQL-10 Which tools help locate ABAP, database, and memory performance problems?

Start with the slow business request and locate where its time or memory goes. Where available, ST03N/STAD help identify costly workloads or requests, SAT traces ABAP execution, and ST05 traces SQL. ST12 combines relevant traces. SQL Monitor helps prioritize SQL across a wider workload; a database plan explains one statement. Memory analysis needs its own evidence.

Reasoning and example

A report is slow, but the longest individual SQL statement may not dominate total time. Use workload/request statistics to choose a representative case, then trace it. Compare call count, transferred rows and total time: a short statement called thousands of times may be the real problem. SQLM can collect a broader SQL profile; SWLT can combine runtime evidence and static findings where supported. Its low monitoring overhead is not zero or a guarantee for every workload: scope collection and permissions appropriately.

For a memory dump, inspect ST22 and compare Memory Inspector snapshots at comparable phases. Look for growing internal tables, retained copies and objects. A database execution plan instead helps explain scans, joins, sorts and large database intermediates. Change one measured bottleneck, then verify the same result, runtime and peak memory again.

Tool access depends on the deployment. Do not promise classic transactions in every ABAP Cloud environment. Use supported ADT profiling/SQL analysis and the available monitoring apps. In SAP BTP ABAP environment, administrator SQL tracing can be started through Capture Request Statistics; detailed trace evaluation uses the Technical Monitoring Cockpit. The exact role, release and tool entry point must be checked.

Avoid this mistake The slowest individual SQL statement may not be the largest total cost if another cheap statement is executed thousands of times.

Read and verify this answer

SAP analyzing SQL statements in ADT

SAP BTP ABAP environment: tracing SQL activities

SAP SQL Monitor

SAP Memory Inspector concepts

SAP analysis tools overview

SQL-11 Does SAP HANA remove the need for indexes and performance tuning?

HANA still needs performance analysis. Its storage and processing are fast, but a query can still read too much data or create huge intermediate results. A secondary index may help a particular lookup, while costing memory and extra update work. I decide from the actual table, workload and execution plan. I do not add indexes just because another database used them.

Reasoning and example

A service repeatedly finds a tiny number of records using the same combination of attributes. I inspect how HANA executes that selection and whether a supported secondary index could help. I compare the lookup gain with memory use and the effect on inserts and updates, because the workload includes writes too.

For a query that legitimately reads much of a column, the same index may offer little benefit. I also avoid describing every primary key as a SQL Server-style clustered index that defines physical row order. HANA has its own storage and index structures. Query correctness and explicit ORDER BY remain necessary regardless of which access path the optimizer selects.

Avoid this mistake Do not say HANA needs no indexes or that every primary key is a clustered index with a predictable physical row order.

Read and verify this answer

SAP HANA secondary indexes

SQL-12 When is table buffering appropriate?

The ABAP table buffer can help with small, rarely changing tables that are read often in supported ways. It is a poor default for constantly changing business data or reads that need immediate freshness across servers. I check the table setting and the actual SQL form, because some statements bypass the buffer. This buffer is separate from HANA caches and internal tables.

Reasoning and example

A small customizing table is read for nearly every document, while its values change only occasionally. Buffering may reduce repeated database access if the table and its reads qualify. I inspect both the configuration and the query forms instead of assuming the buffered setting benefits every SELECT.

An order status that changes continuously has different needs. A buffered value on another application server may not immediately reflect a change, so it may be unsuitable for a decision requiring the latest state. I test the expected consistency and workload before choosing buffering. An internal table used by one ABAP session is another mechanism again; it does not provide shared buffer synchronization.

Avoid this mistake Buffering is not universally faster, and reading buffered data does not guarantee immediate visibility of every change across application servers.

Read and verify this answer

SAP table buffering and synchronization

SQL-13 What are normalization and denormalization, and how do they affect ABAP design?

Normalization stores facts where they belong so one change does not require repairing many copies. First normal form avoids repeating groups; second removes dependence on only part of a candidate key; third removes unsuitable indirect dependencies between non-key facts. Denormalization deliberately stores extra copies or calculated results. I use it for a clear need and define how those copies stay correct.

Reasoning and example

If a customer current address is copied onto every order and all copies are meant to stay current, an address change becomes difficult: some rows may be missed. Keeping that current fact with the customer avoids the repeated update. The order can refer to the customer when it needs the latest address.

An issued invoice may instead need the address exactly as it was when issued. Keeping that historical snapshot is an intentional business rule, not automatically a design error. I clarify which meaning is required before removing duplication. Similarly, a stored total may improve a measured read problem, but it needs a clear update rule so the total cannot silently become stale.

Avoid this mistake NoSQL is not automatically faster or better for large data, and normalization is not a rule against all intentionally stored business snapshots.

Read and verify this answer

SAP ABAP SQL examples and semantics

SQL-14 When is AMDP justified, and what risks must you address?

I consider AMDP when a required HANA operation is difficult to express with supported ABAP SQL or CDS, or measurements justify keeping a large calculation in the database. I check release support, dependencies, client handling and authorization. The team also needs SQLScript skills and suitable tests. AMDP should solve a real problem, not be a default replacement for a simple SELECT.

Reasoning and example

A complex analytical calculation repeatedly transfers a large dataset to ABAP and back. After trying simpler supported SQL and CDS options, I may compare an AMDP implementation that keeps the calculation on HANA. I measure the full operation and compare the actual result, including empty data and unusual values.

I also inspect access rules. Reading an underlying table inside database logic does not automatically enforce the DCL attached to a CDS entity used elsewhere. Transaction ownership still matters: the procedure must not introduce an independent commit into the application operation. A short filtered selection normally gains little from an extra SQLScript layer and may become harder to maintain.

In ABAP for Cloud Development, AMDP procedures must be read-only and client-safe, with permitted dependencies. Standard ABAP has broader procedure options. CDS table functions remain read-only. Check the exact language version and release before reusing an example; AMDP does not allow its own COMMIT or ROLLBACK.

Avoid this mistake AMDP is not a universal performance upgrade, and an AMDP method should not introduce an independent commit into an application transaction.

Read and verify this answer

SAP AMDP use and selection guidance

SAP: AMDP language versions and restrictions

SQL-15 AMDP procedure or CDS table function: what is the difference, and when would you use each?

An AMDP procedure runs SQLScript on HANA and is called from ABAP as a method with input and output parameters. A CDS table function exposes a table-shaped result that ABAP SQL and other CDS entities can read. Its implementation is an AMDP database function. Choose a procedure for an operation your ABAP code calls; choose a CDS table function when a calculation must be a reusable query source. Try supported ABAP SQL or CDS first.

Reasoning and example

For example, an ABAP report passes rows to a multi-stage allocation calculation and receives the calculated rows back. A procedure can fit that calling style. If several CDS consumers need the calculated rows in further joins or filters, a CDS table function may fit better. These are design examples, not proof that SQLScript is faster; first check whether the target release already supports the calculation in ordinary SQL or CDS.

The declarations differ. A procedure uses IMPORTING/EXPORTING or CHANGING parameters passed by VALUE; it has no RETURNING parameter. A CDS table function declares its parameters and result columns in DEFINE TABLE FUNCTION. A public static method declared FOR TABLE FUNCTION implements that contract with BY DATABASE FUNCTION and OPTIONS READ-ONLY. Read the CDS entity with SELECT; do not call its implementation as an ordinary ABAP method. An AMDP table function intended only for other AMDP methods is another variant and is not directly callable from ABAP.

A CDS table function cannot write a helper table. ABAP for Cloud Development also restricts AMDP procedures to read-only, client-safe access and permitted dependencies. Standard ABAP allows broader procedure capabilities, but not an independent COMMIT inside AMDP. Check client handling, authorization, numeric types and empty results explicitly. The small code examples show the different contracts using a trivial calculation; that calculation itself does not require AMDP.

READ-ONLY is not a third kind of AMDP; it restricts database changes. In Code, switch between three parts: a procedure with its ABAP call, the CDS table-function definition, and its implementation with a SELECT consumer. The procedure adds 40 and 60 to produce 100; the table function filters rows by an amount threshold.

Avoid this mistake AMDP is the framework, not a synonym for procedure. Not every AMDP function is a CDS table function, and read-only code is not automatically authorized or client-safe.

Read and verify this answer

SAP: AMDP procedures, functions and ABAP Cloud restrictions

SAP: CDS table functions

SQL-16 What do window functions do in ABAP SQL, and when are they useful?

A window function calculates a value across related result rows while keeping each detail row. Use it for a customer total beside each sale, a running total, or a rank. GROUP BY instead combines rows into one result per group. OVER defines the window; PARTITION BY separates groups, window ORDER BY orders calculations, and a frame selects which rows contribute.

Reasoning and example

Customer A has three sales with quantities 100, 100 and 50. SUM(quantity) OVER(PARTITION BY customer_id) adds 250 beside each of the three rows. GROUP BY customer_id with SUM returns just one row for A. A running ROWS frame ordered by the unique sale ID instead produces 100, 200, 250. Its final value matches the partition total, but the earlier values answer a different question.

For descending quantity, RANK gives 1, 1, 3 and DENSE_RANK gives 1, 1, 2. ROW_NUMBER gives a separate position to each row; add sale_id as a tie-breaker if that position must be repeatable. Do not add this unique tie-breaker to RANK when equal quantities should share a rank. The final SELECT ORDER BY controls displayed row order separately from the window ORDER BY.

WHERE filters the input before the window calculation. To rank first and then keep only selected ranks, use a supported outer query or CTE instead of expecting a window alias to be available in the same WHERE. Check the ABAP release and permitted data source; ABAP SQL support does not prove that the same syntax is available in an ABAP CDS definition. A window can avoid extra ABAP aggregation loops, but it still needs measured database cost, especially for sorting and large partitions.

Avoid this mistake Do not replace GROUP BY with a window without checking the required row count, and do not leave tie ordering or a running-total frame implicit.

Read and verify this answer

SAP ABAP SQL: window expressions and worked examples

SAP ABAP Keyword Documentation: window functions

SQL-17 A large report still runs out of memory with PACKAGE SIZE. What do you check?

PACKAGE SIZE limits rows delivered to the target table per iteration; it does not guarantee bounded total memory. Check whether the program appends every package elsewhere, retains copies or references, or builds a large final result. With FOR ALL ENTRIES, the result can first be materialized in an internal system table, so small packages are not a reliable memory guard.

Reasoning and example

Suppose the report reads 1,000 rows per package but appends each package to lt_all. After a million source rows, lt_all still contains a million rows. Instead, process the current package and keep only the output actually needed: a bounded aggregate, streamed output, or an explicitly designed persisted result. A growing output table, cache, log or deep object graph can defeat an otherwise small package.

Start from the dump or failing phase: use ST22 where available, then compare memory snapshots before and after representative processing steps. Find which internal tables or retained objects grow. Distinguish ABAP session memory from expensive database intermediates; a database plan and an ABAP memory snapshot answer different questions. Select fewer columns and rows, avoid retained duplicates, then measure peak memory and runtime on realistic volumes.

Use INTO TABLE for the current package; APPENDING TABLE keeps adding rows. Do not COMMIT while a SELECT loop still needs its open cursor. If processing needs commits between chunks, design separate restartable reads with a stable continuation key and explicit consistency/transaction rules. Simply adding PACKAGE SIZE is not that design. FOR ALL ENTRIES requires particular care because packaging its target can happen after the full matching result is collected internally.

Avoid this mistake A small package count is not proof of small peak memory. Inspect retained data, database work and the selection variant.

Read and verify this answer

SAP ABAP SQL: SELECT INTO and PACKAGE SIZE

SAP Memory Inspector: compare memory snapshots

SAP ABAP COMMIT WORK and open cursors

Official references

SAP ABAP SQL examples and semantics

SAP FOR ALL ENTRIES selection semantics

SAP HANA secondary indexes

SAP AMDP use and selection guidance

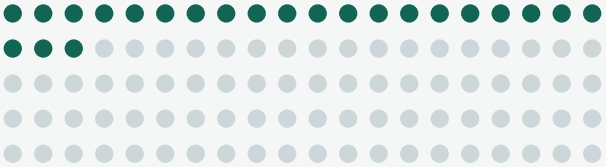
SAP analyzing SQL statements in ADT

OData Gateway and Fiori integration

Separate the OData protocol from SAP implementation technologies. Practice explaining a request, its transaction boundary and the returned metadata or error. Keep legacy Gateway knowledge while choosing supported modern bindings for new services.

Count describes the filtered set; a page is a slice

Example: 100 orders, 23 match, the current page contains 5



100 total orders
23 match the filter

Filter	Order	Page	Return
23 matching records	Stable and deterministic	Skip 10, take 5	5 rows + count 23

Respect requested fields, filters, sorting, paging and count in a custom query provider. Do not count only the returned page.

Count and paging answer different questions. Stable ordering matters when the client requests successive pages.

See the visual atlas for the recall prompt, limits and related questions.

ODATA-01 OData V2 vs V4: what really changes, and what are the benefits of each?

Both let an app read and change business data over HTTP. V4 gives a clearer API contract: separate actions and functions, a date-only type, and more expressive queries. V2 is valuable when existing SAP apps or partners already depend on it. For a new RAP app, I normally choose V4 when the backend and UI support the required features. V4 is not an automatic speed upgrade, and a V2 client cannot simply switch its URL and become a V4 client.

Reasoning and example

One example: a purchase-request app shows 20 orders and lets the user approve one. In V2, an inline count uses $inlinecount=allpages; its JSON collection is inside d.results with __count. In V4, use $count=true; the rows are in value with @odata.count. The screen can look identical, but the client must understand the different contract. Open the two Code examples to compare the same two orders.

Dates and operations: V4 has Edm.Date for a calendar date such as a delivery day. V2 has no equivalent date-only primitive; SAP services commonly use Edm.DateTime with a date-display annotation. V4 also distinguishes a read/calculation function from a potentially modifying action called with POST; V2 exposes service operations through function imports. These differences make intent clearer, but the exact RAP exposure still depends on the supported binding.

Queries: V4 can place query options inside $expand, for example requesting only selected fields of related items. This can avoid unnecessary transferred data when the service supports it. V2 already has $filter, $select, $expand and $batch, so these are not all new V4 features. Both versions support batch changesets; putting requests in one batch does not by itself make the whole batch atomic.

The practical benefit of V2 is compatibility: keep a working partner integration or older Fiori app without forcing a client rewrite. The benefit of V4 is its richer protocol and the matching modern SAPUI5/Fiori elements capabilities on supported releases. SAPUI5 V2 and V4 models have different programming APIs; changing only the service URL is not a migration.

My decision: use a supported V4 binding for the new purchase-request UI, but keep the existing V2 partner endpoint while that consumer migrates. Test metadata, dates, action calls, errors, paging and concurrency. RAP and draft are available in supported V2 scenarios too. Measure real payload size, round trips and backend processing before claiming a performance benefit.

Avoid this mistake Do not claim that V2 only supports XML, V4 introduced $batch, or only V4 supports RAP/draft. Both can use JSON. Protocol support, SAP provider support and UI5 model capabilities are separate checks.

Read and verify this answer

OData V2: JSON format

OASIS: OData V4 JSON format

OASIS: OData V4 protocol

SAP: RAP service binding

SAPUI5: changes compared to the V2 model

ODATA-02 What does $metadata describe?

$metadata tells a client what the service offers. It lists fields, their types, keys, relationships and available operations. It describes the shape of the API, rather than the actual orders or customers. When a field or action is missing in an app, I check this document first.

Reasoning and example

Imagine adding an Approve action to a RAP business object, but no button appears in Fiori. First, I open the metadata of the endpoint the app actually calls. If the action is absent, I check whether the projection and service expose it. Debugging the action handler would not help yet because the client cannot see that action.

If the action is present, I next check the UI annotations, template support and the metadata used by the running app. The same approach works for a missing field. Adding a database column does not automatically add a property to the public service.

Avoid this mistake A database field is not automatically visible in the service just because it exists in persistence.

Read and verify this answer

OASIS OData V4 protocol

ODATA-03 What are the main query options?

OData query options tell the service which data the client needs. $filter chooses rows, $select chooses fields and $orderby sorts them. $top and $skip request a page, while $expand includes related data. I check which options the service supports and keep the request as small as the use case allows.

Reasoning and example

For a list of open orders, I request only the order number, customer and amount. I filter for open status and use a stable order, such as order number, before requesting the first page. This avoids sending full orders with every item when the screen only needs a short list.

If the user opens one order, I can request its items separately or use a supported $expand. Free-text $search is useful only when that service implements it. Count and search details also differ by protocol version. An option appearing in the OData standard does not mean an individual SAP endpoint accepts it.

Avoid this mistake The existence of an OData option in the standard does not guarantee that every SAP service implements it.

Read and verify this answer

OASIS OData V4 protocol

ODATA-04 How do paging and total count work in V2 and V4?

The total count tells the client how many rows match the filter, not how many fit on one page. Filter first, then count and page with a stable order. V2 commonly uses $inlinecount=allpages; V4 uses $count=true for an inline count. When the server returns a next link, the client should follow it.

Reasoning and example

A filter finds 500 open orders, but the screen displays 20 at a time. The response can correctly contain 20 orders and a total of 500. Returning 20 as the total would make the screen think there are no more pages. The /$count resource is another way to request a count.

Sorting matters too. Without a stable order, rows may move between pages even when the underlying data has not changed. Concurrent inserts or deletes can still affect offset paging. In a custom provider, I use the actual response API for that framework; I do not assume every Gateway implementation has the same method for setting the count.

Avoid this mistake Use the response API of the actual provider. Do not assume every Gateway implementation has one generic SET_COUNT method.

Read and verify this answer

OASIS OData V4 protocol

ODATA-05 What is the difference between actions and functions?

In OData V4, a function reads or calculates a result without requesting a business change. An action can change business data and is called with POST. An operation can belong to a particular entity, a collection, or neither. V2 uses function imports for its operation patterns, so I check the version before explaining the endpoint.

Reasoning and example

Consider a request with a calculated delivery estimate and a Submit button. A supported function could return the estimate without saving or approving anything. Submit is an action because it moves the request to a new business state. The action must check whether that transition is allowed.

A bound action might operate on request 4711, while an unbound operation is not attached to a specific entity. These choices are part of the service definition and client contract. I do not implement approval behind GET: a read can be repeated by clients or infrastructure, and it should not unexpectedly approve a document.

Avoid this mistake An HTTP GET should not create or approve business documents.

Read and verify this answer

OASIS OData V4 protocol

ODATA-06 How do ETags prevent lost updates?

An ETag is a version token for the data a client read. The client sends it back in If-Match when changing that data. If someone else has changed the version, the service normally rejects the request with HTTP 412. The user can then reload and resolve the conflict instead of losing the other person's changes.

Reasoning and example

Alice and Ben both read order 4711 with the same ETag. Alice changes the amount and saves, so the server now has a new version. Ben then sends his old ETag with a change to that same order. The service should reject Ben's stale update instead of silently overwriting Alice's work.

I test this with independent clients and inspect both the response and saved data. In RAP, the configured ETag field must change when the relevant data changes. Sending If-Match with a wildcard removes the specific version comparison. An ETag also does not replace authorization: Ben still needs permission to change the order.

Avoid this mistake An If-Match wildcard can skip the intended version comparison. A timestamp also does not give a user permission to update.

Read and verify this answer

OASIS OData V4 protocol

ODATA-07 Is a batch request one database transaction?

A batch is an envelope containing several requests. It does not automatically make them one transaction. Changesets provide atomic groups in multipart batches; OData 4.01 JSON batch has atomicity groups. I check what the service supports and inspect every inner result. An outer success response does not mean every operation succeeded.

Reasoning and example

Suppose a batch contains an order creation and an unrelated address change. These may belong to different groups and finish differently. If creating an order and its items must succeed together, the requests need the supported atomic grouping, or a supported deep-create operation for that structure. Deep create still needs correct backend transaction handling.

I test an invalid item and verify that the intended group leaves no partly created order. I also test a failure in a separate group and inspect each inner response. A service called V4 does not automatically implement every V4.01 JSON batch feature, so I verify its actual protocol and provider capabilities.

Avoid this mistake HTTP 200 for a batch does not mean every operation inside it succeeded.

Read and verify this answer

OASIS OData V4 protocol

OASIS OData 4.01 JSON Format: Batch Requests

ODATA-08 What are MPC_EXT and DPC_EXT in classic Gateway?

In classic Gateway, MPC_EXT describes or extends the service model. DPC_EXT handles data requests and their errors. Customer changes belong in these extension classes rather than generated base classes. I keep reusable business rules in separate classes where practical. RAP uses a different model, with behavior implementations instead of this classic provider structure.

Reasoning and example

In a SEGW service, a list request may reach GET_ENTITYSET in DPC_EXT. I use that method to read the requested collection and map the result to the service response. CREATE_ENTITY, or the relevant deep-create implementation, handles an incoming creation request. Model changes and metadata belong on the model-provider side.

For a pricing rule used by several entry points, I put the calculation in a separate class and call it from the provider. This lets tests check the rule without building an HTTP request each time. It also reduces duplicated logic. I still test the provider itself for request mapping and correct business error responses.

Avoid this mistake Putting all business rules in DPC_EXT makes them harder to reuse and test. MPC_EXT still matters when the model needs extending.

Read and verify this answer

SAP Learning on OData

ODATA-09 How do authentication authorization and CSRF differ?

Authentication tells the system who is calling. Authorization decides what that caller may do. CSRF protection helps stop unwanted requests that misuse a browser's logged-in session. Gateway writes commonly need a fetched CSRF token and the matching session. I check the endpoint's authentication flow because the exact requirements can differ.

Reasoning and example

An app reads an order successfully but gets 403 when saving. I inspect the error first. A missing CSRF token, missing service permission and a denied business action need different fixes, even if they use the same HTTP status. For a token problem, I check how the client fetched the token and kept the required session context.

If the user lacks approval permission, fetching another token will not help. The backend must still deny that action. CSRF protection is not a universal requirement imposed by the OData standard itself, and a valid token never grants the user access to a business operation.

Avoid this mistake CSRF is not a universal OData-standard requirement and a valid token does not grant business permission.

Read and verify this answer

SAP Learning on OData

ODATA-10 How do you design dates media and errors?

I use the service's documented types for dates and timestamps, with a clear time-zone meaning. Files need the supported binary endpoint and correct content type. Errors should tell the client what failed without exposing secrets or internal details. I follow the actual Gateway or RAP error model because their implementation APIs are different.

Reasoning and example

An invoice date such as 12 October is a calendar date; converting it between time zones should not turn it into the previous day. An event timestamp represents a point in time and needs the agreed time-zone handling. V4 has Edm.Date, but V2 does not use the same date representation. I test both against the actual consumer.

For an invoice PDF, I verify the binary content, media type and download behavior. For a rejected update, I use Gateway's supported exception and message facilities, or RAP FAILED and REPORTED as required. The response should identify the problem clearly while keeping credentials and implementation details out.

Avoid this mistake Edm.Date is a V4 type; do not assume identical date serialization in V2.

Read and verify this answer

OASIS OData V4 protocol

ODATA-11 How do you troubleshoot a slow or failing Fiori request?

I start with the exact failing or slow request from the browser. Then I check its response, service metadata and backend logs. For performance, I measure network size, backend time and database work before changing code. I also repeat the request with the affected user's permissions, because a developer preview may use a different setup.

Reasoning and example

A Fiori list takes eight seconds to open. The network trace shows that it requests every order together with all items through $expand. Each database statement may be reasonably fast, but the combined result is huge. I compare a small page request with the expanded request and measure row counts, payload size and total time.

If the request fails instead, I keep the timestamp and correlation ID to find the matching server log. I check routing, identity, authorization and metadata before blaming the UI. Traces belong in an authorized test context. A working ADT preview does not prove that the deployed app reaches the same endpoint under the same user.

Avoid this mistake A working ADT preview does not prove a deployed app's routing, identity, metadata cache and backend authorization are correct.

Read and verify this answer

SAP Learning on OData

ODATA-12 How do you evolve an existing API safely?

I treat an API as an agreement with every app that uses it. Changing keys, field names, types or action behavior can break those apps. I prefer additions that existing clients can tolerate and use a new version for breaking changes. Before retiring an old endpoint, I test the old clients and confirm they have moved.

Reasoning and example

Suppose an existing integration reads a field named CustomerNumber. Renaming it to BusinessPartner might make the new model clearer, but the old client still sends and reads the original name. Saved UI state or generated client code can also depend on that field. I identify these consumers before changing the public contract.

A separate RAP service can run alongside a classic Gateway service while consumers migrate. I document the new behavior, test both contracts and track actual use of the old endpoint. Even an added field needs compatibility checks for strict clients. Creating a new RAP service does not automatically update a V2 consumer or preserve every old business rule.

Avoid this mistake A new RAP service does not automatically update an old V2 client or preserve every behavior that client relies on.

Read and verify this answer

SAP Learning on UI service bindings

Official references

SAP Learning on OData

SAP Learning on UI service bindings

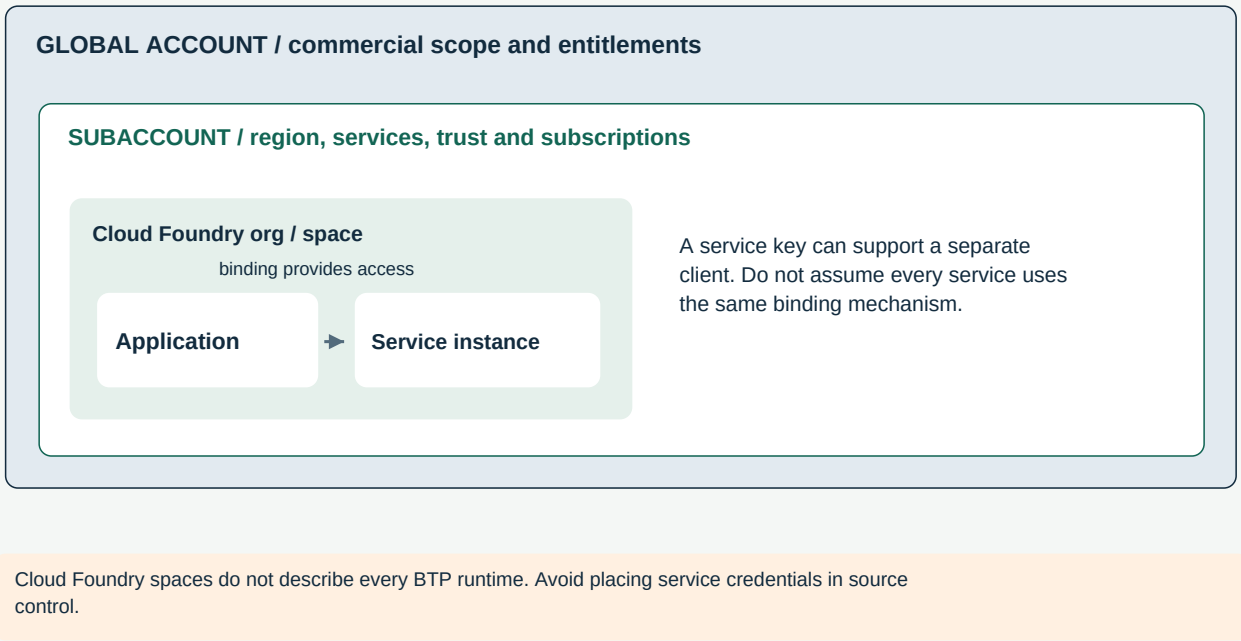
OASIS OData V4 protocol

BTP and cloud integration for ABAP developers

Focus on what an ABAP developer must design, configure or troubleshoot. Separate the ABAP development model from its hosting location, and distinguish Cloud Foundry app mechanics from the managed ABAP environment.

BTP organization and runtime are different axes

A schematic Cloud Foundry example



Cloud Foundry spaces do not describe every BTP runtime. Avoid placing service credentials in source control.

Read the nesting as an organizational example, then distinguish the application, service instance and credentials.

See the visual atlas for the recall prompt, limits and related questions.

BTP-01 What is BTP and how is ABAP Cloud different?

SAP BTP provides services for building apps, connecting systems and handling data. ABAP Cloud is a way of developing ABAP with allowed language features and released APIs. You can use that model in BTP ABAP environment and supported S/4HANA systems. BTP is therefore a platform; ABAP Cloud describes how you develop.

Reasoning and example

A team wants a supplier portal separate from its ERP system. It might build the ABAP part in BTP ABAP environment and call a released ERP API. Another team needs a field and validation close to an S/4HANA document. It might use ABAP Cloud directly in that supported S/4HANA system instead.

Both teams follow ABAP Cloud rules, but their applications run in different places. That difference matters for latency, identity and transactions. BTP ABAP environment is a managed ABAP system with its own development and delivery tools. It is not a regular Cloud Foundry application that you deploy with cf push.

Avoid this mistake Do not call BTP ABAP simply a normal application container deployed with cf push.

Read and verify this answer

SAP ABAP Cloud FAQ

BTP-02 How do global accounts subaccounts entitlements and spaces relate?

A global account is the top-level place for managing your BTP use. Subaccounts separate applications and administration, for example by region or environment. Entitlements say which services and quotas you can use. For Cloud Foundry apps, organizations and spaces organize deployment. These layers have different jobs, so I check them separately when something cannot be created.

Reasoning and example

A developer wants to create a service in a test subaccount but cannot find the required plan. I check whether the global account has that entitlement and whether enough quota was assigned to the subaccount. I also check that the service is available in the selected region and that the developer has the necessary role.

If it is a Cloud Foundry application, I then check its organization and space. I do not apply that same space-based procedure to every BTP service. An entitlement permits consumption; it does not mean a running service instance already exists. The actual creation steps depend on the selected service and account setup.

Avoid this mistake An entitlement is not a runtime container, and every BTP service does not require the same CF-space workflow.

Read and verify this answer

SAP BTP documentation

BTP-03 What is the difference between IaaS PaaS and SaaS?

The difference is how much you operate yourself. With IaaS, you receive infrastructure and manage more of the software above it. With PaaS, the provider runs an application platform and you build on it. With SaaS, you use an existing application. In each case, I check who manages security, configuration and data.

Reasoning and example

Imagine a business needs an approval solution. With infrastructure services, its team may have to manage the operating system and application runtime before building the solution. A managed application platform removes much of that runtime work, but the team still develops and maintains its own approval code. A ready-made business application supplies the business capability itself.

A managed ABAP environment is a useful platform example: developers work within the available ABAP services and rules. They still own their custom application's behavior and access setup. SAP HANA Cloud is a managed database service, so I would not describe it as a finished business application simply because it is delivered as a service.

Avoid this mistake SAP HANA Cloud is a managed database service, not a generic example of a finished business SaaS application.

Read and verify this answer

SAP BTP documentation

BTP-04 What are a service instance binding and service key?

A service instance is a service resource you have created. A binding connects an application to that resource and gives it the required connection information. A service key supplies credentials for a supported external-client use case. These are different objects; creating a binding does not always create a separate service key.

Reasoning and example

Suppose a Cloud Foundry app needs a database service. I create the supported service instance and bind the app to it. The app reads the supplied connection settings rather than keeping a password in source code. In Cloud Foundry, this information is commonly available through VCAP_SERVICES, though other delivery methods exist.

An external tool may instead need a service key if that service supports the use case. I protect and rotate those credentials too. When a binding changes, I check whether the app needs a restart or restage to use the new settings. A local default-env.json file is a development convention, not the universal way Cloud Foundry delivers credentials.

Avoid this mistake default-env.json is a local development convention, not the universal Cloud Foundry credential transport.

Read and verify this answer

Cloud Foundry service bindings

BTP-05 What do containers buildpacks droplets and routes do?

In Cloud Foundry, a buildpack prepares an app with the runtime it needs. Staging produces a runnable package called a droplet. Containers run isolated app instances, and routes send requests to them. One app can have several instances. Reliability still depends on those instances and the services they need, not just on having a container.

Reasoning and example

A Java app is staged with a suitable buildpack, which prepares the runtime and application package. Two app instances then run that package. A route lets users reach the app without knowing which instance handles their request. If one instance fails, the design needs enough healthy capacity and working dependencies to keep serving users.

During blue-green deployment, traffic moves from the old version to the new one. I still check database compatibility and what happens to requests already running on the old version. Changing a route does not repair an incompatible database change. A container is also not the same as a full virtual machine.

Avoid this mistake A container is not a standalone virtual machine, and one application does not always mean exactly one container.

Read and verify this answer

Cloud Foundry: how applications are staged

BTP-06 What do destinations and Cloud Connector solve?

A destination stores connection and authentication settings for a service call. Cloud Connector allows configured cloud access to selected on-premise resources. Together they help an app reach the backend without putting passwords in code. I still check the caller's permissions and the API's release status. Connectivity alone does not make an internal SAP API supported.

Reasoning and example

A BTP app needs to read orders from an on-premise system. I check the destination, the configured Cloud Connector connection and the specific resource exposed through it. Then I verify the certificate or token and the user's backend permissions. A connection can be technically available while the business request is still forbidden.

When troubleshooting, I locate the failing step instead of changing all settings at once. Cloud Connector exposes configured resources; it is not an unrestricted opening into the entire company network. I also confirm that the chosen order API is allowed for this extension. A successful request to an internal endpoint is not proof that SAP supports that use.

Avoid this mistake Cloud Connector is not simply an unrestricted inbound opening of the entire corporate network.

Read and verify this answer

SAP BTP documentation

BTP-07 How do remote transactions differ from local RAP transactions?

A remote API usually saves data in a separate transaction. If my local RAP transaction rolls back, the remote system may still keep its change. I therefore plan how to find the original result, retry safely and handle partial success. A timeout means the outcome may be unknown; it does not prove the remote operation failed.

Reasoning and example

Our app asks a remote payment service to create a payment. The service creates it, but the response is lost. If we simply send a new creation request, we may pay twice. I use the service's documented idempotency mechanism, meaning the same request identifier reuses the original business effect, or query the payment by a stable reference.

The local app keeps a recoverable pending or uncertain status until it knows the result. If one side succeeds and the other cannot complete, a supported cancellation or other compensating business step may be needed. Calling HTTP inside a RAP action does not make the two systems share one rollback.

Avoid this mistake Calling HTTP inside an action does not make the remote change part of the local RAP database transaction.

Read and verify this answer

SAP BTP documentation

BTP-08 What makes event-driven integration reliable?

Reliable event handling assumes messages can be delayed or delivered again. I publish events in a way that matches committed business data and use durable delivery when needed. The consumer remembers a stable event or business identifier so repeating a message does not repeat the effect. I also define retries and how failed messages reach an operator.

Reasoning and example

An order submission should start one approval workflow. If an event is delivered twice, the consumer checks the request and event identifiers and finds the workflow already started. It must not create another process just because the transport delivered another copy. The publication mechanism must also avoid announcing a submission that later rolls back.

I test a temporary outage, a duplicate delivery and a message that repeatedly fails. After bounded retries, the failed message needs a visible recovery path, such as a dead-letter queue where supported. Ordering and retention depend on the configured broker and protocol. I do not promise global ordering or exactly-once business processing without proving the complete design.

Avoid this mistake Do not promise exactly-once end-to-end processing or global ordering without a specific proven design.

Read and verify this answer

SAP BTP documentation

BTP-09 Which tools matter to an ABAP developer on BTP?

I use ADT in Eclipse for ABAP development. Fiori tools support UI development, while the BTP cockpit and service tools manage accounts, access and connections. The correct tool depends on where the application runs. For ABAP work, I should be able to find a released API, run checks and tests, and investigate a failed service call.

Reasoning and example

To build an ABAP extension, I first connect ADT to the intended ABAP system. I check available APIs and their release status there, then develop and test the service. If the project includes a Fiori app, I use supported Fiori tooling for that UI rather than assuming the ABAP deployment also deploys every frontend artifact.

For a connection failure, I may need the BTP cockpit or a service-specific monitor to inspect roles, destinations or communication setup. I verify tool versions against the target system. Older diagrams showing only Neo and Cloud Foundry, or the historical SAP Cloud Platform name, do not fully describe today's BTP services.

Avoid this mistake The historical name SAP Cloud Platform and a Neo-versus-CF-only diagram are not a complete current BTP model.

Read and verify this answer

SAP ABAP Cloud FAQ

BTP-10 How would you secure and operate a side-by-side extension?

I secure both the extension and its calls to other systems. Users and technical identities get only the permissions they need, and credentials stay outside source code. I monitor failures and response times, with a clear owner for recovery. Production readiness also includes safe deployment, API compatibility and a tested way to recover data or service availability.

Reasoning and example

A supplier portal works in development, but production needs more than a successful API call. I check which identity reads each supplier's data, how credentials are stored and revoked, and whether logs hide sensitive fields. Test and production use separate configuration so a test request cannot accidentally post a production order.

For an outage, the operator needs a request identifier, a visible status and instructions for safe retry. I also define who owns failures in the ERP API and who owns portal failures. Before changing a dependency, we test compatibility. Backup and recovery procedures must fit the actual services rather than rely on an assumed platform guarantee.

Avoid this mistake Successful connectivity from a developer machine is not evidence of secure production operation.

Read and verify this answer

SAP BTP documentation

Official references

SAP ABAP Cloud FAQ

Cloud Foundry service bindings

Cloud Foundry environment variables

SAP BTP documentation

Cloud Foundry: how applications are staged

RICEFW, enhancements, and workflow

Use this segment to connect classic delivery skills with current S/4HANA extension choices. Explain the business contract, operational recovery, and edition constraints before naming a transaction or framework. Classic SAP GUI transactions below are diagnostic examples for environments where they exist, not public-cloud instructions.

One business effect, even if the message arrives twice

Idempotency needs a key, a state check and safe persistence

Delivery	Business event ID
Attempt 1	EVT-204
Retry	EVT-204

→ **Consumer transaction**

Check event identity and current state. Apply an allowed transition and record processing safely together.

Current state	Incoming request	Result
Pending / new event	Approve	Apply once
Approved / same event	Approve again	Return prior outcome or no-op
Changed / stale version	Old approval	Reject or reconcile; do not overwrite

A check-then-write race is not solved by an in-memory set. Enforce the concurrency and persistence boundary.

The event identity and current business state decide whether to apply, ignore or reject the request.

See the visual atlas for the recall prompt, limits and related questions.

RICEFW-01 What does RICEFW mean, and how would you scope a deliverable?

RICEFW groups work into Reports, Interfaces, Conversions, Enhancements, Forms and Workflows. I still need a clear business result before estimating any of them. I ask what data is involved, who uses it and what happens on failure. I also check the target system, supported options and who will operate the solution.

Reasoning and example

A request says 'build one supplier conversion.' That sounds small, but it hides several jobs: cleaning source data, mapping fields, checking mandatory values, loading through a supported interface and proving the right suppliers were created. I clarify duplicate rules and expected volume before giving an estimate.

I also define acceptance checks, such as matching supplier counts and recording every rejected row. The cutover plan needs an owner and a safe restart procedure. If standard migration functionality already meets these needs, custom code may be unnecessary. The RICEFW label helps organize delivery; it does not select the technology or tell us the true effort.

Avoid this mistake RICEFW is a delivery classification, not a requirement to build custom code when standard functionality already fits.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

RICEFW-02 How do you choose between a standard API, IDoc, migration tooling, and BDC?

I choose the supported business interface that fits the task and target system. A one-time migration may suit migration tooling; ongoing integration may suit an API or IDoc. BDC can be a classic-system fallback, but screen changes make it fragile. I check required fields, volume and recovery instead of choosing by name alone.

Reasoning and example

For a one-time supplier migration, I first check the delivered migration objects and supported fields. If they cover the requirement, building a permanent custom interface may add needless maintenance. For a continuing partner feed, I compare the available APIs or IDoc scenarios and how they report and recover from individual errors.

BDC drives screens, so its suitability depends on the screen flow and configuration. It is not a general ABAP Cloud option. A BAPI may also lack a required field or be unavailable for direct cloud development. I verify release status and Clean Core classification individually; the word BAPI does not automatically mean Level B or suitability for mass loading.

Avoid this mistake Check each BAPI's classification and supported use. Its name alone proves neither Clean Core Level B nor suitability for mass loading.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

SAP IDoc control record, message type, basic type, and extension

RICEFW-03 How do BAdIs, customer exits, and user exits differ?

A BAdI is an object-oriented extension interface. Classic customer exits can add function, screen or menu behavior through enhancement projects. User exit often means an older application-specific routine. I check the actual extension's timing and allowed use. A BAdI is only suitable if it supports the needed change in the target release.

Reasoning and example

Suppose an order must reject an invalid reference before saving. I look for a supported extension that runs early enough and can report that error. An extension that runs only after posting would not solve the requirement, even if its name sounds relevant. For a BAdI, I also inspect whether filters or multiple implementations are supported by its definition.

In classic systems, SMOD describes customer-enhancement components and CMOD manages their projects. Some SD routines named USEREXIT use another older mechanism. These names are not interchangeable. A released BAdI is a good choice when it fits, but being a BAdI alone does not prove ABAP Cloud availability or eliminate upgrade testing.

Avoid this mistake CMOD/SMOD exits and routines named USEREXIT are not the same mechanism. Filters and multiple implementations depend on the BAdI definition.

Read and verify this answer

SAP explicit and implicit enhancement options

RICEFW-04 What is the difference between implicit and explicit enhancement options?

Explicit enhancement options are places deliberately marked for extensions. Implicit options exist at predefined positions without a marker there. An enhancement point adds code; an enhancement section can replace the defined section's implementation. These are extension mechanisms. Explicit enhancement does not automatically mean editing SAP's original source directly.

Reasoning and example

A standard program contains an explicit enhancement point after a calculation. A customer implementation can add permitted logic at that point. An implicit option may instead be available at a predefined location, such as a procedure boundary, without a dedicated enhancement marker in that location. I inspect the actual option and surrounding behavior before using either.

An enhancement spot organizes explicit options; an enhancement implementation contains the customer logic. Replacing an enhancement section deserves extra care because it can remove standard processing the application needs. These classic mechanisms are also different from released ABAP Cloud extension contracts. I check target support instead of assuming every available editor option is an approved extension.

Avoid this mistake Explicit enhancement does not simply mean modifying SAP source. Source modification is a separate technique.

Read and verify this answer

SAP explicit and implicit enhancement options

RICEFW-05 How would you design a reliable interface?

I define what the message means, who may send it and how success or failure is reported. Synchronous calls fit immediate answers; asynchronous processing can let systems work at different times. Both need clear duplicate and recovery rules. I choose the protocol after understanding these needs, including what to do when the outcome is unknown.

Reasoning and example

An interface posts an invoice, but the sender times out before receiving the result. The receiver may already have saved it. I use a stable request reference and a documented way to query or reuse the original result before retrying. Otherwise, a temporary network problem can become a duplicate invoice.

A temporary outage may justify delayed retries. An invalid company code needs correction instead of repeated automatic posting attempts. I record correlation information so support can follow the same request through both systems. Whether the interface uses OData, SOAP or IDoc, a transport acknowledgment or accepted message is not proof that the business document was posted.

Avoid this mistake A delivered or accepted message does not prove that the receiver posted the business document.

Read and verify this answer

SAP IDoc communication status and application outcome

RICEFW-06 How would you troubleshoot a failed IDoc in a classic system?

I find the IDoc for the business request and read its control data, segments and status history. The detailed error usually shows whether the problem is routing, mapping, master data, permissions or posting logic. I fix that cause before retrying. I also check for an existing business document so recovery does not post it twice.

Reasoning and example

A supplier-order IDoc has an application error because a material is unknown. In a classic system, WE02 or WE05 can show the IDoc and its messages. I inspect the incoming material value and its mapping, then correct the master data or mapping through the approved process. Changing a destination would not fix that particular error.

If the IDoc never reached application processing, partner profiles, ports, process codes or communication logs may be the right next check. Where appropriate and authorized, BD87 supports reprocessing. After retry, I inspect the linked business result. Repeated retries or a manual status change can hide the original problem and may create duplicate effects.

Avoid this mistake Changing a status or repeatedly reprocessing without checking business effects can hide the cause or create duplicates.

Read and verify this answer

SAP IDoc control and message structure

RICEFW-07 What makes a data conversion safe and supportable?

I separate cleaning and checking the data from loading it. Every source record needs a stable identity, a visible result and a safe restart rule. I use supported business interfaces and reconcile business totals after loading. Trial runs reveal bad data and realistic runtime. The cutover plan must also say who stops, restarts or repairs the load.

Reasoning and example

A document migration loads 10,000 source records. Matching that count is not enough: some documents may have wrong currencies, values or relationships. I reconcile counts and amounts by company and currency, and keep rejected records separate from accepted records whose follow-up work is still pending. Trial loads help discover these problems before cutover.

If one package fails, the rerun must skip or safely identify documents already posted. Stable source references help link the source to the actual business results. I also plan load order, data freezes and recovery ownership. Direct table updates may bypass application rules, so a technically completed write is not evidence of a correct migration.

Avoid this mistake A completed load can still contain wrong totals, relationships or business states. Check those results separately.

Read and verify this answer

SAP migration cockpit: objects, mapping, simulation and monitoring

RICEFW-08 How do you choose between SAPscript, Smart Forms, and Adobe output forms?

I start with the output framework supported by the business application. Existing systems may use SAPscript or Smart Forms; other scenarios use Adobe-based forms with the required rendering setup. S/4HANA does not use one universal output solution for everything. I check document needs, languages and delivery channels before choosing or replacing a form technology.

Reasoning and example

An invoice PDF looks correct in one test, but that proves only part of the output process. I separately check how the application decides to generate it, how data reaches the form, how the layout renders and how the result reaches the customer. Email routing or spool processing can fail even when the PDF itself is fine.

My test set includes long item lists, page breaks, totals, several languages, fonts, attachments and reprints. I also verify the application's actual output framework before proposing a BRF+ or Adobe change. An Adobe output form need not be interactive, and not every S/4HANA application follows the same framework or migration path.

Avoid this mistake All S/4HANA output is not automatically BRF+ plus Adobe, and an output form need not be an interactive form.

Read and verify this answer

SAP S/4HANA output control and framework coexistence

RICEFW-09 How do classic Workflow, Flexible Workflow, and SAP Build Process Automation differ?

Classic SAP Business Workflow models processes in the ABAP system. Flexible Workflow offers configurable approval scenarios using SAP workflow concepts. SAP Build Process Automation runs on BTP and supports automation across systems. I first check the delivered business scenario, then choose the option that fits its approval rules, integrations and operating needs.

Reasoning and example

For purchase approval, I first inspect the supported Flexible Workflow scenario and its available start conditions, steps and extensions. It may already cover the business requirement without a custom process. A cross-system approval involving several applications may instead need a BTP-based solution, depending on available connectors and requirements.

These options are not automatic substitutes for each other. Flexible Workflow builds on SAP workflow concepts, and its features vary by business scenario. A custom RAP Submit action also does not create a complete approval process by itself. We still need process creation, approver selection, decisions, callbacks, permissions and a way to handle failures.

Avoid this mistake Flexible Workflow uses SAP workflow concepts, and available features depend on the business scenario.

Read and verify this answer

SAP workflow solution overview

SAP flexible workflow scenario and configuration concepts

RICEFW-10 What are a workflow container, task, work item, and agent?

A workflow container holds the process data. Bindings pass that data between workflow components. A task defines work; a work item is one running instance of that task. Agent determination finds the person who should do it. The process also needs a clear response when nobody can act or a decision cannot be completed.

Reasoning and example

A request for 5,000 EUR starts an approval workflow. The container might carry the request ID and the information needed to choose an approver. Bindings pass the relevant values to the approval task. The created work item is the actual item the selected approver handles; the task is its reusable definition.

I avoid copying every field if that would leave the workflow using stale business data. I also test the case where no valid approver is found and define an error or fallback route. An email notification is only a notification, not the work item itself. Being selected as an agent does not remove the need for business authorization.

Avoid this mistake A notification email is not the work item itself, and being selected as an agent does not replace business authorization checks.

Read and verify this answer

SAP workflow solution overview

RICEFW-11 How should a RAP submit action start an approval process reliably?

Submit should check the request and change its state through RAP's transaction rules. The external approval process should start in a way coordinated with a successful save. I use the supported event or integration mechanism for that release. Stable identifiers and duplicate handling stop retries from starting several workflows for the same submission.

Reasoning and example

A request moves from Created to Submitted. Only after that change is successfully committed should the integration create its external approval process using the supported mechanism. If save fails, we must not leave an approval waiting for a request that never became submitted. If delivery is repeated, the receiver should find the existing process rather than create another.

I link the request version and workflow instance so an authorized callback can later update the correct state. Draft activation is a separate RAP operation unless the application explicitly coordinates it with submission. I verify the target's event capabilities instead of assuming one universal implementation. Calling a workflow endpoint before save completes is not a reliable substitute.

Avoid this mistake Calling an external workflow endpoint before save succeeds can start approval for a document that never committed.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-12 How do you handle duplicate events, stale approvals, and callback retries?

I assume events and callbacks can arrive more than once. The receiver checks a stable event or process identifier before repeating work. An approval also checks the current request version, state and caller permissions. An old decision must not approve a changed or withdrawn request. Repeated successful callbacks should have a stable, safe result.

Reasoning and example

A manager receives approval for version one of a request. Before the manager answers, the requester withdraws it or changes a material detail. When the old approval arrives, the callback compares the stored workflow link and request version with the current record. It rejects or handles the stale decision according to the business rule instead of approving the new data.

A retry of a decision already applied should return a safe result without applying the effect twice. I retain an audit trail and test deadline escalation racing with the original approver. A queue or HTTP success response alone does not provide exactly-once business behavior; the application must enforce the intended transitions.

Avoid this mistake A queue or HTTP success response does not automatically prevent duplicate business effects.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-13 How would you diagnose a workflow that did not start or is stuck?

I find the step where progress stopped: the business save, event delivery, start condition, workflow creation, approver selection or callback. Then I inspect the data and logs for that step. Classic and cloud systems provide different administration tools. I do not assume a missing inbox item means the workflow never started.

Reasoning and example

A request is submitted but nobody sees an approval. First, I confirm that submission actually committed. Then I check whether an event was sent and whether the start conditions selected the intended workflow. If a workflow exists, I inspect its context and agent determination. An existing work item with no recipient needs a different fix from a missing start event.

In classic systems, SWEL can show recorded events when tracing is enabled, and SWI1 can help inspect work items and logs. Cloud scenarios use their supported monitors. No SWEL entry proves little if tracing was off. A session breakpoint may also miss background processing that runs under another user.

Avoid this mistake An absent trace entry proves little if tracing was disabled, and a session breakpoint may not catch background processing under another user.

Read and verify this answer

SAP workflow solution overview

RICEFW-14 How would you test a workflow or enhancement end to end?

I test the full business result, including failures and recovery. For workflow, that means approvals, missing approvers, substitutions, deadlines, duplicate callbacks and stale decisions. For an enhancement, I check when it runs and what it does to the surrounding transaction. Unit tests help, but real integration tests must prove actual delivery, permissions and saved state.

Reasoning and example

A request is submitted, then changed before the manager answers. My test sends the old approval and checks that it cannot silently approve the new request version. I inspect the current business state and audit history, not just the inbox message. Other cases cover a missing approver and a failed callback that is safely retried.

For an enhancement, I test a normal save and a rejected save to check its timing and transaction effects. A mock endpoint is useful for checking request contents and simulated errors. It does not prove the real system's identity setup, event delivery or callback behavior. I record which checks used real integration and which used doubles.

Avoid this mistake A unit-test pass or inbox screenshot does not prove real permissions, saved data or successful process completion.

Read and verify this answer

SAP flexible workflow scenario and configuration concepts

RICEFW-15 Which S/4HANA changes can affect existing ABAP developments?

I check the simplification information and custom-code findings for the target S/4HANA release. Important areas can include MATDOC, ACDOCA, Business Partner and changed output or transactions. I trace how the existing application uses the affected objects. Replacing a table name mechanically is not enough; the business meaning and supported access may have changed.

Reasoning and example

A report reads inventory data directly from old tables. Before changing it, I list its reads, writes and called transactions and check the target release's recommended access. I compare the old report's intended result with the new data model, then test totals, filters and runtime using representative business cases.

The same care applies to finance and Business Partner changes. Business Partner integration does not mean every customer and supplier table simply disappeared. Compatibility mechanisms and migration prerequisites vary. I use release-specific findings and functional tests to decide the change, rather than assume that replacing a reference with MATDOC or ACDOCA preserves the original behavior.

Avoid this mistake Replacing a legacy table name with MATDOC or ACDOCA mechanically is not a migration strategy.

Read and verify this answer

SAP S/4HANA 2025 upgrade guide

Official references

SAP explicit and implicit enhancement options

SAP S/4HANA output control and framework coexistence

SAP workflow solution overview

SAP flexible workflow scenario and configuration concepts

SAP S/4HANA 2025 upgrade guide

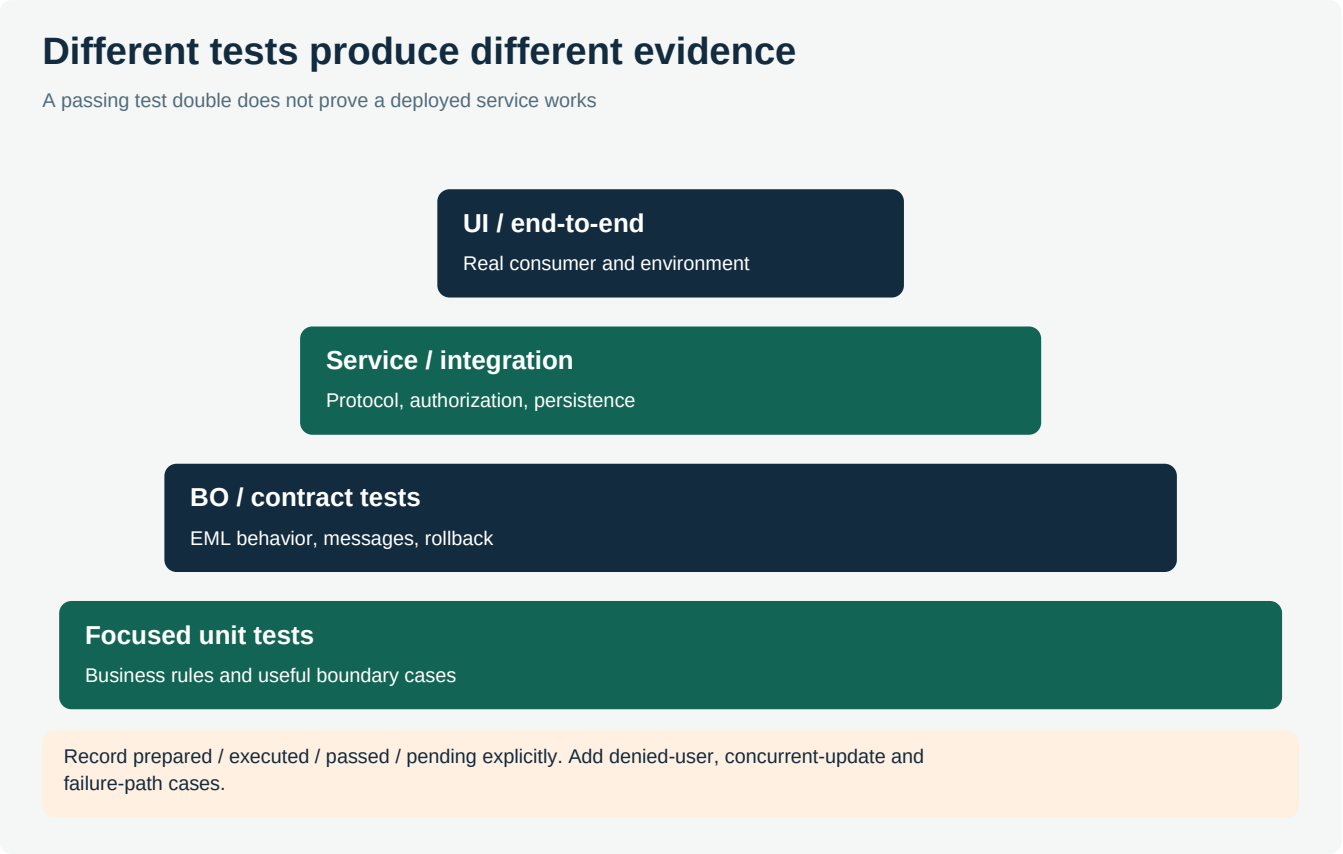
SAP migration cockpit: objects, mapping, simulation and monitoring

SAP IDoc control record, message type, basic type, and extension

SAP IDoc communication status and application outcome

Testing security and delivery

Strong candidates explain how they know a solution works. Distinguish isolated unit tests, database or CDS tests, BO integration tests and real UI or service tests. State precisely which environment and behavior each result covers.



The widening base suggests many focused checks, but there is no mandatory numerical ratio.

See the visual atlas for the recall prompt, limits and related questions.

TEST-01 What makes a useful ABAP Unit test?

A useful test checks a real rule with known inputs and a clear expected result. It should behave the same each time and not depend on another test or shared production data. I cover errors and limits as well as success. A test that only creates an object or reaches a line of code says little about whether the business behavior is correct.

Reasoning and example

For a date-range validator, I prepare a valid range, equal start and end dates, and an end date before the start. The expected result follows the actual business rule: equal dates might be allowed or rejected, so I make that decision explicit. The test name describes the rule that would be broken.

I keep the validator real and control any outside dependency, such as the current date. A failing assertion should explain which result was wrong. High coverage is useful for finding untested areas, but a test can execute every line without checking the right outcome. I therefore review the cases and assertions, not only the percentage. Fresh mutable fixtures belong in setup; do not rely on another test having run first.

Avoid this mistake High statement coverage alone does not prove that assertions or business cases are meaningful.

Read and verify this answer

SAP ABAP Unit documentation

TEST-02 How does dependency injection help?

Dependency injection means giving an object the helpers it needs instead of letting it create all of them internally. A test can supply a fake repository, clock or service client. Production supplies the real one through the same interface. This makes failures and timing predictable in tests. I keep the behavior I am testing real, rather than mocking it away.

Reasoning and example

A retry policy may retry an idempotent remote lookup once. In a unit test, I supply a client that returns a timeout on its first call and customer C20 on its second. A controlled clock or delay seam avoids waiting real minutes. The test checks the actual policy: which failures it retries, how many attempts it allows and what result it returns.

Another case returns a business rejection that should not be retried. A third returns two timeouts: the policy must stop after two calls, request only one delay and return the final failure. If I replaced the retry policy itself with a mock, I would only prove my mock setup. The interface should describe the few operations the policy needs; copying a huge client interface makes both the design and test unnecessarily complicated.

Avoid this mistake Mocking everything, including the object being tested, removes the behavior the test should verify.

Read and verify this answer

SAP Clean ABAP testing guidance

TEST-03 How would you test a RAP business object?

I test the real RAP business object with controlled data, using suitable EML tests where supported. I check successful changes, rejected changes, messages and what is saved. I also cover authorization and conflicting edits at the appropriate test level. Service and UI tests cover different behavior. Available RAP test frameworks depend on the target release.

Reasoning and example

For a Submit action, I create a request in the required initial state and call the real behavior. I check the resulting state and the FAILED and REPORTED responses. I verify persistence only after the intended save, because a change in the transaction buffer is not yet a committed database record.

A second test submits an invalid request and checks that it is rejected without an unintended saved change. Separate tests cover the service protocol and real user permissions. A mocked Fiori success screen or a double replacing the BO cannot prove the implementation works. I state which dependencies were controlled and which real integration paths were actually executed.

Avoid this mistake A mock Fiori success or a doubled BO does not establish that the real RAP implementation works.

Read and verify this answer

SAP RAP documentation

TEST-04 How do SQL and CDS test doubles help?

SQL and CDS test doubles replace supported database dependencies with test-controlled data. That lets a test run with known rows instead of changing production-like tables. I use the framework suited to the component and configure its dependencies carefully. I also clean up the test environment. These tests do not prove real database performance, network behavior or the complete authorization setup.

Reasoning and example

For a CDS total, I load a small fixture with two currencies, an order without a matching text and one record that could match several joined rows. Then I check the expected totals and ensure no extra rows appear. This exposes model errors more clearly than testing against whatever data happens to be in a shared system.

The setup must cover the dependencies the tested component actually reads. An incorrect double configuration can test a different situation from production. After each test, the environment must be reset or destroyed appropriately. I still use real-system checks for the execution plan, deployed access control and integration behavior; a passing double-based test is not a HANA benchmark. A common lifecycle is to create the double environment once in class_setup, clear and load fixture data in setup, and destroy it in class_teardown. Use the actual framework contract; RISK LEVEL HARMLESS does not create the isolation.

Avoid this mistake Passing a double-based test is not a HANA execution-plan benchmark.

Read and verify this answer

SAP ABAP Unit documentation

TEST-05 What does ATC cover and what remains outside it?

ATC runs the static checks configured for the system and check variant. It can find code problems and check supported API or enhancement usage. A pass means those checks found no blocking issue; it does not prove every business case works. Unit tests, security tests and deployment checks are still needed. I record exemptions and the target release used for the checks.

Reasoning and example

A new RAP service may pass syntax and activation but still return the wrong total. It may also pass unit tests while using an API forbidden by the target rules. ATC and runtime tests answer different questions, so I keep their results separate in the handoff instead of summarizing everything as tested.

I record the ATC variant and review any exemptions, including their reason and owner. If the target release changes, I recheck which rules and API classifications apply. An empty finding list is useful evidence, but it is not proof of complete Clean Core compliance, correct authorization for every user or a successful end-to-end business process.

Avoid this mistake Do not say that an ATC pass proves an application is error-free or fully Clean Core compliant.

Read and verify this answer

SAP RAP documentation

TEST-06 What security tests would you include?

I test what a user may do and what they must be prevented from doing. That includes direct API reads and actions, not only visible buttons. I check realistic roles, company or organizational limits, bulk requests and sensitive fields. Login, CDS access control and operation authorization are separate checks. Error messages must also avoid revealing data the user may not see.

Reasoning and example

A user assigned to company A should see A records and be unable to read B records by editing a URL. I also try an action using a known B record key, because hiding B in a list does not prove that the action is protected. A mixed bulk request should behave according to the documented authorization rules.

I test with restricted users as well as an administrator. The UI may hide fields or disable buttons for convenience, but a client can still call the backend directly. I inspect error details too: a rejection must not return another company's sensitive data. These checks make the actual security boundary visible.

Avoid this mistake Feature control, UI.hidden and disabled buttons are not sufficient security boundaries.

Read and verify this answer

SAP Learning on RAP authorization

TEST-07 How do you test concurrency and rollback?

I use separate consumers to reproduce conflicting changes. One saves first, then the other tries to save an older version. I check the expected ETag and lock behavior. I also trigger a failure and inspect whether unintended partial data remains. Remote calls need separate retry and recovery tests, because rolling back the local database cannot undo every external effect.

Reasoning and example

Two clients read the same document version. Client A changes and saves it. Client B then sends a change with its older ETag, and the service should reject it where that concurrency check applies. I inspect both the response and stored document, rather than assume an error message guarantees the state stayed correct.

For rollback, I fail a child operation inside the intended atomic group and check the related records. A separate scenario makes a remote call time out after the remote system may have acted. That case needs reconciliation or safe retry logic; a local rollback alone cannot tell me whether the remote work happened.

Avoid this mistake Sequential happy-path tests cannot establish concurrency safety.

Read and verify this answer

SAP RAP documentation

TEST-08 What would you include in a transport and deployment handoff?

A useful handoff says what must be moved, configured and checked in the target system. I include object dependencies, roles, service setup and required communication settings. I record the source revision, target version, checks actually run and checks still pending. I also explain recovery if deployment fails. A successful transport import is only one part of making the application usable.

Reasoning and example

For a RAP service, I check that the data model, behavior, DCL, metadata and service dependencies arrive together. If a UI is involved, its deployment and roles must also be ready. A missing authorization or endpoint setup can leave a technically imported application unusable. I list these steps with an owner and expected result.

After import, the team should open the real endpoint with a representative user and run the agreed business flow. Local publication in the development system does not prove publication or configuration elsewhere. The handoff identifies the tested revision and a recovery approach, so support can tell what changed and what to do if the rollout is incomplete.

Avoid this mistake Local binding publication or a successful source copy is not proof of cross-system deployment.

Read and verify this answer

SAP RAP documentation

TEST-09 How would you investigate a production defect?

I collect the exact symptom, affected users, time, inputs and deployed revision. Then I use authorized logs, dumps and traces to narrow down the cause. I reproduce safely and test a focused fix against the failing case. I separate code problems from configuration and data problems. Until evidence confirms the cause, I describe my explanation as a hypothesis.

Reasoning and example

A user reports occasional duplicate documents. I compare request identifiers, retries, timeouts and commit points to see whether the same operation was executed twice. A UI delay might hide the symptom without fixing a retry after a network failure. I avoid exposing sensitive payloads while collecting enough evidence to follow the request.

Once the sequence is understood, I add a regression case that reproduces it and verify the expected outcome after the change. I also check the ordinary success path and any duplicate-recovery behavior. The rollout record should distinguish confirmed findings from open questions, so the next person does not treat an attractive but untested explanation as the root cause.

Avoid this mistake A plausible explanation without evidence should not be presented as a confirmed root cause.

Read and verify this answer

SAP RAP documentation

TEST-10 How should you describe tests you have not run?

I say exactly what I checked and what I did not run. Prepared test cases are not passing test results. I name the source revision, environment and observed outcome for actual runs. If I only reviewed local files, I say that clearly and list the SAP checks still needed. This makes the handoff useful and avoids giving false confidence.

Reasoning and example

An honest handoff might say: I reviewed revision [revision] and prepared EML cases for valid submission, missing data and rejected authorization. I have not run them in SAP yet. Activation, ABAP Unit and ATC must still be executed in [target system]. The placeholders must be replaced with the real revision and environment.

If local JSON or frontend checks passed, I list those as their own results. They can prove file structure or browser behavior, but they cannot prove SAP activation or a real RAP save. I include the exact next action and expected result so another developer can finish validation without guessing which evidence is still missing.

Avoid this mistake Never claim a passing SAP syntax check, measured coverage or successful activation without an actual run.

Read and verify this answer

SAP ABAP Unit documentation

Official references

SAP Clean ABAP testing guidance

SAP Learning on RAP authorization

SAP RAP documentation

SAP ABAP Unit documentation

Interview answers and project discussion

Use these structures with your real experience. Keep the first answer to about one minute, then expand when asked. Be explicit about the part you owned, the constraints and the evidence for the outcome.

A strong interview answer has a visible chain of evidence

Build a truthful story that an interviewer can follow

1	**Context**	What business problem and constraints mattered?
2	**Decision**	What did you choose, and which alternative did you reject?
3	**Mechanism**	How did the code or architecture produce the behavior?
4	**Evidence**	What did you measure or test? State actual results.
5	**Limit**	What remains uncertain, release-specific or incomplete?

Use your own experience. If you only prepared a test or proposed a design, say so explicitly.

Practice a 60-second version first, then expand the mechanism and evidence when prompted.

See the visual atlas for the recall prompt, limits and related questions.

STORY-01 How would you introduce yourself for an ABAP role?

I would give a short, truthful summary of my ABAP work and connect it to this role. I would name my main responsibilities, two relevant strengths and one real example. I would clearly separate production work from courses or personal projects. The aim is to give the interviewer useful starting points, rather than list every technology I have heard of.

Reasoning and example

Use this outline with your own facts: My current focus is [real area]. I mainly work on [actual responsibilities]. My strongest relevant skills are [skill one] and [skill two]. In [real project or clearly labeled lab], I handled [your contribution], and the result was [supported outcome]. This role interests me because [honest reason].

Keep the first version short enough to say naturally. Prepare a deeper explanation for each claim, including the system context and what you personally did. If RAP experience comes from a lab, say so and explain what you built and tested there. Do not fill missing experience with invented clients, years or metrics.

Avoid this mistake Do not invent years, clients, performance metrics or RAP production experience.

STORY-02 How do you explain a project you owned?

I start with the problem and who needed it solved. Then I explain my own responsibility, the important constraint and the design I chose. I describe how I tested the work and what actually happened after delivery. Finally, I mention one thing I would improve. I keep the background short so there is time for technical follow-up questions.

Reasoning and example

Prepare a real example using these prompts: [users] needed [business outcome]. I owned [specific part], while [other team] owned [their part]. The main constraint was [actual constraint]. I chose [design] because [reason], and checked it with [tests actually run]. The observed result was [truthful outcome].

Be ready to draw the request and save flow, explain one rejected option and discuss a failure case. If you have a classic ABAP example and a modern service example, prepare both. A learning project is still useful when labeled accurately. Listing transaction codes alone does not explain ownership; your decisions and evidence show what you contributed.

Avoid this mistake Listing transactions and technologies without explaining the result does not show ownership.

STORY-03 How do you answer a question when the SAP release is unknown?

I explain the part of the answer that is generally true, then identify what depends on the SAP edition or release. I ask for that missing context and say how I would verify support. I can also describe a supported alternative if the feature is unavailable. This is more useful than guessing syntax or just saying it depends.

Reasoning and example

For a RAP question, I can first explain the business need: for example, reject an invalid change before saving. Then I identify the exact feature or BDL syntax whose availability needs checking in the target release. For an API, I check whether the required operation is released for the intended use, not just whether an object with that name exists.

A spoken answer can use placeholders honestly: On [known environment] I used [actual feature]. For your release, I would check [specific documentation or repository release information] before choosing it. If unavailable, I would evaluate [real alternative]. This shows a verification method without pretending all SAP systems support the same implementation.

Avoid this mistake Saying it depends without explaining what it depends on is not a useful answer.

STORY-04 How do you discuss a performance improvement?

I explain what was slow, how I measured it and what data volume the measurement used. Then I describe the bottleneck, the change and the measured result. I also explain how I checked that the result stayed correct. I use only figures from my actual work. If no reliable numbers were recorded, I say that instead of inventing a speedup.

Reasoning and example

Prepare a real sequence: [operation] was slow with [representative workload]. I used [tool actually used] and found [observed cause]. I changed [specific access or processing step]. Under [comparable test conditions], the result changed from [measured baseline] to [measured outcome]. I checked correctness using [actual comparison or tests].

Be ready to explain why the change helped. Fewer database round trips, fewer transferred rows or an appropriate internal-table key are clearer than saying the code was optimized. Include a limitation, such as a workload not yet measured, if it is real. If your example is a lab benchmark, identify it as one and do not present its results as production evidence.

Avoid this mistake Do not claim a speedup from code appearance alone or memorize an invented percentage.

STORY-05 How would you defend an architecture trade-off?

I compare options using the actual requirement: what must save together, which APIs exist, who uses the result and how failures are handled. I state my choice and its main cost. I also explain what would make me choose differently. A good answer shows why the decision fits this situation, not loyalty to a particular SAP product.

Reasoning and example

For a real extension decision, describe [business need] and the change that must happen reliably. If validation must reject a local save, explain which supported extension point can run at the right time. If a separate portal is involved, explain its remote calls, login needs and recovery when one system is unavailable.

Then state the evidence you would check: API availability in [target product], transaction behavior and a realistic failure test. Use your actual experience or label the example as a proposed design. Saying always use BTP or always use RAP skips the reason for the choice. A convincing answer includes the downside of the selected option and when it stops being suitable.

Avoid this mistake Always use RAP or always use BTP is not an architecture argument.

STORY-06 What should you ask the interviewer?

I would ask about the SAP systems, the work I would own and how the team delivers changes. I want to know the mix of classic ABAP and RAP, the Clean Core rules, and the testing and release process. I would also ask who handles UI, integration and production incidents. Three focused questions are usually better than a long technology checklist.

Reasoning and example

Choose questions that affect your daily work: Which editions and releases do you support? What would I build or maintain in the first few months? How does a change move from development to production? These invite concrete answers about the role, rather than only a list of products.

Then follow one answer with a practical example: Can you describe a recent Clean Core decision or a difficult production issue and how the team handled it? Listen for ownership, review and testing responsibilities. The title ABAP Developer can cover maintenance, new services or migration work. Use the discussion to understand the actual job and compare it with your real experience and learning goals.

Avoid this mistake Do not assume the role title tells you whether the work is maintenance, migration or new development.

STORY-07 How do you estimate an ABAP or RAP task when some requirements are still unclear?

First agree what finished means, then split the work into small parts. Compare familiar parts with similar completed work and estimate them from the bottom up. Use a range for uncertain parts and a short investigation for the biggest unknown. Include testing, review and delivery. Effort in person-days is not the same as a calendar deadline.

Reasoning and example

Suppose the task is a RAP Submit action. A teaching example for the most likely effort is: clarify the API and rules, 0.5 day; implement the action, 1 day; authorization and validation, 1 day; automated and integration tests, 1 day; review and handoff, 0.5 day. That is 4 person-days under the stated assumptions. These are illustrative numbers, not standard durations for RAP work. Waiting for an API owner or a test system affects the delivery date separately.

Choose the technique for the information available. Analogy compares a genuinely similar completed task and adjusts for differences. Bottom-up estimation adds the work packages. Three-point estimation records optimistic, most likely and pessimistic cases, for example 2, 4 and 8 person-days. Explain what causes each case; these values are not guaranteed bounds or a promised probability. Planning Poker helps a team compare relative size and discuss different assumptions. Story points are not a fixed number of hours and should not be used to compare developer productivity.

If the key uncertainty is whether an API supports the required save behavior, time-box a small spike before committing to a delivery forecast. Agree its question, owner and time limit. Record assumptions, exclusions, dependencies and acceptance checks, then update the estimate when evidence changes. Include the spike effort explicitly; do not hide it in an unexplained buffer. A date forecast also needs team availability, dependencies and the planned release window.

Avoid this mistake Do not estimate only coding time, convert story points into fixed hours, or present an uncertain forecast as a guaranteed deadline.

Read and verify this answer

SAP Activate: release planning and Planning Poker

PMI: Practice Standard for Scheduling, three-point estimates

Scrum.org: story-point estimation pitfalls

Official references

SAP Activate: release planning and Planning Poker

PMI: Practice Standard for Scheduling, three-point estimates

Scrum.org: story-point estimation pitfalls

HTTP, REST, JSON and XML

Explain the protocol, architectural style and payload independently. Practice concrete requests, payload edge cases and ABAP integration failure handling.

Read an HTTP exchange as a contract

Illustrative HTTP/1.1 layout; HTTPS protects the transport

REQUEST: client to service

GET /orders/4711 HTTP/1.1

Host: api.example.test

Accept: application/json

Method + target identify the request.

Accept asks for a representation.

Credentials are intentionally omitted.

RESPONSE: service to client

HTTP/1.1 200 OK

Content-Type: application/json

ETag: "v7"

{"id":"4711","status":"Open"}

ETag is an opaque version validator.

Transport / HTTP → **Representation / JSON**

REST is an architectural style; JSON and XML are payload formats. OData adds service conventions on top of HTTP.

Read the method, target, status and headers before interpreting the body. The sample domain is deliberately non-production.

See the visual atlas for the recall prompt, limits and related questions.

HTTP-01 What is the difference between HTTP, REST, OData and a payload format?

HTTP defines how requests and responses work. REST is a set of design rules for an API. OData adds rules for data services, such as querying related records. JSON and XML are formats for the data you send. Using JSON over HTTP does not, by itself, make an API RESTful.

Reasoning and example

An order app sends an HTTP GET request and receives a response with a status code. The response body might use JSON to describe order 4711. If the service uses OData, the client can use its supported query rules to request selected fields or related items. These choices describe different parts of the same call.

REST adds architectural rules, including stateless requests. Here, stateless means the request carries the context the server needs to understand it. It does not mean the server cannot store orders in a database. Keeping these ideas separate makes it easier to explain why a payload problem differs from a protocol or API-design problem.

Avoid this mistake Do not use REST, HTTP and JSON as interchangeable names.

Read and verify this answer

HTTP semantics: IETF RFC 9110

REST constraints: Fielding dissertation

HTTP-02 What is inside an HTTP request and response?

An HTTP request contains a method, a target address, headers and sometimes a body. The response contains a status, headers and sometimes content. Headers carry information such as authentication or content type. HTTPS protects the connection, but the response still needs checking: a secure connection does not mean the business operation succeeded.

Reasoning and example

For an order update, I inspect the target order URL, the method and the fields in the body. I also check credentials, Content-Type and any If-Match version token. In the response, I examine both the status code and the returned data or error message. This shows whether the server accepted the requested business change.

For example, TLS can connect successfully while the server returns 403 because the user cannot change that order. I would then investigate permissions rather than the certificate. HTTP/1.1 is often shown as readable text for teaching; HTTP/2 and HTTP/3 carry the same broad message concepts using different framing.

Avoid this mistake A successful TCP/TLS connection does not prove that the business request succeeded.

Read and verify this answer

HTTP semantics: IETF RFC 9110

HTTP-03 How do GET, POST, PUT, PATCH and DELETE differ?

GET reads data. POST asks the target to process submitted content. PUT creates or replaces the identified resource's state, while PATCH applies a described change. DELETE asks to remove the target resource association. I follow the endpoint's documented meaning; these methods are not interchangeable labels for the same operation.

Reasoning and example

To change only an order's delivery note, I check whether the service supports PATCH and which patch format it accepts. Leaving a field out may mean keep its value, while sending null or an empty string may mean something different. The API contract decides those details. PUT should not be assumed to mean an arbitrary partial update.

POST might create an order or invoke a business action; it does not always insert one row. DELETE also does not promise physical erasure of every database record. Repeating PATCH is not inherently safe: setting a value can be repeatable, while an instruction to increment a value may change it again.

Avoid this mistake PUT is not an arbitrary partial update, and POST does not always mean creating exactly one row.

Read and verify this answer

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

HTTP-04 What do safe and idempotent mean, and how do they affect retries?

Safe means the requested operation is read-oriented. Idempotent means repeating the same request has the same intended effect as doing it once. PUT and DELETE are idempotent by HTTP semantics, although their responses may differ. POST and PATCH need a specific guarantee before I repeat a business change after a timeout.

Reasoning and example

A client sends CreateOrder and times out before receiving a response. The server may already have saved the order. Sending a new request without checking could create a duplicate. I use a documented idempotency key, which identifies repeated attempts at the same request, or query the result using a stable business reference.

For DELETE, the first request might report success and the second might report that the resource is absent. That does not contradict idempotency: the intended final state is still that the resource is removed. Safe methods can also cause incidental logging; the key point is that the client did not request a business change.

For retryable failures, use a maximum attempt count, a total time limit and increasing delays with jitter: a small random spread prevents many clients from retrying at once. Respect Retry-After when supplied. Set connection and response time limits using the APIs available on the target platform. A timeout limits waiting; it does not prove that the receiver cancelled its work.

Avoid this mistake Do not blindly retry a potentially committed business creation because the response was lost.

Read and verify this answer

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

AWS Builders Library: timeouts, retries and jitter

HTTP-05 Which HTTP statuses should an ABAP API developer explain?

I distinguish successful completion from acceptance for later processing: 200 is success, 201 creation, 202 accepted and 204 success without content. Common errors are 400 invalid request, 401 missing or invalid authentication, 403 refusal and 404 not found. I also know 409 conflict, 412 failed precondition and 415 unsupported content type.

Reasoning and example

A payment API returns 202. I cannot tell the user that payment has finished; I need the documented status check or callback to learn the final result. If an update returns 412, I investigate the stale version condition instead of treating it as a server outage. The error body often explains more than the status alone.

For 429 or 503, the contract may allow a delayed, bounded retry, sometimes using Retry-After. I still consider whether the original operation might have succeeded. A 500 needs diagnosis too. I keep a correlation ID for support without logging credentials, and inspect inner results when a batch envelope returns HTTP 200.

A 413 Content Too Large response means the request content exceeds what the server is willing or able to process. Reduce or split the payload only if the API supports it; sending the same oversized request repeatedly is not a useful default. Distinguish this from 429 Too Many Requests, where the issue is request rate rather than the size of one payload.

Avoid this mistake HTTP 200 on a batch envelope does not prove every contained operation succeeded.

Read and verify this answer

HTTP status reference: MDN

IETF RFC 9110: 413 Content Too Large

HTTP-06 How do Accept and Content-Type differ?

Content-Type says what format a message actually contains. Accept says which response formats the client can understand. A client can send JSON and ask for JSON back, but those are two separate settings. I check the actual response type before parsing it, especially when the call fails.

Reasoning and example

An app sends a JSON order with Content-Type: application/json and Accept: application/json. If the endpoint only accepts a different request format, it can return 415. If it cannot provide an acceptable response format, it can return 406. These are format problems, not automatically password or authorization problems.

An intermediary might return an HTML error page instead of the expected JSON. Trying to parse that page as an order produces a confusing second error. I therefore inspect the status and response Content-Type first. Merely changing the header from XML to JSON does not convert the body; the serialized data must actually match the declared format.

Avoid this mistake Changing Content-Type does not convert an XML body into JSON.

Read and verify this answer

HTTP semantics: IETF RFC 9110

HTTP-07 What must you preserve when mapping JSON to ABAP?

I preserve field names, data types and the meaning of missing values. JSON distinguishes an absent property, null and an empty string; an ABAP update may need to distinguish them too. Arrays keep their order, but object-property order is not a reliable contract. I also protect number precision and identifiers with leading zeros.

Reasoning and example

An external customer ID is "0004711". If I read it as a number, the leading zeros disappear, so the contract should treat it as a string. For an amount, I preserve the agreed decimal precision and currency. Whether that amount is sent as a JSON number or string depends on the API.

A PATCH request with no Phone property might leave the phone unchanged, while Phone set to null might clear it. Mapping both to the same initial ABAP value can destroy that distinction. JSON itself has no native date type, comments, NaN or Infinity. I also avoid relying on duplicate object-property names because parsers may handle them differently.

Avoid this mistake Treating missing and null fields as the same initial ABAP value can accidentally change or clear data.

Read and verify this answer

JSON format: IETF RFC 8259

SAP JSON transformations

HTTP-08 What XML concepts matter in an integration interview?

XML represents data with elements, attributes and nested content. Well-formed XML follows syntax rules; schema validation checks further rules about its structure and values. Namespace prefixes are aliases, so I identify names using the namespace URI and local name. I use a real XML parser rather than splitting the text by hand.

Reasoning and example

One partner sends a:Order and another sends b:Order. If both prefixes refer to the same namespace URI, they can identify the same element name. A parser that only looks for the literal text a:Order would wrongly reject the second message. The namespace URI plus the local name Order identifies what the element means.

A default namespace applies to unprefixed elements, not unprefixed attributes. I also check how the contract distinguishes a missing element, an empty element and an explicit schema-defined nil value. Encoding, escaping and secure parser settings matter. Valid XML syntax alone does not prove that the order contains a valid customer or amount.

Avoid this mistake String splitting is not an XML parser. Correct XML syntax also does not prove valid business data.

Read and verify this answer

XML specification: W3C

SAP sXML parsing

W3C Namespaces in XML

HTTP-09 How do you choose ABAP JSON and XML processing tools?

I choose a parser, serializer or transformation allowed in the target ABAP environment. The tool must produce the partner's actual format and handle bad input clearly. CALL TRANSFORMATION, sXML and JSON libraries serve different needs. I test real sample payloads and check release status; a familiar classic helper may not be allowed in ABAP Cloud.

Reasoning and example

A partner expects a small JSON object with its own property names. The identity transformation can produce canonical ABAP formats such as asJSON or asXML, but that is not automatically the partner's format. I check the output and choose a supported mapping approach that matches the contract, including numbers, missing values and errors.

For a very large XML file, a streaming parser can process sections without loading the whole document into a DOM tree. I compare that with the navigation needs of the task. I never build XML or JSON by simply joining unescaped user values into strings, because quotes and special characters can corrupt the message.

Avoid this mistake Do not promise that a familiar on-premise helper class is released for ABAP Cloud.

Read and verify this answer

SAP JSON transformations

SAP sXML parsing

HTTP-10 How would you implement and troubleshoot an outbound HTTP call in ABAP?

I use an allowed destination and HTTP client, with credentials stored in configuration. I send the documented request, set a timeout and handle connection failures separately from HTTP error responses. Then I parse and validate the response. Retries need an explicit rule, because the remote system may have saved a change before the connection failed.

Reasoning and example

An outbound supplier call fails. I first locate the failing step: name resolution, TLS trust, destination configuration, authentication or the application request. If I receive HTTP 400 with a useful error body, repeatedly changing the certificate is unlikely to help. I compare the headers and payload with the API contract instead.

In ABAP Cloud, I use the released client and destination APIs and the communication setup supported by that environment. Classic systems may use different APIs. Logs should help find the request without exposing passwords, tokens or sensitive business data. I also verify the retry policy: rolling back the local transaction does not undo a remote commit.

If users can configure the target URL, also consider SSRF: the server could be tricked into calling an unintended internal or privileged endpoint. Prefer approved destinations and allowlisted targets. Revalidate redirects or disable them as the client permits; validating only the original hostname is insufficient. Keep network restrictions and credential scope aligned with the destination.

Avoid this mistake Do not hard-code a password, suppress all exceptions, or assume a remote call shares the local rollback.

Read and verify this answer

SAP ABAP integration and connectivity

OWASP: server-side request forgery prevention

HTTP-11 How do authentication, authorization, CORS and CSRF differ?

Authentication answers who is calling; authorization answers what they may do. CORS controls which cross-origin responses browser scripts can access. CSRF defenses help stop forged requests that misuse browser credentials in relevant flows. These protections solve different problems. Allowing an origin does not give its users permission to approve an order.

Reasoning and example

The same API call works from an ABAP backend but fails from a browser app hosted on another origin. I inspect the browser's CORS error and any preflight request. The backend call is not subject to the browser's CORS enforcement in the same way, so its success does not prove that the browser configuration is correct.

After fixing CORS, a user may still receive an authorization error when approving an order. That check must remain. In a session-based browser flow, the endpoint may also require CSRF protection for writes. CORS is neither a general server-to-server firewall nor a replacement for authentication, authorization or the applicable CSRF defenses.

Avoid this mistake CORS is not a general server-to-server firewall or a replacement for authentication.

Read and verify this answer

CORS: MDN

HTTP-12 How would you test an HTTP integration contract?

I test what the integration does, not just whether it can send JSON. Cases include bad input, missing values, business errors, expired credentials and timeouts. I also test repeated requests and conflicting updates. Controlled doubles help reproduce failures, but a real integration test must check the actual destination, identity and partner behavior.

Reasoning and example

For an order creation API, I test a normal order, an invalid customer and a price that needs exact decimal handling. I also simulate a lost response after creation. Recovery should find or safely reuse the first result rather than create a second order. A wrong Content-Type or malformed body should produce a clear error.

I use a test double to make these failures repeatable, then run authorized tests against the real endpoint for TLS, authentication, serialization and permissions. The evidence includes a redacted request, correlation ID and final business outcome. Serializing an object and reading it back locally does not prove the partner accepts that representation.

Avoid this mistake A serializer round-trip test alone does not prove that the partner accepts your representation.

Read and verify this answer

HTTP semantics: IETF RFC 9110

HTTP-13 How do you send a GET request with released ABAP HTTP APIs?

First configure an approved connection. Resolve an IF_HTTP_DESTINATION, create a client with CL_WEB_HTTP_CLIENT_MANAGER, set the path and headers, then execute GET. Check the status before using the response body. Close the client after success or failure, and keep connection errors separate from HTTP error responses.

Reasoning and example

For example, read order 4711 from a partner API. A custom communication scenario contains the HTTP outbound service. An administrator configures the communication system and arrangement with the host and authentication. CL_HTTP_DESTINATION_PROVIDER=>CREATE_BY_COMM_ARRANGEMENT resolves that setup; passwords stay out of the program. This method is for your developed scenario, not an arbitrary SAP-delivered scenario. Newer supported systems also offer communication targets, which SAP now recommends.

The code cards separate connection setup from the GET operation. This example expects HTTP 200 and JSON. A received 404 means the resource was not found; a client exception means the call itself failed. After a 200, verify the content type and required fields rather than assuming every body is valid order data. Close the client in either case and report any cleanup failure separately. Adapt the path, timeout and success contract to the real endpoint.

Avoid this mistake A configured destination does not prove that the remote business user has permission. A returned HTTP response does not prove that the request succeeded.

Read and verify this answer

SAP: Outbound HTTP and released client APIs

IETF: HTTP conditional requests and If-Match

HTTP-14 How do you change remote data safely with PATCH or POST from ABAP?

Follow the API contract: choose the method and body format, use its authentication, and fetch a CSRF token if required. For a versioned update, send the ETag you actually read in If-Match. Inspect the response, close the client on every path, and treat a timeout as an unknown outcome. Do not automatically repeat a business change.

Reasoning and example

Suppose order 4711 has an ETag from a successful GET. The PATCH example changes only its note. This fictional endpoint accepts JSON Merge Patch, fetches a CSRF token with HEAD, and uses cookies. Keep the same HTTP client for the token and the modifying request. SET_CSRF_TOKEN fetches and sets the token; If-Match separately prevents changing an obsolete version. A 412 response requires a fresh read and conflict handling, not replacing the ETag with *.

For this API, 200 or 204 confirms the completed update; 204 has no body to parse. A 202 response, if the real API supports it, means work was accepted and still needs tracking. A POST to create an order instead needs its documented create payload and success result, often a 201 with Location. A lost response does not prove that either operation failed: reconcile by a stable business reference or use the API's documented duplicate protection. Local ROLLBACK cannot undo a completed remote save.

CSRF is not required by every API, and HEAD-based token fetching is not universal. Follow the service's documented flow. Raw JSON Merge Patch rules are not automatically OData PATCH rules; use the real media type, entity metadata and ETag contract.

Avoid this mistake CSRF token, ETag and an idempotency key are not interchangeable: they address request-session protection, concurrent updates and duplicate attempts respectively.

Read and verify this answer

SAP: Outbound HTTP and released client APIs

IETF: HTTP conditional requests and If-Match

IETF: JSON Merge Patch

Official references

HTTP semantics: IETF RFC 9110

PATCH method: IETF RFC 5789

HTTP status reference: MDN

JSON format: IETF RFC 8259

XML specification: W3C

REST constraints: Fielding dissertation

CORS: MDN

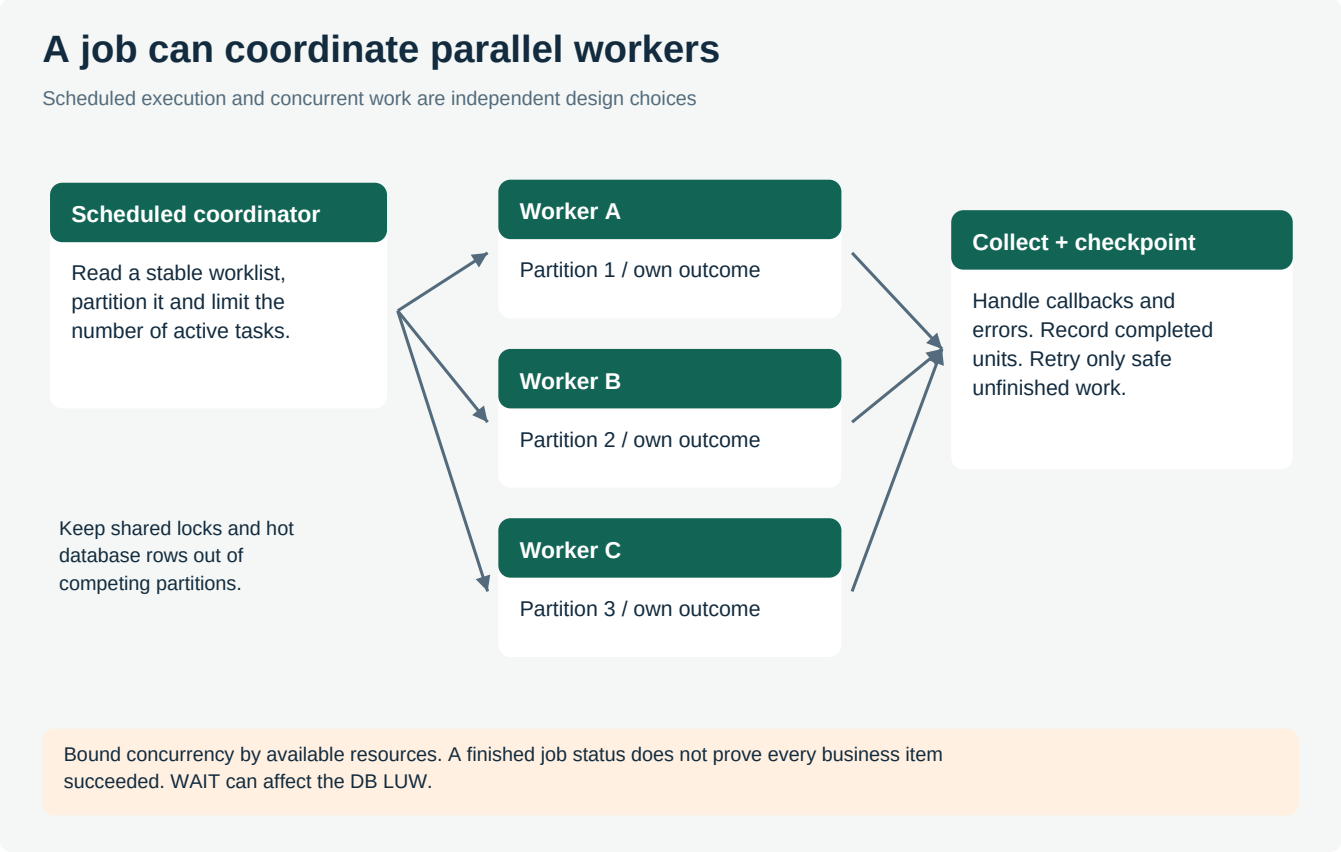
SAP JSON transformations

SAP sXML parsing

SAP ABAP integration and connectivity

Function modules, RFC, BAPI, parallel processing, and jobs

Separate the callable unit, the communication mechanism, and the business transaction. Classic RFC and job transactions remain important interview topics, but their statements and APIs are not automatically available in ABAP Cloud. Explain ownership, restart behavior, and observable outcomes before promising reliability or faster processing.



A coordinator schedules bounded independent work and collects explicit outcomes; a scheduler does not create parallelism automatically.

See the visual atlas for the recall prompt, limits and related questions.

RFC-01 How does a function module differ from an ABAP method?

A function module is a procedure in a function group; modules in that group can share state. A method belongs to a class or interface and fits object-oriented design. I usually put new business rules in testable classes. Existing function modules can remain as required API or framework entry points and call those classes.

Reasoning and example

An integration already calls a function module to calculate an order charge. I can keep that public interface and move the calculation into a class with clear inputs and outputs. The function module then passes the request to the class and returns the result. Existing callers keep working, while unit tests can exercise the calculation directly.

I still check the function module's actual attributes. A normal function module is not automatically remote-enabled or an update module; those are different execution models. Moving code into a method also does not make it stateless, remotely callable or released for ABAP Cloud. Those properties need their own design and checks.

Avoid this mistake A method is not automatically stateless, remotely callable or released for ABAP Cloud.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

RFC-02 How do synchronous, asynchronous, transactional, queued, and background RFC differ?

sRFC waits for a reply; aRFC lets the caller continue and collect results later. tRFC records a unit for processing after commit, with durable retry. qRFC adds ordering through queues, while bgRFC supports transactional and queued units with newer infrastructure. I choose by response, retry and ordering needs, within the target system's supported options.

Reasoning and example

A tRFC call is first registered in the current SAP LUW. COMMIT WORK triggers processing of that registered unit; ROLLBACK WORK removes the current registrations. The registration itself is not proof that the receiver has finished. sRFC and aRFC do not provide that same durable retry model.

The documented unit guarantees are Exactly Once for tRFC and bgRFC type T, with ordering in the configured scope for qRFC and bgRFC type Q. That does not guarantee one business effect across an entire distributed process: two separate submissions may still create two orders. bgRFC is the successor technology. I let the runtime manage the unit's transaction and do not assume unrelated queues share one global order.

Avoid this mistake STARTING NEW TASK does not provide durable message retry. Separate queues do not automatically have one shared processing order.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP: CALL FUNCTION IN BACKGROUND TASK

SAP: CALL FUNCTION IN BACKGROUND UNIT

RFC-03 What is the difference between a BAPI and an RFC-enabled function module?

RFC is a mechanism for calling remote-enabled function modules. A BAPI is a business API, traditionally implemented by such a module for a business object or interface. A remote-enabled module is not automatically a BAPI. I check the particular API's business rules, return messages, transaction handling and release status before using it.

Reasoning and example

A classic integration may call a BAPI to create a sales order. The BAPI defines the business input, supported behavior and returned messages; RFC provides the remote call mechanism. A custom remote-enabled utility function has the mechanism too, but that alone does not give it a BAPI business contract.

Release status is another separate question. A BAPI supported in a classic integration may not be released for direct ABAP Cloud use. A modern released business API may use HTTP or OData instead of a classic BAPI. I therefore check the target development model and Clean Core classification rather than infer them from the name or remote-enabled flag.

Avoid this mistake Remote-enabled, BAPI, released API and Clean Core Level A mean different things. Verify each relevant property.

Read and verify this answer

SAP BAPI transaction model and documented exceptions

RFC-04 When should BAPI_TRANSACTION_COMMIT or BAPI_TRANSACTION_ROLLBACK be called, and what does WAIT mean?

The code that owns the BAPI transaction decides when to commit or roll back after checking the business results. For a remote BAPI, transaction control must reach the correct remote context. WAIT requests completion of high-priority updates; it does not wait for every later workflow or interface. I also check the specific BAPI's documented exceptions.

Reasoning and example

A client calls two BAPIs that are meant to form one transaction. It checks their relevant return messages and exceptions before asking the correct transaction context to commit. If the operation fails, it follows the documented rollback path. A local COMMIT WORK is not a way to commit an unrelated remote session.

After BAPI_TRANSACTION_COMMIT with WAIT, I check the commit result as well. Downstream messages, lower-priority work or workflows may still be pending. Some documented legacy BAPIs commit internally, so I verify the actual API behavior before combining calls. Inside RAP behavior code, inserting this commit function is not a permitted shortcut around framework transaction ownership.

Avoid this mistake Check the BAPI's actual success rules and return messages. Local COMMIT WORK cannot commit an unrelated remote BAPI context.

Read and verify this answer

SAP BAPI transaction model and documented exceptions

RFC-05 Does a remote call participate in the caller's local SAP LUW?

I do not assume that the caller and remote system share one rollback. Remote execution has its own context, and each side's transaction owner must be clear. Calls that must share a remote transaction need the correct session handling. If the remote side has already committed, local rollback cannot undo it; recovery needs an explicit business step.

Reasoning and example

A program remotely creates a document, then fails while recording its local reference. If the remote document was committed, rolling back the caller leaves that document in place. The program must find it using a stable reference and either complete the local record or use a supported cancellation process. Sending another creation request may duplicate it.

For two remote BAPI calls intended to commit together, I use the connector's supported transaction context and send transaction control to that context. Reusing a destination name alone is not a complete session design. An aRFC worker also cannot simply inherit the caller's uncommitted transaction buffer or memory.

Avoid this mistake ROLLBACK WORK in the caller cannot undo an already committed remote business document or external side effect.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP BAPI transaction model and documented exceptions

RFC-06 How would you collect results from CALL FUNCTION STARTING NEW TASK?

I start each aRFC call with a task ID and an end-of-task callback. In the callback, RECEIVE RESULTS FROM FUNCTION collects the result and handles communication or system errors. I track which tasks started, completed, failed or timed out. Starting a task successfully only proves dispatch, not that its business work succeeded.

Reasoning and example

A coordinator splits six invoices into three packages of two and launches remote-enabled function calls with STARTING NEW TASK within its concurrency limit. Each task ID points back to its package. CALLING or PERFORMING ON END OF TASK registers the callback, which uses RECEIVE RESULTS FROM FUNCTION to collect the output and record the outcome.

Package three may finish before package one, so I never match results by arrival order. The caller must stay alive and allow callbacks to run, using an appropriate bounded wait where needed. I also handle tasks that never return in time. The usual synchronous IMPORTING pattern is not how an aRFC caller receives its eventual results.

Avoid this mistake An immediate successful dispatch does not mean business success, and the normal synchronous IMPORTING pattern does not collect an aRFC result.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

RFC-07 How do you prevent parallel RFC processing from exhausting resources?

I limit how many tasks can run at once and give each task a useful-sized package. In classic ABAP, DESTINATION IN GROUP can use a configured RFC server group. Resource availability can change, so I handle RESOURCE_FAILURE and reduce or delay dispatch. I measure the result: more parallel tasks can increase waiting and make the job slower.

Reasoning and example

A report needs to process 100,000 independent records. Starting one task per record wastes resources. I choose packages large enough to justify call overhead, use the agreed RFC group and cap outstanding tasks. RZ12 defines the relevant groups in classic systems, and SPBT_INITIALIZE can help inspect currently available resources.

That resource count is only a snapshot, not a reservation. If dispatch fails because capacity disappeared, the coordinator must throttle or retry within a limit. I also measure locks and database load. A background coordinator may start aRFC workers in dialog work processes, so its own job type does not mean every worker uses a background process.

Avoid this mistake A resource count is a snapshot, not a reservation; hard-coded maximum concurrency can overload the system or increase lock contention.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

RFC-08 Why are WAIT and implicit database commits important in parallel processing?

WAIT is not always a harmless pause. Classic WAIT can cause a database commit, so I avoid placing it around unfinished writes or an open database cursor without checking the exact rules. RFC boundaries can matter too. A database commit does not mean every update task or remote business operation has completed successfully.

Reasoning and example

A coordinator writes some rows and then waits for parallel workers. If that wait causes an unexpected database boundary, the program may save part of its work earlier than intended. An open database cursor can also be affected. I therefore design the commit boundaries before dispatch and waiting instead of relying on timing to make the program work.

Workers should receive self-contained input rather than depend on the caller's pending writes or transaction buffer. COMMIT WORK AND WAIT, WAIT UNTIL and collecting an aRFC callback solve different problems. I check the exact statement and framework context; neither 'RFC never commits' nor 'every RFC commits the whole business transaction' is a safe general rule.

Avoid this mistake Neither 'RFC never commits' nor 'every RFC commits the whole business transaction' is a safe rule; statement, context, and framework restrictions matter.

Read and verify this answer

SAP WAIT UNTIL and database commit implications

RFC-09 How would you design parallel workers that can be restarted safely?

Each worker needs a clear package, transaction boundary and restart rule. I use stable business keys so a retry can find work already completed. Different workers should avoid changing the same locked object at the same time. After a timeout, I check the actual business result rather than assume the package failed and run everything again.

Reasoning and example

Six workers process six packages. Five report success and the sixth times out. Restarting all six could repeat work that already committed. I record completed, pending and uncertain outcomes and use stable references to locate each package's business documents. Independent worker commits mean the overall job can be partly complete.

There is a harder case: the sixth worker saved its document but failed before recording completion. A package status alone cannot prove that nothing happened. Recovery must check the business effect or use a documented idempotent operation. I also test two packages that accidentally contain the same object, because lock contention can remove the benefit of parallel processing.

For a custom work queue, define how two workers claim one item atomically. One design is to lock the item, re-read its pending state, record the owner and claim, then release the lock under a documented transaction boundary. Every worker must follow that protocol. Recovery of an abandoned claim must check whether the original worker is still active and whether its business effect already exists; an expired timer alone must not authorize duplicate processing.

Avoid this mistake Separately committed workers are not one all-or-nothing transaction. Extra workers can also make lock contention worse.

Read and verify this answer

SAP parallel processing, callbacks, and resource groups

SAP classic background scheduling and monitoring

SAP: lock ownership

RFC-10 How do you schedule and troubleshoot a classic background job?

In a classic system, I schedule job steps, variants, user and start conditions, commonly in SM36. In SM37, I inspect status, job log, steps and spool. I then follow application errors or dumps. Before restarting a canceled job, I check what it already saved; cancellation does not undo earlier commits.

Reasoning and example

A nightly posting job is canceled halfway through its input. I inspect the job log and application records to find the last confirmed business results. The rerun must use a stable selection and avoid posting the completed records again. If the job is periodic, I also check whether the next run could overlap with recovery.

Job states such as scheduled, released, ready, active, finished and canceled describe execution progress. Finished only means the job completed technically; the report may still have rejected individual business records. I reconcile accepted and failed records and define who handles missed runs, authorization errors and unavailable external systems.

Avoid this mistake SM37 Finished is not proof that every business record succeeded, and canceling a job does not reverse its earlier commits.

Read and verify this answer

SAP classic background scheduling and monitoring

RFC-11 How do application jobs differ from classic report scheduling in ABAP Cloud?

Application jobs use the execution framework supported by the target product. A catalog entry connects executable logic with parameter metadata; a template provides scheduling defaults. Users operate jobs through supported apps or APIs. I check the actual interfaces and permissions rather than copy a classic report-scheduling pattern into ABAP Cloud.

Reasoning and example

A cloud application needs a nightly cleanup for completed requests. I implement the supported executable class and connect it to the appropriate job catalog entry. A template can supply the usual retention period, while another template uses a different business selection. The scheduler still needs permission to run the job and process the selected data.

I verify how the class, catalog, template and Application Jobs app fit together on the target release. I do not assume JOB_OPEN, SUBMIT or an interface signature from another release is available. Application jobs also exist in supported on-premise products, so the concept is not limited to cloud systems.

Avoid this mistake Do not copy an unreleased JOB_OPEN or SUBMIT pattern into ABAP Cloud, or assume an interface signature from a different release.

Read and verify this answer

SAP application job implementation, catalog entries, and templates

RFC-12 Which operational and security checks make RFC or job processing production-ready?

I check who calls the RFC or runs the job, what they may do and how connections are secured. Operators need useful logs, alerts and a safe recovery procedure. Tests include outages, duplicate requests, locks, bad input and timeouts. A scheduling or transport acknowledgment is only one step; I verify the final business result too.

Reasoning and example

A queued posting stops because the receiver is unavailable. An operator needs to find the affected business requests, understand whether anything was posted and restart only through the supported recovery path. In classic systems, relevant tools can include SM58 for tRFC, SMQ1 or SMQ2 for queues and SBGRFCMON for bgRFC.

I test both temporary technical failures and permanent business errors using realistic volumes. The technical user should have only the required access, and operators should not receive unrestricted payload-editing rights just to inspect failures. RFC authorization does not replace application authorization. Durable queues also need monitoring; persistence alone does not guarantee that someone resolves a stuck message.

Avoid this mistake RFC permission does not replace business permission. A durable queue still needs someone to monitor and recover failures.

Read and verify this answer

SAP RFC variants and their quality-of-service scope

SAP classic background scheduling and monitoring

Official references

SAP RFC variants and their quality-of-service scope

SAP BAPI transaction model and documented exceptions

SAP parallel processing, callbacks, and resource groups

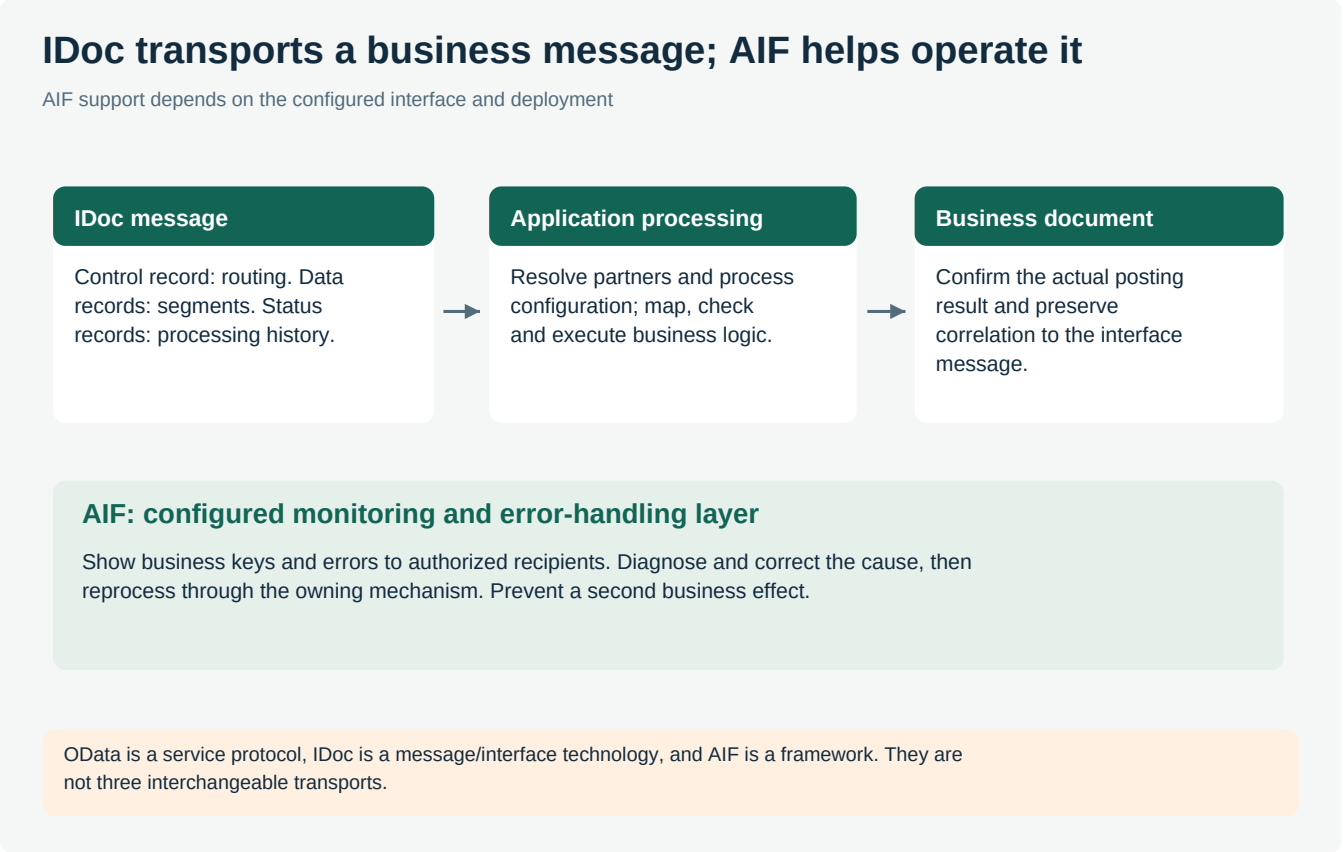
SAP WAIT UNTIL and database commit implications

SAP classic background scheduling and monitoring

SAP application job implementation, catalog entries, and templates

IDoc, ALE, AIF, and interface operations

Follow an integration message from business intent through routing, validation, posting, and recovery. IDoc is a structured message and processing technology; AIF supplies interface processing and operational capabilities for configured scenarios rather than replacing the transport. Classic transaction examples apply only where available; AIF capabilities, installed content, and usage rights depend on the product and contract.



Follow the message to the business outcome. The AIF band represents supported configured capabilities, not an obligatory stage for every IDoc.

See the visual atlas for the recall prompt, limits and related questions.

INTF-01 What does an IDoc contain, and how do message type, basic type, and extension differ?

An IDoc has control data, business data in segments and a processing-status history. The message type says what business message it represents. The basic type defines the standard structure, and an extension adds supported customer segments. Sender and receiver must agree on these parts; adding a segment definition does not automatically fill or process its values.

Reasoning and example

An order message needs an extra customer reference. I check whether a supported IDoc extension is appropriate, then arrange for the sender to fill the new segment and the receiver to understand it. Middleware mappings may need an update too. Declaring the extension in configuration alone does not transfer the business value into the target order.

When diagnosing the message, the control record helps identify routing and structure, data records hold the actual values, and status records show processing progress. The technical IDoc number identifies that message. It is not automatically the number of the order created by the receiver, so I keep the business reference for correlation.

Avoid this mistake Message type and basic type have different jobs. An IDoc number is also not automatically the target business document number.

Read and verify this answer

SAP IDoc control record, message type, basic type, and extension

INTF-02 What do ALE distribution models, partner profiles, ports, and process codes do?

The ALE distribution model describes configured distribution relationships. A partner profile says how a partner's messages are handled. A port defines the technical communication mechanism, and an inbound process code selects processing logic. I check these settings together. A valid payload cannot post correctly if it reaches the wrong partner rule or processor.

Reasoning and example

An inbound order IDoc arrives, but the receiver cannot select the expected processing. I inspect the partner and message settings, then the assigned process code and application implementation. In a classic landscape, WE20 shows partner profiles and WE21 shows ports; BD64 is relevant where an ALE distribution model controls the scenario.

Not every interface uses the same model-driven routing. A logical or business partner identifies a participant, while a port and an RFC destination serve different technical roles. A successful connection test only proves part of the path. It does not prove that the right partner-profile entry matches or that the inbound processor can post this business data.

Avoid this mistake A successful connection test proves neither correct partner-profile matching nor that the selected inbound process can post the payload.

Read and verify this answer

SAP IDoc partner and port configuration

INTF-03 Walk through outbound and inbound IDoc processing.

Outbound processing creates an IDoc from a business trigger, chooses a receiver and sends it through the configured channel. Inbound processing receives it and passes it to the appropriate application logic. Each stage records progress. Receiving the message is not the same as posting its business document, especially when processing waits for a background run.

Reasoning and example

A sales process produces an outbound order message. Depending on the scenario, the trigger may be application output, change pointers or an application API. The sender creates the IDoc and dispatches it. The receiver can store the incoming IDoc immediately but postpone application processing until a scheduled job runs.

I therefore track creation, dispatch, receipt and posting as separate milestones. If the sender says it dispatched the message, I still inspect the receiver's status and linked document. Immediate, collected and background settings affect timing. Sender and receiver may also use different technical IDs, so I correlate them using message context and business references rather than assume the numbers match.

Avoid this mistake Creation, dispatch, receipt, and business posting are separate milestones; success at one does not establish success at all later stages.

Read and verify this answer

SAP IDoc partner and port configuration

INTF-04 How do you interpret common IDoc statuses without overstating success?

I read the status together with direction, history and its detailed message. Common inbound statuses are 51 for a posting error, 53 for a posted application document and 64 for pending application processing. Outbound 03 means the data reached the port. It does not prove that the receiving system created the business document.

Reasoning and example

An outbound IDoc shows 03, while the customer says no order exists. That is possible: the sender has handed data to the port, but later transport or receiver processing may have failed. I follow the message into the receiver and inspect its status and application messages. Outbound 30 commonly means it is still ready for dispatch.

For an inbound 51, I read the error instead of guessing from the number alone. For 53, I verify the linked application document and any later business steps the requirement includes. Even a successful posting status does not prove that every downstream approval or delivery process has completed.

Avoid this mistake Read status by direction and meaning, not by whether its number looks low. Outbound success does not prove receiver posting.

Read and verify this answer

SAP IDoc communication status and application outcome

INTF-05 How would you investigate an IDoc that is missing or has failed?

I locate the message using sender, receiver, message type, business key and time. Then I inspect control data, segments and status history. This tells me whether generation, routing, transport or application posting failed. I follow the evidence to the relevant monitor rather than repeatedly checking connectivity for every kind of error.

Reasoning and example

The sender shows dispatch, but no matching IDoc appears on the receiver. I first verify the search criteria, including partner and time range. If the message is genuinely missing, I inspect the technical channel, middleware or relevant RFC queue. That is different from a receiver that already has the IDoc in application-error status 51.

For status 51, I read the business error and check mappings, master data and permissions. WE02 or WE05 can help with classic IDoc inspection, while communication and application monitors show failures outside that record. Repeated destination tests will not repair an invalid material number that the application is correctly rejecting.

Avoid this mistake Repeatedly testing a destination will not fix an application validation error, and an empty IDoc search may simply use the wrong partner or time range.

Read and verify this answer

SAP IDoc partner and port configuration

SAP IDoc communication status and application outcome

INTF-06 How do you reprocess an IDoc without creating duplicate business documents?

I correct the cause and check whether the business document already exists before reprocessing. Then I use the supported restart path for that message and status. After retry, I verify both the IDoc and the business result. Duplicate prevention needs a stable business reference or documented application rule, not blind trust in a technical message number.

Reasoning and example

An IDoc failed during order processing, but the outcome is unclear after a timeout. I search for the expected order using the external business reference before trying again. If it already exists, recovery must follow the documented process rather than create another order. Where appropriate and authorized, BD87 can reprocess the original IDoc.

Creating a test copy or sending a new message gives it a new technical identity. That can bypass duplicate checks that only remember the old IDoc number. I preserve the original history and correlation, then inspect the final status and linked document. Manually making a status look successful does not fix the failed business step.

Avoid this mistake Changing a status to look successful, or creating another IDoc blindly, does not repair the original error and may duplicate its business effect.

Read and verify this answer

SAP IDoc communication status and application outcome

INTF-07 What is SAP Application Interface Framework, and what is it not?

AIF helps teams process, monitor and recover interface messages with useful business context. Depending on configuration, it can provide mappings, checks and actions. It does not replace the underlying transport or automatically cover every interface. I verify the target product's supported features and usage rights before proposing a custom AIF solution.

Reasoning and example

A business operator needs to find failed messages by order number rather than decode a technical queue entry. A configured AIF interface can expose that business key, explain a mapping error and provide an authorized recovery path. The message may still travel through an underlying technology such as IDoc; AIF adds processing and operating support around the configured scenario.

Before designing custom interfaces, I confirm the current product scope and the customer's entitlement. SAP distinguishes full custom-interface use from limited runtime rights included in particular products. Seeing an AIF screen does not prove unrestricted custom-development rights or identical features in every cloud and on-premise edition.

Avoid this mistake AIF does not replace the transport. Its presence does not prove unrestricted custom-use rights or the same features in every product.

Read and verify this answer

SAP AIF monitoring and error handling

SAP AIF licensing and limited runtime usage distinction

INTF-08 How do AIF structure mappings, value mappings, checks, and actions work together?

Structure mapping changes the shape of a message. Value mapping translates codes, such as an external company code into an internal one. Checks validate data, and actions perform the configured work. I keep these responsibilities clear and check the actual interface sequence. Passing AIF checks does not remove the target application's own validation or transaction rules.

Reasoning and example

A partner sends company code EXT01, but SAP expects 1000. Value mapping translates the code, while structure mapping puts it into the field expected by the target. A check can reject a missing or invalid mapping before the posting action calls the application. The operator should see which source value needs attention and who owns that mapping.

I verify the configured engine and processing order rather than assume every AIF interface follows the same steps. The posting action still needs application authorization, duplicate protection and correct commit handling. Adding arbitrary commits can leave partially completed processing that cannot be recovered as intended. Changes to mapping rules also need version-aware testing.

Avoid this mistake The target application still validates its business data. Extra commits inside an action can make recovery inconsistent.

Read and verify this answer

SAP AIF monitoring and error handling

SAP AIF: Interface Creation

INTF-09 How would you configure useful and secure AIF monitoring?

I make messages searchable by useful business keys and send alerts to the people responsible for them. Viewing a payload, editing it and restarting processing need separate permissions. The monitor should explain the failure without exposing unnecessary sensitive data. Recipient assignment helps route work, but it does not replace authorization checks.

Reasoning and example

A finance operator needs to investigate a failed cost-center mapping. The monitor should show the source value, affected business reference and mapping error. That does not mean the operator should see every other interface or be able to change invoice amounts. I test view, edit, restart and cancel permissions separately.

In classic AIF, permissions can be scoped by namespace, interface, version and activity, with relevant field restrictions also considered. Tools such as /AIF/IFMON and /AIF/ERR apply where installed; cloud products use their supported apps and roles. I test access with real restricted identities, because a helpful screen layout alone does not enforce the allowed data scope.

Avoid this mistake Being a monitor recipient does not grant permission to every payload. Viewing and changing sensitive data need explicit access checks.

Read and verify this answer

SAP AIF error-handling authorization

INTF-10 What must you establish before resubmitting or editing a message in AIF?

Before restarting, I check which processing engine owns the message and which steps already succeeded. I also check what the restart will repeat and which fields an operator may change. The correction and retry must preserve the audit trail. AIF restart behavior varies by interface; it does not undo a business document that was already committed.

Reasoning and example

An AIF monitor shows an error for an interface that uses IDoc processing underneath. Another interface may instead run AIF-controlled actions. Their restart behavior can differ, so I inspect the configuration and supported recovery procedure before choosing a button. I avoid starting retries simultaneously in AIF and in the underlying technical monitor.

If the error followed a timeout, I first check for an existing target document. An authorized operator can then correct permitted fields or retry the appropriate failed step. Afterward, I verify the business result and message status. Resubmission is neither a general rollback nor automatically a request to create a completely new business transaction.

Avoid this mistake Resubmission cannot undo an earlier commit. Check what it actually repeats before treating it as a new request.

Read and verify this answer

SAP AIF monitoring and error handling

INTF-11 When would you use OData, IDoc, or AIF?

OData provides HTTP-based queries and business operations. IDoc provides structured messages for established application-integration scenarios. AIF adds processing, monitoring and recovery around supported interfaces. They can work together because they solve different needs. I choose the available business contract and delivery behavior first, then decide how operators will monitor it.

Reasoning and example

A Fiori app needs to read the current status of one request, so a supported OData service can fit. An established partner already sends order messages through an IDoc scenario with asynchronous processing. Replacing that working contract just because OData sounds newer may add cost without improving the business process.

AIF may help operators investigate and recover those messages if the interface is configured for it. It is not a third wire protocol competing with HTTP or IDoc transport. OData also supports system integration, not only UI screens, and IDocs need not wait for batch-only dispatch. The required response timing and supported application interface drive the choice.

Avoid this mistake AIF is not a third wire protocol, OData is not limited to UI use, and IDoc is not restricted to batch-only dispatch.

Read and verify this answer

SAP AIF monitoring and error handling

INTF-12 What security and end-to-end tests would you require for an IDoc or AIF interface?

I test the complete path from incoming message to the business document. That includes permissions, data checks, duplicate handling and recovery after failures. I also test who may view, edit or restart messages. Final checks compare business counts and amounts, because a green monitor or transport acknowledgment alone does not prove the right result.

Reasoning and example

A controlled test sends an order, loses the response and retries it. I verify that recovery finds or safely reuses the first result instead of creating another order. Other tests cover missing mappings, invalid segments, unavailable receivers and messages whose processing depends on order. The actual target document is part of the evidence.

I also log in as an operator who may inspect errors but must not edit amounts or resubmit a restricted interface. Those attempts should be denied. Results should identify the business key, interface version and execution time. A mocked transport can reproduce errors but cannot prove the real receiver's persistence, authorization or duplicate protection.

Avoid this mistake A mock transport or successful technical acknowledgment does not verify the real receiver's persistence, authorization, duplicate handling, or operational recovery.

Read and verify this answer

SAP AIF error-handling authorization

Official references

SAP IDoc control record, message type, basic type, and extension

SAP IDoc partner and port configuration

SAP IDoc communication status and application outcome

SAP AIF monitoring and error handling

SAP AIF error-handling authorization

SAP AIF licensing and limited runtime usage distinction

Supporting skills: HANA decisions and source-to-release delivery

Three focused senior code-review decisions complement the main ABAP Cloud track: choose database implementation from semantics and evidence, protect client and authorization boundaries, and distinguish source history from tested runtime delivery. The fixtures are original; target product and release remain required inputs.

Correct grain before faster execution

Fictional Order 4711: two items; each accidentally joins to two language texts

Item	Amount EUR
10	40
20	60

Correct total: 100 EUR

join →

Item	Language	Amount EUR
10	EN	40
10	DE	40
20	EN	60
20	DE	60

Wrong total: 200 EUR

Review gates before comparing SQL/CDS and AMDP

1 Restore result grain, client isolation and business authorization.
2 For a CDS table function, verify read-only and target-specific client safety.
3 Measure database work, returned rows and end-to-end time on equal results.

One million detail rows -> grouped results can reduce transfer; measure the actual plan.

A faster wrong aggregate is still wrong. General AMDP capabilities differ from the CDS table-function contract; eligible APIs and release support still apply.

A two-language text join duplicates both item amounts. Repair the relationship before accepting a performance rewrite, then verify security, client isolation and the chosen implementation contract.

See the visual atlas for the recall prompt, limits and related questions.

SQLX-01 A CDS table function joins client-dependent purchasing data in SQLScript, writes a helper table, and assumes the outer CDS DCL protects every internal read. Would you approve it?

No. A CDS table function's AMDP database function must be read-only under its documented rules. I remove the write and check every source and join for correct client handling. Business authorization is a separate check: an outer CDS rule does not automatically protect every internal SQLScript read. I also verify target support and allowed dependencies.

Reasoning and example

Two clients contain purchase order 4711. If a SQLScript join matches only the order number, it can combine records from both clients and produce the wrong total or expose data. Filtering the final rows may not undo a total already contaminated by that join. I check client handling in the actual sources and joins, then test overlapping keys.

I separately test users with different purchasing permissions through the real supported read path. SQLScript does not automatically reproduce ABAP SQL or CDS access checks. I also review return types, intended rows and USING dependencies. If supported CDS or ABAP SQL already solves the task, it may be simpler. General AMDP capabilities are broader than this read-only table-function case.

Avoid this mistake The CDS table function is read-only. ABAP Cloud also restricts AMDP procedures to reads; Standard ABAP procedure capabilities are broader. None of these options grants permission to read any SAP table.

Interviewer probes How can joining data from two clients produce a wrong total even if final rows are filtered? / Which tests prove both client separation and the user's purchasing permissions?

Strong answer criteria Distinguishes CDS table-function rules from general AMDP capabilities; Checks client separation and user permissions independently; Verifies target support, sources and returned results

Verification evidence Activate on the target release and inspect the real read path. Test overlapping keys across clients and users with allowed and denied purchasing access.

Read and verify this answer

SAP CDS table functions: SQLScript, read-only and client safety

SAP ABAP data models and access-control concepts

SQLX-02 A report transfers one million item rows to ABAP to calculate totals. A rewrite uses AMDP but doubles amounts after a text join. Which implementation would you recommend?

I fix the wrong total before comparing speed. Then I try a supported ABAP SQL or CDS aggregation and use AMDP only if its extra capabilities are needed. The database should filter and total suitable data before sending it to ABAP. I preserve currencies, permissions and correct joins, and measure the full request rather than assume SQLScript is faster.

Reasoning and example

Order 4711 has two items: 40 EUR and 60 EUR. Its total should be 100 EUR. If each item joins to two language texts, the joined result has four rows and totals 200 EUR. I fix the relationship or aggregation order according to the required result. DISTINCT or dividing by two merely hides the current symptom.

Only after proving the totals do I compare alternatives using realistic data. I test missing texts, duplicate texts, different currencies and null values. Traces should show database work, transferred rows and end-to-end time. Returning fewer rows can reduce transfer, but it does not by itself prove that the database did less work.

Avoid this mistake Do not hide a wrong join with DISTINCT or a fixed division factor. HANA being available is not a reason by itself to choose AMDP.

Interviewer probes Which measurements distinguish too much data transfer from expensive database work? / How would you test several currencies, missing texts and duplicate texts?

Strong answer criteria Proves the totals are correct before accepting faster code; Uses the simplest supported database calculation that fits; Measures database work, transferred rows and total time

Verification evidence Use data with known totals and realistic larger datasets. Compare results, duplicated rows, SQL trace records and full request timings.

Read and verify this answer

SAP Learning: code pushdown with ABAP SQL and CDS

SAP SQL trace analysis

DELIV-01 A pull request passes local linting and its ABAP sources are pushed with abapGit. The team calls the cloud feature deployed. What evidence and delivery steps are missing?

Git stores source history, and abapGit transfers supported development objects. Neither proves that the feature is deployed and usable. I choose the delivery route supported by the actual ABAP product, then verify the target version, activation, tests and configuration. A real service check must show that the intended user can work and a denied user cannot.

Reasoning and example

A pull request passes local lint, but the target system may still reject an object during activation or lack a required dependency. I link the source revision to the component versions, delivery identifiers and target import evidence. For BTP ABAP environment, supported transport/gCTS or add-on scenarios may apply; other ABAP Cloud products have their own delivery routes.

The target checks include syntax, activation, ATC, meaningful ABAP Unit tests and relevant integration tests. I also verify service access, communication setup and recovery ownership. Local lint is not SAP activation, and mocks do not prove real permissions. Technical identities and secrets stay out of Git. A source export alone is not a complete application deployment.

Avoid this mistake An abapGit push is not a transport release, and gCTS is not another name for abapGit. Delivery routes differ across ABAP Cloud products.

Interviewer probes How do you prove the target received the same source revision that was tested? / Which missing configuration can break an active service after a successful import?

Strong answer criteria Separates Git history, source transfer and deployment; Chooses the delivery route for the actual product; Requires target-system results and working configuration

Verification evidence Record the commit, component and dependency versions, and delivery IDs. Keep target activation, ATC, test and import logs plus real allowed-user and denied-user service checks.

Read and verify this answer

SAP: working with abapGit

SAP BTP ABAP environment: delivery via add-on or gCTS

Official references

SAP CDS table functions: SQLScript, read-only and client safety

SAP ABAP data models and access-control concepts

SAP Learning: code pushdown with ABAP SQL and CDS

SAP SQL trace analysis

SAP: working with abapGit

SAP BTP ABAP environment: delivery via add-on or gCTS

Core development mechanisms

Build reliable checks, tests and data flows with small examples. Shared Objects guidance is specific to classic Standard ABAP; verify the release and API contract before using any excerpt.

DEVCORE-01 How do roles, authorization objects and AUTHORITY-CHECK work together?

An administrator assigns roles that give a user the required authorizations. An authorization object groups the fields to check, such as activity and company. AUTHORITY-CHECK compares the requested values with the current user's authorizations. Check sy-subrc immediately: zero means success; any other result must stop the protected operation. The statement reports the result; your code must enforce it.

Reasoning and example

A user may display orders in company 1000 but cannot change them. Checking ACTVT 02 and BUKRS 1000 against the application authorization object fails for that user. The application must then reject the change before it writes anything. Checking display activity 03 would answer the wrong question.

The object and its fields must already exist and match the application security design. Read the organizational value from trusted data, not an unchecked client claim. Do not run another statement that changes sy-subrc before evaluating it. Test an allowed user and a denied user; a hidden button is not the check.

In classic AS ABAP, an administrator maintains authorizations in PFCG roles and generates authorization profiles for assigned users. SAP BTP ABAP environment uses IAM apps, business catalogs and business roles with appropriate restrictions. Administration depends on the product; PFCG is not a universal Cloud setup screen. A role named Approver does not by itself prove that the required field values are present.

In RAP, separately check service access, read restrictions through CDS DCL and operation permissions through global/instance authorization. These layers complement one another. An accessible Fiori tile does not automatically grant every order, and an allowed read does not permit Submit. ADV-02 and TEST-06 explain those boundaries further.

Avoid this mistake AUTHORITY-CHECK only reports its result. Ignoring a nonzero sy-subrc leaves the protected operation open.

Read and verify this answer

SAP Learning: AUTHORITY-CHECK

SAP Learning: roles and authorization profiles

SAP BTP ABAP environment: authorization basics

DEVCORE-02 When are ABAP Shared Objects useful, and how do you access them safely?

In classic ABAP, Shared Objects can hold reusable objects in shared memory on an application server. They are useful for a read-heavy cache, not as a durable work queue. Define the area in SHMA, attach with the required access mode, and detach correctly. Plan rebuilding and invalidation so readers do not rely on stale data.

Reasoning and example

Suppose many sessions read the same tax configuration on server A. ATTACH_FOR_READ accesses an available area version. ATTACH_FOR_WRITE creates a new version; ATTACH_FOR_UPDATE copies the active version when versioning is enabled, or attaches to the active version without versioning. Release read handles with DETACH; finish changes using DETACH_COMMIT or discard them with DETACH_ROLLBACK. Handle missing versions and lock conflicts.

If configuration changes from version 4 to 5, readers need an invalidation and rebuild policy. Server B does not automatically share server A's memory contents. Restart or invalidation must not lose the only record of pending business work. DETACH_COMMIT concerns the area handle; it is not a general database COMMIT substitute, and transactional areas have additional rules. This is classic Standard ABAP guidance, not a promise of ABAP Cloud API availability.

Avoid this mistake A cache is replaceable. Do not use application-server shared memory as the only durable record of messages or unfinished transactions.

Read and verify this answer

SAP: Access to shared object area instances

SAP: Shared Objects example

SAP: Shared Objects introduction

DEVCORE-03 How do you page through changed records without skipping equal timestamps?

Use a stable order with a tie-breaker, such as changed_at plus a unique ID. Continue after the last complete pair, not after the timestamp alone. Limit each page and keep a fixed upper boundary for the run. This avoids one common paging gap, but timestamps alone do not guarantee a complete or consistent delta feed.

Reasoning and example

Orders 7 and 9 both changed at 09:00; order 10 changed at 09:01. If a page ends at (09:00, 7), the next page must include (09:00, 9), then (09:01, 10). A condition changed_at > 09:00 would lose order 9. Use the same pair in the continuation condition and ORDER BY, and advance the extraction checkpoint after the page and continuation are durably staged atomically, or after downstream confirmation when no durable stage exists. Track staged delivery separately.

A transaction can receive a timestamp before it commits and become visible after the reader has passed that position. Rows may also change between pages. A fixed upper time does not create a database snapshot or capture deletions. For reliable extraction, prefer a supported delta or change-log contract; otherwise define overlap, deduplication, deletion handling and reconciliation from the source guarantees. The accompanying example illustrates ordering on fixed in-memory data only.

Avoid this mistake A timestamp plus ID fixes ties, not late commits. Do not describe keyset paging alone as exactly-once delivery or a consistent snapshot.

Read and verify this answer

SAP: ABAP SQL selection and ordering

DEVCORE-04 When do setup, class_setup and teardown run in ABAP Unit?

setup runs before each test method; teardown runs afterwards. class_setup runs once before the class tests, and class_teardown once after them. Use setup for fresh mutable test data. Keep tests independent of execution order. RISK LEVEL HARMLESS describes the intended risk; it does not turn unsafe code into a sandbox.

Reasoning and example

Two tests check a total calculator. setup creates a new input table containing 40 and 60 before each test. One test expects 100. The other clears its input and expects 0. Because each gets fresh data, running the empty-input test first must not break the total test.

Use class_setup for an expensive shared test environment when appropriate, then reset its mutable data in setup. Destroy that environment in class_teardown and release per-test resources in teardown. The example needs no external cleanup because it only uses local memory. A real database write or remote call still needs deliberate isolation; the HARMLESS label does not intercept it.

Avoid this mistake Do not make one test prepare data for another. Shared mutable fixtures and unsafe dependencies remain problems even with RISK LEVEL HARMLESS.

Read and verify this answer

SAP: ABAP Unit fixtures and examples

SAP: Test attributes

DEVCORE-05 How do logs, metrics and traces help explain a failing request?

Logs describe individual events. Metrics summarize measurements, such as request counts and duration. Traces connect the stages of one request across components. Use them together: a metric reveals a pattern, a trace locates a slow stage, and a log explains a specific failure. Correlation IDs connect evidence without recording entire business payloads.

Reasoning and example

Request Q7 reads the database quickly, then its HTTP call times out and retries once. A log records the timeout and correlation ID. A retry counter increases by one. A trace with database and HTTP spans shows where the time went. Together they distinguish a slow remote dependency from slow SQL.

Keep metric dimensions bounded, such as operation and outcome; avoid a new series for every request ID. Put suitable IDs in logs or traces instead, with access controls and redaction. Distinguish event time from the time telemetry was observed or collected: delayed delivery must not make yesterday's failure look like today's new event.

Avoid this mistake Logging every payload creates noise and can expose secrets. A request ID is useful for correlation but usually a poor metric dimension.

Read and verify this answer

OpenTelemetry: Signals

OpenTelemetry: Metrics

OpenTelemetry: Traces

OpenTelemetry: Log timestamps

Official references

SAP Learning: AUTHORITY-CHECK

SAP: Access to shared object area instances

SAP: Shared Objects example

SAP: ABAP SQL selection and ordering

SAP: ABAP Unit fixtures and examples

SAP: Test attributes

OpenTelemetry: Signals

OpenTelemetry: Metrics

OpenTelemetry: Traces

SAP: Shared Objects introduction

OpenTelemetry: Log timestamps

SAP Learning: roles and authorization profiles

SAP BTP ABAP environment: authorization basics